FROM SCRATCH

BUILD AN AI Agent

Agents that reason, plan, and act autonomously

Jungjun Hur
Younghee Song

MANNING

Build an AI Agent (From Scratch)

Agents that reason, plan, and act autonomously

Jungjun Hur
Younghee Song

MANNING
Shelter Island

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity.

For more information, please contact

Special Sales Department
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964
Email: orders@manning.com

© 2026 Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The author and publisher have made every effort to ensure that the information in this book was correct at press time. The author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.

Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Development editor:	Rebecca Johnson
Technical editor:	Vaibhav Tupe
Review editors:	Radmila Ercegovac, Angelina Lazukić
Production editor:	Keri Hales
Copy editor:	Keir Simpson
Proofreader:	Katie Tennant
Technical proofreader:	Ninoslav Cerkez
Typesetter:	Tamara Švelić Sabljić
Cover designer:	Marija Tudor

ISBN 9781633434615
Printed in the United States of America

contents

preface *ix*
acknowledgements *xi*
about this book *xiii*
about the authors *xvii*
about the cover illustration *xviii*

PART 1 BUILDING YOUR FIRST LLM AGENT 1

1 *What is an AI agent?* 3

1.1 The age of AI agents 4

1.2 Understanding LLM agents 5

What is an LLM? 5 ▪ *What is an LLM agent? 6*

1.3 Workflow vs. agent 8

Workflow: Developer-defined flow 9 ▪ *Agent: LLM-directed flow 9* ▪ *Combining workflows and agents in practice 10*

1.4 Tasks that require agents 11

Tasks that require an LLM 11 ▪ *Conditions for using agents 12*
GAIA: An agent gym 13

1.5 Context engineering 13

Why agents fail 14 ▪ From prompt engineering to context engineering 14 ▪ Bigger context isn't always better 15 Five context-engineering strategies 15 ▪ From context engineering to harness engineering 17 ▪ The journey of this book 17

1.6 Prerequisites for reading this book 18

2 *The brain of AI agents: LLMs 20*

2.1 Choosing LLMs for agents 21

Starting with closed LLMs 21 ▪ Expanding to open LLMs 22 Essential LLM capabilities for agents 22

2.2 LLM API basics for building agents 24

Setting up the development environment 24 ▪ Getting started with the OpenAI API 25 ▪ Unifying providers with LiteLLM 26 Conversation management: Handling stateless APIs 27 Generating structured output 28 ▪ Asynchronous calls 29

2.3 Enhancing agent intelligence: Prompt engineering 31

The role of system prompts 31 ▪ Guidelines for agent prompts 33

2.4 Experiencing LLM limitations with GAIA 35

Why GAIA: Setting goals for agent development 35 ▪ Experiment setup 37 ▪ Results and analysis: Why LLMs need tools 42

3 *Enabling actions: Tool use 45*

3.1 LLM tools 47

Why do we need LLM tools? 47 ▪ Types of LLM tools 47

3.2 How LLMs use tools 48

How tool calling works 49 ▪ How an LLM chooses tools 54 Guidelines for effective tool calling 54

3.3 Building tools and tool definitions for LLMs 58

Implementing a web search tool 58 ▪ Converting to tool definitions 61 ▪ End-to-end tool execution 64 The challenges of custom tools 66

3.4 MCP: Standardizing tools 67

The core of MCP: Server–client architecture 68 ▪ Hands-on: Running an MCP server 70 ▪ Understanding the MCP client 72 ▪ Hands-on: Implementing an MCP server 75

4 *Implementing a basic ReAct agent 79*

4.1 How ReAct agents work 80

The think-act cycle 80 ▪ From text parsing to tool calling 81

4.2 Agent architecture overview 82

The completed agent 82 ▪ Information flow: The core design 83
Components we have to build 85

4.3 ExecutionContext: The agent's central storage 85

What happens during agent execution 85 ▪ ExecutionContext implementation 87

4.4 Tool abstraction 88

Why we need a unified tool interface 88 ▪ BaseTool: The foundation 89 ▪ FunctionTool: Wrapping functions 90
Integrating MCP tools 91

4.5 LLM communication layer 93

Reasons for adding a communication layer 93 ▪ LlmRequest: Selecting what to send 94 ▪ LlmResponse: Standardizing what we receive 95 ▪ LlmClient: The provider adapter 95
Putting it together 98

4.6 Implementing the agent 99

Agent class structure 100 ▪ The run() method 101
The step() method 102 ▪ The think() and act() methods 103

4.7 Adding structured output 105

The approach: Tools as output formatters 105 ▪ Modifying the agent 106 ▪ Using structured output in practice 108

4.8 Testing with the GAIA benchmark 109

From LLM to agent 109 ▪ Results 110

PART 2 DEVELOPING ADVANCED AGENT CAPABILITIES..113

5 *Building knowledge bases with RAG 115*

5.1 The problem of using internal data 116

A simple case: Single file 117 ▪ Multiple files 117
Large or extensive data 117

5.2 Types of search methods 118

Keyword search 119 ▪ Vector search 120 ▪ Graph search 121
Structure-based search 122

5.3 Practicing vector search 123

Embedding: Converting text to vectors 123 ▪ Chunking: Dividing long text into search units 124 ▪ Implementing vector search 126 ▪ Why vector databases are necessary 127
Exercise: Finding relevant information from web search results 127

5.4 Structure-based search 131

Preparing the GAIA dataset 131 ▪ Implementing filesystem tools 133 ▪ Connecting tools to the agent 138 ▪ Solving GAIA zip file problems 139

5.5 Extending agents with callbacks 140

The need for agent extension 140 ▪ Implementing tool callbacks 141 ▪ Human-in-the-loop tool-execution approval 144
Compressing search results 146

6 *Adding memory to your agent 149*

6.1 The anatomy of agent memory 150

Limitations of the current memory architecture 150
Three challenges of the current memory architecture 151
Context engineering and memory 153

6.2 Managing context during execution 154

Separating storage from presentation 154 ▪ Sliding-window strategy 156 ▪ Token counting 157 ▪ Compaction strategy 159 ▪ Summarization strategy 162 ▪ Hierarchical context management 164

6.3 Continuous execution: Session and state management 167

The Session class 167 ▪ Managing sessions with SessionManager 169 ▪ Integrating sessions into the agent 171
Basic example: Multiturn conversation 173 ▪ Data structures for tool confirmation 175 ▪ Extending tools for confirmation 177
Implementing pause and resume in the agent 178 ▪ Complete example: HITL workflow 184

6.4 Long-term memory: Accumulating knowledge across sessions 186

The structure of long-term memory 187 ▪ Information extraction: Structured output 188 ▪ Building a vector store with ChromaDB 190 ▪ Implementing TaskMemoryManager 191
Retrieving memories 196

7 *Planning and reflection for complex tasks 201*

7.1 Giving agents time to think 202

The limitations of ReAct 202 ▪ How human experts work 203 Why time to think matters 204

7.2 Planning: Setting direction 204

When planning is necessary 205 ▪ Implementing the planning tool 205 ▪ Planning-tool example 207 Extension directions 208

7.3 Reflection: Checking and correcting 209

When reflection is necessary 210 ▪ Implementing the reflection tool 210 ▪ The real value of reflection: Failure recovery 212 ▪ Running an agent that uses reflection for research synthesis 212

7.4 Integrating planning and reflection 213

8 *Empowering agents with code execution 216*

8.1 Giving agents a computer 217

Limitations of the predefined-tool approach 217 What code execution brings 219 ▪ The effectiveness of code-based actions 222

8.2 Connecting code environments 224

Why sandboxes are necessary 224 ▪ Introduction to E2B 225 Connecting the sandbox to the agent 227 ▪ Testing the code-execution agent 230

8.3 Porting tools to sandboxes 230

The need for tool portability 231 ▪ Extending FunctionTool 232 Implementing the Wikipedia revision tool 233 ▪ Handling sandbox tools in the Agent 234 ▪ Practical testing 236

8.4 workspace: Using filesystems and the CLI 238

The workspace concept 238 ▪ Implementing workspace tools 239 Practical example: Analyzing an Excel file 241

8.5 Agent skills 242

The agent skills concept 242 ▪ Progressive tool-information disclosure 243 ▪ Implementation 245 ▪ Practical example: PDF merge skill 248 ▪ Hierarchical tool structure 249

9 *Orchestrating multi-agent systems 252*

9.1 Why multi-agent? 253

9.2 Three collaboration patterns 255

9.3 Agent workflow: Defining order in code 257

Workflow design principles 258 ▪ Sequential: Step-by-step execution 259 ▪ Parallel: Concurrent execution 261 Loop: Iterative execution 263 ▪ Composing workflows 266

9.4 Agent as Tool: Calling agents as tools 266

Making agents into tools 267 ▪ Implementing AgentTool 267 Context isolation 269 ▪ Practical example: Researcher and coder collaboration 270

9.5 Agent Transfer pattern 271

Differences between Agent as Tool and Transfer 271 ▪ Transfer is also a tool 273 ▪ Agent tree 273 ▪ Implementing the transfer tool 278 ▪ Implementing agent transfer 279

9.6 A2A: Collaborating across networks 281

The core of A2A: Agent card and task request/response 282 Server: Exposing an agent to the network 283 ▪ Client: Calling remote agents 285

10 *Evaluating agents 288*

10.1 Observing an agent 289

Pillars of observability: Metrics, traces, and logs 290 Generating, collecting, and exporting telemetry: OpenTelemetry 291

10.2 Building datasets and establishing evaluation criteria 292

What should we evaluate? 293 ▪ Creating a dataset 294 Analyzing errors 296 ▪ Designing rubrics and metrics 297

10.3 Evaluating with LLM-as-a-judge 298

Types of LLM-as-a-judge 300 ▪ Building a rubric-based evaluation system 301

10.4 Operations: CI/CD 304

Evaluation-gated deployment 304 ▪ Improving the test set and evaluator: Agent quality flywheel 305

appendix OpenAI API Key 307

index 309

preface

When we started building agents, the frameworks were a blessing. LangGraph, CrewAI, AutoGen, and many others made it possible to stand up a working agent in a handful of lines. But every time we tried to go deeper, either to understand why an agent behaved the way it did or to fix a failure that surfaced in production, we ended up reading through the framework's source code. When we were inside, we kept noticing the same thing. Beneath every framework lies the same small set of ideas: message loops, tool calls, execution contexts, memory, planning, and collaboration. The names and defaults differ, but the patterns are shared.

That realization became the seed of this book. Rather than teach any single framework, we wanted to surface the patterns that sit underneath all of them so that readers could build their own agents from scratch and then, armed with that understanding, read any framework's code as though it were their own.

Writing about a field this young has unique difficulties. The landscape kept shifting while we wrote. As the concepts and tooling matured and people's attention widened, new terms emerged, and two ideas in particular reshaped the book as it came together. The first was context engineering. It grew from the realizations that an agent is really a loop of repeated LLM calls and that now that models are capable enough, an agent's quality turns on how well you manage the input that flows through that loop.

The second idea was harness engineering. On one side, agent developers had been looking for ways to raise an agent's performance through the elements that sit outside the model. On the other side, as people began handing agents a far wider range of tasks and real products built on agents (such as coding agents) appeared, interest grew in how users could get the most out of an agent. Harness engineering grew where these two currents met. Today, the environment in which a model runs—that is, the

information an agent can reach—matters as much as the model itself, and how we steer an agent toward the behavior we want has become a central question. Each new concept sent us back to chapters we thought were finished. The process wasn't an easy one, but we believe that the book is better for it, and we hope that readers find the journey as worthwhile as we did.

The scope of this book is narrow on purpose. Rather than survey every framework or document any one of them in detail, it walks through the components of every agent, building them in Python with tools that run on a laptop and hold up in production. Python was the natural choice because the large language model (LLM) ecosystem already lives there, and the supporting libraries, such as LiteLLM for model access and Model Context Protocol (MCP) for tool interoperability, were chosen because they don't obscure the ideas.

We hope that reading this book does for you what writing it did for us: makes agents feel less like black boxes and more like things you can take apart on your desk. Happy building.

acknowledgements

Jungjun Hur: This book would not have been possible without the constant support and encouragement of my family. I would also like to thank Younghee Song for the collaboration, feedback, and support throughout this journey.

Younghee Song: First, I want to thank my family. To my constant source of support—my father, Baek-young Song; my mother, Sun-ee Youn (my forever best friend!); and my brother, Young-han Song—I could not have given myself fully to this work without you. I hope to make you proud as a daughter and a sister.

To my coauthor, Jung-jun Hur: your brilliant insight, persistence, and talent continue to inspire me. I count myself lucky to have written this book with you.

I am also grateful to Yong-min Cho, Dae-il Yang, and Sung-won Shin at PwC for their trust, support, and encouragement.

We are grateful to the team at Manning for making this book real. Vaibhav Tupe's technical review caught issues that would have embarrassed us in print, and Rebecca Johnson's editorial care turned a sprawling draft into something readers can use. We thank Michael Stephens, who believed in this project from the beginning and gave it a home at Manning. Also, we thank the members of the production team who made the book possible.

Thanks go to all the reviewers: Adam Wan, Adriano Freitas, Alain Couniot, Balaji Salem Balasundram, Cass Petrus, Christopher Shehu, David Bluestein II, Frances Buontempo, Ioannis Atsonios, James Liu, Jean Lazarou, Jesús A. Juárez Guerrero, Jiri Pik, Kevin Etienne, Kristina Emelianova, Louis Luangkesorn, Maja Ferle, Maksim Nechaev, Maris Sekar, Mark Winteringham, Matt Ferderer, Meetu Malhotra, Michael Anderson, Micheal Lanham, Nancy Al Kalach, Ninoslav Cerkez, Noel Welsh, Pavel Anni, Peter

Zhang, Rakesh Surapaneni, Richard Meinsen, Richard Vaughan, Robert T Walsh, Roberto Infante, Sanny Mulyono, Saverio Patruno, Sayarnil Banerjee, Seth Weidman, Theo Despoudis, and Victoria Lo. Your suggestions helped make this book better.

Any remaining mistakes are ours alone.

about this book

This book is a practical guide to building an AI agent from the ground up. Its goal is not to teach any single framework but to give readers a complete mental model of what an agent is made of by implementing each layer themselves. By the end, readers should be able to open the source code of any popular agent framework and follow along, as well as design and operate an agent tailored to their own needs.

Who should read this book

This book was written with the following readers in mind:

- Developers who have used LLM APIs but remain unclear about the internals of an agent
- Engineers who want to see what lies behind the magic of frameworks
- Teams who need to build agents tailored to their own data, tools, and workflows
- Practitioners who must operate agents reliably in production

What this book asks of you is a grasp of Python basics and some experience calling APIs. A deep background in machine learning or natural-language processing isn't required. What is required is the willingness to read, run, and modify the code yourself as you learn. These are the only prerequisites for this book.

How this book is organized: A road map

The book is divided into two parts that span 10 chapters. Part 1 builds the smallest working agent from the ground up. Part 2 grows that agent into a practical system and then shows how to evaluate, operate, and improve it over time. Each chapter extends the code of the preceding one, so the agent isn't rewritten along the way—only extended.

- Part 1, "Building your first LLM agent"
  - Chapter 1 examines what an agent is and why it matters. It draws the boundary between workflows and agents and lays out the full map of components the book will cover.
  - Chapter 2 calls the LLM API directly to understand the lowest layer of the agent, which is its communication channel. Using LiteLLM, we also set up an abstraction that isn't locked in to any single provider.
  - Chapter 3 implements tool calling—the way an LLM interacts with the outside world. Using web search and a calculator as examples, we explore the mechanics of function calling. We also look at Model Context Protocol (MCP) as a standard for tools.
  - Chapter 4 implements the ReAct loop of thinking and acting, completing our first fully functional agent. The core abstraction introduced here, `ExecutionContext`, becomes the foundation that every subsequent chapter extends.
- Part 2, "Developing advanced agent capabilities"
  - Chapter 5 implements retrieval-augmented generation (RAG), using embeddings and vector search to work with external knowledge, and adds tools for navigating the filesystem. We also show how to use the callback pattern to control tool behavior flexibly.
  - Chapter 6 tackles the context problem that arises as conversations grow longer. We refine short-term memory with sliding windows, compression, and summarization, and give conversations continuity through sessions and long-term memory.
  - Chapter 7 introduces planning and reflection to handle problems that ReAct alone can't solve. The agent gains a form of metacognition, decomposing tasks on its own and checking its own progress.
  - Chapter 8 moves beyond tool calling, implementing the CodeAct approach, in which the LLM writes and executes code directly. Inside a sandbox environment, we meet a true execution agent that can reach into the filesystem and the command-line interface (CLI).
  - Chapter 9 addresses problems too large for a single agent by coordinating specialized agents. We cover three collaboration patterns—Workflow, Agent as Tool, and Agent Transfer—along with the Agent2Agent (A2A) protocol, which enables collaboration across network boundaries.
  - Chapter 10 completes the agent by showing how to evaluate and operate it. We establish how to measure whether the agent actually works, then use automated evaluation and a disciplined loop of observing, analyzing, and improving to keep the agent improving over time rather than treating it as finished once it runs.

The chapters are designed to be read in order. Because the code of each chapter builds directly on the code of the preceding one, skipping chapters will likely leave gaps in the implementation. If you're already comfortable with LLM APIs or RAG, you may move quickly through chapters 2 and 5, but you should still skim them to see how those pieces connect to the agent you're assembling. The real value of the book comes from running the code alongside the reading, so keep the companion notebooks open as you go.

About the code

The book comes with a companion code repository that contains the full implementation developed across the chapters. The code is organized as a single Python package called `scratch_agents`, together with a set of Jupyter Notebooks (notebooks/ch02 through notebooks/ch10) that mirror the progression of the book. Source code appears in every chapter from chapter 2 onward.

The code was written and tested in Python 3.13. Dependencies are managed with `uv`, and running `uv sync` from the project root will install everything required. The main libraries used throughout the book include OpenAI, Anthropic, and LiteLLM for model access; MCP and FastMCP for tool protocols; Pydantic for typesafe data models; tavily-python for web search; ChromaDB for vector storage; and e2b-code-interpreter for sandboxed code execution. The book also uses `numpy`, scikit-learn, tiktoken, pandas, `openpyxl`, `pymupdf`, `httpx`, Hugging Face datasets, `tqdm`, `python-docx`, and `jupyterlab` for supporting tasks. The examples run on an ordinary laptop and don't require a graphics processing unit (GPU).

Several chapters call paid or quota-limited external services, so a few API keys are required to run the examples. The full list and each key's purpose are documented in the `.env.example` file at the root of the repository.

This book contains many examples of source code, both in numbered listings and inline with normal text. In both cases, source code is formatted in a `fixed-width font like this` to separate it from ordinary text. In rare cases, code is also in **`bold`** to highlight changes from previous steps in the chapter, such as when a new feature is added to an existing line of code.

In many cases, the original source code has been reformatted; we've added line breaks and reworked indentation to accommodate the available page space in the book. In some cases, even this was not enough, and listings include line-continuation markers (➥). Additionally, comments in the source code were often removed from the listings when the code is described in the text.

You can get executable snippets of code from the liveBook (online) version of this book at https://livebook.manning.com/book/build-an-ai-agent-from-scratch. The complete code for the examples in the book is available for download from the Manning website at https://www.manning.com/books/build-an-ai-agent-from-scratch and from GitHub at https://github.com/shangrilar/ai-agent-from-scratch.

liveBook discussion forum

Purchase of *Build an AI Agent (From Scratch)* includes free access to a private web forum run by Manning Publications where you can make comments about the book, ask technical questions, and receive help from the authors and other users. To access the forum and subscribe to it, go to https://livebook.manning.com/book/build-an-ai-agent-from-scratch/discussion.

Manning's commitment to our readers is to provide a venue where meaningful dialogue between individual readers and between readers and the authors can take place. It is not a commitment to any specific amount of participation on the part of the authors, whose contributions to the forum remain voluntary (and unpaid). We suggest that you try asking the authors some challenging questions lest their interest stray! The forum and the archives of previous discussions will be accessible on the publisher's website as long as the book is in print.

about the authors

JUNGJUN HUR is an independent AI consultant and educator specializing in LLM applications and AI agents. He is also the author of *Practical AI Application Development Using LLMs* (Onlybook, 2024), a best-selling book in Korea.

YOUNGHEE SONG is an AI consultant at PwC, where she helps organizations develop and implement practical AI strategies.

Both authors have strong backgrounds in innovating with LLMs and communicating complex technologies to technical audiences. Their combined expertise positions them well to teach readers how to construct LLM agents from the ground up.

about the cover illustration

The figure on the cover of *Build an AI Agent (From Scratch)* is "Femme de Stirie," or "Woman of Styria," taken from a collection by Jacques Grasset de Saint-Sauveur, published in 1788. Each illustration is finely drawn and colored by hand.

In those days, it was easy to identify where people lived and what their trade or station in life was by their dress alone. Manning celebrates the inventiveness and initiative of the computer business with book covers based on the rich diversity of regional culture centuries ago, brought back to life by pictures from collections such as this one.

Part 1

Building your first LLM agent

The term *AI agent* is used broadly, but the underlying idea is simple. An LLM serves as the brain, tools give it a way to act on the world, and a loop ties the two together so the agent can keep working until a task is done. This part focuses on assembling those three elements from scratch so that by the end, you have a small working agent. We skip frameworks on purpose. After you've built the internals yourself, any framework you pick up afterward becomes easy to read because you already know what's happening underneath.

Chapter 1 defines an agent and draws a clear line between agents and the chatbots or automation scripts they're often confused with. It establishes the mental model that carries through the rest of the book.

Chapter 2 is about talking to an LLM directly. You'll learn how to send messages, shape behavior through a system prompt, and parse what comes back, building the thin communication layer that every later chapter depends on.

Chapter 3 gives the LLM a way to reach into the outside world through tools and function calling. You'll convert ordinary Python functions to specifications that the model can understand and invoke.

Chapter 4 ties everything together with a ReAct loop—the cycle of thinking, acting, and observing that turns a one-shot model call into an agent that keeps going until the job is finished.

By the end of part 1, you'll have a working agent of your own and with it a feel for the pieces every agent needs, whatever labels future frameworks stick on them: the conversation record that tracks what has been said and done, the

tool abstraction that lets the model take real actions, the execution state that carries across turns of the loop, and the layer that speaks to the model itself. That small agent becomes the foundation for everything in part 2, and it keeps running unchanged no matter how much we add on top of it.

1 What is an AI agent?

This chapter covers

- The landscape of AI agents today
- LLMs as the decision-making core of agents
- Workflows versus agents and when to use each
- GAIA benchmark for measuring agent performance
- Context engineering for building effective agents

You may have heard of agent-building frameworks such as LangGraph, CrewAI, AutoGen, and OpenAI Agents. These frameworks make it easy to build agents quickly, but they also hide what's happening inside. This book takes a different approach: we'll build agents from scratch, understanding every component before relying on any framework.

Why build from scratch? Agent development is fundamentally about debugging failures. When your agent gives a wrong answer or gets stuck in a loop, you have to understand exactly what went wrong. Did the large language model (LLM) misinterpret the context? Did a tool return unexpected results? Was crucial information

missing? Without understanding how agents work internally, diagnosing these problems is difficult regardless of which tools you use. By building each component yourself, you'll develop the mental model you need to troubleshoot any agent system, whether you built or inherited it.

This chapter establishes the foundation for everything that follows. We'll start by surveying the landscape of AI agents, from personal assistants to specialized coding tools, to understand what we're building toward. Then we'll examine how LLMs serve as the brain of an agent and what distinguishes a true agent from a simple workflow. We'll introduce General AI Assistants (GAIA), the benchmark we'll use throughout this book to measure progress, and explore context engineering, the discipline that determines whether an agent succeeds or fails. By the end, you'll have a clear mental model of what agents are, when to use them, and the principles that will guide your implementation journey.

1.1 The age of AI agents

AI agents are rapidly transforming how we interact with technology. From personal assistants that help with everyday tasks to sophisticated systems that handle complex professional work, agents are becoming integral parts of both consumer products and enterprise solutions. Before diving into how to build them, let's survey the landscape of AI agents to understand what we're working toward:

- *Personal AI agents*—These agents are the most familiar form of AI agents today. Services such as ChatGPT, Claude, and Gemini started as conversational chatbots; users asked them a question, and they provided an answer. But these systems have evolved dramatically. Modern personal agents can analyze uploaded documents, search the web for current information, generate images, write and execute code, and even help with shopping decisions. They serve as general-purpose assistants that adapt to whatever task you bring them, learning your preferences and communication style over time.
- *Customer-facing agents*—These agents operate on behalf of businesses, interacting directly with customers in real time. Going beyond simple FAQ chatbots, they can access company policies, retrieve customer data, process transactions, and make decisions based on business rules. When you interact with a support agent that checks your order status, processes a refund, or helps troubleshoot a product issue, you're increasingly likely to be communicating with an AI agent. These systems must balance helpfulness with strict adherence to company guidelines and regulatory requirements.
- *Specialized agents*—These agents tackle domain-specific tasks that require deep expertise or extended processing time. Coding agents like Claude Code, Cursor, and Codex CLI can navigate codebases, implement features, fix bugs, and refactor code, which are tasks that previously required hours of developer time. Deep Research agents can conduct comprehensive investigations across hundreds of sources, synthesizing findings into detailed reports. Because these tasks often

take significant time to complete, specialized agents frequently operate asynchronously, notifying users when results are ready. When connected to proprietary enterprise data, these agents become vertical solutions tailored to specific industries or use cases.

Despite their different applications and interfaces, these agents have a common foundation: they're powered by LLMs. Understanding how LLMs work and how they enable agent capabilities is essential for anyone who wants to build effective agents. In this book, we use *AI agent* and *LLM agent* interchangeably because virtually all modern AI agents are powered by LLMs.

1.2 Understanding LLM agents

All the agents we explored share something remarkable: they rely on LLMs as their decision-making core. But how does a system designed to predict the next word become capable of completing multistep tasks autonomously? First, let's unpack what makes LLMs uniquely suited to power agents; then we'll examine the three essential components that transform an LLM into an agent. This material helps you understand the agent loop that forms the foundation for everything you'll build in this book.

1.2.1 What is an LLM?

An LLM is a language model trained on nearly all publicly available text from the internet. Although modern LLMs have evolved into multimodal models that process images, audio, and video, this book uses the term *LLM* to refer to all these variants because they share a foundational architecture.

The core principle of LLMs is simple: learning to predict the next word by processing massive amounts of text data. Given the sentence "Because it's summer vacation, I didn't go to _____," most people would expect the blank to be filled with the word *school*. Through large-scale training on this simple principle, LLMs develop an understanding of language structure, context, and meaning. This ability to apply learned patterns to new examples is a form of generalization, as shown in figure 1.1. LLMs process text by breaking it into *tokens*, which are typically words or word fragments. The model predicts the next token based on all preceding tokens.

Recently, reasoning models have emerged that analyze tasks or formulate plans before producing results. The reasoning model doesn't answer immediately; first, it generates thinking tokens starting

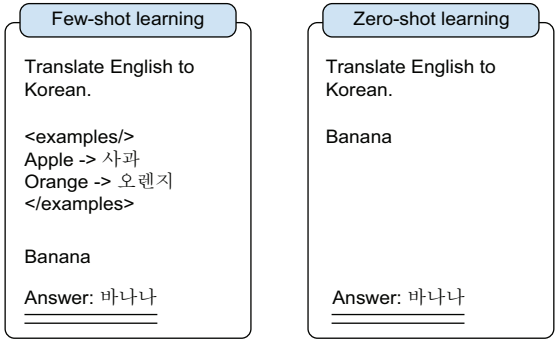

Figure 1.1 Example of a language model's generalization capability

with <think>. This advance enables LLM agents to tackle complex and unfamiliar problems effectively.

1.2.2 What is an LLM agent?

A representative example of an LLM agent is a research agent. When a user asks, "Summarize the 2024 Nobel Physics Prize winners' research," the agent gathers information from multiple sources, analyzes findings, and produces a comprehensive report.

As shown in figure 1.2, the research agent doesn't simply generate an answer from its training data; it also searches for information about the 2024 Nobel Prize, identifies the winners (Geoffrey Hinton and John J. Hopfield), explores academic papers, and consults Wikipedia. Then it synthesizes all this information into a coherent report detailing the winners' achievements and their effect on AI.

What's worth noticing is who decided each step: which source to check first, whether the search results were sufficient, what to look for next, and when to start writing the report. These decisions weren't hardcoded; the LLM made them on the fly, based on the context at each moment.

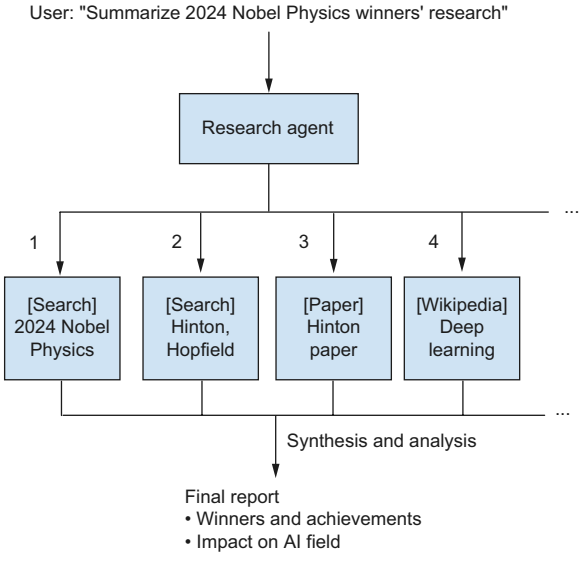

Figure 1.2 User requests flow through the research agent, which branches into multiple searches and synthesis.

CORE DEFINITION: AUTONOMOUS CONTROL FLOW

This example reveals the essence of an agent. An LLM agent is a program that hands part of its control flow to an LLM, letting the model decide what actions to take and when to stop based on its current context and goals.

By *control flow*, we mean the sequence of "what to do next." In traditional software, this flow is spelled out in the code. `if` statements, `for` loops, and the order of function calls

were all decided by the developer in advance. In an LLM agent, part of this decision-making authority is handed over to the LLM.

ELEMENTS THAT REALIZE AUTONOMY: LLM + TOOL + LOOP

For autonomous control flow to work in practice, three structural elements come together. The *LLM* serves as the agent's brain. It understands the current situation and decides what to do next. *Tools* extend the agent's action space into the external world, including web search, code execution, and database access. The *loop* is the execution structure that lets these decisions happen repeatedly until the goal is achieved. To summarize, the LLM makes autonomous decisions possible, tools expand the action space, and the loop unfolds it over time.

HOW THE AGENT LOOP WORKS

The loop is necessary because it's difficult to know in advance which tools will be needed or how many steps will be required to complete a task. Figure 1.3 shows the research task on the 2024 Nobel Prize winners from figure 1.2, viewed through the lens of the agent loop.

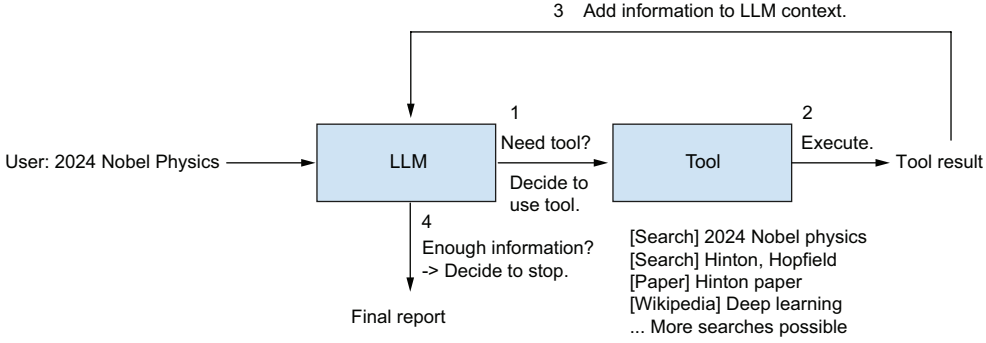

Figure 1.3 The LLM agent's decision loop is an iterative process of LLM decision-making and tool use.

The process follows a continuous loop:

1. The LLM evaluates the current context and determines whether a tool is needed. If more information is required, it decides which tool to use based on what's missing.
2. The selected tool is executed, whether it's searching for "2024 Nobel Physics," looking up "Hinton, Hopfield," finding academic papers, or accessing Wikipedia for background knowledge.
3. The tool's results are added back to the LLM's context, enriching its understanding of the topic. This accumulated context allows the LLM to make more-informed decisions in subsequent iterations.

4 The LLM decides whether to continue or stop. If it determines that sufficient information has been gathered to answer the user's question comprehensively, it generates the final report; otherwise, it returns to step 1 for another iteration.

This iterative process of reasoning, acting, and observing enables the agent to handle tasks of unpredictable complexity, from simple queries that require a single search to complex research that requires dozens of information sources.

1.3 Workflow vs. agent

When integrating LLMs into applications, developers face a fundamental architectural choice: how much control of the execution flow the LLM should have. This decision shapes everything from system reliability to operational costs. Figure 1.4 illustrates seven distinct levels of agency, progressing from left to right with increasing autonomy. The three rows in the figure represent different aspects of control: who produces the current output (current step), who decides what happens next (next step), and who defines the available options (available steps). As we move rightward, the LLM gains more control of each dimension.

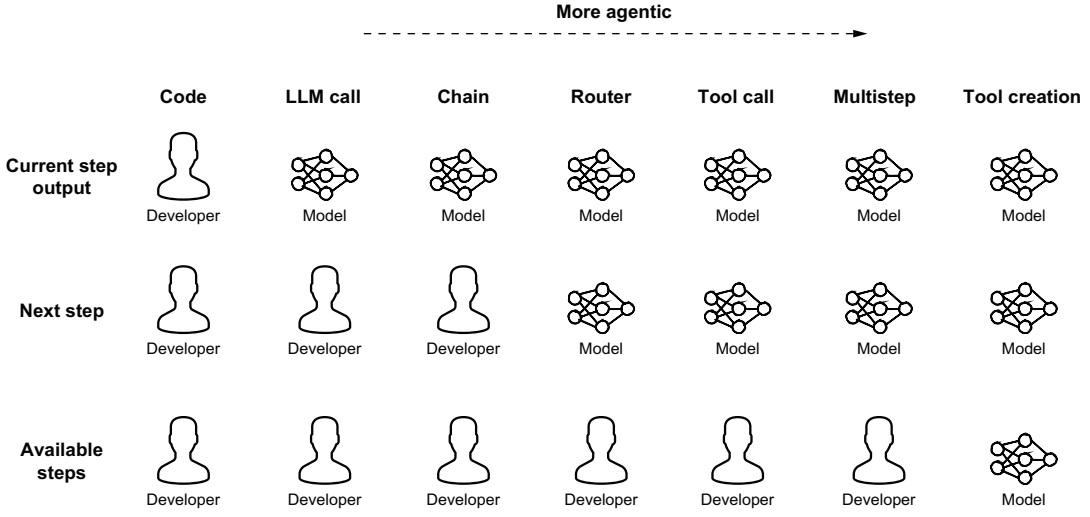

Figure 1.4 Progression of agency levels in LLM applications

These seven levels can be broadly categorized into two architectural approaches: workflows and agents. In *workflows*, developers predefine the execution flow and use LLMs to perform specific steps within that structure. In *agents*, LLMs determine their own processes dynamically, deciding which actions to take and when to stop. Understanding this distinction is essential in choosing the right approach for your use case.

1.3.1 Workflow: Developer-defined flow

A *workflow* is a system in which developers explicitly design the sequence of operations, with LLMs executing specific steps within that predefined structure. The key characteristic is predictability: given the same input, the system follows the same path through the workflow.

LLM CALL

The most basic pattern is a single *LLM call* in which one prompt goes in and one response comes out. This pattern delegates a specific task to the model while keeping everything else under developer control. Examples include text classification, summarization, and answering a straightforward question. Despite its simplicity, a well-crafted single call can handle surprisingly complex tasks.

CHAIN

A *chain* connects multiple LLM calls in a predefined sequence, with the output of one step becoming the input for the next. The developer designs this flow in advance, and the LLM executes each step sequentially.

This pattern follows a fundamental principle: LLMs perform best with focused, well-defined tasks. Rather than ask a model to "analyze this document and create a presentation," a chain might break this request into discrete steps: extract key points, organize by theme, generate slide content, and refine for clarity. Each step is simpler and more likely to succeed than an attempt to do everything at the same time.

ROUTER

A *router* introduces conditional logic that the LLM uses to decide which predefined path to take next. Given a user query, the model might classify it and route it to the appropriate handler. Billing questions go to one workflow; questions about technical support go to another.

Although the LLM makes the decision, it's choosing among a fixed set of options that the developer explicitly defined. The available paths and what happens along each path remain under developer control, making routers a workflow pattern rather than an agent pattern. The LLM influences the route but doesn't control the journey.

1.3.2 Agent: LLM-directed flow

An *agent* is a system in which the LLM dynamically directs its own processes and tool use, maintaining control of how it accomplishes tasks. Rather than follow a predetermined path, the agent decides what to do next based on the current context and its progress toward the goal.

TOOL USE WITH A MULTISTEP LOOP

The defining pattern of an agent combines two capabilities: the ability to use external tools and the autonomy to continue operating until the task is complete. Because LLMs can only generate text, they can't interact directly with external systems, access real-time information, or perform precise calculations. Tools bridge this gap by exposing

external functionalities (such as web search, code execution, database queries, and API calls) as callable functions. The LLM examines the available tools and decides whether one would help; if so, it specifies which tool to call with what parameters.

What transforms tool use into an agent is the loop. Rather than making a single tool call and stopping, the agent operates in a cycle: assess the current state, decide on an action, observe the result, and repeat until the task is complete. The LLM itself determines when to continue and when to stop.

This multistep loop is the core mechanism that enables agents to handle tasks of unpredictable complexity. A simple query might resolve in one step; a complex research task might require dozens of iterations across multiple tools.

TOOL CREATION

At the highest level of autonomy, the agent doesn't just choose among available tools; it creates new ones. This typically involves generating code to implement functionality that doesn't exist in the predefined toolkit.

If, for example, an agent has to process data in a specific format that no existing tool handles, it might write a custom parsing function, execute it, and use the results to continue its task. This capability allows agents to extend their own abilities dynamically, adapting to requirements that weren't anticipated when the system was designed.

1.3.3 Combining workflows and agents in practice

The distinction between workflows and agents isn't always binary in production systems. In practice, the most effective architectures often combine both approaches, using workflows to provide structure and predictability, and embedding agents at specific points where flexibility is needed.

AGENTS AS NODES IN A WORKFLOW

A common pattern is to design an overall workflow with well-defined stages but implement one or more of those stages as an agent. Consider a document-processing pipeline:

1. *Document intake (workflow)*—Validate format and extract metadata.
2. *Content analysis (agent)*—Research context, gather related information, and synthesize findings.
3. *Quality review (workflow)*—Check against compliance rules and format output.
4. *Delivery (workflow)*—Route to the appropriate destination.

The agent operates freely within its designated stage, making multiple tool calls, following chains of inquiry, and adapting to what it discovers. But the overall process maintains the predictability of a workflow. If the agent fails or produces unexpected results, the workflow can catch them at the quality-review stage.

WHY COMBINE?

This hybrid approach offers several advantages:

- *It contains complexity.* Agent behavior is inherently less predictable, so limiting where agents operate makes the overall system easier to reason about and debug.

- *It optimizes costs.* Agent loops can be expensive due to multiple LLM calls, so using them only where their flexibility is genuinely needed keeps operational costs manageable.
- *It enables graceful degradation.* If an agent component fails, the workflow structure allows for fallback behaviors or human escalation at defined points.

The architectural choice isn't workflow versus agent but where in this system agent behavior provides enough value to justify its costs. This pragmatic framing will serve you well as you design your own LLM-powered applications.

1.4 Tasks that require agents

Before jumping into agent development, we have to ask the right questions in the right order. Two key decisions shape whether an agent is the appropriate solution:

- Does this task require an LLM?
- If an LLM is needed, does it require an agent, or will a workflow suffice?

Answering no at either stage means choosing a simpler, more cost-effective approach. Let's examine the criteria for each decision and then look at a benchmark that captures tasks at which agents truly excel.

1.4.1 Tasks that require an LLM

Before considering the use of an LLM agent, you must determine whether the task itself requires an LLM. This foundational question matters because LLMs introduce significant overhead in terms of computational cost and potential for errors. This overhead may be unnecessary if the task can be solved with simpler, more deterministic approaches. If your task involves basic data processing with predictable inputs and outputs, for example, traditional programming logic will be faster, cheaper, and more reliable than any LLM-based solution. You use two main criteria to make this decision:

- *Tasks involving unstructured data*—If a task requires analyzing unstructured data such as text, images, or audio, it's a strong candidate for LLM use. Traditional programming excels at analyzing structured data with clear schemas but struggles with the ambiguity inherent in natural language or visual content. Although models that handle diverse data formats are generally called *large multimodal models* (LMMs), they're fundamentally based on LLMs, so this book collectively refers to them as LLMs.
- *Input diversity*—If user input is limited or the requested tasks are narrowly scoped, using a specialized machine language (ML) model or rule-based system might be more cost effective. Similarly, if the number of potential tasks is small and predefined, it's more efficient in terms of both cost and latency for a developer to hardcode the logic than to use an LLM. LLMs shine when inputs are unpredictable and varied, requiring flexible interpretation that can't be anticipated.

1.4.2 Conditions for using agents

When you've determined that an LLM is necessary, the next question is how to use it. Will a single LLM call suffice, or does the task require the iterative reasoning and tool use that define an agent? Consider the following question:

> *If Eliud Kipchoge could maintain his marathon world record pace indefinitely, how long would it take him to run from Earth to the Moon?*

Answering this question requires multiple steps. First, you have to search to find out who Kipchoge is and what his marathon world record is. Next, you check Wikipedia for the distance between Earth and the Moon. Finally, you perform calculations to divide the distance by the speed and derive the final answer.

Anyone can perform this task manually. But navigating multiple websites to gather information, convert units, and perform calculations is time consuming and error prone. This kind of multistep research task is exactly the kind at which agents shine.

As excitement about agents grows, people fall into a common trap: applying agents to every task simply because they're new and powerful. But you don't need a complex web framework to build a simple static website. Choosing the right tool is key to building an efficient system.

Agents come with inherent tradeoffs:

- *Costs are higher.* Multiple LLM calls multiply API expenses compared with single requests.
- *Latency increases.* Response time accumulates with each reasoning step.
- *Errors propagate.* Mistakes in early steps cascade through the entire process.

Consider a customer service query that could be handled by a single LLM call that costs 1 cent. Processing the same query with an agent that requires 10 calls would cost 10 cents—a 10× difference. When you're handling thousands of requests daily, this decision isn't just technical but also business.

When should you use an agent? You can decide based on these three criteria:

- *Task complexity*—If it's difficult to predict how many steps a task will require, agents have the advantage. "Find the population of region A" is straightforward. By contrast, "Analyze how LLM agents will shape the future" requires gathering diverse information and synthesizing complex relationships. This task is hard to predefine with static logic.
- *Task value*—Because agents require multiple calls and dynamic decision-making, they cost more and respond more slowly. Therefore, the value of completing the task should outweigh the additional cost and latency.
- *Error cost and detectability*—LLMs are prone to make incorrect decisions, and the risk increases with more calls. If errors lead to critical consequences, an agent may not be the best choice. Detectability also matters. In domains that require specialized knowledge, users or developers may not even realize that an agent has produced faulty results.

1.4.3 GAIA: An agent gym

The Kipchoge question we examined earlier is from a benchmark called GAIA, a dataset that systematically collects tasks requiring agents. GAIA consists of question-and-answer pairs. Each question requires multistep reasoning, web searches, calculations, and more. The questions are difficult to solve with a single LLM call, and it's not easy to predefine a clear workflow, either. These problems naturally require an agentic approach.

Meta and Hugging Face released GAIA in 2023. Since then, model capabilities have improved significantly, with more challenging benchmarks emerging. Still, GAIA's problems range from straightforward to genuinely difficult even for current models, making it well suited to learn agent development. This book uses GAIA for three reasons:

- *Clear answers enable fast feedback.* The core of agent development is identifying and reducing failures. When answers are clear, you can verify immediately whether you're heading in the right direction. Then you can iterate rapidly through cycles of experimentation and improvement without using ambiguous evaluation criteria.
- *It's ideal for practicing the agent development cycle.* The process of building an agent goes like this:
  - Determine whether an agent would help solve a specific problem. If there's potential, develop a prototype.
  - Observe when the agent fails, analyze the cause, and improve it.
  - Repeat this observe-analyze-improve cycle. GAIA provides an appropriate level of difficulty for experiencing this cycle from start to finish.
- *No domain knowledge is required.* Most GAIA problems center on web search and information synthesis. We all perform tasks in daily life that involve searching, gathering information, and organizing it. You can intuitively understand the problems and the solution processes without specialized knowledge in medicine or law.

Throughout this book, we'll use GAIA to track changes in agent performance as we add new techniques in each chapter. You'll see firsthand how each component, such as tool use, memory, and planning, improves the agent's problem-solving capabilities.

GAIA gives us a way to measure agent performance. But measurement alone doesn't improve an agent. What makes an agent better at solving these problems? The answer lies in how we engineer the context that the LLM receives.

1.5 Context engineering

Before diving into agent development, we'll clarify two terms that are often used interchangeably. A *prompt* is the input text that a user sends to an LLM. It can be further divided into *system prompts*, which specify how the model should respond and behave (such as "You are a friendly travel guide") and *user prompts* (or messages), which contain the user's actual requests.

Context is a broader concept that encompasses all the information an LLM references when generating a response. This information includes the system prompt, user

messages, conversation history, tool execution results, and retrieved documents. In this sense, prompts are one component of context.

Think of context as the LLM's working memory. Just as you can't solve a math problem if someone gives you only half the equation, an LLM can't complete a task if crucial information is missing from its context.

An LLM predicts the next token based solely on the information within its context window. The model generates text by using patterns learned during training to employ that information. This simple fact is why context engineering is so critical in agent development.

1.5.1 Why agents fail

Agent failures can be attributed to two main causes:

- *Insufficient model intelligence*—This includes cases in which the model fails to solve complex mathematical problems or makes errors in logical reasoning. Addressing this issue requires using a more powerful model or waiting for advances in model capabilities.
- *Lack of necessary information*—Even if a model is intelligent enough, it can't produce correct answers if the information needed to complete the task isn't in the context. If someone asks about the company's vacation policy and the company policy document isn't in the context, even the most capable model can't provide an accurate answer.

Interestingly, a significant portion of real-world agent failures stems from information deficiency. An LLM can be only as smart as the context it's given. This is why what information to include in the context is a key factor in determining agent performance.

1.5.2 From prompt engineering to context engineering

In the early days of LLM adoption, prompt engineering was the primary focus. Practitioners concentrated on how to write system prompts that would make models respond more accurately and on which instructions would yield outputs in the desired format. Techniques like "Let's think step by step" and "You are an expert" emerged during this period.

But the landscape changed as agents began using tools. When an agent performs a web search, the results are added to the context. When the agent executes code, the execution results are added. When the agent queries a database, the query results are added. Context began to change dynamically, evolving beyond static system prompts.

This shift raised a new question: "How do we maintain only the essential information needed for task completion in the context?" The approach that answers this question is context engineering (figure 1.5).

Context engineering is the discipline of providing the information an LLM needs to perform its tasks at the right time and in the right form. It goes beyond simply writing good prompts to comprehensively designing how tool-execution results should be processed, how much conversation history to retain, and when to retrieve external knowledge.

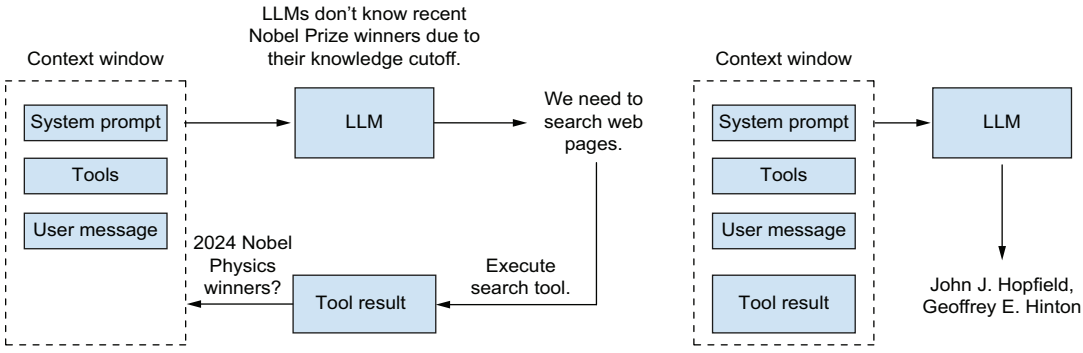

Figure 1.5 LLMs can produce accurate, high-quality responses only when sufficient information is provided in the context.

1.5.3 *Bigger context isn't always better*

The context windows of recent LLMs have grown dramatically. Most models now support context windows ranging from hundreds of thousands to a million tokens. Would filling the entire context window with all available data yield the best responses?

Unfortunately, no. Recent research has reported that model performance degrades as context length increases (figure 1.6). This phenomenon is sometimes called *context rot*.

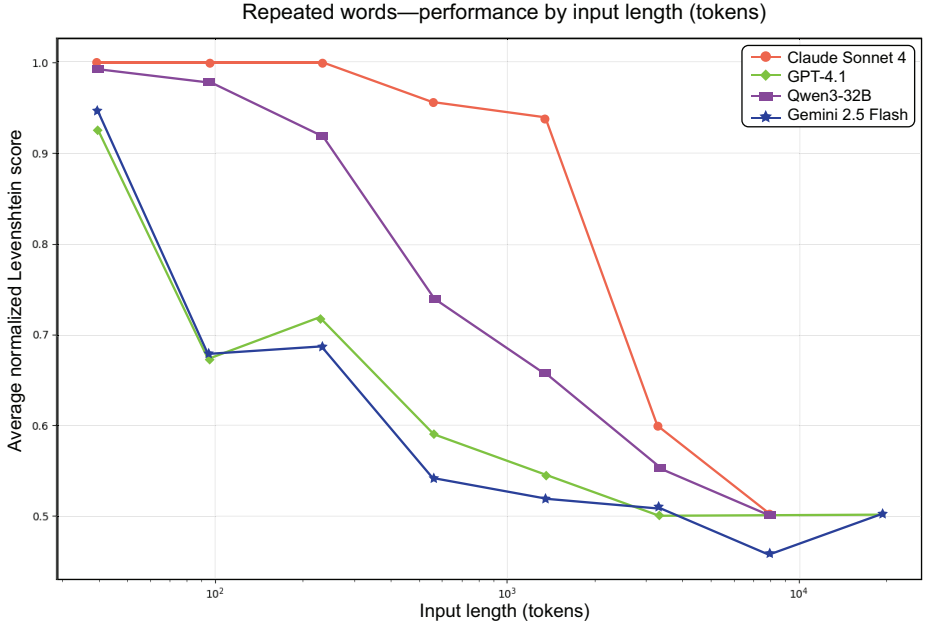

Figure 1.6 Even with large context windows, longer inputs can degrade model performance. (Source: https://research.trychroma.com/context-rot)

The "lost in the middle" effect is particularly well documented; in this effect, models tend to miss important information located in the middle of long contexts.

This provides an important insight: to maximize model performance, we should not simply provide more information but selectively provide only highly relevant information. When the context is filled with unnecessary information, crucial details may get buried, or the model's attention could be dispersed.

1.5.4 Five context-engineering strategies

Context engineering can be broadly categorized into five strategies (table 1.1), each of which is covered in detail in various chapters of this book:

- *Generate*—Using LLM-generated outputs within the context, which includes generating plans for complex tasks or reflecting on completed work to revise strategies. Beyond simply retrieving external information, this strategy lets the LLM structure and evolve the context on its own.
- *Retrieve*—Fetching necessary information from external sources and adding it to the context, which includes web searches, database queries, file reading, and similar document retrieval from vector databases. This core strategy enables agents to access up-to-date information or domain-specific knowledge that isn't present in their training data.
- *Write*—Recording information from the context to external storage, which includes saving important conversation content to long-term memory, recording intermediate calculation results to a scratch pad, and saving generated code to files. This strategy overcomes the limitations of the context window and enables persistent storage of information.
- *Reduce*—Shrinking the context, which includes summarizing old conversations, deleting unnecessary information, and filtering information based on importance. This strategy is essential for preventing context rot and focusing the model's attention on key information.
- *Isolate*—Separating tasks or tools into distinct environments, which includes executing complex code in sandbox environments and assigning different specialized domains to separate agents. This strategy prevents any single context from becoming overly complex and enables the construction of optimized contexts for each task.

Table 1.1 The five major categories of context engineering

Strategy	Description	Related chapters
Generate	Constructing context with LLM-generated outputs	Chapter 7 (planning and reflection)
Retrieve	Bringing information from external sources into context	Chapter 3 (tools), Chapter 5 (knowledge base), Chapter 6 (memory)
Write	Saving from context to external storage	Chapter 6 (memory), Chapter 8 (workspace)

Table 1.1 The five major categories of context engineering (*continued*)

Strategy	Description	Related chapters
Reduce	Compressing and deleting context	Chapter 6 (memory)
Isolate	Separating tasks/tools into distinct environments	Chapter 8 (workspace), Chapter 9 (multi-agent)

Chapter 6 is a hub where three strategies converge: retrieve, write, and reduce. The reason is that memory systems involve retrieving old experiences (retrieve), storing new experiences (write), and summarizing or deleting old memories (reduce).

1.5.5 From context engineering to harness engineering

Earlier, we watched attention move from prompt engineering to context engineering. As the field matured, that attention widened one step outward. The reason is that agents left the benchmark and arrived as real products. As more of them run directly on a user's computer, concrete demands on everything surrounding the model have come into focus. Here are a few examples:

- Users want behavior tuned to their own environment and projects, so an agent has to remember their context and learn the way they repeatedly work (memory and skills).
- As agents began executing code on a user's machine, isolating that execution safely became important (code execution).
- Work no longer stops at a single agent; running and coordinating several agents has become common (multi-agent).

Everything that supports an agent from outside the model is called the *harness*, and designing it is *harness engineering*. The harness is more than just components such as memory and tools. The user's working environment in which the agent runs—that is, the files, projects, and systems it can reach—is part of the harness too. In one line:

$$\text{Agent} = \text{Model} + \text{Harness}$$

Context engineering is part of this harness as well. A new term doesn't replace it; it takes its place within the larger picture surrounding the model.

One thing is worth making clear: harness engineering isn't entirely new. Attaching memory, executing tools safely, and wiring several agents together are problems that agent developers have worked on all along and ones that we will build piece by piece in this book. What changed is that agents have begun to weave into people's lives and work, and as they do, the question of what an agent must do well is taking an increasingly concrete shape. *Harness engineering* is the name given to those crystallizing demands.

1.5.6 The journey of this book

Starting from a basic agent loop, this book applies context-engineering strategies one by one and grows the harness on top of them, building a single complete agent (figure 1.7). We verify the effect of each step quantitatively with the GAIA benchmark.

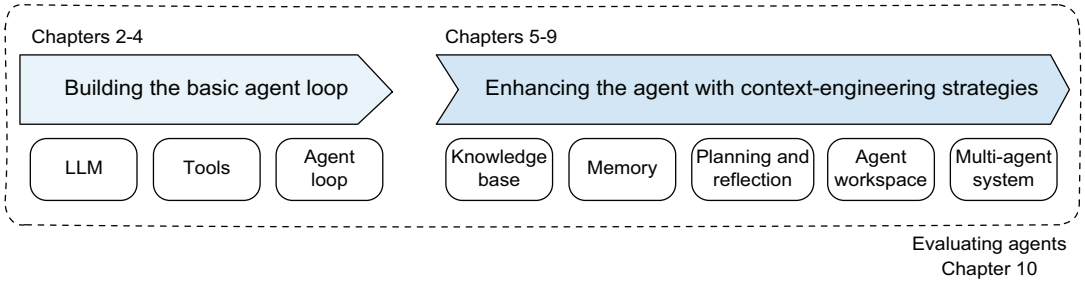

Figure 1.7 An overview of the journey through the book

Chapters 2 through 4 implement the basic agent loop. In chapter 2, you learn to use LLMs and test whether an LLM alone can solve GAIA problems. Some are solvable with an LLM alone, but others need tools such as web search and file reading. Chapter 3 shows how to build tools and make LLMs invoke them, as well as how to integrate tools through MCP without building them from scratch. Chapter 4 combines these tools into an agent loop that uses tools iteratively, built as an abstract `Agent` class. This loop is the foundation of the harness.

Chapters 5 through 9 add new elements to this basic loop, filling in the real-world demands described in this chapter one at a time. Chapter 5 adds retrieval by giving the agent data sources to draw on. Chapter 6 implements conversation management during a task and long-term memory so that the agent can remember the user's context. Chapter 7 sets the direction of complex tasks through planning and reflection. Chapter 8 provides a workspace and skills so the agent can execute code safely, and chapter 9 creates several specialized agents that work together. Memory and skills, safe code execution, and multi-agent setups are demands that took shape as agents became real products, and all are parts of the harness.

Chapter 10 covers how to observe and evaluate agents. It's the final piece for confirming that a well-built agent performs well.

1.6 Prerequisites for reading this book

This book covers the full process of implementing LLM agents from scratch using plain Python. To fully understand and follow the content and hands-on exercises in this book, readers should meet the following basic prerequisites:

- *Basic knowledge of Python*—All examples and exercises in this book are written in Python. Therefore, readers should be familiar with the following Python fundamentals:
  - Basic data structures such as variables, data types, lists, and dictionaries
  - Control flow structures such as `if` statements and `for`/`while` loops
  - Defining and calling functions
  - Basic object-oriented programming (OOP) concepts such as classes

If you're unfamiliar with these topics, it's best to study a beginner-level Python guide beforehand.

- *Development-environment setup*—To complete the exercises effectively, a local Python development environment is necessary:
  - Install the latest version of Python in your local environment (recommended: Python 3.12 or later).
  - Prepare a code editor or IDE (recommended: Visual Studio Code).
  - It's helpful to learn how to install Python packages using `pip` or `uv`.

  API keys and external services:
  - OpenAI API key for accessing GPT models (primary LLM used in examples)
  - Tavily API key for web search functionality in research agents
  - Basic understanding of REST API concepts and HTTP requests

  Cost considerations:
  - *OpenAI*—The total cost of completing all exercises should be less than $10.
  - *Tavily Search API*—Free tier includes up to 1,000 searches per month, which is sufficient for all book exercises.
  - We'll provide tips on cost-effective development and testing throughout the book.

When you've completed these preparations, you're ready to begin your journey into LLM agent development with this book. Chapter 2 establishes the foundational understanding of LLMs that powers everything we'll build throughout the book.

Summary

- AI agents represent a wide spectrum, from personal assistants such as ChatGPT and Claude to customer-facing agents and specialized tools such as Claude Code and Cursor. These agents have a common foundation: LLMs are their decision-making core.
- An LLM agent consists of three elements: the LLM (brain), tools (means of interacting with the external world), and a loop (an iterative process that continues until goal completion). The LLM decides which tool to use and when to stop.
- Workflows are developer-defined execution flows in which LLMs perform specific steps. Agents are LLM-directed flows in which the model determines its own process dynamically. Production systems often combine both approaches.
- Use agents when tasks require multiple unpredictable steps, provide sufficient value to justify costs, and allow error detection. The GAIA benchmark provides ideal practice problems for agent development.
- Context engineering is the discipline of providing the right information at the right time. Five strategies (generate, retrieve, write, reduce, and isolate) form the framework for building effective agents throughout this book.

2 The brain of AI agents: LLMs

This chapter covers

- Selecting the right LLM for agent development
- Using LLM APIs with LiteLLM
- Using prompt engineering for agents
- Experiencing LLM limitations through the GAIA benchmark

Throughout this book, we'll build a research agent—a system that can interpret requests such as "survey recent work on X" and "extract key findings from these PDFs," gather information from multiple sources, and synthesize findings into comprehensive answers. At the heart of this agent is the large language model (LLM), which serves as its decision-making brain. But before we can build an effective agent, we have to answer some fundamental questions. Which LLM should we use, and how do we communicate with it programmatically? What can an LLM do on its own, and where does it fall short?

We'll address these questions head-on in this chapter (figure 2.1), starting by exploring how to choose an LLM for agent development; then we'll get hands-on with APIs, using LiteLLM to work seamlessly across providers. We'll examine prompt

engineering principles that transform a general-purpose LLM into a reliable agent. Finally, we'll put LLMs to the test using the General AI Assistants (GAIA) benchmark, directly experiencing their limitations when faced with real-world problems. This experiment will reveal exactly why agents need tools, setting the stage for everything we build in chapter 3 and beyond.

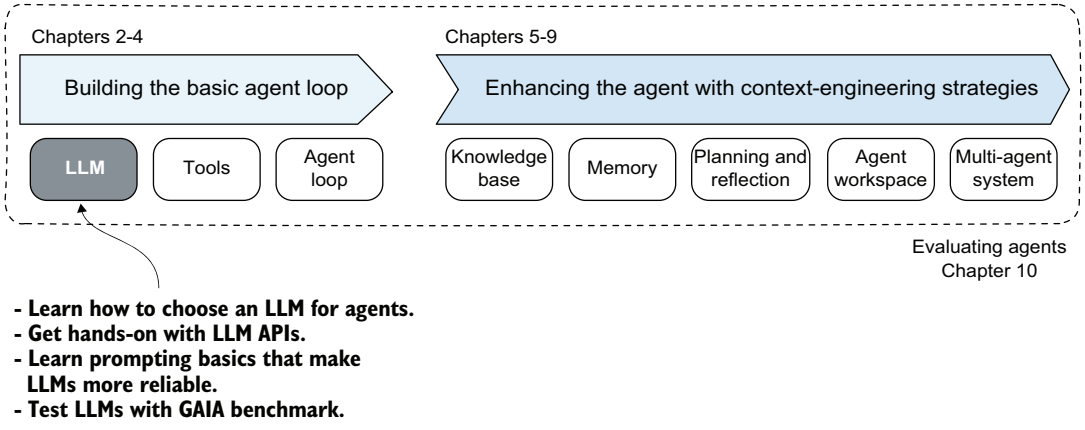

Figure 2.1 Journey through the book: chapter 2

2.1 Choosing LLMs for agents

When starting agent development, the first question you face is, "Which LLM should I use?" This choice directly affects your agent's performance, cost, and development speed. Let's explore how to choose an LLM from an agent developer's perspective.

2.1.1 Starting with closed LLMs

LLMs fall into two broad categories. *Open LLMs* are models whose weights can be downloaded, run locally, and fine-tuned. Meta's Llama, Mistral AI's Mistral, Alibaba's Qwen, and DeepSeek fall into this category. *Closed LLMs* are accessible only through APIs; OpenAI's GPT, Anthropic's Claude, and Google's Gemini are the leading examples. This book focuses on building agents with closed LLMs. This section explores why closed LLMs are the best choice for starting agent development.

Rapid experimentation and validation

The core of agent development is validating ideas and iterating quickly. With closed LLMs, you can start immediately with only an API key. Without preparing graphics processing unit (GPU) servers or deploying model infrastructure, you can focus directly on implementing agent logic. Because you use the same environment from prototype to production, behaviors validated during development work identically in the actual service.

WHEN YOU NEED TOP PERFORMANCE

An agent's performance ultimately depends on the LLM's intelligence. To perform complex tasks, an agent must accurately understand user intent, select appropriate tools, and derive results through multistep reasoning. The best-performing models today are still closed LLMs. Although open LLMs are advancing rapidly, closed LLMs maintain an edge in complex reasoning and tool-use accuracy. When the agent's brain is smart, the entire system works properly.

PRODUCTION-READY FEATURES

Modern agent development requires more than simple text generation. Closed LLM providers offer essential features for agents through stable APIs:

- *Tool calling*—Naturally invokes external tools and APIs
- *Structured output*—Generates responses in structured formats such as JSON
- *Cost optimization*—Uses tokens efficiently through prompt caching and batch processing

These capabilities can be added to open LLMs but require custom implementation or third-party libraries. With closed LLMs, you can use battle-tested features immediately.

2.1.2 Expanding to open LLMs

When you've validated your agent's design with closed LLMs, open LLMs become worth considering.

COST OPTIMIZATION

When you're handling high-volume traffic, self-hosting can be cheaper than using APIs. Simple tasks such as classification and extraction can often be handled well enough by smaller open models. This enables a strategy in which core reasoning stays with high-performance closed LLMs while auxiliary tasks are distributed to open LLMs.

STABILITY AND CONTROL

Depending on external APIs means that their outages become your outages. Self-hosted open LLMs eliminate this external dependency. In addition, when regulations require sensitive data to stay in your infrastructure, open LLMs may be the only option.

A REALISTIC APPROACH

Transitioning to open LLMs is best done gradually. First, validate your agent's design and behavior with closed LLMs; then port the same design to open LLMs and test whether performance degradation falls within acceptable bounds. A hybrid configuration that keeps closed LLMs for core reasoning while replacing only simple tasks with open LLMs is also a good strategy.

2.1.3 Essential LLM capabilities for agents

Regardless of which LLM you choose, certain capabilities are essential for agent development.

TOOL CALLING (FUNCTION CALLING)

For an agent to interact with the external world, it must be able to invoke tools. An LLM's ability to appropriately select and call tools such as web search, code execution, and database queries is central to an agent. We'll cover this capability in detail in chapter 3.

STRUCTURED OUTPUT

LLM outputs often have to integrate with other systems, not just be delivered to humans. The ability to generate structured output matching JSON or specific schemas is essential for integrating agents with real software systems.

LONG CONTEXT SUPPORT

Agents accumulate various types of information in their context, including conversation history, tool-execution results, and retrieved documents. Processing this information effectively requires a sufficiently long context window. Most recent LLMs support hundreds of thousands to a million tokens, but a longer context isn't always better. The key to context engineering is providing the right information at the right time in the right form.

DESIGNING FOR MODEL AGILITY IN A RAPIDLY CHANGING LANDSCAPE

The LLM market evolves at an astonishing pace. New models are released monthly, performance rankings shift regularly, and pricing structures are updated frequently. What's ideal today may be outdated tomorrow, so it's important to design with model flexibility from the start. To stay informed, we can rely on trusted sources:

- *Artificial Analysis* (https://artificialanalysis.ai)—Comprehensive comparison of quality, speed, and cost
- *Arena Leaderboard* (https://lmarena.ai/leaderboard)—User preference rankings based on real interactions
- *Official announcements from providers*—New model releases and pricing updates

This is also why we use LiteLLM (figure 2.2) throughout this book. With the same code, you can easily switch among different providers' models so you can test and transition quickly when new models are released.

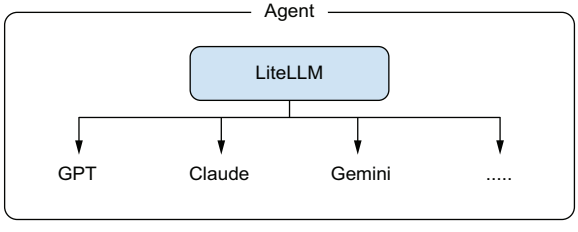

Figure 2.2 LiteLLM, a unified wrapper that lets you call multiple LLM providers through a single interface

OUR CHOICE FOR THIS BOOK: OPENAI API

Throughout this book, we'll primarily use OpenAI's GPT models. For simple examples, we'll use GPT-5 mini to reduce costs, and agent implementations will center on

GPT-5. The OpenAI API is widely used and supported by abundant learning resources, provides stable support for tool calling and structured output, and offers an easy-to-use Python library. It lets us focus on agent logic itself.

Still, we design our code to avoid vendor lock-in. Using LiteLLM (introduced in section 2.2), the same code can easily switch among OpenAI, Anthropic, Google, and even self-hosted open LLMs.

> **Reasoning models**
>
> You may have heard of reasoning models like OpenAI's o series, which spend additional thinking tokens to work through complex problems step by step. These models excel at tasks requiring deep analysis but come with higher latency and cost.
>
> This distinction is becoming increasingly blurred, however. Anthropic's Claude now offers hybrid models that can switch between standard and reasoning modes. OpenAI's GPT-5 takes things further by automatically determining whether to engage in reasoning based on the query, so users no longer have to choose among model types. This trend toward unified models that adapt their reasoning depth to the task at hand simplifies the choice for agent developers.

2.2 LLM API basics for building agents

Now let's get hands-on with LLM APIs. We'll set up the development environment, learn the basics of OpenAI's API, and then use LiteLLM to unify multiple providers under a single interface. We'll also cover conversation management, structured output, and asynchronous calls—all essential capabilities for agent development.

2.2.1 Setting up the development environment

All code examples are available on GitHub at https://mng.bz/vZvJ. Start by cloning the repository:

```
git clone https://github.com/shangrilar/ai-agent-from-scratch
cd ai-agent-from-scratch
```

This book uses `uv` as the Python package manager. `uv` is faster than `pip` and handles virtual environment management and dependency resolution in one step. If you don't have `uv` installed, you can install it with the following command:

```
# macOS/Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
# Windows
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

The repository includes a `pyproject.toml` file, so a single command creates the virtual environment and installs all dependencies. When installation is complete, launch Jupyter Lab to follow along with the examples:

```
uv sync
uv run jupyter lab
```

Next, configure your API keys. Create a `.env` file in the project root directory, and add your API keys. See appendix A for instructions on obtaining API keys from each provider. At minimum, you need an OpenAI API key to follow the examples in this book. You also need a Hugging Face token (`HF_TOKEN`) to access the GAIA benchmark dataset in section 2.4.

To use these environment variables in Python, load the `.env` file using the `python-dotenv` library, which is included in the project dependencies. Because our notebook files are in subdirectories such as `chapter_02/`, we use `find_dotenv()` to locate the `.env` file in the project root.

Listing 2.1 Loading environment variables

```
from dotenv import load_dotenv, find_dotenv
load_dotenv(find_dotenv())
```

The `find_dotenv()` function searches parent directories to find the `.env` file, so it works regardless of which subdirectory you run your code from. When loaded, LLM clients automatically read their API keys from these environment variables. This snippet appears at the top of every notebook in the companion repository, so the `.env` file is loaded before any LLM call in that chapter.

2.2.2 Getting started with the OpenAI API

OpenAI's Chat Completions API has become the industry standard. Most LLM providers offer similar interfaces, so understanding the OpenAI API makes it easy to work with other providers. Let's initialize the OpenAI client and send a simple request.

Listing 2.2 Basic OpenAI API call

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-5-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"}
    ]
)

print(response.choices[0].message.content)
```

The two most important parameters in `chat.completions.create` are `model` and `messages`. The `model` parameter specifies which model to use, and `messages` contains the conversation.

The `messages` parameter is a list of message objects, each with a `role` and `content`. There are three roles:

- `developer` (*previously*, `system`)—The system prompt that defines the model's overall behavior
- `user`—Messages from the user
- `assistant`—Responses generated by the model

The actual response text is in `response.choices[0].message.content`. The reason why `choices` is a list is that you can request multiple responses with the `n` parameter, though typically, you use only the first one.

One common parameter is `temperature`, which ranges from `0` to `2`. Lower values produce more deterministic and consistent responses; higher values generate more creative and varied outputs. Use `0`–`0.3` for tasks that require accuracy, such as data extraction or classification, and `0.7`–`1.0` for creative writing or brainstorming. The `max_tokens` parameter limits the number of tokens generated.

Other providers work similarly. The following listing shows how to use Anthropic's Claude.

Listing 2.3 Anthropic API call

```
from anthropic import Anthropic

client = Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "What is the capital of France?"}
    ]
)

print(response.content[0].text)
```

Although the OpenAI and Anthropic APIs are similar, subtle differences exist, such as method names (`chat.completions.create` versus `messages.create`), response structure (`message.content` versus `content[0].text`), and required parameters (whether `max_tokens` is mandatory). Managing these differences across providers makes development cumbersome. In the next section, we'll see how to solve this problem.

2.2.3 Unifying providers with LiteLLM

When developing agents, you often have to compare models or switch among them based on cost and performance requirements. If each provider has a different interface, these tasks become tedious. LiteLLM solves the problem. It's a library that lets you call more than 100 LLMs through a unified interface.

Listing 2.4 Unified API calls with LiteLLM

```
from litellm import completion

response = completion(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "Hello!"}]
)

response = completion(
    model="claude-sonnet-4-5",
    messages=[{"role": "user", "content": "Hello!"}]
)
```

A single `completion` function handles all providers. Change the model name, and the response format is unified to the OpenAI format. API keys for each provider are read automatically from environment variables.

2.2.4 Conversation management: Handling stateless APIs

When you use web services such as ChatGPT and Claude, they appear to remember previous conversations. But LLM APIs are stateless, so each API call is independent and has no memory of previous calls. You can verify this with the following code.

Listing 2.5 Demonstrating API statelessness

```
from litellm import completion

# First call
response1 = completion(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "My name is Jungjun."}]
)
print(response1.choices[0].message.content)

# Second call
response2 = completion(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "What is my name?"}]
)
print(response2.choices[0].message.content)
```

Even though we introduced ourselves in the first call, the second call has no memory of it. To maintain conversation continuity, you must manage the conversation history manually.

Listing 2.6 Managing conversation history

```
from litellm import completion

messages = []
```

```
# First exchange
messages.append({"role": "user", "content": "My name is Jungjun."})
response1 = completion(model="gpt-5-mini", messages=messages)
assistant_message1 = response1.choices[0].message.content
messages.append({"role": "assistant", "content": assistant_message1})
print(assistant_message1)

# Second exchange - includes previous conversation history
messages.append({"role": "user", "content": "What is my name?"})
response2 = completion(model="gpt-5-mini", messages=messages)
assistant_message2 = response2.choices[0].message.content
print(assistant_message2)
```

We accumulate all conversation content in the `messages` list and pass the entire history with each call. User messages are added with the `user` role, and model responses are added with the `assistant` role. This allows the model to understand previous context and maintain a coherent conversation.

This approach is even more important in agent development. Agents must add not only conversation history but also tool-call results, search results, and intermediate reasoning steps to the context. As conversations grow longer, token use increases, raising costs and slowing response times. Chapter 6 covers efficient context-length management techniques in detail.

2.2.5 Generating structured output

The natural-language text generated by LLMs is great for humans to read but inconvenient for programs to process. To call tools, an agent must use a structured format to output the tool to call and the arguments to use. Structured output makes LLMs generate responses in a defined format, such as JSON.

You can define the desired output format using Pydantic, a Python library for data validation that lets you define expected data structures as classes. By inheriting from `BaseModel`, you create a schema that specifies field names and their types. Pydantic automatically validates data against this schema, raising clear errors if the data doesn't match, which makes it easy to enforce data contracts and catch malformed inputs early. In this example, `ExtractedInfo` defines three fields. The `name` and `email` fields are required strings, and `phone` is an optional string that defaults to `None` if not provided.

Listing 2.7 Structured output with Pydantic

```
from pydantic import BaseModel
from litellm import completion

class ExtractedInfo(BaseModel):
    name: str
    email: str
    phone: str | None = None

response = completion(
    model="gpt-5-mini",
```

```
    messages=[{
        "role": "user",
        "content": "My name is John Smith, my email is john@example.com, and
     my phone is 555-1234."
    }],
    response_format=ExtractedInfo
)

result = response.choices[0].message.content
```

When you pass a Pydantic model to `response_format`, the LLM generates JSON matching that schema. You can use the parsed object directly from the response's `content` attribute.

Structured output plays a crucial role in agent development. In tool calling (chapter 3), the LLM must use a structured format to output which tool to call and which arguments to use. Converting user intent into appropriate actions is also an application of structured output.

2.2.6 Asynchronous calls

When developing agents, you often process multiple LLM requests simultaneously. Examples include evaluating dozens of problems in benchmark tests, comparing responses from multiple models, and running multiple agents concurrently in a multi-agent system.

If you send requests sequentially, the total execution time equals the sum of each request's time. With asynchronous execution, you can start multiple requests simultaneously and process each response as it arrives.

Python handles asynchronous programming with `async/await` syntax and the `asyncio` library. LiteLLM supports asynchronous calls through the `acompletion` function.

Listing 2.8 Asynchronous LLM calls

```
import asyncio
from litellm import acompletion

async def get_response(prompt: str) -> str:
    response = await acompletion(
        model="gpt-5-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

prompts = [
    "What is 2 + 2?",
    "What is the capital of Japan?",
    "Who wrote Romeo and Juliet?"
]

# Execute all requests concurrently
```

```
tasks = [get_response(p) for p in prompts]
results = await asyncio.gather(*tasks)

for prompt, result in zip(prompts, results):
    print(f"Q: {prompt}")
    print(f"A: {result}\n")
```

Functions defined with `async def` are called *coroutines*. The `await` keyword waits for an asynchronous operation to complete and allows other tasks to run in the meantime. `asyncio.gather` executes multiple coroutines concurrently and returns all results together.

If sending three requests sequentially takes 1 second per request, the total time is 3 seconds. By sending requests asynchronously, you can receive all responses in about 1 second, which is the duration of the longest request.

When you send many requests simultaneously, two problems arise: rate limits from the API provider and transient failures from network problems or server overload. LiteLLM's `num_retries` parameter handles transient failures with automatic exponential backoff. For rate limiting, you can use Python's `asyncio.Semaphore` to limit how many requests run concurrently.

The key insight is that `Semaphore` controls access at the `async with` block level. By wrapping the `acompletion` call itself, we ensure that no matter how many tasks are running, the number of actual API calls that can happen at the same time is limited.

Listing 2.9 Rate-limited concurrent requests

```
import asyncio
from litellm import acompletion

# Limit to 10 concurrent requests
semaphore = asyncio.Semaphore(10)

async def call_llm(prompt: str) -> str:
    """LLM call with rate limiting and automatic retry."""
    async with semaphore:
        response = await acompletion(
            model="gpt-5-mini",
            messages=[{"role": "user", "content": prompt}],
            num_retries=3  # Automatic retry with exponential backoff
        )
        return response.choices[0].message.content

# Even with 100 concurrent tasks, only 10 API calls run at a time
prompts = [f"What is {i} + {i}?" for i in range(100)]
tasks = [call_llm(p) for p in prompts]
results = await asyncio.gather(*tasks, return_exceptions=True)
```

The `call_llm` function wraps `acompletion` with both protections. The semaphore ensures that even if we create 100 tasks at the same time, only 10 API calls run simultaneously. When one task completes, the next waiting task proceeds. The `num_retries=3`

parameter tells LiteLLM to retry failed requests automatically up to three times with exponential backoff, which is particularly useful for handling temporary rate-limit errors.

The `return_exceptions=True` argument in `asyncio.gather` prevents a single failure from canceling all other tasks. Instead, exceptions are returned as values in the results list, allowing us to handle failures gracefully and still get results from successful calls. This pattern is essential for benchmark evaluations such as GAIA, in which we need to process hundreds of problems efficiently without overwhelming the API.

2.3 Enhancing agent intelligence: Prompt engineering

After you select an LLM and learn how to use its API, you have to teach the LLM to function properly as an agent. This task is the domain of prompt design.

Prompts fall into two main categories. *User prompt*s are messages that users type in the chat interface, changing every turn and staying beyond the developer's control. *System prompts* are developer-defined instructions that persist throughout the conversation, setting the agent's personality, permissions, constraints, and tool-use policies. This book focuses on system prompt design. A well-crafted system prompt transforms a general-purpose LLM into a reliable agent for specific tasks. Although we can't control user prompts, system prompts enable us to build agents that respond consistently to a wide range of user inputs.

2.3.1 The role of system prompts

System prompts are particularly important for agents because agents act autonomously across multiple steps. System prompts define the behavioral rules that guide all these decisions.

From a context-engineering perspective, a system prompt is information that's always included in the context. Every time the agent calls a tool, analyzes results, or decides on its next action, the system prompt sits at the front of the context, guiding the agent's judgment. This is why the quality of the system prompt determines the quality of the agent's overall behavior.

What does a production agent's system prompt look like? Anthropic publishes Claude's system prompts (https://mng.bz/4nPa). Analyzing these prompts reveals the core principles of agent system-prompt design. Note that the system prompt discussed here is the one that's applied automatically when you use Claude through the website (https://claude.ai) or the mobile app's chat interface. When calling Claude through the API, developers write their own system prompts, so this prompt doesn't apply.

Claude's system prompt serves four main roles: defining the product's identity, specifying output format and style, setting boundaries on prohibited behaviors, and clarifying the limits of its knowledge. The following sections discuss these roles in detail.

Defining product identity

The most fundamental role of a system prompt is to define "who" the agent is. Claude's system prompt includes a product-information section that enables Claude to answer

questions about itself accurately. It specifies, for example, that the current model belongs to the Claude 4.5 model family; that there are three versions (Opus, Sonnet, and Haiku); and that various access methods exist, including the API and Claude Code.

Why is this information necessary? The LLM's training data may contain information about previous versions. By clearly providing current product information in the system prompt, you can prevent the model from stating outdated or inaccurate information. In addition, by instructing the model to direct users to official documentation for features it doesn't know about, you prevent it from providing incorrect information.

The same applies to developing agents. An agent must accurately understand its own role and capabilities to communicate clearly to users what it can and can't do. If you're building a customer service agent, for example, you should define in the system prompt what types of requests the agent can handle and when it should escalate to a human.

SPECIFYING OUTPUT FORMAT AND STYLE

A significant portion of Claude's system prompt is devoted to instructions about tone and formatting. Particularly notable are the detailed guidelines on bullet points and list use. The prompt instructs Claude to avoid excessive formatting and to respond naturally in sentences and paragraphs during typical conversations. Lists are restricted to cases in which the user explicitly requests them or the information is multifaceted enough that bullet points are essential.

Why are such detailed instructions necessary? LLMs tend to follow patterns from their training data, and because bullet points are heavily used in technical documentation and blog posts, models naturally prefer list formats. But excessive formatting feels unnatural in everyday conversation. By explicitly adjusting this instruction in the system prompt, you encourage situation-appropriate responses.

SETTING BEHAVIORAL BOUNDARIES

The most important yet easily overlooked part of a system prompt defines what not to do. Claude's system prompt includes a `refusal_handling` section that clearly specifies types of requests that the model should refuse. These requests include content that threatens child safety; information about making chemical, biological, or nuclear weapons; and instructions for writing malicious code. It also specifies restrictions on creative content featuring real public figures and persuasive content.

Boundary-setting is particularly important in agent development for a specific reason: because agents act autonomously, they can behave in unintended ways if no clear boundaries are set for them. If a customer service agent has the authority to process refunds but not to delete accounts, this authority must be made clear in the system prompt. If a financial agent can provide information but shouldn't give investment advice, that boundary must also be clearly defined.

Claude's prompt has a separate section for legal and financial advice. Instead of making confident recommendations, it instructs the model to provide factual information so users can make their own decisions and remind users that it isn't a lawyer or financial adviser. Defining what not to do is as important as defining what to do.

CLARIFYING KNOWLEDGE LIMITS

Another important task is handling the model's knowledge limitations. Claude's system prompt specifies the knowledge cutoff date and instructs the model to honestly say it can't know about events after that date. It also instructs the model to suggest turning on the web search tool for questions that require current information.

This provides a crucial lesson for agent development: agents must know their own limitations. If a request exceeds those limits, the agent should either use appropriate tools or inform the user honestly. An agent that admits what it doesn't know and suggests alternatives is more trustworthy than one that pretends to know everything.

2.3.2 Guidelines for agent prompts

Now, let's explore prompt-design principles specific to agents. What is the fundamental difference between a chatbot and an agent? A chatbot converses with users, whereas an agent works on their behalf. This difference calls for a fundamentally different approach to prompt design.

ACTING AUTONOMOUSLY

What happens when you ask a typical chatbot, "What's the weather like in Seoul today?" If the chatbot has access to a web search tool, it might respond, "Would you like me to search for that?" This answer seems polite, but from an agent's perspective, it's inefficient. When users ask about the weather, they want an answer, not a decision about whether to search.

Agents should act before asking questions. When the user's intent is clear, execute immediately without asking for permission. If they ask about the weather, search. If a calculation is necessary, calculate. If they ask to analyze a file, open the file. This principle distinguishes agents from chatbots.

That doesn't mean agents should act blindly in every situation. When the user's intent is too ambiguous to determine, clarification is necessary. The key principle is "If it's not ambiguous, don't ask; just execute." When you reflect this principle in a system prompt, you might write something like this:

QS
```
When the user's intent is clear, execute immediately without confirmation.
Only when intent is unclear, ask minimal questions to clarify.
```

The last line is important. It allows the agent to act autonomously while providing a safety net to prevent it from going down the wrong path when the situation is genuinely ambiguous.

USING TOOLS STRATEGICALLY

Acting autonomously essentially means using tools appropriately. An agent's capabilities depend on which tools it can access and how well it uses them. How you design tool-use guidelines in the system prompt determines your agent's performance.

SETTING THE DEFAULT ATTITUDE

First, establish the default attitude toward tool use. "Use when necessary" is vague; "use proactively, without asking permission" is clearer. For an agent with a search tool, you might specify this: "For rapidly changing information (stock prices, breaking news), search immediately. For slower-changing topics (government positions, company CEOs), always search if the question asks about the current status." Clearly defining when to use tools is key.

DEFINING TRIGGER PATTERNS

Providing guidance that helps the LLM identify when to invoke tools based on linguistic signals is effective. If your agent has a tool that searches past conversations, for example, you could define trigger patterns like these:

QS Use the conversation search tool when you see expressions like these:

- Explicit references: "What we discussed...", "As I mentioned before..."
- Time references: "What did we talk about yesterday?" "Show me last week's chat."
- Implicit signals: "You suggested", "We decided", "My project", "That bug".

Providing specific linguistic patterns helps the LLM judge tool invocation timing more accurately. When you're developing agents, it's helpful to list how users typically express situations that require a particular tool.

SPECIFYING NEGATIVE GUIDES

Knowing when not to use tools is equally important. For a search tool, you might specify, "Don't search for timeless information, fundamental concepts, definitions, or well-established technical facts." Questions like "How do I write a for loop in Python?" and "What is the Pythagorean theorem?" don't require searching.

Unnecessary tool calls increase costs and latency. The bigger problem is that if an agent searches for everything, user experience suffers. Nobody wants to use an agent that makes them wait several seconds for answers to simple questions. Specifying when not to use tools is an important part of agent-prompt design.

ANCHORING WITH EXAMPLES

Finally, providing concrete input–output pairs as examples helps the LLM understand the expected behavior much better:

QS Example 1:
User: "What's Samsung Electronics' current stock price?"
Action: [web_search: Samsung Electronics stock price]
Response: Samsung Electronics' current stock price is XX won.
Example 2:
User: "What is the Pythagorean theorem?"
Action: Respond directly without searching
Response: The Pythagorean theorem states that in a right triangle...

With examples like these, the LLM clearly understands how to behave in similar situations. Concrete examples serve as more powerful guides than abstract rules.

Using tool information in the prompt

We've explored how to instruct agents how to use tools. But for an agent to use tools effectively, another element is necessary: descriptions of the tools themselves.

When providing tools to an LLM, you must explain what each tool does, what parameters it requires, and what results it returns. The quality of these descriptions directly affects the LLM's tool-selection accuracy. Even tools with identical functionality can be called at the wrong time or not called at all if their descriptions are vague.

Chapter 3 covers how to write tool descriptions and how the mechanism by which LLMs invoke tools (tool calling) works. For now, remember that designing tool-use guidelines in the system prompt and defining the tools themselves are separate tasks, both of which are critical to agent performance.

2.4 Experiencing LLM limitations with GAIA

We've explored the use of LLM APIs and the principles of prompt design. Now let's apply LLMs to real problems and directly observe their capabilities and limitations. This experiment will make clear why agents need tools.

2.4.1 Why GAIA: Setting goals for agent development

Before diving into the experiment, let's discuss a crucial principle of agent development: you need a clear target to aim for. When you're building any software system, measurable goals are important. Goals become even more critical for agents because their behavior is inherently unpredictable. Unlike traditional software, in which you can trace through deterministic logic, agents make decisions dynamically based on context. Without a concrete benchmark, you might spend weeks tweaking prompts and adding features without knowing whether you're making progress.

Throughout this book, we're building a research agent that can gather information from multiple sources, analyze findings, and produce comprehensive answers. How do we know whether our agent is getting better? We need test cases that meet certain criteria:

- Have clear, verifiable answers (so we can measure accuracy objectively)
- Represent realistic user requests so improvements translate to real-world value
- Cover a range of difficulty levels (so we can track progress as we add capabilities)

This is where GAIA comes in. Chapter 1 introduced GAIA as an ideal benchmark for practicing agent development. Now we'll use it as our development compass.

Thinking of GAIA as simulated user inputs

Here's a useful mental model: imagine each GAIA problem as a request from a real user of our research agent. This user asks, "If Eliud Kipchoge could maintain his marathon pace indefinitely, how long would it take him to run to the Moon?" The user

expects the agent to figure out how to answer, not to ask where to find Kipchoge's pace or the distance between Earth and Moon.

This framing helps us think about agent development the right way. We're not only trying to pass a benchmark; we're also building a system that can handle the kinds of questions real users would ask. GAIA problems happen to have verified answers, which let us measure our progress objectively. But the goal is always to build an agent that genuinely helps users.

LOADING THE GAIA DATASET

Let's load the GAIA dataset from Hugging Face. GAIA is a gated dataset, so you need a Hugging Face account and must accept the dataset's terms of use before accessing it. Visit https://mng.bz/Qwow and follow the on-page instructions to accept the dataset's terms of use. Then ensure that your `HF_TOKEN` environment variable is set as described in section 2.2.1. We'll focus on level-1 problems in this chapter to establish a baseline for what LLMs can do without tools.

Listing 2.10 Loading GAIA level-1 problems

```
from datasets import load_dataset

level1_problems = load_dataset("gaia-benchmark/GAIA", "2023_level1",
split="validation")
print(f"Number of Level 1 problems: {len(level1_problems)}")
```

GAIA problems are categorized into three difficulty levels. Level-1 questions generally require no tools, or at most one tool, with no more than five steps. Level-2 questions involve more steps—roughly between 5 and 10—and require combining tools. Level-3 questions are designed for a near-perfect general assistant, requiring arbitrarily long sequences of actions, any number of tools, and access to the world in general. We start with level 1 because our goal is to establish what LLMs can do before we give them tools.

UNDERSTANDING THE PROBLEM STRUCTURE

Each GAIA problem has a consistent structure. Let's examine the Kipchoge problem from chapter 1:

```
{
    'task_id': 'e1fc63a2-da7a-432f-be78-7c4a95598703',
    'Question': 'If Eliud Kipchoge could maintain his record-making marathon
pace '
                'indefinitely, how many thousand hours would it take him to
run '
                'the distance between the Earth and the Moon at its closest
approach? '
                'Please use the minimum perigee value on the Wikipedia page
for the Moon '
                'when carrying out your calculation. Round your result to the
nearest '
```

```
                '1000 hours and do not use any comma separators if
necessary.',
    'Level': 1,
    'Final answer': '17',
    'file_name': '',
    'Annotator Metadata': {
        'Steps': '1. Googled Eliud Kipchoge marathon pace...',
        'Tools': '1. A web browser.\n2. A search engine.\n3. A calculator.',
        'Number of tools': '3'
    }
}
```

The `Question` field contains what a user would ask our agent. `Final answer` is the ground truth we'll compare against—in this case, `17`. The `file_name` field indicates whether the problem includes an attached file; an empty field means that the problem has no attachment. The `Annotator Metadata` field reveals what the creators of the problem needed to solve it, including a web browser, a search engine, and a calculator.

This metadata is particularly revealing. Even though this problem is at level 1, the easiest category, the annotators needed three tools to solve it. They had to search for Kipchoge's marathon pace, look up the Earth–Moon perigee distance on Wikipedia, and perform calculations.

2.4.2 Experiment setup

To systematically evaluate LLMs on GAIA problems, we have to set up several components. We must define the response format, create an API calling function with proper error handling, select models to test, and establish how we'll judge correctness.

DEFINING THE RESPONSE FORMAT

We'll use structured output, covered in section 2.2.5. This approach ensures that we can reliably extract the model's answer for comparison with the ground truth.

Listing 2.11 Pydantic model for GAIA responses

```
from pydantic import BaseModel
class GaiaOutput(BaseModel):
    is_solvable: bool
    unsolvable_reason: str = ""
    final_answer: str = ""
```

The `is_solvable` field indicates whether the model believes that it can solve the problem with its current capabilities. When `is_solvable` is `False`, the `unsolvable_reason` field captures why the model can't provide an answer (e.g., "I need current information but cannot search the web"). The `final_answer` field contains the response we'll compare with the ground truth. Making both `unsolvable_reason` and `final_answer` optional, with empty-string defaults, allows clean handling of both solvable and unsolvable cases.

BUILDING THE EVALUATION PIPELINE

We use GAIA's standard evaluation prompt, which instructs models to provide answers in a consistent format:

QS You are a general AI assistant. I will ask you a question.

First, determine if you can solve this problem with your current capabilities and set "is_solvable" accordingly.

If you can solve it, set "is_solvable" to true and provide your answer in "final_answer".

If you cannot solve it, set "is_solvable" to false and explain why in "unsolvable_reason".

Your final answer should be a number OR as few words as possible OR a comma-separated list of numbers and/or strings.

If you are asked for a number, don't use a comma to write your number; also don't use units such as $ or a percent sign unless specified otherwise.

If you are asked for a string, don't use articles, neither abbreviations (e.g., for cities), and write the digits in plain text unless specified otherwise.

If you are asked for a comma-separated list, apply the above rules depending on whether the element is a number or a string.

This prompt is designed to elicit answers that can be compared directly against GAIA's ground truth. The formatting rules (no commas in numbers and no units unless specified) ensure consistent comparison.

With our response format and system prompt defined, we have to build the infrastructure to evaluate multiple models across many problems efficiently. This task requires handling several practical concerns: rate limits from different API providers, error handling for failed requests, and parallel execution to keep evaluation time reasonable.

MANAGING API RATE LIMITS

Different providers impose different rate limits on API requests; your specific limits depend on your API tier and usage history. Rather than use a single semaphore for all requests, we create a separate semaphore for each provider. This approach prevents a burst of requests to one provider from blocking requests to another.

Listing 2.12 Provider-specific rate limiting

```
PROVIDER_SEMAPHORES = {
    "openai": asyncio.Semaphore(30),
    "anthropic": asyncio.Semaphore(10),
}

def get_provider(model: str) -> str:
    """Extract provider name from model string."""
    return "anthropic" if model.startswith("anthropic/") else "openai"
```

The semaphore values here (`30` for OpenAI and `10` for Anthropic) are conservative starting points that work for most API tiers. Adjust them based on your specific rate

limits, which you can find in each provider's documentation or dashboard. If you encounter rate-limit errors during execution, reduce the values; if your tier allows higher concurrency, you can increase the values for faster throughput.

THE CORE SOLVING FUNCTION

The `solve_problem` function handles a single API call with proper rate limiting and error handling. It acquires the appropriate semaphore before making the request, ensuring that we never exceed our concurrent-requests limit for any provider.

Listing 2.13 Core problem-solving function

```
async def solve_problem(model: str, question: str) -> GaiaOutput:
    """Solve a single problem and return structured output."""
    provider = get_provider(model)

    async with PROVIDER_SEMAPHORES[provider]:
        response = await acompletion(
            model=model,
            messages=[
                {"role": "system", "content": gaia_prompt},
                {"role": "user", "content": question},
            ],
            response_format=GaiaOutput,
            num_retries=2,
        )
        finish_reason = response.choices[0].finish_reason
        content = response.choices[0].message.content

        if finish_reason == "refusal" or content is None:
            return GaiaOutput(
                is_solvable=False,
                unsolvable_reason=f"Model refused to answer (finish_reason:
{finish_reason})",
                final_answer=""
            )
        return GaiaOutput.model_validate_json(content)
```

The function uses `async with` to acquire and release the semaphore automatically. Inside this block, we call LiteLLM's `acompletion` with our system prompt and the problem question. The `response_format=GaiaOutput` parameter enables structured output, and `num_retries=2` handles transient failures automatically.

We also handle cases in which a model refuses to answer. Some models decline certain questions for safety reasons, returning a `"refusal"` finish reason. Rather than raise an exception, we capture this response as an unsolvable problem with an appropriate reason.

VALIDATING ANSWERS

GAIA uses exact-match evaluation, meaning that the model's answer must match the ground truth exactly (ignoring case and whitespace). The following simple validation function handles the comparison.

Listing 2.14 Answer validation

```
def is_correct(prediction: str | None, answer: str) -> bool:
    """Check exact match between prediction and answer (case-insensitive)."""
    if prediction is None:
        return False
    return prediction.strip().lower() == answer.strip().lower()
```

The case-insensitive comparison and whitespace stripping account for minor formatting differences that shouldn't count as errors. If the prediction is `None` (due to an error or refusal), we return `False`.

EVALUATING INDIVIDUAL PROBLEMS

The `evaluate_gaia_single` function wraps `solve_problem` to capture all relevant metadata about each evaluation attempt. This metadata includes not only whether the answer was correct but also what the model predicted, whether it considered the problem solvable, and any errors that occurred.

Listing 2.15 Single-problem evaluation

```
async def evaluate_gaia_single(problem: dict, model: str) -> dict:
    """Evaluate a single problem-model pair and return result."""
    try:
        output = await solve_problem(model, problem["Question"])
        return {
            "task_id": problem["task_id"],
            "model": model,
            "correct": is_correct(output.final_answer, problem["Final
answer"]),
            "is_solvable": output.is_solvable,
            "prediction": output.final_answer,
            "answer": problem["Final answer"],
            "unsolvable_reason": output.unsolvable_reason,
        }
    except Exception as e:
        return {
            "task_id": problem["task_id"],
            "model": model,
            "correct": False,
            "is_solvable": None,
            "prediction": None,
            "answer": problem["Final answer"],
            "error": str(e),
        }
```

Capturing this metadata is valuable for later analysis. When we examine failures, we can distinguish between problems that the model attempted but got wrong and problems that it recognized as unsolvable. The `unsolvable_reason` field will help us categorize the capabilities the model is missing.

RUNNING THE FULL EXPERIMENT

Finally, we have to orchestrate evaluation across all problems and models. The `run_experiment` function creates tasks for every problem–model combination and executes them concurrently using `asyncio.gather`. We use `tqdm_asyncio.gather` to display a progress bar during execution.

Listing 2.16 Running the full experiment

```
async def run_experiment(
    problems: list[dict],
    models: list[str],
) -> dict[str, list]:
    """Evaluate all models on all problems."""
    tasks = [
        evaluate_gaia_single(problem, model)
        for problem in problems
        for model in models
    ]

    all_results = await tqdm_asyncio.gather(*tasks)

    # Group results by model
    results = {model: [] for model in models}
    for result in all_results:
        results[result["model"]].append(result)

    return results
```

The function returns results grouped by model, making it easy to calculate per-model accuracy and analyze failure patterns.

SELECTING MODELS AND RUNNING THE EXPERIMENT

For this experiment, we test four models from two providers; these models represent different capability tiers. From OpenAI, we use GPT-5 (flagship model) and GPT-5 mini (cost optimized). From Anthropic, we test Claude Sonnet 4.5 (balanced performance) and Claude Haiku 4.5 (fast and economical).

Listing 2.17 Model selection and execution

```
MODELS = [
    "gpt-5",
    "gpt-5-mini",
    "anthropic/claude-sonnet-4-5",
    "anthropic/claude-haiku-4-5"
]

subset = level1_problems.select(range(20))
results = await run_experiment(subset, MODELS)
```

We start with a subset of 20 problems to keep initial experimentation fast and cost effective. When we've verified that the pipeline works correctly, we can expand to the

full dataset. The `level1_problems.select(range(20))` call creates a subset containing the first 20 problems from our level-1 dataset.

2.4.3 Results and analysis: Why LLMs need tools

Let's examine what happened when we ran four models against 20 level-1 GAIA problems. In table 2.1, the Accuracy column lists the fraction of all 20 problems answered correctly, and the Judged solvable column counts how many problems the model attempted instead of declining to answer them as unsolvable. As we'll see, some models score well above the 5/20 (25%) ceiling that tool-free reasoning would predict, for reasons we examine later.

Table 2.1 Experiment results across models

Model	Accuracy	Judged solvable
GPT-5	10/20 (50%)	11/20 (55%)
GPT-5 mini	4/20 (20%)	8/20 (40%)
Claude Sonnet 4.5	3/20 (15%)	4/20 (20%)
Claude Haiku 4.5	3/20 (15%)	4/20 (20%)

ONLY FIVE PROBLEMS ARE TRULY SOLVABLE WITHOUT TOOLS

The most important finding is that only 5 of these 20 problems can be solved without external tools. For these problems, the answer can be derived purely from reasoning or from knowledge reliably present in training data. This means that the theoretical maximum accuracy for any LLM without tools is 25%. Yet we see models attempting far more problems and sometimes succeeding. How?

MODELS VARY IN SELF-ASSESSMENT ACCURACY

One interesting pattern is how models differ in judging whether they can solve a problem. Some models were conservative, judging only four problems as solvable, which is close to the true count of five. These models showed good calibration, meaning that when they said, "I cannot solve this," they were usually correct about needing external information.

Other models were more ambitious, attempting 8 to 11 problems. Surprisingly, they often succeeded on problems that seemingly required external information. The likely explanation is training data. Modern LLMs are trained on vast amounts of web content, including Wikipedia articles and reference materials. When a problem asks about a well-documented fact, the model may have encountered that information during training.

But this "knowledge" is unreliable for agent development. The model can't distinguish between information that it knows accurately and information that it is confabulating. We can't depend on training data to provide current or precise information.

Model refusals

Another observation is that some models refused to answer certain problems, returning a refusal rather than attempting an answer. One GAIA problem contains text written backward:

```
.rewsna eht sa "tfel" drow eht fo etisoppo eht etirw ,ecnetnes siht
dnatsrednu uoy fI
```

When reversed, this text simply asks the model to write the opposite of "left" as the answer. But some models flagged it as a potential attempt to bypass safety filters through obfuscation and refused to engage with it.

Providers and models have varying safety policies that determine what triggers a refusal. A question that one model answers without hesitation might cause another to refuse to answer. These policies evolve as providers update their safety guidelines, and the specific triggers aren't always documented publicly in detail.

For production agents, you should test your specific use case across your target models and have fallback strategies when refusals occur. Check each provider's use policies and safety documentation to understand its general approach, but expect that edge cases will require empirical testing rather than reliance on documentation alone.

Why 15 problems remain unsolvable

Analyzing the problems that models correctly identified as unsolvable reveals clear patterns in what capabilities are missing. Some problems require both web search and file reading, which is why the counts sum to more than 15 (table 2.2).

Table 2.2 Required capabilities for unsolvable problems

Missing capability	Count
Web search and web page access	11
File reading	6

The Kipchoge problem from chapter 1 exemplifies why web access matters. Even if the model happens to know Kipchoge's marathon pace from training, the problem explicitly requires checking "the minimum perigee value on the Wikipedia page for the Moon." The user expects the agent to look this information up, not to guess it from training data.

File-dependent problems present an even clearer limitation. When a problem includes an attached spreadsheet or document, no amount of training data can help. The model simply can't access information it was never given.

Key takeaways

This experiment reveals three critical insights for agent development:

- *LLMs have reasonable self-awareness of their limitations.* When properly prompted, models can often identify when they can't solve a problem. This capability is valuable because an agent that knows what it can't do can use appropriate tools instead.
- *Training data is no substitute for tools.* Even when models "know" information from training, that knowledge is unreliable for precise, current, or verifiable answers. Agents need tools to access authoritative sources.
- *Most real-world questions require external capabilities.* In our sample, 75% of even the easiest GAIA problems required tools. For a research agent to be genuinely useful, it must be able to search the web, read files, and perform calculations.

Now we have a baseline: without tools, even capable LLMs solve only a fraction of level-1 problems. In chapter 3, we'll give our agent the ability to use tools, starting with web search. Then we'll return to these same GAIA problems and measure how much our agent has improved.

Summary

- When you're starting agent development, closed LLMs offer practical advantages: out-of-the-box features such as tool calling and structured output, stable APIs that let you focus on agent logic, and top-tier performance for complex reasoning tasks. Open LLMs become relevant later for cost optimization.
- LiteLLM provides a unified interface across providers, enabling model flexibility in a rapidly evolving landscape. Key API patterns include conversation management, structured output with Pydantic, and asynchronous calls with rate limiting.
- System prompts define agent behavior: product identity, output format, behavioral boundaries, and knowledge limits. For agents, prompts should emphasize taking autonomous action and using tools strategically rather than asking for permission.
- The GAIA benchmark reveals that even capable LLMs solve only 25% of easy problems without tools. The other 75% of problems require external capabilities such as web search and file reading.
- Models show reasonable self-awareness of limitations when they're properly prompted, but training-data knowledge is unreliable for precise, current, or verifiable answers. Agents need tools to access authoritative sources.

3 Enabling actions: Tool use

This chapter covers

- Understanding the limitations of LLMs and why they need tools
- Calling and executing tools
- Building and integrating custom tools
- Using MCP for tool standardization

On their own, large language models (LLMs) can't access external data or interact with external systems. They need tools and the ability to select and use those tools, which is central to implementing agents. A basic agent performs tasks step by step by repeatedly selecting the next action to take, and tools make this possible.

First, we'll explore how to build tools and inform the LLM how to use them. We'll also learn how the LLM chooses the appropriate tool for a given task and relays tool executions back to the LLM. Then we'll examine the Model Context Protocol (MCP) by Anthropic, an initiative that standardizes the development and use of tools, as shown in figure 3.1.

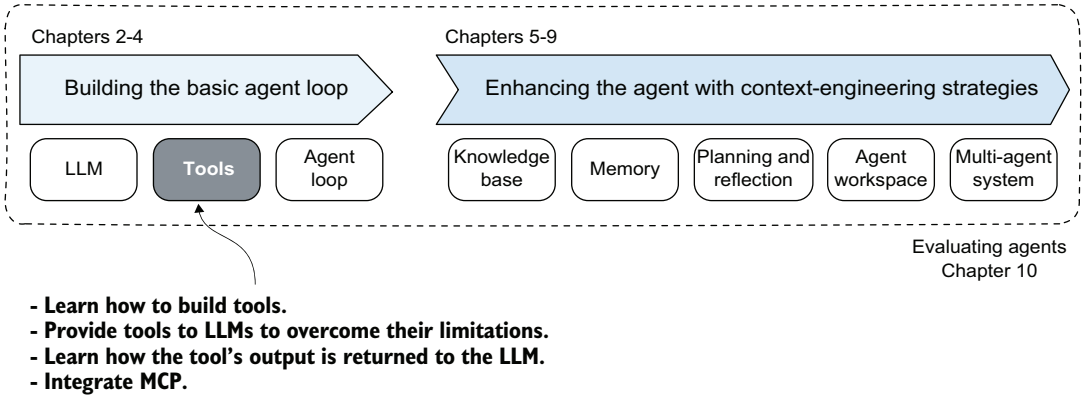

Figure 3.1 Book structure overview: chapter 3

As shown in figure 3.2, we'll construct an LLM system that takes real-world actions using tools, focusing on the tool-calling and tool-execution components. We'll give the LLM context that includes information about the available tools and how to use them (labeled "Tool info" in the figure). If the LLM determines that a tool is needed (1), the tool is executed (2), and the result is added to the LLM's context (3). If the LLM concludes that no further tool use is necessary and that it can generate a final answer directly, it outputs that response (4), which is added to the context and returned to the user (5).

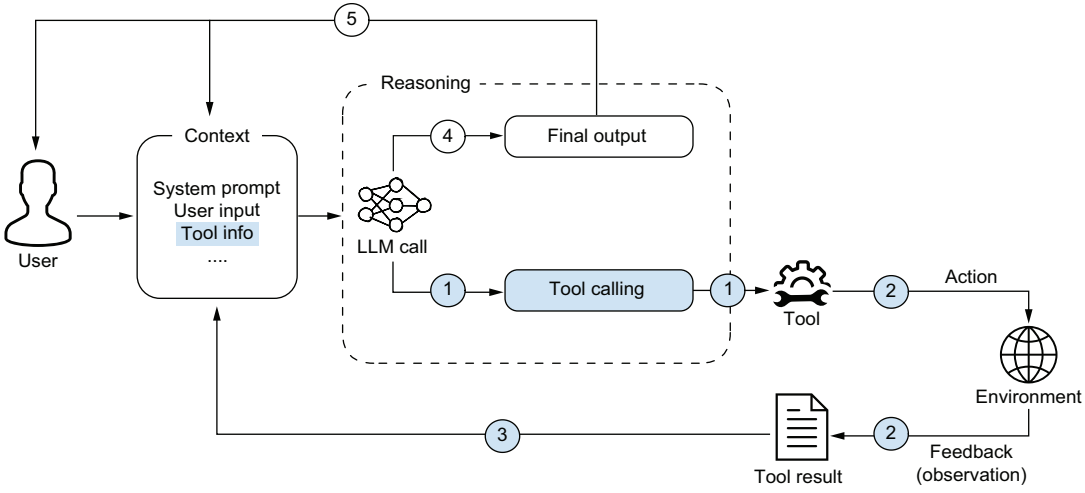

Figure 3.2 The LLM receives user input, reasons about it, selects the necessary tool, receives the execution result, and produces a final response.

What does it take for an LLM to become an agent that interacts with the external world and achieves goals? Let's take a closer look at the types of tools that help LLMs overcome their limitations.

3.1 LLM tools

Despite their impressive capabilities, LLMs have inherent limitations that prevent them from functioning as fully autonomous agents. These limitations fall into three categories, all of which tools address:

- *Temporal*—LLMs face a temporal limitation. They can't access information beyond their training data or retrieve data that changes in real time. This limitation makes them unreliable for tasks that require current knowledge, such as answering questions about recent events or live market conditions.
- *Interaction*—LLMs have an interaction limitation, meaning that they can't influence the external world directly. Although they can generate text describing an action, they can't book a flight, send an email, or control a device.
- *Functional*—LLMs suffer from functional limitations in specialized domains. Tasks that require precise computation, code execution, or media generation often exceed what language models can reliably accomplish through text prediction alone.

3.1.1 Why do we need LLM tools?

Information-augmenting tools bridge the temporal gap by injecting up-to-date knowledge into the agent's context. These tools include web search engines, domain-specific retrieval systems such as arXiv and custom databases, real-time APIs for weather or stock prices, and internal knowledge systems that query company repositories. As shown in figure 3.2 earlier, these tools deliver results that extend the LLM's awareness beyond its training cutoff.

Action-executing tools overcome the interaction limitation by enabling agents to affect the real world. Examples include booking systems for hotels and flights, communication tools for emails and calendar management, automation tools for document generation, and Internet of Things (IoT) controllers for smart devices. These tools trigger actions and receive observations in return, forming a feedback loop with the environment.

Domain-specialized tools address functional limitations by handling tasks where LLMs fall short. Calculators, code interpreters, format converters, image generators, and voice synthesis engines allow hybrid problem solving, combining the LLM's reasoning with reliable execution in specialized domains.

3.1.2 Types of LLM tools

Tools are categorized as custom-built by the developer or predefined by the LLM provider. Custom tools are developed or adapted by the developer to fit a specific environment. Examples include APIs that integrate with internal company systems or scripts for specialized data analysis. Custom tools can be fully tailored to project requirements.

But this also means that the developer is responsible for building, maintaining, and securing them, so bugs and security problems are the developer's responsibility.

Noncustom tools are predefined and offered directly by the LLM provider. Examples include OpenAI's web search feature and Anthropic's web search tool. These tools can be used immediately without any development effort, and the provider manages stability and performance. The tradeoffs are that they can't be easily modified or extended and can be changed or removed at the provider's discretion. You can find the list of available noncustom tools in the LLM provider's official documentation.

LLM providers are continuously expanding their built-in tool offerings. OpenAI provides built-in tools such as web search, file search, and code interpreter through the Responses API. Anthropic offers computer use and web search tools. Google's Gemini includes code execution and Google Search integration. These tools are production ready and well maintained, and they require no implementation effort. Given this trend, you may wonder why you should build custom tools at all.

This book focuses on custom tools for two reasons:

- *Understanding how tools work internally is essential for debugging agent failures.* When an agent produces unexpected results, you need to know whether the problem lies in the tool's implementation, the LLM's tool selection, or the way results are processed. Built-in tools are black boxes that obscure these details.
- *Real-world agents often require specialized capabilities that providers don't offer.* Querying your company's internal database, integrating with proprietary APIs, or implementing domain-specific logic requires custom tools.

That said, provider tools have their place. When you're doing rapid prototyping or a built-in tool matches your needs, using provider tools makes sense. The skills you develop building custom tools will help you evaluate when to build and when to use what's already available.

3.2 How LLMs use tools

How does a model that can only generate text select and execute a tool? The answer lies in a mechanism called *tool calling*. To better understand tool calling, consider an analogy. Imagine the LLM as a wise sage locked in a room with no tools. Though it has learned a vast amount from countless texts, it can communicate only by passing notes under the door. It doesn't know what tools exist outside, how they work, or how to use them directly. That's where we come in: we send a note explaining what tools are available, how each tool functions, and what task must be completed.

We can think of this note as a *tool definition*—a structured description that tells the LLM what tools (functions, APIs, or actions) are accessible, what parameters they expect, and what they do. When the LLM receives this information, it can begin reasoning purely through text generation about which tool is appropriate for a given task and how to use it. After the LLM decides which tool is required, it generates a *tool call*—a structured output that includes the name of the tool and the arguments required to invoke it. This tool call is passed to an external executor, which runs the tool or API in the real world.

Next, let's take a closer look at how a tool definition is structured and provided to the model and how the tool call generation and execution process works. You can find all the code examples for this section in our GitHub repository at https://github.com/shangrilar/ai-agent-from-scratch.

3.2.1 How tool calling works

Tool calling involves a multistep process, illustrated in figure 3.3. To initiate tool use, the developer sends a message (representing user input) to the LLM, along with tool definitions. This is step 1.

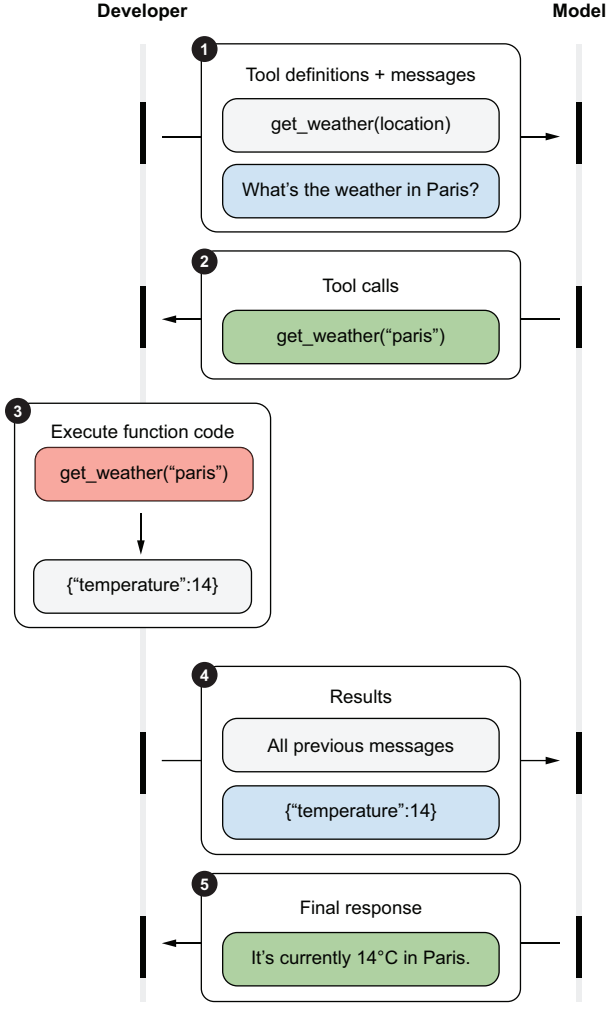

Figure 3.3 Key steps in tool calling: tool definition, delivery, and LLM tool selection

One important thing to note is that the LLM doesn't execute tools itself. Rather, it generates a textual specification indicating which tool should be called and with what parameters. In this sense, the LLM acts as a mediator between the user's request and the available tools. The LLM-based application or agent handles the execution of the tool, which corresponds to step 3 in figure 3.3. When the tool is executed, the result is returned to the LLM, which interprets the output and continues the conversation. The final response to the user incorporates this result.

Therefore, to enable tool calling, we must do the following:

1. Define the tool definitions provided to the LLM.
2. Define the tools.
3. Give the LLM the task to perform and the tool definitions from step 1.
4. Build a system that can execute the tools based on the tool calls generated by the LLM.
5. Reflect the execution results back into the LLM's context.

Next, let's look at how to implement these steps.

STEP 1: SPECIFY TOOL DEFINITIONS

The first step is creating a structured definition that tells the LLM what tools are available and how to use them. Think of this step as an instruction manual for the LLM. Let's examine how it works by looking at a calculator tool definition from the outside in.

Starting at the outermost level, we specify `"type": "function"` to indicate that this tool is callable. Inside, we define the `function` object with three essential components:

- `name`—The tool's identifier that the LLM will use to call it (`"calculator"`)
- `description`—When and why to use this tool ("Perform basic arithmetic operations")
- `parameters`—The specification of inputs required to use the tool

The `parameters` section follows a JSON schema format, defining each input the tool expects. For our calculator, we need three inputs: an operator (limited to adding, subtracting, multiplying, or dividing through the `enum` constraint) and two numbers. The `required` field ensures that the LLM always provides all necessary inputs.

Listing 3.1 Calculator tool definition schema

```
calculator_tool_definition = {
    "type": "function",
    "function": {
        "name": "calculator",
        "description": "Perform basic arithmetic operations.",
        "parameters": {
            "type": "object",
            "properties": {
                "operator": {
                    "type": "string",
                    "description": "Arithmetic operation to perform",
```

```
                    "enum": ["add", "subtract", "multiply", "divide"]
                },
                "first_number": {
                    "type": "number",
                    "description": "First number for the calculation"
                },
                "second_number": {
                    "type": "number",
                    "description": "Second number for the calculation"
                }
            },
            "required": ["operator", "first_number", "second_number"],
        }
    }
}
```

STEP 2: SET UP THE TOOL

We can define a Python function such as `calculator()` to match the input schema provided earlier. Based on the function name and arguments generated by the LLM, we invoke the correct function with the appropriate parameters.

Listing 3.2 Implementation of calculator function

```
def calculator(operator: str, first_number: float, second_number: float):
   if operator == 'add':
       return first_number + second_number
   elif operator == 'subtract':
       return first_number - second_number
   elif operator == 'multiply':
       return first_number * second_number
   elif operator == 'divide':
       if second_number == 0:
           raise ValueError("Cannot divide by zero")
       return first_number / second_number
   else:
       raise ValueError(f"Unsupported operator: {operator}")
```

STEP 3: EXECUTING THE TOOL CALLING

The LLM receives the tool definitions and the user's question. Then it decides whether a tool is needed and, if so, which one. Consider two queries: "What's the capital of South Korea?" and "What is 1234 x 5678?" Using a tool definition that defines a calculator, the LLM responds to the first question with a plain-text answer but responds to the second by generating a tool call, including the correct operator (`multiply`) and operands (`1234, 5678`).

Listing 3.3 Tool-calling execution example

```
tools = [calculator_tool_definition]

response_without_tool = completion(
```

```
        model='gpt-5-mini',
        messages=[{"role": "user",
➥"content": "What is the capital of South Korea?"}],
        tools=tools
)
print(response_without_tool.choices[0].message.content)
print(response_without_tool.choices[0].message.tool_calls)

response_with_tool = completion(
        model='gpt-5-mini',
        messages=[{"role": "user", "content": "What is 1234 x 5678?"}],
        tools=tools
)
print(response_with_tool.choices[0].message.content)
print(response_with_tool.choices[0].message.tool_calls)
```

The output is

```
# Capital question doesn't need tool call
The capital of South Korea is Seoul.
None
# Multiplication question needs tool call
None
[ChatCompletionMessageFunctionToolCall(id='call_viaOEiQJ5VEB9YvKl95qlDjM',
function=Function(arguments='{"operator":"multiply","first_
number":1234,"second_number":5678}', name='calculator'), type='function')]
```

As you can see from the output, the model demonstrates intelligent tool selection. For the capital question, the model provides a direct answer ("The capital of South Korea is Seoul") without invoking any tools, indicated by `tool_calls` being `None` because this information is already present in its training data. For the multiplication question, the model recognizes that it needs computational assistance and generates a tool call with the appropriate parameters; the `content` field becomes `None`. This selective behavior shows that LLMs can discern when tools are necessary and when they can answer directly from their knowledge base.

STEP 4: RUNNING THE TOOL

This step executes the tool that the LLM requested. Because the LLM itself can't run any code, we, as the host system, must perform the execution.

The function name and arguments returned by the LLM through tool calling—`function_name` and `function_args`—are extracted. If the selected function is the calculator, the function is `executed` by passing in the arguments generated by the LLM.

Listing 3.4 Extracting and executing the tool from the response

```
ai_message = response_with_tool.choices[0].message

if ai_message.tool_calls:
   for tool_call in ai_message.tool_calls:
       function_name = tool_call.function.name
```

```
        function_args = json.loads(tool_call.function.arguments)

        if function_name == "calculator":
            result = calculator(**function_args)
```

Step 5: Feeding results back to the LLM

Finally, we feed the tool result back into the LLM by appending a new message to the conversation with a role of `"tool"` and including the output in the content. The updated context is passed back to the LLM, which generates the final response.

Listing 3.5 Feeding tool results back to the LLM

```
ai_message = response_with_tool.choices[0].message
messages.append({
    "role": "assistant",
    "content": ai_message.content,
    "tool_calls": ai_message.tool_calls
})

if ai_message.tool_calls:
    for tool_call in ai_message.tool_calls:
        function_name = tool_call.function.name
        function_args = json.loads(tool_call.function.arguments)

        if function_name == "calculator":
            result = calculator(**function_args)

            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": str(result)
            })

final_response = completion(
    model='gpt-5-mini',
    messages=messages
)
print("Messages: ", messages)
print("Final Answer:", final_response.choices[0].message.content)
```

Append the AI's message with tool calls to the conversation history.

Extract the tool call details and execute the calculator function.

Add the tool execution result to the messages with the tool role.

The output is

```
Messages: [{'role': 'user', 'content': 'What is 1234 x 5678?'},
➥{'role': 'assistant', 'content': None, 'tool_calls':
➥[ChatCompletionMessageFunctionToolCall(id='call_TN8z8oUZ4g8hLwRfYTYFhDGD',
➥function=Function(arguments='{"operator":"multiply",
➥"first_number":1234,"second_number":5678}', name='calculator'),
➥type='function')]},
➥{'role': 'tool', 'tool_call_id': 'call_TN8z8oUZ4g8hLwRfYTYFhDGD',
➥'content': '7006652'}]
Final Answer: 1234 × 5678 = 7,006,652
```

The execution follows a three-step feedback loop:

1. The assistant's tool call request is added to the message history.
2. The `calculator` function is executed with the parsed arguments (`multiply`, `1234`, `5678`), returning `7006652`. This result is appended as a `"tool"` role message with the matching `tool_call_id`.
3. When this complete conversation history is sent back to the LLM, the LLM interprets the raw result and formats it into a clear, human-readable answer with proper number formatting.

In our calculator example, the result from the calculator is the final answer, so feeding it back to the LLM may seem redundant. But for more complex tools, such as web search, this step is crucial. Search results, for example, must be returned to the LLM so that it can synthesize the final response from relevant sources. This loop of *call* > *execute* > *return* > *reason* is essential to consistent tool use.

3.2.2 How an LLM chooses tools

How does the LLM learn to select tools and generate the required inputs? The answer lies in how the LLM is trained on tool-use examples.

During training, LLMs are exposed to datasets containing tool definitions paired with appropriate tool calls. Given a request like "Please set the living room blinds to 75% openness," the training data shows the model that it should call the `set_blind_openness` function with the arguments `room='living room'` and `openness_percentage=75`. By learning from such examples, the model develops the ability to do the following:

1. Recognize when a tool is needed based on the user's request
2. Select the appropriate tool from the available options
3. Extract the correct parameter values from the user's input
4. Format the tool call in the expected structure

This training process is similar to how LLMs learn other capabilities through pattern recognition across large amounts of data. The key difference is that tool calling requires the model to output structured data (function names and arguments) rather than free-form text.

> **TIP** If you're curious what this training data looks like, you can explore datasets like NousResearch's hermes-function-calling-v1 on Hugging Face (https://huggingface.co/datasets/NousResearch/hermes-function-calling-v1), which contains examples of tool definitions and corresponding tool calls used to train models for function calling.

3.2.3 Guidelines for effective tool calling

The following best practices are adapted from OpenAI's official documentation on function calling. For LLMs to call tools reliably, a tool's schema must make the when, what,

and how crystal clear. Each pair of examples in the following sections includes a problematic schema, its corrected version, and a brief explanation of why the fix is effective.

Clear and explicit function definitions

LLMs learn when and how to use tools based on function names and parameter descriptions. Define the tool's purpose, parameter types, and expected outputs explicitly. A good rule of thumb: if an intern can understand and use the tool from the schema alone, so can an LLM.

Listing 3.6 Antipattern: Vague tool definition

```
{
  "name": "book",
  "description": "Book something",
  "parameters": {
    "type": "object",
    "properties": { "when": { "type": "string" } }
  }
}
```

This schema hides the object of the action: the function name `book` does not say what is being booked. The model can't infer which inputs are mandatory or how to format them.

Listing 3.7 Improved: Explicit tool definition

```
{
  "name": "reserve_table",
  "description": "Create a restaurant table reservation.",
  "parameters": {
    "type": "object",
    "properties": {
      "restaurant_id": {
        "type": "string",
        "description": "Internal ID, e.g., rst_123"
      },
      "datetime": {
        "type": "string",
        "format": "date-time",
        "description": "ISO-8601 in the user's local time"
      },
      "party_size": {
        "type": "integer",
        "minimum": 1,
        "maximum": 20
      },
      "notes": {
        "type": "string",
        "description": "Allergies, occasion, etc. Optional."
      }
    },
```

```
    "required": ["restaurant_id", "datetime", "party_size"],
  }
}
```

The improved version states its intent plainly through the name and description, distinguishes between required and optional parameters, and constrains formats and ranges so invalid inputs are easier to detect and handle in code.

APPLYING SOFTWARE ENGINEERING BEST PRACTICES

Avoid ambiguous or logically conflicting parameters. `toggle_light(on: bool, off: bool)`, for example, allows contradictory inputs. Replace mutually exclusive Booleans with a single enum. Also, validate ranges and formats in the schema so that you can reject requests early with clear error codes.

Listing 3.8 Antipattern: Conflicting parameters

```
{
  "name": "toggle_light",
  "parameters": {
    "type": "object",
    "properties": {
      "on": { "type": "boolean" },
      "off": { "type": "boolean" }
    }
  }
}
```

Here, the model can produce impossible combinations, such as `on=true` and `off=true`. That ambiguity forces extra prompt logic and increases the likelihood of call failures.

Listing 3.9 Improved: Single enum parameter

```
{
  "name": "toggle_light",
  "description": "Control the light state",
  "parameters": {
    "type": "object",
    "properties": {
      "state": {
        "type": "string",
        "enum": ["on", "off"],
        "description": "The desired state of the light"
      }
     },
    "required": ["state"],
  }
}
```

This design removes contradiction by modeling state as a single enum. It gives the model one clear parameter to fill and lets your application reject unsupported values deterministically.

Reducing model load and using code effectively

Don't make the model resupply identifiers you already have. Let code remember context, and let the model focus on the decision itself.

It is inefficient to have the model regenerate information that has already been obtained. If an `order_id` was retrieved in a previous step, for example, it's more efficient to provide a parameterless function such as `submit_refund()` and pass the `order_id` automatically through code rather than ask the model to call a function that requires `order_id` as an input.

Listing 3.10 Antipattern: Model-supplied identifiers

```
{
  "name": "submit_refund",
  "parameters": {
    "type": "object",
    "required": ["order_id", "reason"],
    "properties": {
      "order_id": {
        "type": "string",
        "description": "The order ID to refund"
      },
      "reason": {
        "type": "string",
        "description": "Reason for the refund"
      }
    }
  }
}
```

This approach forces the model to recall and pass `order_id` correctly on every call. This increases error risk and token usage, especially in multistep flows, where identifiers can drift or be confused.

Listing 3.11 Improved: Decision-only parameters

```
{
  "name": "submit_refund",
  "parameters": {
    "type": "object",
    "required": ["reason"],
    "properties": {
      "reason": {
        "type": "string",
      }
    }
  }
}
```

Now the model supplies only the decision input—the reason—while the calling code injects the correct `order_id` from context. This reduces the chance of refunding the

wrong order, lowers prompt cost, and improves success rates across multiturn workflows. By *drift*, we mean the model recalling the wrong identifier across turns when several similar IDs appeared earlier in the conversation. We return to this topic in chapter 6 when we discuss session state.

LIMITING THE NUMBER OF FUNCTIONS

As a general guideline, keep the number of tools under 20. Using too many tools can lead to selection errors. If your task requires many tools or complex logic, consider fine-tuning the model to improve tool-calling precision.

3.3 Building tools and tool definitions for LLMs

Now that we understand how tool calling works, let's build a real tool from scratch. We'll implement a web search tool, one of the most essential capabilities for any research agent. Through this process, you'll experience firsthand what it takes to create a production-ready tool.

3.3.1 Implementing a web search tool

For web search, we'll use Tavily instead of alternatives such as Google Search API and Bing API because it's optimized for LLMs and delivers results that are easier to process. To use Tavily, sign up at https://www.tavily.com/; then retrieve your API key from https://app.tavily.com/home. Tavily allows up to 1,000 free API calls per month.

Before continuing, install the Tavily Python client:

```
uv add tavily-python
```

Then store your API key in a `.env` file using the `TAVILY_API_KEY` variable:

```
TAVILY_API_KEY=<Your Tavily API KEY>
```

STARTING SIMPLE

Let's start with the simplest possible implementation to verify that everything works. At this stage, our goal is to confirm that we can connect to the API and retrieve results. We'll keep the configuration minimal and add complexity only when necessary.

The code follows a straightforward pattern. First, we load environment variables using `load_dotenv()`, which reads our API key from the `.env` file. Then we create a `TavilyClient` instance with that key. The `search_web` function itself is minimal: it takes a query string, calls the Tavily API's `search()` method, and returns the results. We default `max_results` to 2 to keep responses concise during development and reduce API costs.

Listing 3.12 Basic web search function using Tavily

```
import os
from tavily import TavilyClient
```

```
from dotenv import load_dotenv

load_dotenv()

tavily_client = TavilyClient(os.getenv("TAVILY_API_KEY"))

def search_web(query: str, max_results: int = 2) -> list:

    response = tavily_client.search(query, max_results=max_results)
    return response.get("results")
```

Let's try searching for Eliud Kipchoge's marathon record to check whether `search_web` works. Each result includes a title, URL, content snippet, and relevance score. The first result is Kipchoge's Wikipedia page, and the second is a BBC article. From these snippets, we see that his official marathon best time is 2:01:09 (Berlin 2022). Our basic tool works.

Listing 3.13 Testing the web search function

```
search_web("Kipchoge's marathon world record")
```

The output is

```
[
    {
        'title': 'Eliud Kipchoge - Wikipedia',
        'url': 'https://en.wikipedia.org/wiki/Eliud_Kipchoge',
        'content': 'Eliud Kipchoge is a Kenyan long-distance runner who
        ➥competes in the marathon and formerly specialized in the 5000 metres.
        ➥Kipchoge is the 2016 and 2020 Olympic marathon champion, and was the
        ➥world record holder in the marathon from 2018 to 2023, until that
        ➥record was broken by Kelvin Kiptum at the 2023 Chicago Marathon.
        vKipchoge has run 4 of the 10 fastest marathons in history...
        ➥Personal bests Marathon: 2:01:09 (Berlin 2022)...',
        'score': 0.92125154,
        'raw_content': None
    },
    {
        'title': 'Eliud Kipchoge breaks two-hour marathon mark by 20 seconds',
        'url': 'https://www.bbc.co.uk/sport/athletics/50025543',
        'content': 'Eliud Kipchoge has become the first athlete to run a
        ➥marathon in under two hours, beating the mark by 20 seconds. The
        ➥Olympic champion - who holds the official marathon world record of
        ➥2:01:39, set in Berlin, Germany in 2018 - missed out by 25 seconds
        ➥in a previous attempt at the Italian Grand Prix circuit at Monza in
        ➥2017...',
        'score': 0.9014448,
        'raw_content': None
    }
]
```

ADDING SEARCH OPTIONS

Our basic function retrieves results, but real-world use often demands more control. Consider these scenarios: a user wants only recent news articles about a topic, results specifically from their country, or deeper research with more results. Each request requires additional parameters. Tavily's API supports these parameters through optional arguments.

Let's expand our function to expose the most common options. We've added four parameters. The `topic` parameter lets users focus on specific content types: `"general"` for broad web search, `"news"` for recent articles, and `"finance"` for financial information. The `time_range` parameter filters results by recency, which is useful when users want current information rather than historical content.

Listing 3.14 Web search function with additional options

```
def search_web(
    query: str,
    max_results: int = 2,
    topic: str = "general",
    time_range: str | None = None,
    country: str | None = None,
) -> list:

    response = tavily_client.search(
        query,
        max_results=max_results,
        topic=topic,
        time_range=time_range,
        country=country,
    )
    return response.get("results")
```

But this example just scratches the surface. Tavily's API supports nearly 20 parameters, such as filtering by specific domains (including or excluding certain websites), controlling the depth of search (basic versus advanced), including images in results, and retrieving raw HTML content. You have to understand the purpose of each parameter, as well as its valid values and how it interacts with other options. The more parameters you expose, the more complex your tool definition becomes and the harder it is for the LLM to use correctly.

HANDLING ERRORS GRACEFULLY

So far, we've assumed that everything works perfectly. But what happens when things go wrong? An invalid API key returns a 401 authentication error. Exceeding your monthly use limit triggers a 429 rate-limit error. Network problems cause connection timeouts. The API might return malformed data, or the service could be temporarily unavailable.

A minimal approach catches all exceptions and returns an error message. The expanded `search_web` function in listing 3.14 wraps the API call in a `try-except` block and returns an error string if anything goes wrong. Now the return type is `list | str`,

indicating that the function returns either a list of results or an error-message string. This simple approach ensures that the agent can continue operating even when the search fails.

Listing 3.15 Web search function with error handling

```
def search_web(
    query: str,
    max_results: int = 5,
    topic: str = "general",
    time_range: str | None = None,
) -> list | str:
    try:
        response = tavily_client.search(
            query,
            max_results=max_results,
            topic=topic,
            time_range=time_range,
        )
        return response.get("results")
    except Exception as e:
        return f"Error: Search failed - {e}"
```

This catchall approach, however, treats every error the same way. In production systems, different errors often require different responses. Authentication errors (401) might indicate a configuration problem that needs human attention. Rate-limit errors (429) might benefit from automatic retry with backoff. Timeout errors might warrant a retry with a simpler query. Network errors might need handling different from API errors.

For our learning purposes, this simple error handling suffices. But as you build tools for production agents, you'll likely need to distinguish among error types and handle each one appropriately. The complexity of robust error handling is yet another dimension of tool development that goes beyond simply calling an API.

3.3.2 Converting to tool definitions

We've created a web search tool. Earlier in this chapter, we also built a `calculator` function to perform computations. To make these tools accessible to an LLM, we have to define them using a standardized tool-definition format.

Because the functions we've written are simple, converting them to tool definitions manually won't take much time. But during development, functions may change frequently or grow more complex, making manual updates error prone and inefficient. To address this problem, we can create and use a utility function that automatically converts a Python function to a tool definition.

To generate a tool definition from a function, we extract the function's name, docstring, and parameter details. In Python, we do this by using the `inspect` module.

To illustrate the process, let's consider a sample function called `example_tool`. This function takes two inputs: `input_1`, which is a string, and `input_2`, which is an integer with a default value of `1`.

Listing 3.16 Extracting tool metadata using `inspect`

```
import inspect

def example_tool(input_1:str, input_2:int=1):
    """docstring for example_tool"""
    return

print(f"function name: {example_tool.__name__}")
print(f"function docstring: {example_tool.__doc__}")
print(f"function signature: {inspect.signature(example_tool)}")
```

The output is

```
function name: example_tool
function docstring: docstring for example_tool
function signature: (input_1: str, input_2: int = 1)
```

We can extract the function's name using the `__name__` attribute and its description using the `__doc__` attribute. We use `inspect.signature` to determine the parameter types and whether they're required. From the output, we see that `input_1` is a required string and `input_2` is an optional integer with a default value of `1`. Using this information, we can implement a utility function as shown in the following listing.

Listing 3.17 Generating input schema from Python functions

```
def function_to_input_schema(func) -> dict:
    type_map = {
        str: "string",
        int: "integer",
        float: "number",
        bool: "boolean",
        list: "array",
        dict: "object",
        type(None): "null",
    }

    try:
        signature = inspect.signature(func)
    except ValueError as e:
        raise ValueError(
            f"Failed to get signature for function {func.__name__}:
            ➥{str(e)}"
        )

    parameters = {}
    for param in signature.parameters.values():
        try:
            param_type = type_map.get(param.annotation, "string")
        except KeyError as e:
            raise KeyError(
                f"Unknown type annotation {param.annotation} for parameter
```

Extracts the function signature using the inspect module

Maps Python types to JSON Schema types with string as the default

```
                ➥{param.name}: {str(e)}"
            )
        parameters[param.name] = {"type": param_type}

    required = [
        param.name
        for param in signature.parameters.values()
        if param.default == inspect._empty
    ]

    return {
            "type": "object",
            "properties": parameters,
            "required": required,
        }
```

Maps Python types to JSON Schema types with string as the default

Parameters without default values are marked as required.

In the listing code, `inspect.signature` extracts function parameters and converts their types to the appropriate format for a tool definition. It also checks whether each parameter has a default value to determine whether it's required and stores the required arguments in a list. Finally, it organizes all the information according to the standard tool-definition format. To verify that the `function_to_tool_definition` utility works correctly, we can try converting our web search tool as follows.

Listing 3.18 Converting functions to tool definitions

```
def format_tool_definition(name: str, description: str, parameters: dict) ->
dict:
    return {
        "type": "function",
        "function": {
            "name": name,
            "description": description,
            "parameters": parameters,
        },
    }

def function_to_tool_definition(func) -> dict:
    return format_tool_definition(
        func.__name__,
        func.__doc__ or "",
        function_to_input_schema(func)
    )

function_to_tool_definition(search_web)
# {'type': 'function', 'function': {'name': 'search_web',
# ➥'description': 'Search the web for the given query.',
# ➥'parameters': {'type': 'object', 'properties':
# ➥{'query': {'type': 'string'}}, 'required': ['query']}}}
```

The resulting schema should include the function name and description, along with parameter types and whether each parameter is required:

```
search_tool_definition = function_to_tool_definition(search_web)
print(search_tool_definition)
```

The output is

```
{'type': 'function', 'function': {'name': 'search_web', 'description':
'Search the web for the given query.', 'parameters': {'type': 'object',
'properties': {'query': {'type': 'string'}, 'max_results': {'type':
'integer'}, 'topic': {'type': 'string'}, 'time_range': {'type': 'string'}},
'required': ['query']}}}
```

Now we can use this utility to convert all the tools we've created to schemas and prepare to solve our target problem.

3.3.3 End-to-end tool execution

We have a working web search function and a utility to convert it to a tool definition. The final piece is building the execution infrastructure that connects the LLM's tool calls to actual function execution. We need two components: a function to execute tools based on LLM output and a control loop to manage the conversation between the LLM and tools.

BUILDING THE TOOL EXECUTION SYSTEM

First, we define a utility function that executes a tool and returns its result. This function takes a *toolbox* (a dictionary that maps tool names to functions) and a tool call from the LLM; then it executes the appropriate function with the provided arguments. It extracts the tool name and arguments from the LLM's response, looks up the corresponding function in the toolbox, and calls it with the parsed arguments. This simple pattern works for any tool as long as it's registered in the toolbox.

Listing 3.19 Tool-execution utility function

```
def tool_execution(tool_box, tool_call):
    function_name = tool_call.function.name
    function_args = json.loads(tool_call.function.arguments)

    tool_result = tool_box[function_name](**function_args)
    return tool_result
```

Next, we build a control loop to manage the interaction between the LLM and tools. This function sends the user's question to the LLM along with available tool definitions. If the LLM requests a tool call, we execute it and feed the result back into the context. This cycle continues until the LLM returns a final response without requesting any tools. Let's trace what happens in this loop:

1. The LLM receives the system prompt, user question, and available tool definitions.
2. If the LLM determines that it needs external information, it generates a tool call.

3. We append the assistant's message (which contains the tool call) to the conversation history.
4. We execute the requested tool and append the result as a "tool" message.
5. The loop continues, sending the updated conversation back to the LLM.
6. When the LLM has enough information to answer, it returns a response without tool calls, and we exit the loop.

Listing 3.20 Simple agent loop for tool execution

```
from litellm import completion

def simple_agent_loop(system_prompt, question):
    tools = [search_web]
    tool_box = {tool.__name__: tool for tool in tools}
    tool_definitions = [function_to_tool_definition(tool) for tool in tools]

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": question}
    ]

    while True:
        response = completion(
            model="gpt-5-mini",
            messages=messages,
            tools=tool_definitions
        )

        assistant_message = response.choices[0].message

        if assistant_message.tool_calls:
            messages.append(assistant_message)
            for tool_call in assistant_message.tool_calls:
                tool_result = tool_execution(tool_box, tool_call)
                messages.append({
                    "role": "tool",
                    "content": str(tool_result),
                    "tool_call_id": tool_call.id
                })
        else:
            return assistant_message.content
```

TESTING THE AGENT LOOP

Let's test our system. The question in the following listing requires current information.

Listing 3.21 Testing the agent loop

```
system_prompt = """You are a helpful assistant.
Use the search tool when you need current information."""

result = simple_agent_loop(
```

```
    system_prompt,
    "Who won the 2025 Nobel Prize in Physics?"
)
print(result)
```

The output is

```
The 2025 Nobel Prize in Physics was awarded jointly to John Clarke, Michel
H. Devoret, and John M. Martinis. They were recognized for experiments
demonstrating macroscopic quantum tunnelling and energy quantisation in
electric circuits — work underpinning superconducting quantum technologies
(announced Oct. 7, 2025)
```

The LLM recognizes that it needs current information that isn't in its training data. It calls the `search_web` tool with an appropriate query, receives the search results, and synthesizes a final answer. Without the tool, the LLM would have to admit that it doesn't know the answer or risk providing outdated information.

3.3.4 The challenges of custom tools

Building the web search tool in section 3.3.1 was straightforward. We installed the Tavily client, wrote a simple function, and converted it to a tool definition. The entire process took a few minutes. But this simplicity is deceptive. As you develop more tools and scale to real-world projects, several challenges emerge that make developing custom tools surprisingly difficult.

UNDERSTANDING UNFAMILIAR APIS

Tavily was designed specifically for LLM applications, which made integration easy. The API is simple, the response format is already optimized for RAG, and the documentation is concise. But most services you want to integrate aren't built with LLMs in mind.

Consider building a tool that searches your company's Slack workspace, retrieves files from Google Drive, or queries a Salesforce database. Each service has its own API conventions, rate-limiting policies, pagination patterns, and response formats. Before writing any code, you must spend hours reading documentation, understanding the data models, and figuring out how to transform responses into something useful for an LLM. This up-front investment multiplies with every new tool you build.

HANDLING ERRORS ROBUSTLY

Look back at our `search_web` function, which has minimal error handling. In a production environment, this function would be inadequate. Real-world tools must handle network timeouts gracefully, manage API rate limits with appropriate backoff strategies, deal with service outages, validate inputs before making requests, and provide meaningful error messages that help the LLM (and ultimately the user) understand what went wrong.

Implementing robust error handling often requires more code than the core functionality itself. A simple web search function might be 10 lines, but a production-ready version with proper error handling, retries, and logging could easily be 50 lines or more.

Duplicating efforts across teams

In any organization that builds AI agents, multiple teams often need similar capabilities. The marketing team wants a web search tool for its content-generation agent. The customer support team needs one for its FAQ bot. The research team requires one for its analysis pipeline.

Without a standard way to share tools, each team builds their own implementation. Teams solve the same problems independently, make similar mistakes, and maintain separate codebases. From an organizational perspective, this duplication wastes significant engineering resources. The same tool might exist across the company in three or four slightly different versions.

You may think that the solution is simple: one team builds the tool, and the others reuse it. In practice, this approach rarely works smoothly. Different teams structure their tool definitions differently. Some use dictionaries; others use Pydantic models. Execution patterns vary between synchronous and asynchronous implementations. Return formats are inconsistent. Documentation quality differs dramatically.

When you try to adopt a tool from another team, you often spend more time adapting it to your codebase than you would have spent building it from scratch. Dependencies conflict, interfaces don't match, and assumptions embedded in the code don't apply to your use case. The friction of integration often outweighs the benefit of reuse.

Carrying the ongoing burden of maintenance

Tool building isn't a one-time effort. APIs evolve, and your tools must evolve with them. Services deprecate endpoints, change response formats, and introduce new required parameters. Security vulnerabilities in dependencies require updates. Users request new features and report edge cases you didn't anticipate.

When you maintain one or two tools, this maintenance is manageable. But agents typically need many tools, such as web search, file operations, database queries, email, calendar, and domain-specific integrations. Each tool becomes a maintenance responsibility. As your tool collection grows, the cumulative maintenance burden becomes substantial.

These challenges aren't unique to LLM tools. The software industry has solved similar problems through standardization. REST APIs standardized how web services communicate. Package managers standardized how code is shared and versioned. Containerization standardized how applications are deployed.

The LLM tool ecosystem needs similar standardization: a common way to define tools, share them across projects, and integrate them regardless of which LLM provider you use. This is exactly what MCP provides.

3.4 MCP: Standardizing tools

As we learned in section 3.3, developing tools that LLM agents can use is time consuming, and inconsistencies in implementation make them difficult to reuse. In November 2024, Anthropic introduced MCP to bring this kind of standardization to the world of LLM tool integration. Just as APIs established a common language for developers

to communicate with services, MCP established a common language for LLM agents to communicate with tools. Just as you can call any REST API using the same HTTP patterns, an MCP-compatible agent can use any MCP-compatible tool without custom integration code.

MCP gained growing attention throughout early 2025. It opened the door for developers to use tools built by companies or communities instead of building them from scratch. As shown in figure 3.4, MCP lets agents easily integrate tools from services such as Google Drive, Slack, Tavily, Brave Search, GitHub, and Figma without custom tool wrappers.

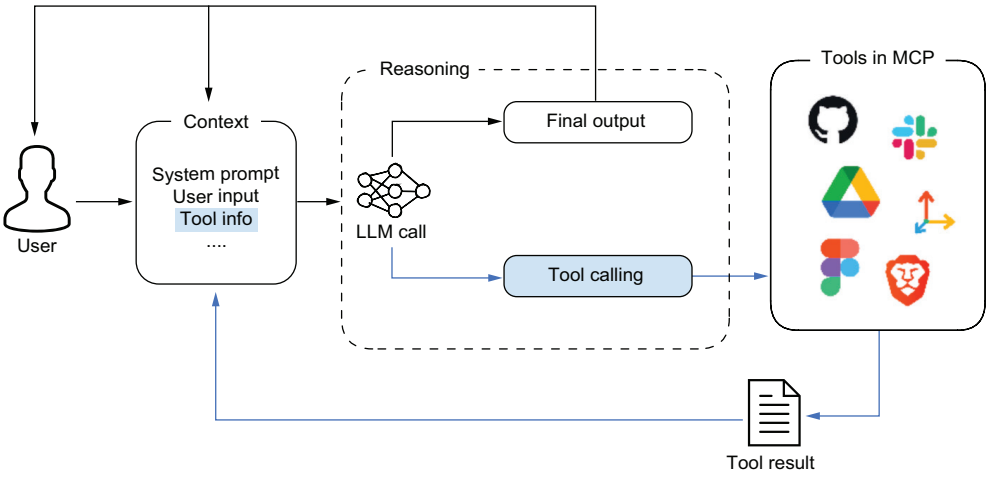

Figure 3.4 MCP makes it easy to extend an agent's toolset by integrating third-party tools.

3.4.1 The core of MCP: Server–client architecture

At its core, MCP separates the brain of the agent from its arms and legs (tools) through a three-component architecture: host, client, and server. Figure 3.5 illustrates how these components work together. Let's clarify each role:

- *Host*—The host is the application that interacts with users and makes decisions using an LLM. It receives user input, performs reasoning, and decides which tools to use and when. Think of it as the brain of the system. Examples include Claude Desktop, an IDE with AI features, and the agent application you're building.
- *Client*—The MCP client acts as a bridge between the host and the MCP servers. When the host decides that it needs a tool, the client handles the communication, requesting available tool definitions from servers and forwarding tool-execution requests. Each client maintains a connection to one MCP server.
- *Server*—The MCP server hosts the tools and executes them on demand. It responds to two types of requests: "What tools do you have?" (returning tool definitions) and

"Execute this tool with these arguments" (running the tool and returning results). Servers can connect to external services such as databases and APIs.

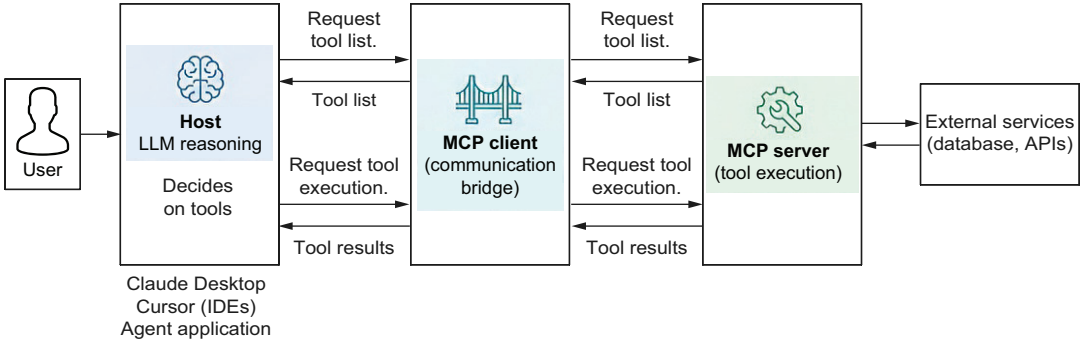

Figure 3.5 MCP server–client architecture. The host contains the LLM for reasoning and user interaction, connecting through MCP clients, which manage server communication. MCP servers expose tool definitions and handle execution, optionally integrating with external services such as databases and APIs.

MCP's scope beyond tools

MCP defines three types of capabilities that servers can provide: tools (functions the LLM can call), prompts (reusable prompt templates), and resources (data the LLM can access). In this book, we focus exclusively on tools because they're the most essential capability for building agents.

Two standardized interfaces

Regardless of which transport you use, MCP standardizes two key interfaces:

- *Tool discovery*—In section 3.3, we manually defined schemas for each tool. With MCP, the client can request available tool definitions, and the server returns them in a standardized format. Tool developers implement the schema once; then any MCP-compatible client can automatically retrieve and use it.
- *Tool execution*—Previously, we built our own `tool_execution` function to bridge LLM output and actual function calls. MCP standardizes this function too. The client sends a tool-execution request to the server, which handles execution and returns the result in a consistent format.

Transport mechanisms

MCP defines two standard transport mechanisms for communication between clients and servers:

- *Stdio (standard I/O)*—This approach is the simplest one. The client launches the server as a subprocess and communicates through standard input/output

streams. Because everything runs locally on the same machine, there's no network overhead. This setup is ideal for local development and for tools that don't have to be shared across machines.

- *Streamable HTTP*—This approach enables remote communication over the network. The client sends requests using HTTP POST and GET, and the server can stream responses when needed.

Throughout this chapter, we'll use the stdio transport because it's simple to set up and sufficient for learning MCP concepts. When you need remote or shared tool servers in production, you can switch to streamable HTTP without changing your tool implementations.

3.4.2 Hands-on: Running an MCP server

Before diving into implementation details, let's experience MCP firsthand by running an existing server. We'll use the official Tavily MCP server, which provides the web search capability we built manually in section 3.3 but packages it as a ready-to-use MCP server.

MCP servers in the ecosystem are typically distributed as npm packages run using npx. If you don't have Node.js installed, you'll have to install it first. For macOS and Windows, download the installer for your operating system from https://nodejs.org/, and follow the installation steps. Use the current Node.js LTS release (22.x or later). The MCP Inspector we use in this section requires Node 22.7.5 or later; older versions such as 10.x, 18.x, and 20.x will fail at install time.

Verify your installation by checking the version:

```
node --version
```

If the Node version is displayed, the installation was successful.

RUNNING THE TAVILY MCP SERVER

With Node.js installed, we can launch the Tavily MCP server using a single command. First, set your Tavily API key as an environment variable:

```
export TAVILY_API_KEY=<your tavily api key>
```

Now, launch the server with the MCP Inspector, a browser-based tool for testing MCP servers:

```
npx @modelcontextprotocol/inspector npx -y tavily-mcp@latest
```

This command does several things at the same time. The `npx` command runs `npm` packages without installing them globally. The `@modelcontextprotocol/inspector` package provides a web interface for interacting with MCP servers. The `-y` flag automatically confirms any prompts. Finally, `tavily-mcp@latest` is the official Tavily MCP server package (figure 3.6).

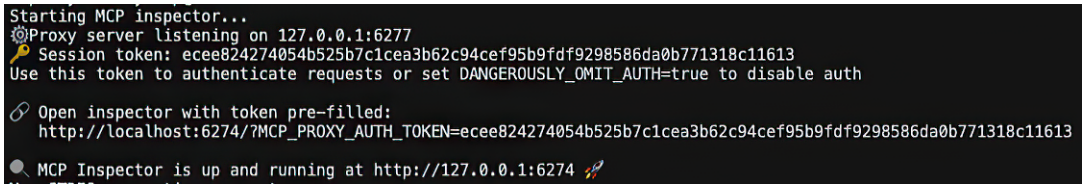

Figure 3.6 MCP Inspector startup log showing the server initialization and connection URL

Open the URL shown in your terminal. If the URL includes a session token, you can connect without additional configuration.

Exploring the server with MCP Inspector

When the Inspector UI loads in your browser, click the Connect button. After a successful connection, several tabs appear: Resources, Prompts, and Tools. These tabs reflect the capabilities exposed by the MCP server.

Select the Tools tab, and click List Tools. You'll see the tools that the Tavily MCP server provides:

- `tavily_search`—Provides web search functionality (similar to what we built in section 3.3)
- `tavily_extract`—Extracts content from specific URLs
- `tavily_crawl`—Crawls websites and extracts structured data
- `tavily_map`: Generates site maps

Click `tavily-search` to expand its details. You'll see the tool's description and parameter schema, formatted in the standard MCP format. Enter a query like "2025 Nobel Physics" and then click Run Tool. The server executes the search and returns results directly in your browser (figure 3.7).

What we've accomplished

Take a moment to appreciate what happened. With a single command, we did the following:

- Downloaded and ran a production-ready MCP server
- Connected to it using a standard client interface
- Discovered available tools automatically
- Executed a tool and received results

Compare this result with the one in section 3.3, where we spent considerable effort implementing the basic search functionality. The Tavily MCP server includes multiple tools, robust error handling, and optimized response formatting, all ready to use without requiring us to write any code.

This is the power of standardization. Tool developers can focus on building reliable, feature-rich servers. Agent developers can focus on reasoning and orchestration, knowing that any MCP-compatible tool will work with their systems.

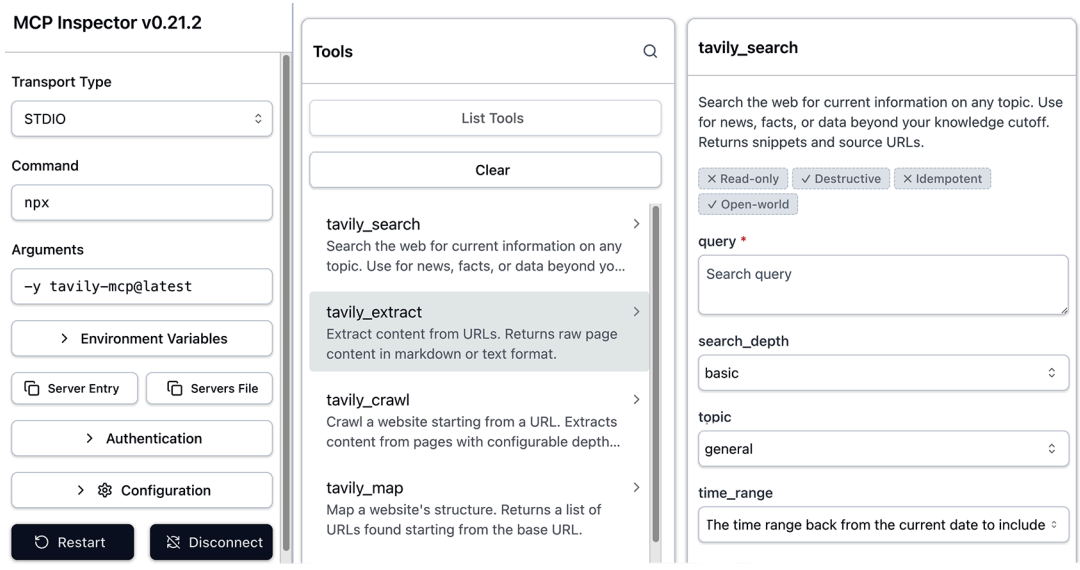

Figure 3.7 MCP Inspector interface showing the official Tavily MCP server, which provides a richer set of tools, including `tavily_search`, `tavily_extract`, `tavily_crawl`, `tavily_map`

Next, we'll look at the other side of this architecture: building an MCP client that can discover and use these tools programmatically within our agent.

3.4.3 Understanding the MCP client

In section 3.4.2, we interacted with the Tavily MCP server through the inspector's web interface. The interface is useful for testing, but production agents need to interact with MCP servers programmatically, which is where the MCP client comes in.

Your agent uses the MCP client to discover and invoke tools from MCP servers. It handles the protocol details, such as establishing connections, requesting tool definitions, executing tools, and parsing results. Understanding how the client works is essential for integrating MCP tools into your own agents.

TIP Install the MCP Python SDK for hands-on practice: `uv add mcp`.

CONNECTING TO AN MCP SERVER

Let's write a simple client that connects to the Tavily MCP server, lists available tools, and performs a search. The following things happen in this code:

1. `StdioServerParameters` specifies how to launch the server. The command is `npx`, which runs npm packages without installing them globally. The args include `-y` to confirm prompts automatically and `tavily-mcp@latest` to use the latest version. Environment variables pass the API key and system `PATH`.

2 `stdio_client` launches the server as a subprocess. The `async with` ensures proper cleanup when we're done and returns read and write streams that the session uses for communication.

3 `ClientSession` provides the high-level API. After calling `initialize()`, we can interact with the server using standard methods like `list_tools()` and `call_tool()`.

4 `session.list_tools()` requests all available tools from the server. The server returns tool definitions, including names, descriptions, and parameter schemas.

5 `session.call_tool()` executes a specific tool. We pass the tool name and a dictionary of arguments. The server runs the tool and returns the result.

Listing 3.22 Connecting to an MCP server

```
import asyncio
import os
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from dotenv import load_dotenv

load_dotenv()

server_params = StdioServerParameters(
    command="npx",
    args=["-y", "tavily-mcp@latest"],
    env={
        "TAVILY_API_KEY": os.getenv("TAVILY_API_KEY"),
    }
)

async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()

        # List available tools
        tools_result = await session.list_tools()
        print("Available tools:")
        for tool in tools_result.tools:
            print(f"  - {tool.name}: {tool.description[:60]}...")

        result = await session.call_tool(
                "tavily_search",
                arguments={"query": "2025 Nobel Physics"}
            )
        print("Search Result:")
        print(result.content)
```

Running this code produces output similar to this example:

```
Available tools:
  - tavily_search: A powerful web search tool that provides comprehensive, real...
```

```
  - tavily_extract: A powerful web content extraction tool that retrieves and
    pr...
  - tavily_crawl: A powerful web crawler that initiates a structured web
    crawl...
  - tavily_map: A powerful web mapping tool that creates a structured map
    of...
Search Result:
[TextContent(type='text', text='Detailed Results:\n\nTitle: Press release:
Nobel Prize in Physics 2025...
```

CONVERTING MCP TOOLS FOR LLM USE

The tools that `list_tools()` returns use MCP's schema format. To use these tools with LLMs, we have to convert them to whatever format the LLM provider expects.

One natural question at this point is this: if MCP is the standard, why doesn't every LLM provider accept MCP's format directly? MCP standardizes the server side of the contract: how tools are described, discovered, and invoked across processes. The wire format that the LLM API itself expects in its `tools` parameter is set by each provider (OpenAI, Anthropic, and Google) and predates MCP. MCP can't unilaterally change those public APIs, so it standardizes the parts it can own (tool discovery and execution over a transport) and leaves the last-mile translation to the client. In practice, this last mile is small and mechanical. MCP's `inputSchema` is already JSON schema, which is exactly what each provider's `parameters` field expects, so the converter simply wraps it in the provider's outer envelope. As providers increasingly add native MCP support, this conversion step may disappear, but for now, the client owns it.

Fortunately, we built the foundation for this in section 3.3.2. The `format_tool_definition` function we created takes a name, description, and parameters dictionary and returns a properly structured OpenAI tool definition. MCP tool objects provide exactly these three pieces of information through their `name`, `description`, and `inputSchema` attributes, making the conversion straightforward.

Listing 3.23 Converting MCP tools to OpenAI format

```
def mcp_tools_to_openai_format(mcp_tools) -> list[dict]:
    """Convert MCP tool definitions to OpenAI tool format."""
    return [
        format_tool_definition(
            name=tool.name,
            description=tool.description,
            parameters=tool.inputSchema,
        )
        for tool in mcp_tools.tools
    ]
```

With this conversion function, we can retrieve tools from any MCP server and use them immediately with OpenAI's API. The following listing shows how the process works.

Listing 3.24 Retrieving tools in OpenAI format

```
async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()

        # List available tools
        tools_result = await session.list_tools()
        openai_format_tools = mcp_tools_to_openai_format(tools_result)
        for tool in openai_format_tools:
            print(tool)
```

3.4.4 Hands-on: Implementing an MCP server

So far, you've used external MCP servers built by others. Although you can build effective agents using only community-provided servers, understanding how to create your own server gives you the confidence to navigate the MCP ecosystem. You'll know exactly what's happening under the hood, making it easier to debug problems, extend existing servers, and build custom tools for your specific needs.

The good news is that building an MCP server is surprisingly simple. Using the FastMCP library, you can convert an existing Python function to an MCP-compatible tool by adding a decorator. In this section, you'll transform the `search_web` function you built in section 3.3 into a fully functional MCP server.

Building the server

Start by installing the required libraries:

```
uv add mcp
```

Next, create a new file called `tavily_mcp_server.py`. We'll take our existing `search_web` function and wrap it with the MCP infrastructure. The `FastMCP` instance creates our server with a name that identifies it to clients. The `@mcp.tool()` decorator registers our function as an MCP tool. FastMCP automatically extracts the function name, parameters, and type hints to generate the tool schema. The docstring becomes the tool's description, which helps LLMs understand when and how to use the tool. Finally, `mcp.run(transport='stdio')` starts the server using standard input/output for communication.

Listing 3.25 Custom Tavily MCP server implementation

```
import os
from tavily import TavilyClient
from dotenv import load_dotenv
from mcp.server.fastmcp import FastMCP

load_dotenv()

tavily_client = TavilyClient(os.getenv("TAVILY_API_KEY"))
```

```
mcp = FastMCP("custom-tavily-search")

@mcp.tool()
def search_web(query: str, max_results: int = 5) -> str:
    """
    Search the web using Tavily API.

    Args:
        query: Search query string
        max_results: Maximum number of results to return (default: 5)

    Returns:
        Search results as formatted string
    """
    try:
        response = tavily_client.search(
            query,
            max_results=max_results,
        )
        results = response.get("results", [])
        return "\n\n".join(
            f"Title: {r['title']}\nURL: {r['url']}\nContent: {r['content']}"
            for r in results
        )
    except Exception as e:
        return f"Error searching web: {str(e)}"

if __name__ == "__main__":
    mcp.run(transport='stdio')
```

Notice how little code we added to our original function. The core logic remains unchanged; we simply wrapped it with the MCP infrastructure.

CONNECTING WITH THE CLIENT

Now let's verify that the server works by connecting to it with the client code from section 3.4.3. The only change we have to make is updating `StdioServerParameters` to point to our custom server instead of the official Tavily package.

Listing 3.26 Connecting to the custom MCP server

```
server_params = StdioServerParameters(
    command="uv",
    args=["run", "tavily_mcp_server.py"],
    env={
        "TAVILY_API_KEY": os.getenv("TAVILY_API_KEY"),
    }
)

async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()

        # List available tools
```

```
        tools_result = await session.list_tools()
        print("Available tools:")
        for tool in tools_result.tools:
            print(f"  - {tool.name}: {tool.description}")
```

Running the preceding code produces output like this:

```
Available tools:
  - search_web:
Search the web using Tavily API.

Args:
    query: Search query string
    max_results: Maximum number of results to return (default: 5)

Returns:
    Search results as formatted string
```

Compare this result to the one in section 3.4.2, in which the official Tavily server exposed four tools: `tavily-search`, `tavily-extract`, `tavily-crawl`, and `tavily-map`. Our custom server exposes only one tool, `search_web`, but the client code is nearly identical. Through the power of standardization, whether you're connecting to a sophisticated official server or a simple custom one, the client interface remains the same.

In this section, we completed the full MCP cycle:

- We converted our custom `search_web` function to an MCP server with minimal changes.
- We connected to our server using the same client patterns we used for the official Tavily server.
- We verified that tool discovery and execution work correctly.

This demonstrates the core value proposition of MCP: tool implementation and tool use are decoupled. Server developers focus on building reliable tools. Agent developers focus on reasoning and orchestration. The protocol handles everything in between.

With tools and MCP, we have all the building blocks we need for a complete agent. In chapter 4, we'll combine these capabilities into a robust agent framework, implementing proper tool abstraction, error handling, and the iterative reasoning loop that transforms an LLM with tools into a true agent.

Summary

- Tools extend LLMs' capabilities, enabling them to access APIs, perform calculations, and retrieve external information. Tools fall into three categories: information-augmenting, action-executing, and domain-specialized.
- Tool calling is the mechanism by which LLMs generate structured outputs that specify which tool to use and with what parameters. The LLM acts as a mediator between user requests and available tools but doesn't execute the tools.

- Tool execution is handled externally. The LLM generates specifications that a separate system executes, with the results being fed back into the LLM's context for further reasoning.
- Tool definitions are structured schemas that describe available tools, their parameters, and expected outputs. Clear, explicit definitions are essential for reliable tool selection by the LLM.
- Building custom tools involves implementing the tool function, creating tool definitions, and building execution infrastructure. Although using them is straightforward for simple cases, custom tools create a maintenance burden and inconsistencies as projects scale.
- MCP standardizes tool development through a server–client architecture. Servers host and execute tools; clients discover and invoke them. This separation enables tool reuse across projects and providers.

4 Implementing a basic ReAct agent

This chapter covers

- Understanding ReAct, the core pattern behind modern AI agents
- Managing agent state with `ExecutionContext`
- Unifying local functions and MCP tools
- Building the LLM communication layer and implementing a `ToolCallingAgent` with the think-act loop
- Adding structured output and testing agent performance against the GAIA benchmark

We've explored how LLMs work and given them the ability to use tools. Now it's time to combine these pieces into something more powerful: an agent that can reason about what it needs, take action to get it, and continue until it solves the problem.

As shown in figure 4.1, we'll complete the foundation of the basic agent loop by building an agent using the ReAct pattern, chaining multistep reasoning and actions to solve problems end to end, and adding robust error handling. ReAct (reasoning

and acting) isn't a framework but a way of designing agents that mirrors how humans solve problems: assess the situation, decide what information or action is required, execute that action, observe the results, and repeat.

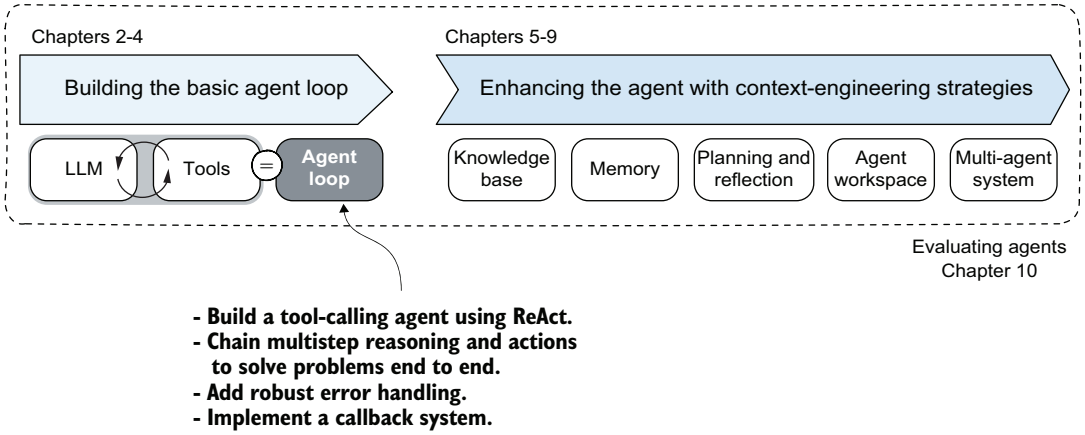

Figure 4.1 Book structure overview: chapter 4

By the end of this chapter, you'll have a working agent that can search the web for real-time information, chain multiple steps of reasoning to solve problems, handle errors gracefully, and return structured output in any format you specify. You'll test it on the GAIA benchmark. This agent will serve as the foundation for everything that follows: memory systems, planning strategies, knowledge retrieval, and multi-agent collaboration.

4.1 How ReAct agents work

Think about how you'd answer this question: "If marathon runner Eliud Kipchoge could maintain his world record pace indefinitely, how long would it take him to reach the Moon?" You wouldn't try to answer immediately. Instead, you'd think, "I need Kipchoge's marathon pace and the Earth–Moon distance." Then you'd search for that information, calculate the result, and formulate your answer. This natural cycle of thinking and acting is exactly how ReAct agents operate.

ReAct is the foundational pattern behind most modern AI agents. The core idea is simple: instead of trying to answer everything at the same time, an agent alternates between reasoning about what it needs to reach an answer and taking actions to get it.

4.1.1 The think-act cycle

Let's see how a ReAct agent would tackle the Kipchoge problem:

```
Thought: I need to find Kipchoge's marathon world record pace.
Action: search_web("Kipchoge marathon world record")
Observation: Kipchoge's record is 2:01:09 for 42.195 km (Berlin 2022)

Thought: Now I need the Earth-Moon distance at closest approach.
Action: search_wikipedia("Moon perigee distance")
Observation: The Moon's perigee (closest approach) is 356,500 km

Thought: I can now calculate the time needed.
Action: calculator("356500 / (42.195 / 2.0186) * 1")
Observation: 17,034 hours

Thought: I have enough information to answer.
Final Answer: Approximately 17,000 hours
```

Notice the pattern. At each step, the agent explicitly states what it's thinking, takes a concrete action, and observes the result. This cycle repeats until the agent has gathered enough information to provide a final answer.

What makes this approach powerful is its adaptability. If the first search returned unhelpful results, the agent could try a different query. If the calculation seemed wrong, the agent could verify it by searching for additional information. Unlike a fixed script, which breaks when something unexpected happens, a ReAct agent adjusts its approach based on what it discovers. This behavior mirrors how humans naturally solve problems. We don't execute a predetermined sequence of steps; we think, act, observe the results, and decide what to do next. The LLM handles the reasoning, and tools handle the actions. Together, they form a system that can tackle problems that neither could solve alone.

4.1.2 From text parsing to tool calling

Early ReAct implementations faced a frustrating problem: they relied entirely on text parsing. The agent had to output responses in an exact format, such as `Action: search["query"]`, and any deviation would break the system. A missing bracket, an extra space, or a slightly different keyword could cause parsing failures. Developers spent enormous effort crafting prompts that begged the LLM to follow precise formatting rules, yet errors remained common.

Tool calling eliminates this fragility. Instead of parsing free-form text, the LLM generates structured outputs that specify exactly which tool to call and which arguments to use. The format is guaranteed by the API, which doesn't simply hope that the LLM follows instructions perfectly.

Figure 4.2 illustrates this evolution. On the left, the traditional approach requires explicit "thought" and "action" text that must be parsed. On the right, modern tool calling handles this task natively.

If there's no explicit "thought" step now, is it still ReAct? Yes. The reasoning didn't disappear; it moved inside the LLM. As we saw in chapter 3, modern LLMs are trained on extensive tool-calling examples, learning to recognize which tool is appropriate for

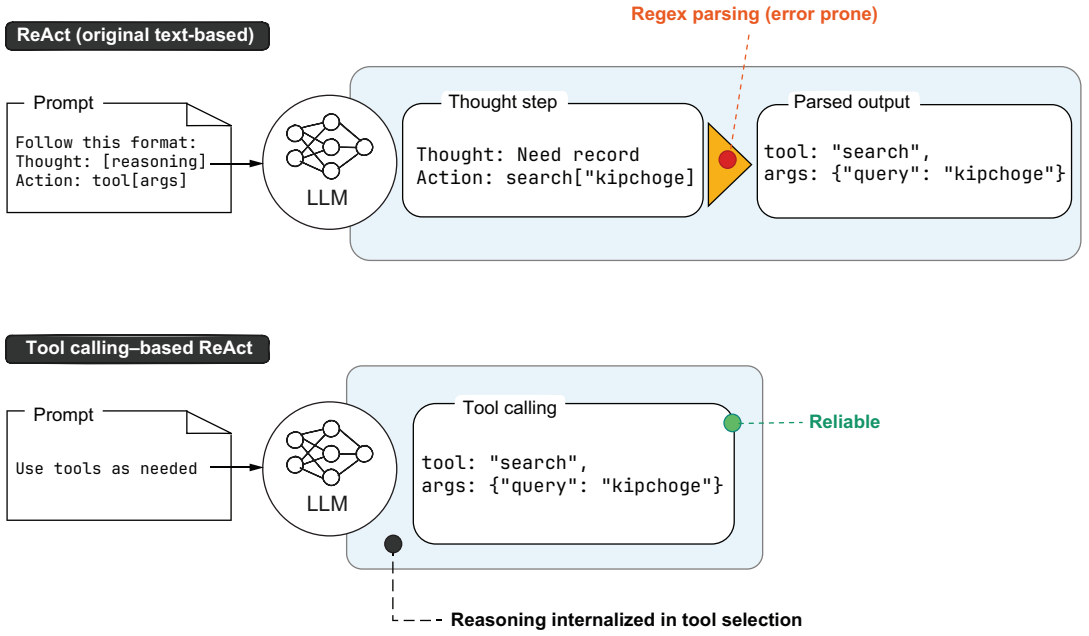

Figure 4.2 Evolution from text-based ReAct to tool-calling ReAct. The explicit "thought" step has been internalized in the LLM's tool-selection process, eliminating the need for fragile text parsing.

each situation. When the LLM selects a tool and specifies its arguments, it has already performed the reasoning internally. The pattern remains the same: reason about what's needed, act to get it, observe the result, and repeat.

This is why tool calling–based agents are still called ReAct agents across major frameworks such as LangGraph, Google ADK, and Hugging Face `smolagents`. The core principles remain intact: adaptive problem solving through iterative reasoning and action, with the LLM dynamically selecting tools based on context rather than following a predetermined script.

Now we're ready to build our own agent. Let's look at the complete architecture of what we're going to implement, starting with the result and working backward to understand each component we need to build.

4.2 Agent architecture overview

Before diving into implementation details, let's see what we're building toward. Understanding the complete picture first will help you appreciate why each component exists and how the components work together. The code for this chapter is available here: https://mng.bz/X7MG.

4.2.1 The completed agent

The following listing shows how our finished agent will look in action.

Listing 4.1 Agent example

```
from scratch_agents import Agent, LlmClient
from scratch_agents.tools import calculator, search_web

agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[calculator, search_web],
    instructions="You are a helpful assistant"
)

result = await agent.run("What is 1234 * 5678?")
```

This simple interface hides significant complexity. The agent receives a user question, decides whether to use tools, executes them if necessary, and returns a final answer. For the multiplication question, it would call the calculator tool and return `"7,006,652"`.

But what happens behind this clean API? To build an agent that works, we must understand the information flow and the components that make it possible.

4.2.2 Information flow: The core design

An agent's job is straightforward in concept: receive user input, ask the LLM what to do next, execute tools if asked to do so, feed the results back to the LLM, and repeat until done. This cycle continues until the LLM decides that it has enough information to provide a final answer.

WHY MESSAGES ALONE AREN'T ENOUGH

You may think that maintaining a simple list of messages (the conversation history with the LLM) would suffice. After all, that's what we did in chapter 2 when managing conversations. But running an agent requires tracking much more information:

- `events`—The history of all interactions, including user messages, LLM responses, and tool-execution results
- `current_step`—A counter that prevents infinite loops (what if the agent keeps calling tools forever?)
- `execution_id`—A unique identifier for debugging and logging (which execution produced this error?)
- `state`—A scratch pad for dynamic data during execution, such as task progress, intermediate results, or user preferences that tools may need
- `final_result`—A flag that indicates when execution is complete

If we scatter this information across different variables and pass them to each method individually, the code becomes unwieldy. Every method would need a long parameter list, and adding new information (such as memory in chapter 6) would require modifying every method signature.

The solution is to consolidate everything in a single container. All methods receive this one container, and when new information is required, we modify only the container

definition. This container is `ExecutionContext`, the central storage for all execution states.

THE EXECUTIONCONTEXT-CENTERED DESIGN

Figure 4.3 illustrates how information flows through our agent, with `ExecutionContext` at the center.

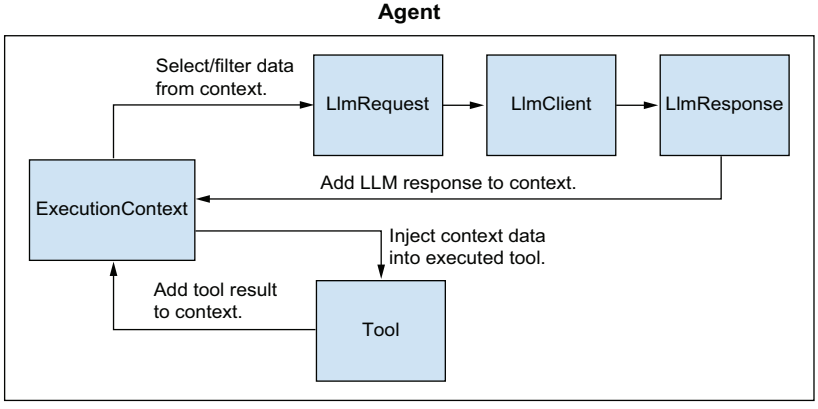

Figure 4.3 Information flow in the agent architecture. `ExecutionContext` sits at the center, managing all execution state. Arrows show (1) context to `LlmRequest` to `LlmClient` for outgoing LLM calls, (2) `LlmResponse` back to context for incoming responses, and (3) bidirectional flow between context and tools for tool execution.

`ExecutionContext` serves as the central storage where all information converges. When the agent needs to call the LLM, it extracts relevant information from `ExecutionContext` into an `LlmRequest`. When the LLM responds, the response flows back into `ExecutionContext`. When tools execute, the agent passes `ExecutionContext` to them so that they can access any execution state they need, and their results are recorded back in `ExecutionContext`.

This architecture provides two key benefits:

- It simplifies method signatures because every method receives only the context.
- It enables *context propagation*, in which the agent can pass `ExecutionContext` to a tool that needs access to execution state (such as a tool that checks user permissions before acting).

Notice the separation between `LlmRequest` and `ExecutionContext`. Whereas `ExecutionContext` stores everything, `LlmRequest` is a curated subset containing only what the LLM needs for a specific call. This separation is where context engineering happens: deciding what information to include, what to omit, and how to format it for optimal LLM performance.

4.2.3 Components we have to build

With the architecture clear, table 4.1 shows our implementation road map. Each component has a specific role in managing the information flow discussed in the preceding section.

Table 4.1 Agent components and implementation roadmap

Order	Component	Section	Role
1	`ExecutionContext`	4.3	Central storage that manages all information during execution
2	Tool abstraction	4.4	Unifies tools from chapter 3 under a consistent interface that can receive context
3	LLM communication layer	4.5	`LlmRequest` selects information for the LLM; `LlmClient` handles API calls; `LlmResponse` standardizes responses
4	`Agent`	4.6	Orchestrator that creates context, coordinates information flow, and implements the think–act loop

We'll build from the foundation up. `ExecutionContext` comes first because everything else depends on it. Tool abstraction follows because we need a consistent way to define and execute tools. The LLM communication layer standardizes how we talk to language models. Finally, the `Agent` class brings everything together into the think–act loop that solves problems.

4.3 ExecutionContext: The agent's central storage

We need `ExecutionContext` to avoid scattering state across multiple variables and simplify how components share information. Now let's determine *what* we have to store by examining what happens when an agent runs.

4.3.1 What happens during agent execution

Recall the Kipchoge problem from section 4.1. When an agent solves this problem, a series of events unfolds:

1. User asks, "How long would it take Kipchoge to run to the Moon?"
2. LLM decides that it needs information and requests the `search_web` tool.
3. Tool executes and returns "Kipchoge's record is 2:01:09 for 42.195km."
4. LLM needs more information and requests the `search_wikipedia` tool.
5. Tool executes and returns "Moon's perigee is 356,500 km."
6. LLM can calculate now and requests the `calculator` tool.
7. The tool executes and returns: "17034."
8. LLM provides the final answer: "Approximately 17,000 hours."

Looking at this sequence, we can identify three distinct types of occurrences:

- *Messages*—Text exchanges in a conversation. The user's initial question and the LLM's final answer are both messages. Each message has a role (user, assistant, or system) and text content.
- *Tool calls*—Calls that occur when the LLM decides to use a tool. The LLM specifies which tool to call and with what arguments. In step 2, the LLM requests `search_web` with the query "Kipchoge marathon world record."
- *Tool results*—Results that capture what happens when tools execute. They include the tool's output and whether execution succeeded or failed. Step 3 would be a successful result containing Kipchoge's marathon time.

Let's define these occurrences as data types.

Listing 4.2 Content types for agent communication

```
class Message(BaseModel):
    """A text message in the conversation."""
    type: Literal["message"] = "message"
    role: Literal["system", "user", "assistant"]
    content: str

class ToolCall(BaseModel):
    """LLM's request to execute a tool."""
    type: Literal["tool_call"] = "tool_call"
    tool_call_id: str
    name: str
    arguments: dict

class ToolResult(BaseModel):
    """Result from tool execution."""
    type: Literal["tool_result"] = "tool_result"
    tool_call_id: str
    name: str
    status: Literal["success", "error"]
    content: list

ContentItem = Union[Message, ToolCall, ToolResult]
```

The `type` field in each class serves as a discriminator, making it easy to identify what kind of content we're working with. The `tool_call_id` links a `ToolResult` back to its originating `ToolCall`, which is essential when multiple tools execute in parallel.

These content types capture *what* happened, but for debugging and analysis, we also have to know *who* produced each piece of content and *when*. We wrap content items with metadata using an `Event` class.

Listing 4.3 `Event` class for execution tracking

```
class Event(BaseModel):
    """A recorded occurrence during agent execution."""
```

```
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    execution_id: str
    timestamp: float = Field(default_factory=lambda: datetime.now().timestamp())
    author: str  # "user" or agent name
    content: List[ContentItem] = Field(default_factory=list)
```

The `execution_id` groups all events from a single agent run, enabling us to trace an entire problem-solving session. The `author` field distinguishes between user input and agent actions. Here's how one step from the Kipchoge problem would look as an `Event`:

```
Event(
    execution_id="abc-123",
    author="research_agent",
    content=[ToolCall(
        tool_call_id="call-1",
        name="search_web",
        arguments={"query": "Kipchoge marathon world record"}
    )]
)
```

Each step in the agent's execution becomes an `Event`, creating a complete audit trail that is invaluable for debugging.

4.3.2 ExecutionContext implementation

Now that we know what to store, implementing `ExecutionContext` is straightforward.

Listing 4.4 Implementing `ExecutionContext`

```
@dataclass
class ExecutionContext:
    """Central storage for all execution state."""

    execution_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    events: List[Event] = field(default_factory=list)
    current_step: int = 0
    state: Dict[str, Any] = field(default_factory=dict)
    final_result: Optional[str | BaseModel] = None

    def add_event(self, event: Event):
        """Append an event to the execution history."""
        self.events.append(event)

    def increment_step(self):
        """Move to the next execution step."""
        self.current_step += 1
```

Each field serves a specific purpose:

- `execution_id`—Unique identifier for this execution session, generated automatically

- `events`—Chronological list of all events that occur during execution
- `current_step`—Counter to prevent infinite loops (agent stops after `max_steps`)
- `state`—Flexible key-value store for custom data that tools may need, such as API configurations or intermediate results
- `final_result`—Holds the agent's answer when execution completes

`ExecutionContext` can also be passed to tools that need access to the execution state. A tool might read from `state` or examine previous events in the execution history. For this to work, we need a consistent way to define tools that can receive context. In the next section, we'll implement tool abstraction to unify the function-based tools under a common interface that supports this capability.

4.4 Tool abstraction

In chapter 3, we built tools as simple Python functions and created a utility to convert them to tool definitions for the LLM. This approach worked well for learning the basics of tool calling. But as we prepare to build a complete agent framework, several practical challenges emerge, and our function-based approach doesn't handle them well.

4.4.1 Why we need a unified tool interface

Let's revisit what we built in chapter 3. We created functions such as `calculator()` and `search_web() and` then used `function_to_tool_definition()` to generate the schemas that LLMs need:

```
def search_web(query: str, max_results: int = 5) -> str:
    """Search the web for information."""
    return tavily_client.search(query, max_results=max_results)
search_tool_definition = function_to_tool_definition(search_web)
search_web(query="Python tutorials")
```

This code works, but the function and its metadata exist as separate entities. When building an agent, we pass definitions to the LLM and map names back to functions for execution, keeping everything in sync manually. With tool abstraction, everything stays bundled together:

```
@tool
def search_web(query: str, max_results: int = 5) -> str:
    return tavily_client.search(query, max_results=max_results)

search_web.tool_definition
search_web.execute(query="Python tutorials")
```

Let's implement this with two classes. `BaseTool` defines the interface, and `FunctionTool` wraps existing functions.

4.4.2 BaseTool: The foundation

Every tool in our framework inherits from `BaseTool`, an abstract base class that defines what it means to be a tool. Let's examine what this interface provides.

A tool must have three essential properties: a `name` for identification, a `description` explaining its purpose, and a `tool_definition` schema that tells the LLM how to use it. The tool must also be executable, which we'll represent with an async `execute` method that receives `ExecutionContext` and any arguments.

The initialization provides sensible defaults. If no name is given, it uses the class name; if no description is given, it uses the docstring. These defaults reduce boilerplate when you're creating simple tools. The `tool_definition` property returns the schema that tells the LLM what parameters the tool accepts. Store it as `_tool_definition` to allow subclasses to generate it lazily if necessary.

Listing 4.5 `BaseTool` abstract class

```
class BaseTool(ABC):
    """Abstract base class for all tools."""

    def __init__(
        self,
        name: str = None,
        description: str = None,
        tool_definition: Dict[str, Any] = None,
    ):
        self.name = name or self.__class__.__name__
        self.description = description or self.__doc__ or ""
        self._tool_definition = tool_definition

    @property
    def tool_definition(self) -> Dict[str, Any] | None:
        return self._tool_definition

    @abstractmethod
    async def execute(self, context: ExecutionContext, **kwargs) -> Any:
        pass

    async def __call__(self, context: ExecutionContext, **kwargs) -> Any:
        return await self.execute(context, **kwargs)
```

The `execute` method is abstract because every tool must implement its own logic. Notice that it always receives `context` as its first parameter. This is the key design decision that enables context propagation: the `Agent` passes `ExecutionContext` to every tool, and tools can use it if they need access to execution state. Tools that don't need context simply ignore it.

Finally, `__call__` provides syntactic convenience. Instead of writing `tool.execute(context, query="test")`, we can write `tool(context, query="test")`.

With `BaseTool` defined, we have a contract that all tools must follow. Next, we'll implement `FunctionTool` to wrap the simple functions we created in chapter 3.

4.4.3 FunctionTool: Wrapping functions

`FunctionTool` acts as an adapter that wraps existing functions with the `BaseTool` interface. The key challenge is context propagation. The agent passes `ExecutionContext` to every tool, but most functions don't expect a `context` parameter. They have signatures like `search_web(query: str)`, not `search_web(context: ExecutionContext, query: str)`.

The solution is to inspect each function's signature at initialization. If the function has a `context` parameter, we pass it; if not, we omit it. Simple functions work unchanged, and context-aware functions can access execution state when necessary.

During initialization, we check whether the function accepts a `context` parameter. The `execute` method uses this information to decide whether to forward the context. The agent treats all tools identically, passing context to every one, and each tool decides whether to use it. The `_generate_definition` method reuses `function_to_input_schema` from chapter 3 to automatically create the tool schema from the function's type hints.

Listing 4.6 `FunctionTool` implementation

```
class FunctionTool(BaseTool):
    """Wraps a Python function as a BaseTool."""

    def __init__(
        self,
        func: Callable,
        name: str = None,
        description: str = None,
        tool_definition: Dict[str, Any] = None
    ):
        self.func = func
        self.needs_context = 'context' in inspect.signature(func).parameters

        name = name or func.__name__
        description = description or (func.__doc__ or "").strip()
        tool_definition = tool_definition or self._generate_definition()

        super().__init__(
            name=name,
            description=description,
            tool_definition=tool_definition
        )

    async def execute(self, context: ExecutionContext, **kwargs) -> Any:
        """Execute the wrapped function."""
        if self.needs_context:
            result = self.func(context=context, **kwargs)
        else:
            result = self.func(**kwargs)

        # Handle both sync and async functions
        if inspect.iscoroutine(result):
```

```
            return await result
        return result

    def _generate_definition(self) -> Dict[str, Any]:
        """Generate tool definition from function signature."""
        parameters = function_to_input_schema(self.func)
        return format_tool_definition(self.name, self.description, parameters)
```

The following code snippet shows `FunctionTool` in action. The function's name, docstring, and parameters are extracted automatically to create a fully functional tool:

```
def calculator(expression: str) -> float:
    """Calculate mathematical expressions."""
    return eval(expression)

calc_tool = FunctionTool(calculator)

print(calc_tool.name)           # "calculator"
print(calc_tool.description)    # "Calculate mathematical expressions."
await calc_tool(context, expression="1234 * 5678")  # 7006652
```

The @tool decorator

For convenience, we provide a decorator that creates `FunctionTool` instances. The following code is equivalent to writing `calculator = FunctionTool(calculator)` but uses cleaner syntax:

```
@tool
def calculator(expression: str) -> float:
    """Calculate mathematical expressions."""
    return eval(expression)
```

The decorator also accepts optional parameters that override the tool's metadata. Instead of using the function's name and docstring, you can specify custom values that appear in the tool definition sent to the LLM:

```
@tool(name="web_search", description="Search the internet for information")
def search_web(query: str) -> str:
    """Search the web."""
    return tavily_client.search(query)
```

Throughout this book, we'll use `@tool` for its cleaner syntax. The implementation is a thin wrapper around `FunctionTool`. See the GitHub repository for details.

4.4.4 Integrating MCP tools

We've explored MCP and connected to MCP servers to discover and execute tools. Now we'll use those tools in our agent framework. The challenge is that MCP tools

live on external servers and are executed via `session.call_tool()`, whereas our agent expects `BaseTool` instances with an `execute()` method.

The solution is to create a wrapper function for each MCP tool that calls the server and then wraps that function with `FunctionTool`. This approach reuses everything we've built.

The `load_mcp_tools` function connects to an MCP server, retrieves all available tools, and converts each one to a `FunctionTool`. It takes connection parameters, such as the command to start the server, and returns a list of tools ready for use with our agent.

Listing 4.7 Loading tools from MCP servers

```
async def load_mcp_tools(connection: dict) -> list[BaseTool]:
    """Load tools from an MCP server and convert to FunctionTools."""
    tools = []

    async with stdio_client(StdioServerParameters(**connection)) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            mcp_tools = await session.list_tools()

            for mcp_tool in mcp_tools.tools:
                func_tool = _create_mcp_tool(mcp_tool, connection)
                tools.append(func_tool)

    return tools
```

The core logic lives in `_create_mcp_tool`. For each MCP tool, it creates a wrapper function called `call_mcp` that establishes a connection to the server and executes the tool with whatever arguments are passed. Then this wrapper function is wrapped with `FunctionTool`. Notice that we pass `tool_definition` explicitly rather than let `FunctionTool` generate it because the MCP server already provides a complete schema via `mcp_tool.inputSchema`.

Listing 4.8 Creating `FunctionTool` from the MCP tool

```
def _create_mcp_tool(mcp_tool, connection: dict) -> FunctionTool:
    """Create a FunctionTool that wraps an MCP tool."""

    async def call_mcp(**kwargs):
        async with stdio_client(StdioServerParameters(**connection)) as
     (read, write):
            async with ClientSession(read, write) as session:
                await session.initialize()
                result = await session.call_tool(mcp_tool.name, kwargs)
                return _extract_text_content(result)

    tool_definition = {
        "type": "function",
        "function": {
            "name": mcp_tool.name,
```

```
            "description": mcp_tool.description,
            "parameters": mcp_tool.inputSchema,
        }
    }

    return FunctionTool(
        func=call_mcp,
        name=mcp_tool.name,
        description=mcp_tool.description,
        tool_definition=tool_definition
    )
```

With this adapter, MCP tools integrate seamlessly with our agent. The following example loads tools from the Tavily MCP server and combines them with a local calculator tool. The agent doesn't know or care that some tools are local functions and others call remote MCP servers. All the tools implement the same `BaseTool` interface:

```
connection = {
    "command": "npx",
    "args": ["-y", "tavily-mcp@latest"],
    "env": {"TAVILY_API_KEY": os.getenv("TAVILY_API_KEY")}
}
mcp_tools = await load_mcp_tools(connection)

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[calculator, *mcp_tools],
    instructions="You are a helpful assistant."
)
```

NOTE This implementation creates a new connection for each tool call. Although simple to understand, this approach adds overhead. In production systems with frequent MCP tool use, you might maintain a persistent session to avoid repeated connection setup.

4.5 LLM communication layer

With `ExecutionContext` storing our execution state and tools unified under `BaseTool`, we need one more piece: a way to communicate with the LLM. This communication layer bridges the gap between our internal data structures and the LLM's API requirements.

4.5.1 Reasons for adding a communication layer

Consider what happens when our agent has to call the LLM. `ExecutionContext` contains a list of `Events`, each holding `ContentItems` such as `Messages`, `ToolCalls`, and `ToolResults`. But LLM APIs expect a specific message format: a list of dictionaries with roles and content. Someone must translate between these representations.

We also face the challenge of provider diversity. Although we use LiteLLM to abstract away most provider differences (as we learned in chapter 2), we still have to structure our requests consistently and parse responses into a standard format. Without this layer, translation logic would be scattered throughout the `Agent` class, making it harder to maintain and extend. The solution involves three components working together:

- `LlmRequest`—Packages what we want to send: instructions, conversation contents, and available tools. It serves as a staging area where we select and organize information before the API call.
- `LlmClient`—Handles the API communication. It transforms `LlmRequest` into the format that LLM APIs expect, makes the call, and converts the response back.
- `LlmResponse`—Standardizes what we receive. Regardless of which provider we use, responses come back in the same format that our agent can process.

This separation keeps each component focused. `LlmRequest` knows nothing about API formats; `LlmClient` knows nothing about `ExecutionContext`. Each piece does one job well.

4.5.2 LlmRequest: Selecting what to send

`LlmRequest` is the outbound gate from our agent to the LLM. It holds everything required for a single LLM call organized in four components.

Listing 4.9 `LlmRequest` class

```
class LlmRequest(BaseModel):
    """Request object for LLM calls."""
    instructions: List[str] = Field(default_factory=list)
    contents: List[ContentItem] = Field(default_factory=list)
    tools: List[BaseTool] = Field(default_factory=list)
    tool_choice: Optional[str] = None
```

The `instructions` field holds system prompt fragments. Rather than use a single monolithic prompt, we allow multiple instruction strings that get combined. This flexibility is useful when instructions come from different sources: base agent instructions, task-specific guidance, or dynamically generated context.

The `contents` field stores the conversation history as `ContentItem` objects: messages from users and assistants, `ToolCalls` that the LLM requested, and `ToolResults` from executions. The LLM uses this core context to understand the current situation.

The `tools` field lists available tools as `BaseTool` instances. `LlmClient` extracts their definitions when building the API request.

The `tool_choice` field controls how the LLM selects tools. Setting it to `"auto"` lets the LLM decide freely. Setting it to `"required"` forces tool use, which becomes important for structured output in section 4.6.

Notice what `LlmRequest` doesn't contain: it has no reference to `ExecutionContext` or `Events`. The agent is responsible for extracting relevant information from

ExecutionContext and packaging it into LlmRequest. This separation is intentional. It makes LlmRequest a simple data container while keeping context-selection logic in the agent, where it belongs.

This is where context engineering happens. In section 4.6, we'll implement _prepare_llm_request() in the Agent class, which decides what information from ExecutionContext goes into each LlmRequest. For now, we flatten all events into contents. In chapter 6, this becomes the extension point for memory and context-management strategies.

4.5.3 LlmResponse: Standardizing what we receive

LlmResponse is the inbound gate, standardizing LLM responses regardless of which provider generated them.

Listing 4.10 LlmResponse class

```
class LlmResponse(BaseModel):
    """Response object from LLM calls."""
    content: List[ContentItem] = Field(default_factory=list)
    error_message: Optional[str] = None
    usage_metadata: Dict[str, Any] = Field(default_factory=dict)
```

The content field stores what the LLM produced, represented as ContentItem objects. A response might include a Message with text, one or more ToolCalls requesting tool execution, or both. Using the ContentItem types defined in section 4.3 keeps our data model consistent throughout the system.

The error_message field captures failures. When an API call fails due to network problems, rate limits, or invalid requests, we store the error here rather than raise an exception. This allows the agent to handle failures gracefully, perhaps by retrying or informing the user.

The usage_metadata field tracks token consumption. Knowing how many input and output tokens each call uses helps with cost management and debugging. For now, we simply store this information. Chapter 10 covers monitoring and cost tracking in detail.

4.5.4 LlmClient: The provider adapter

LlmClient bridges LlmRequest and LlmResponse with LLM APIs. Thanks to LiteLLM (introduced in chapter 2), we can support multiple providers through a single implementation.

Listing 4.11 LlmClient class

```
class LlmClient:
    """Client for LLM API calls using LiteLLM."""

    def __init__(self, model: str, **config):
        self.model = model
```

```
        self.config = config

    async def generate(self, request: LlmRequest) -> LlmResponse:
        """Generate a response from the LLM."""
        try:
            messages = self._build_messages(request)
            tools = [t.tool_definition for t in request.tools] if request.
tools else None

            response = await acompletion(
                model=self.model,
                messages=messages,
                tools=tools,
                **({"tool_choice": request.tool_choice}
                   if request.tool_choice else {}),
                **self.config
            )

            return self._parse_response(response)
        except Exception as e:
            return LlmResponse(error_message=str(e))
```

The constructor takes a model identifier (such as `"gpt-5-mini"` or `"anthropic/claude-sonnet-4-5"`) and optional configuration parameters such as `temperature` or `max_tokens`.

The `generate` method orchestrates the API call in three steps:

1. It builds the messages list from the request.
2. It extracts tool definitions with a simple list comprehension.
3. It calls LiteLLM's `acompletion` and parses the response. If anything fails, it returns an `LlmResponse` with the error captured rather than crashing.

BUILDING MESSAGES

The `_build_messages` method transforms `LlmRequest` contents into the message format that LLM APIs expect. The main complexity is that OpenAI's API requires assistant messages and their tool calls to appear together in a single message object while we store them as separate `ContentItems`.

Listing 4.12 Building messages for the LLM API

```
def _build_messages(self, request: LlmRequest) -> List[dict]:
    """Convert LlmRequest to API message format."""
    messages = []

    for instruction in request.instructions:
        messages.append({"role": "system", "content": instruction})

    for item in request.contents:
        if isinstance(item, Message):
            messages.append({"role": item.role, "content": item.content})
```

```
        elif isinstance(item, ToolCall):
            tool_call_dict = {
                "id": item.tool_call_id,
                "type": "function",
                "function": {
                    "name": item.name,
                    "arguments": json.dumps(item.arguments)
                }
            }
            # Append to previous assistant message if exists
            if messages and messages[-1]["role"] == "assistant":
                messages[-1].setdefault("tool_calls", []).append(tool_call_dict)
            else:
                messages.append({
                    "role": "assistant",
                    "content": None,
                    "tool_calls": [tool_call_dict]
                })

        elif isinstance(item, ToolResult):
            messages.append({
                "role": "tool",
                "tool_call_id": item.tool_call_id,
                "content": str(item.content[0]) if item.content else ""
            })

    return messages
```

The logic handles each `ContentItem` type differently. `Messages` become standard message objects. `ToolCalls` are appended to the preceding assistant message's `tool_calls` array because our agent stores them consecutively from the same LLM response. If a `ToolCall` appears without a preceding assistant message, we create one with null content. `ToolResults` become tool-role messages linked to their originating call via `tool_call_id`.

Parsing responses

The `_parse_response` method converts API responses back to our standard format. The method handles both `TextResponses` and `ToolCalls`. When the LLM returns text, we wrap it in a `Message`; when it requests tools, we create `ToolCall` objects. Both can appear in the same response, and our `ContentItem` list accommodates them naturally.

Listing 4.13 Parsing the LLM API's response

```
def _parse_response(self, response) -> LlmResponse:
    """Convert API response to LlmResponse."""
    choice = response.choices[0]
    content_items = []

    if choice.message.content:
        content_items.append(Message(
```

```
                role="assistant",
                content=choice.message.content
            ))

        if choice.message.tool_calls:
            for tc in choice.message.tool_calls:
                content_items.append(ToolCall(
                    tool_call_id=tc.id,
                    name=tc.function.name,
                    arguments=json.loads(tc.function.arguments)
                ))

        return LlmResponse(
            content=content_items,
            usage_metadata={
                "input_tokens": response.usage.prompt_tokens,
                "output_tokens": response.usage.completion_tokens,
            }
        )
```

4.5.5 *Putting it together*

With all three components defined, let's see how they work together. We create an `LlmClient` with a model identifier, build an `LlmRequest` containing our instructions and conversation contents, and call `generate()` to get an `LlmResponse`. The response's content list contains the LLM's output as `ContentItem` objects, which we can iterate through to extract the answer.

Listing 4.14 Using the LLM communication layer

```
from scratch_agents import LlmClient, LlmRequest, Message

# Create client
client = LlmClient(model="gpt-5-mini")

# Build request
request = LlmRequest(
    instructions=["You are a helpful assistant."],
    contents=[Message(role="user", content="What is 2 + 2?")],
)

# Generate response
response = await client.generate(request)

# Response contains the answer
for item in response.content:
    if isinstance(item, Message):
        print(item.content)  # "4"
```

The communication layer provides clean separation of concerns. `LlmRequest` packages what to send. `LlmClient` handles how to send it. `LlmResponse` standardizes what comes back.

4.6 Implementing the agent

With `ExecutionContext` storing our state, tools unified under `BaseTool`, and `LlmClient` handling LLM communication, we're ready to build the `Agent` class that orchestrates everything. The `Agent` brings together all the components we've built, coordinating the flow of information between the LLM and tools to solve problems step by step.

Figure 4.4 shows the complete picture of what we're implementing. The `Agent` initializes with `_setup_tools()`; then `run()` creates an `ExecutionContext` and repeatedly calls `step()` until completion. Each `step()` performs one think–act cycle: `think()` calls the LLM to decide what to do next, and `act()` executes any requested tools.

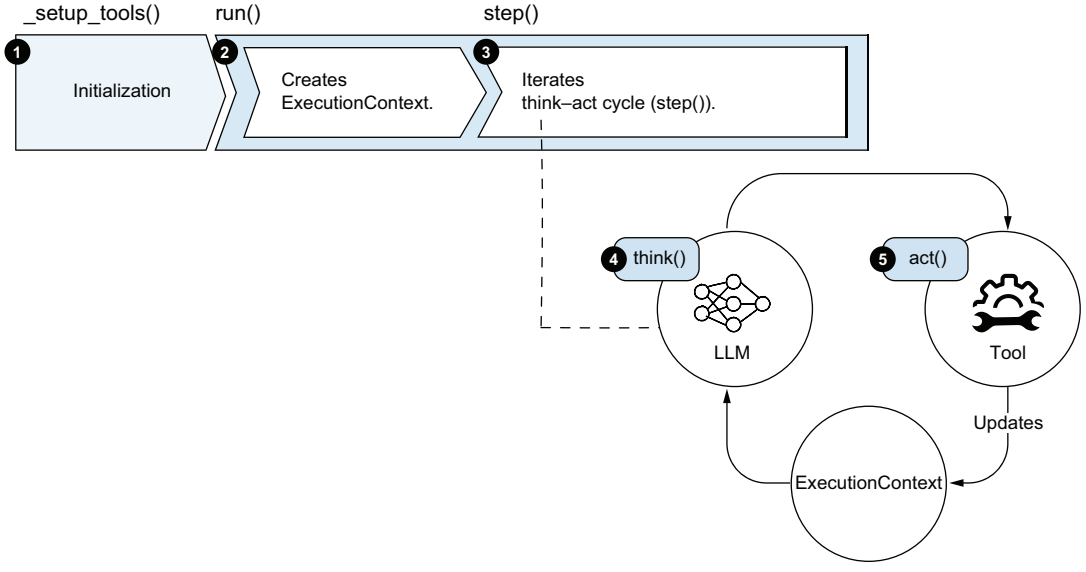

Figure 4.4 Agent implementation road map showing the five core methods and their execution flow. The numbered sequence illustrates how the components work together to solve problems.

We'll implement these five methods in logical order:

1. `_setup_tools()`—Registers the tools the agent will use, converting the list to a dictionary for efficient lookup
2. `run()`—The entry point; receives user requests, manages the execution loop, and returns the final result
3. `step()`—Performs one complete think–act cycle, updating `ExecutionContext` with each interaction
4. `think()`—Calls the LLM to analyze the situation and decide the next action
5. `act()`—Executes the tools selected by the LLM and captures their results

By the end of this section, you'll have a working agent that can solve multistep problems. You'll also add structured output capabilities, allowing the agent to return typed Pydantic models instead of free-form text when your application requires predictable data formats.

4.6.1 Agent class structure

Let's start with the `Agent` class constructor and the `_setup_tools()` method. The constructor receives all the components we've built in previous sections and stores them for use during execution.

Listing 4.15 `Agent` class constructor

```
class Agent:
    def __init__(
        self,
        model: LlmClient,
        tools: List[BaseTool] = None,
        instructions: str = "",
        max_steps: int = 10,
        name: str = "agent",
    ):
        self.model = model
        self.instructions = instructions
        self.max_steps = max_steps
        self.name = name
        self.tools = self._setup_tools(tools or [])
```

The parameters serve distinct purposes:

- `model`—The `LlmClient` instance that handles LLM communication
- `tools`—List of `BaseTool` instances the agent can use
- `instructions`—System prompt that defines the agent's behavior
- `max_steps`—Safety limit to prevent infinite loops when the agent keeps calling tools without reaching a conclusion

The `_setup_tools()` method prepares the tools list for use. For now, it returns the list unchanged, but this method serves as an extension point for future enhancements, such as adding default tools.

Listing 4.16 Setting up agent tools

```
def _setup_tools(self, tools: List[BaseTool]) -> List[BaseTool]:
    return tools
```

With the basic structure in place, let's implement the `run()` method, which orchestrates the entire execution.

4.6.2 The run() method

The `run()` method is the entry point for agent execution. It creates the execution environment, manages the think–act loop, and returns the result.

To give callers access to the full execution trace, we define `AgentResult`, which bundles the final output with its `ExecutionContext`. This allows inspection of every step the agent took, which is invaluable for debugging and analysis.

Listing 4.17 `AgentResult` data class

```
@dataclass
class AgentResult:
    """Result of an agent execution."""
    output: str | BaseModel
    context: ExecutionContext
```

First, the method checks whether an `ExecutionContext` was provided, creating one if not. Then it wraps the user's input in an `Event` and adds it to the context. The main loop calls `step()` repeatedly until a final result is obtained or the maximum step limit is reached. After each step, we check whether the last event represents a final response and, if so, extract the result.

Listing 4.18 Agent `run()` method

```
async def run(
    self,
    user_input: str,
    context: ExecutionContext = None
) -> AgentResult:
    # Create or reuse context
    if context is None:
        context = ExecutionContext()

    # Add user input as the first event
    user_event = Event(
        execution_id=context.execution_id,
        author="user",
        content=[Message(role="user", content=user_input)]
    )
    context.add_event(user_event)

    # Execute steps until completion or max steps reached
    while not context.final_result and context.current_step < self.max_steps:
        await self.step(context)

        # Check if the last event is a final response
        last_event = context.events[-1]
        if self._is_final_response(last_event):
            context.final_result = self._extract_final_result(last_event)

    return AgentResult(output=context.final_result, context=context)
```

The helper methods handle completion detection and result extraction. The `_is_final_response()` method checks whether an event represents a final answer by examining its contents. An event is final when it contains neither tool calls nor tool results, meaning that the LLM provided a direct answer. The `_extract_final_result()` method iterates through the event's content to find the assistant's message.

Listing 4.19 Completion-detection helpers

```
def _is_final_response(self, event: Event) -> bool:
    """Check if this event contains a final response."""
    has_tool_calls = any(isinstance(c, ToolCall) for c in event.content)
    has_tool_results = any(isinstance(c, ToolResult) for c in event.content)
    return not has_tool_calls and not has_tool_results

def _extract_final_result(self, event: Event) -> str:
    for item in event.content:
        if isinstance(item, Message) and item.role == "assistant":
            return item.content
    return None
```

This approach is straightforward. Access the answer via `result.output`, and examine execution details through `result.context` when necessary:

```
result = await agent.run("What is 1234 * 5678?")
print(result.output)                        # "7006652"
print(result.context.current_step)
```

4.6.3 The step() method

The `step()` method performs one complete think–act cycle. It prepares a request for the LLM, gets the LLM's decision, and executes any tools that the LLM requests.

The method starts by calling `_prepare_llm_request()` to package the current context in a format suitable for the LLM. Then it calls `think()` to get the LLM's response and wraps that response in an `Event`, recording it in the context. If the response contains tool calls, the method calls `act()` to execute them and records the results as another `Event`. Finally, it increments the step counter.

Listing 4.20 Agent `step()` method

```
async def step(self, context: ExecutionContext):
    # Prepare what to send to the LLM
    llm_request = self._prepare_llm_request(context)

    # Get LLM's decision
    llm_response = await self.think(llm_request)

    # Record LLM response as an event
    response_event = Event(
        execution_id=context.execution_id,
```

```
            author=self.name,
            content=llm_response.content,
        )
        context.add_event(response_event)

        # Execute tools if the LLM requested any
        tool_calls = [c for c in llm_response.content if isinstance(c, ToolCall)]
        if tool_calls:
            tool_results = await self.act(context, tool_calls)
            tool_event = Event(
                execution_id=context.execution_id,
                author=self.name,
                content=tool_results,
            )
            context.add_event(tool_event)

        context.increment_step()
```

The `_prepare_llm_request()` method extracts information from `ExecutionContext` and packages it in an `LlmRequest`. It flattens all events into a list of content items, combines the items with the agent's instructions and available tools, and sets the tool choice to `"auto"` so the LLM can decide whether to use tools.

Listing 4.21 Preparing the LLM request from context

```
def _prepare_llm_request(self, context: ExecutionContext) -> LlmRequest:
    # Flatten events into content items
    flat_contents = []
    for event in context.events:
        flat_contents.extend(event.content)

    return LlmRequest(
        instructions=[self.instructions] if self.instructions else [],
        contents=flat_contents,
        tools=self.tools,
        tool_choice="auto" if self.tools else None,
    )
```

Notice the separation between `ExecutionContext` and `LlmRequest`. `ExecutionContext` holds the complete execution history, whereas `LlmRequest` contains only what we choose to send to the LLM. Currently, we send everything, but this separation is where context engineering happens. We could summarize old messages, omit irrelevant tool results, or inject additional information by modifying how we build `LlmRequest` without touching the original context.

4.6.4 The think() and act() methods

The `think()` and `act()` methods handle the two core operations in each step: getting the LLM's decision and executing the requested tools.

The `think()` method is deliberately simple. It takes the prepared `LlmRequest` and passes it to the `LlmClient`, returning whatever response comes back. The complexity

of building the request happens in `_prepare_llm_request()`, and the complexity of parsing the response happens in `LlmClient`. This keeps `think()` focused on a single responsibility.

Listing 4.22 Agent `think()` method

```
async def think(self, llm_request: LlmRequest) -> LlmResponse:
    return await self.model.generate(llm_request)
```

The `act()` method executes the tools that the LLM requested. First, it builds a dictionary mapping tool names to tool instances for efficient lookup. Then it iterates through each `ToolCall`, retrieves the corresponding tool, executes it with the provided arguments, and collects the results. The method passes `ExecutionContext` to each tool execution, allowing tools to access execution state if they need it. Each execution is wrapped in a `try-except` block, capturing failures as error results rather than crashing the entire agent.

Listing 4.23 Agent `act()` method

```
async def act(
    self,
    context: ExecutionContext,
    tool_calls: List[ToolCall]
) -> List[ToolResult]:
    tools_dict = {tool.name: tool for tool in self.tools}
    results = []

    for tool_call in tool_calls:
    if tool_call.name not in tools_dict:
        results.append(ToolResult(
            tool_call_id=tool_call.tool_call_id,
            name=tool_call.name,
            status="error",
            content=[f"Tool '{tool_call.name}' not found"],
        ))
        continue

    tool = tools_dict[tool_call.name]

        try:
            output = await tool(context, **tool_call.arguments)
            results.append(ToolResult(
                tool_call_id=tool_call.tool_call_id,
                name=tool_call.name,
                status="success",
                content=[output],
            ))
        except Exception as e:
            results.append(ToolResult(
                tool_call_id=tool_call.tool_call_id,
                name=tool_call.name,
                status="error",
```

```
                content=[str(e)],
            ))

    return results
```

When tool execution fails, the error message becomes part of the conversation history. The LLM sees this failure in the next step and adapts its approach, perhaps trying a different tool or rephrasing its query. This graceful error handling is one of the advantages of the ReAct pattern: the agent can recover from failures rather than stop entirely.

4.7 Adding structured output

In chapter 2, we tested LLMs on General AI Assistants (GAIA) benchmark problems using structured output. We defined a `GaiaOutput` Pydantic model with fields such as `is_solvable`, `unsolvable_reason`, and `final_answer`; then we passed it as the `response_format` parameter to ensure that the LLM returned data in a predictable structure. This approach worked well for standalone LLM calls.

The agent we built in section 4.6, however, returns only free-form text. When the LLM finishes reasoning and provides its final answer, `_extract_final_result()` simply extracts the text content from the assistant message. This works fine for conversational interactions, but many production scenarios require structured data that downstream systems can parse reliably.

Consider an agent that analyzes customer feedback. Instead of returning "The sentiment is positive with a confidence of 85%," you may need it to return a structured object with `sentiment: "positive"` and `confidence: 0.85`. Or imagine a research agent that must return findings in a specific format for a report-generation pipeline. Free-form text would require fragile parsing logic that could break whenever the LLM phrases things differently.

To address this possibility, we'll add structured output capability to our agent. This exercise serves two purposes: it reinforces your understanding of the agent architecture by modifying its core methods, and it introduces a pattern for extending agents with new features.

4.7.1 The approach: Tools as output formatters

The key insight is that we can use our existing tool-calling infrastructure to enforce structured outputs. Instead of having the LLM generate free text and then trying to parse it into a structure, we create a special tool that accepts our desired output schema as its input. When the agent has to provide its final answer, it calls this tool with properly structured data, guaranteeing type safety and format compliance.

This approach has a couple of advantages:

- It reuses our existing tool-calling mechanism, keeping the implementation simple and consistent.

- It uses the LLM's extensive training on tool calling to ensure proper formatting. The LLM already knows how to generate valid JSON matching a schema; we simply give it a schema that matches our desired output structure.

The implementation requires modifications of five methods, each of which plays a specific role in the agent's architecture (table 4.2).

Table 4.2 Changes required to add structured output support to the `Agent` class

Method	Role	Modification
`__init__`	Stores configuration	Adds `output_type` parameter
`_setup_tools()`	Prepares available tools	Creates `final_answer` tool dynamically
`_prepare_llm_request()`	Packages context for LLM	Forces tool usage when structured output needed
`_is_final_response()`	Detects completion	Checks for `final_answer` tool success
`_extract_final_result()`	Extracts the answer	Returns Pydantic model from `ToolResult`

4.7.2 Modifying the agent

First, we add an `output_type` parameter to the constructor. When provided, this parameter should be a Pydantic model class that defines the expected output structure. We also add `output_tool_name` to track the name of our dynamically created tool.

Listing 4.24 Agent constructor with structured output

```
class Agent:
    def __init__(
        self,
        model: LlmClient,
        tools: List[BaseTool] = None,
        instructions: str = "",
        max_steps: int = 10,
        output_type: Optional[Type[BaseModel]] = None,  # New parameter
    ):
        self.model = model
        self.instructions = instructions
        self.max_steps = max_steps
        self.output_type = output_type
        self.output_tool_name = None  # Will be set if output_type provided
        self.tools = self._setup_tools(tools or [])
```

The key logic happens in `_setup_tools()`. When an `output_type` is specified, we dynamically create a `final_answer` tool that accepts the specified Pydantic model as its input. This tool simply returns whatever structured data it receives, but making it a tool ensures that the LLM must format its response according to our schema.

Notice that we use Python's closure to capture `self.output_type` in the tool definition. The `@tool` decorator we implemented earlier handles the Pydantic model validation automatically, ensuring that any call to `final_answer` provides data that matches our schema.

Listing 4.25 Dynamic output tool creation

```
def _setup_tools(self, tools: List[BaseTool]) -> List[BaseTool]:
    if self.output_type is not None:
        @tool(
            name="final_answer",
            description="Return the final structured answer matching the
required schema."
        )
        def final_answer(output: self.output_type) -> self.output_type:
            return output

        tools = list(tools)  # Create a copy to avoid modifying the original
        tools.append(final_answer)
        self.output_tool_name = "final_answer"

    return tools
```

When structured output is required, we must ensure that the agent uses that `final_answer` tool instead of providing a free-text response. The OpenAI API and most other providers support a `tool_choice` parameter that can force the model to use tools. We modify `_prepare_llm_request()` to set this parameter appropriately.

The logic is straightforward. If we have an output tool (meaning that structured output is needed), we set `tool_choice="required"` to force the LLM to use a tool; otherwise, we fall back to the original behavior.

Listing 4.26 Forcing tool use for structured output

```
def _prepare_llm_request(self, context: ExecutionContext) -> LlmRequest:
    flat_contents = []
    for event in context.events:
        flat_contents.extend(event.content)

    # Determine tool choice strategy
    if self.output_tool_name:
        tool_choice = "required"  # Force tool usage for structured output
    elif self.tools:
        tool_choice = "auto"
    else:
        tool_choice = None

    return LlmRequest(
        instructions=[self.instructions] if self.instructions else [],
        contents=flat_contents,
        tools=self.tools,
        tool_choice=tool_choice,
    )
```

To detect when the agent has produced its final structured output, we modify `_is_final_response()`. When structured output is required, we look for a successful result from our `final_answer` tool. Only when this specific tool returns successfully do we consider the agent's work complete. For free-text responses, we maintain the original behavior.

Listing 4.27 Detecting structured output completion

```
def _is_final_response(self, event: Event) -> bool:
    if self.output_tool_name:
        # For structured output: check if final_answer tool succeeded
        for item in event.content:
            if (isinstance(item, ToolResult)
                and item.name == self.output_tool_name
                and item.status == "success"):
                return True
        return False

    # Original logic for free-text responses
    has_tool_calls = any(isinstance(c, ToolCall) for c in event.content)
    has_tool_results = any(isinstance(c, ToolResult) for c in event.content)
    return not has_tool_calls and not has_tool_results
```

Finally, we update `_extract_final_result()` to extract structured data from tool results. When a required output tool is specified, we look for its successful execution and return the validated Pydantic model instance. This gives the calling code a strongly typed object to work with rather than raw text.

Listing 4.28 Extracting structured results

```
def _extract_final_result(self, event: Event) -> str | BaseModel:
    if self.output_tool_name:
        # Extract structured output from final_answer tool result
        for item in event.content:
            if (isinstance(item, ToolResult)
                and item.name == self.output_tool_name
                and item.status == "success"
                and item.content):
                return item.content[0]

    # Original logic for free-text responses
    for item in event.content:
        if isinstance(item, Message) and item.role == "assistant":
            return item.content
    return None
```

4.7.3 Using structured output in practice

With these modifications in place, using structured output becomes straightforward. The following listing shows how you might build an agent that analyzes customer feedback and returns structured sentiment data.

Listing 4.29 Using structured output in practice

```
from pydantic import BaseModel
from typing import Literal, List

class SentimentAnalysis(BaseModel):
    sentiment: Literal["positive", "negative", "neutral"]
    confidence: float
    key_phrases: List[str]

agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[],
    instructions="Analyze the sentiment of the provided text.",
    output_type=SentimentAnalysis
)

result = await agent.run("This product exceeded my expectations! Highly
recommend.")
print(f"Sentiment: {result.output.sentiment}")      # "positive"
print(f"Confidence: {result.output.confidence}")    # 0.92
print(f"Key phrases: {result.output.key_phrases}")  # ["exceeded
expectations", "highly recommend"]
```

The agent processes the input and reasons about the sentiment; when it's ready to respond, it calls the `final_answer` tool with a properly structured `SentimentAnalysis` object. The agent's output is the Pydantic model itself, not a string, and it is available through result.output, so you can access fields with full type safety.

This pattern extends naturally to more complex scenarios. You could define output schemas for research summaries, data-extraction results, or any structured format your application requires. By integrating structured output through the existing tool-calling mechanism, you've added powerful type safety and format guarantees without complicating the agent's core logic.

4.8 Testing with the GAIA benchmark

In chapter 2, we tested LLMs directly on GAIA problems and observed their limitations. Most problems required information that the LLM didn't have, leading to low accuracy. Now let's see how our agent performs when it's equipped with tools.

4.8.1 From LLM to agent

The evaluation setup remains largely the same as in chapter 2. We reuse `GaiaOutput`, `gaia_prompt`, `is_correct()`, and the experiment runner structure. The key difference lies in what solves each problem. Instead of making a direct LLM call, now we use an agent with tools. First, we load tools from the Tavily MCP server, which we explored in chapter 3.

Listing 4.30 Loading MCP tools for GAIA

```
from scratch_agents.tools import load_mcp_tools

tavily_connection = {
```

```
    "command": "npx",
    "args": ["-y", "tavily-mcp@latest"],
    "env": {"TAVILY_API_KEY": os.getenv("TAVILY_API_KEY")}
}

mcp_tools = await load_mcp_tools(tavily_connection)
```

Next, we create an agent configured for GAIA evaluation. The agent receives the MCP tools for web search and uses `GaiaOutput` as its structured output type.

Listing 4.31 Creating the GAIA evaluation agent

```
def create_gaia_agent(model: str, tools: list) -> Agent:
    return Agent(
        model=LlmClient(model=model),
        tools=tools,
        instructions=gaia_prompt,
        output_type=GaiaOutput,
        max_steps=15,
    )
```

The core change is in `solve_problem`. Whereas we called `acompletion()` directly in chapter 2, now we call `agent.run()`. The returned `AgentResult` contains both the output and the full execution context.

Listing 4.32 Solving problems with an agent

```
SEMAPHORE = asyncio.Semaphore(3)

async def solve_problem(agent: Agent, question: str) -> AgentResult:
    async with SEMAPHORE:
        return await agent.run(question)
```

Now the semaphore limits concurrent `Agent` executions rather than individual LLM calls. Each `Agent` execution may involve multiple internal LLM calls as it reasons and uses tools, but limiting parallel problem solving prevents those calls from overwhelming the APIs.

The rest of the evaluation pipeline, including `evaluate_single()` and `run_experiment()`, follows the same structure as in chapter 2. The only adjustment is accessing the answer via `result.output` instead of parsing the LLM response directly. See the GitHub repository for the complete implementation.

4.8.2 Results

The experiment compares an agent that has web search tools with an agent that doesn't have those tools on 10 GAIA problems that require web access. Both agents use GPT-5 as the underlying model.

Listing 4.33 Running the GAIA experiment

```
models = [
    "gpt-5",
]

results = await run_experiment(
    problems=list(problems),
    models=models,
    tools=mcp_tools,
)
```

DEBUGGING WITH EXECUTION TRACES

When problems fail, we need to understand why. Because `AgentResult` includes the full `ExecutionContext`, we can examine what happened during execution. For convenient visualization in Jupyter Notebooks, we provide a `display_trace()` utility.

Listing 4.34 Debugging with execution traces

```
from scratch_agents.utils import display_trace

agent = create_gaia_agent(models[0], mcp_tools)
result = await agent.run(
    "If Eliud Kipchoge could maintain his marathon pace, "
    "how many thousand hours to reach the Moon?"
)

print(f"Answer: {result.output.final_answer}")
print(f"Steps: {result.context.current_step}")

display_trace(result.context)
```

This trace reveals the agent's reasoning: it identified the need for two pieces of information, searched for each piece, and produced the final answer. When debugging failures, you can pinpoint where things went wrong.

THE EFFECT OF WEB SEARCH TOOLS

To quantify the value of web search capabilities, we ran a controlled experiment comparing agents with and without search tools on GAIA level-1 problems that require web access. We selected 10 problems for which the official GAIA annotations indicated web search as a required tool(table 4.3).

Table 4.3 GAIA level-1 results with and without web search tools

Condition	Correct	Total cost
With web search	7/10	$1.89
Without web search	2/10	$0.26

The results demonstrate a 50 percentage-point improvement when web search tools are available. Without search capabilities, the agent could answer only questions involving information that happened to exist in its training data. With search tools, the agent successfully retrieved current information from Wikipedia, academic papers, and other web sources.

Examining failures reveals interesting patterns. The agent without web search either admitted that it couldn't solve the problem (returning `is_solvable=False`) or hallucinated incorrect answers. When it was asked about Mercedes Sosa's studio albums from 2000 to 2009, the agent with search correctly found three albums on Wikipedia; the agent without search guessed four.

REMAINING CHALLENGES

Although web search dramatically improves performance on information retrieval tasks, many GAIA problems require additional capabilities. A significant portion of level-1 problems includes attached files (such as PDFs, spreadsheets, images, and audio files) that the agent must process to find the answer. Our current agent can't handle these file-based problems.

In chapter 5, we'll extend our agent with file-processing tools that can read PDFs, parse spreadsheets, analyze images, and transcribe audio. Chapter 5 will unlock the remaining GAIA problems and move us closer to creating a truly general-purpose AI assistant.

Summary

- ReAct is the foundational pattern behind modern AI agents, in which the LLM alternates between reasoning about what it needs and taking actions to get it. Modern implementations use tool calling instead of fragile text parsing.
- `ExecutionContext` centralizes all execution state, including events, step count, execution ID, and flexible state storage. This design simplifies method signatures and enables context propagation to tools that need access to execution state.
- Tool abstraction through `BaseTool` and `FunctionTool` provides a consistent interface for all tools. The `@tool` decorator wraps simple functions, whereas `load_mcp_tools()` integrates external MCP servers, allowing the agent to use both local and remote tools seamlessly.
- The LLM communication layer separates concerns cleanly. `LlmRequest` packages what to send, `LlmClient` handles API communication via LiteLLM, and `LlmResponse` standardizes what comes back. This separation is where context engineering happens.
- The `Agent` class orchestrates everything through five core methods. `_setup_tools()` prepares available tools, `run()` manages the execution loop, `step()` performs one think–act cycle, `think()` calls the LLM, and `act()` executes requested tools.
- Structured output enables agents to return type-safe Pydantic models instead of free-form text. By creating a dynamic `final_answer` tool from the output schema, we use existing tool-calling infrastructure to guarantee format compliance.

Part 2

Developing advanced agent capabilities

The agent you built in part 1 runs beautifully inside a notebook, but hand it a realistic request, and its limits show up fast. Its knowledge is frozen to the training data, so without a way to fetch fresh documents, the answers drift out of date. Between turns, nothing is remembered, which rules out any longer conversation. Hard problems throw it off because it jumps in without a plan. Code is something it can discuss but not run. Finally, tasks that demand several skills tend to overwhelm a single agent trying to do everything at the same time. This part closes each of those gaps one chapter at a time and ends by showing how to evaluate whether the resulting agent works.

Chapter 5 introduces retrieval augmented generation (RAG) and file tools so that the agent can reach for knowledge that lives outside the model's training data. You'll implement embeddings, chunking, and vector search from scratch and give the agent a way to read from local files.

Chapter 6 adds memory in three layers. You'll build a way to manage ever-growing conversations, session handling that so different users keep their own history, and long-term memory that survives across runs. The same chapter introduces human-in-the-loop (HITL) approval for sensitive tool calls.

Chapter 7 teaches the agent to break a complex problem into a plan and review its own intermediate results so that instead of charging at the problem one step at a time, it works with a sense of the whole task.

Chapter 8 enables the agent to write and execute Python inside a sandbox. It also introduces a reusable skill system that the agent can use to assemble its own higher-level capabilities from existing tools.

Chapter 9 takes the final step out of the single-agent world into multiagent systems. You'll build workflow orchestration, wrap agents as tools for other agents, let one agent transfer control to another, and connect agents across the network with the A2A protocol.

Chapter 10 closes the loop by showing how to evaluate agents systematically. You'll learn why agent evaluation must look beyond the final answer, how observability data (such as metrics, traces, and logs) supports evaluation, how to build datasets and rubrics from real failure patterns, and how to use LLM-as-a-Judge and CI/CD gates to keep agent quality improving after deployment.

One design principle runs through every chapter in this part: each new capability lands as an optional extension to the agent you've already built, not a rewrite of it. The agent from chapter 4 still runs at the end of chapter 10, but with more powers available when they're needed and with an evaluation process for checking whether those powers help. This is how healthy production codebases grow, and practicing it here means that the habit will transfer when you build agents outside this book. By the end of part 2, you'll have an agent that has outgrown the demo stage and is ready for real workflows.

5 Building knowledge bases with RAG

This chapter covers

- Understanding why agents need external knowledge bases
- Using keyword, vector, graph, and structure-based search methods
- Implementing vector search from scratch with embeddings, chunking, and similarity calculations
- Building structure-based search tools for filesystem exploration
- Extending agent capabilities through the callback pattern

We've built the foundation of an AI agent: connecting to large language models (LLMs), implementing tool use, and creating the agent loop. Now we enter a new phase. As figure 5.1 shows, we'll enhance our basic agent with context-engineering strategies. This chapter tackles the first of these enhancements: building knowledge bases with retrieval-augmented generation (RAG).

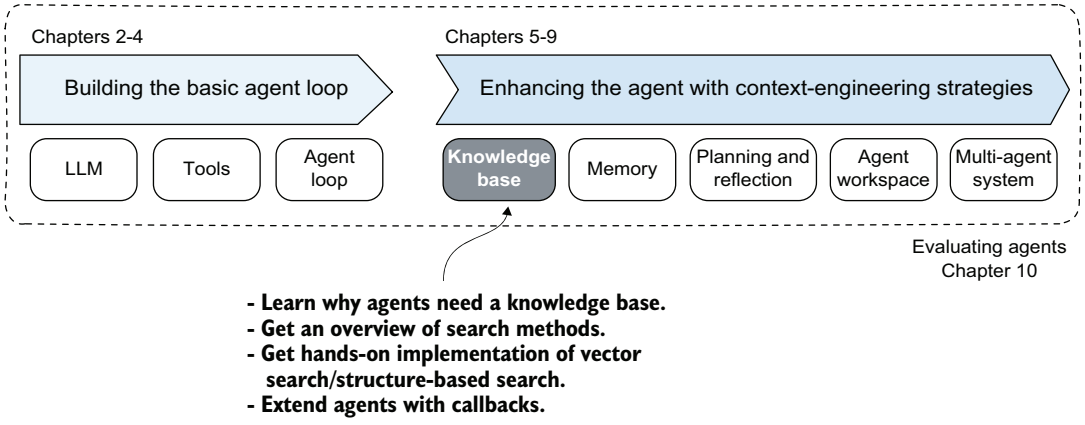

Figure 5.1 Book structure overview: chapter 5 in focus

We'll begin with the basic components of vector search: how embeddings capture text meaning, why chunking is necessary, and how vector databases operate. We'll implement a mini vector search system to experience the full flow. Next, we'll extend this system to structure-based search using General AI Assistants (GAIA) benchmark zip file problems, in which the agent navigates folder structures and reads files like a human developer. Finally, we'll see how to use the callback pattern to extend agent behavior, as well as to implement human-in-the-loop (HITL) approval and automatic search result compression.

> **NOTE** Because this book focuses on LLM agents, it doesn't deeply cover all search methods in RAG. Detailed discussions of keyword search techniques (such as BM25 and TF-IDF), graph search implementations (such as knowledge graph construction and query languages), and score combination strategies for hybrid search are beyond the scope of this book. Our emphasis is on how agents use knowledge base tools, focusing on vector search and structure-based search through practical exercises. If you want to learn more, refer to *A Simple Guide to Retrieval Augmented Generation*, by Abhinav Kimothi (Manning, 2025; https://mng.bz/eBWz), and *Essential GraphRAG*, by Tomaž Bratanič and Oskar Hane (Manning, 2025; https://mng.bz/pZAG).

5.1 The problem of using internal data

Real work environments present many problems that web search can't solve. In these cases, we need to use data that isn't publicly available, such as internal company documents, project codebases, and personal files. These internal data sources follow the same principle as web search: in a vast amount of data, we have to find only the parts that are relevant to the current question and put them in the context. We'll implement search functionality so that agents can use internal data effectively.

5.1.1 A simple case: Single file

Let's start with the simplest situation. The GAIA benchmark includes problems with attached files. Suppose that a CSV file is given with the question "What month had the highest sales in this data?"

The solution here is straightforward: read the file and put the entire thing into the context. If the file is a few hundred lines, it fits easily within modern LLMs' context windows. We can solve such problems simply by adding a file-reading tool to the agent we implemented in chapter 4.

5.1.2 Multiple files

The GAIA benchmark also includes problems with attached zip files rather than single files. When you extract them, multiple files and folders appear. The problem is that you don't know in advance which file contains the answer. Suppose that you're given a code archive along with the question "What type of database is used in this project?" When you extract the provided zip file, you get this structure:

```
project/
├── README.md
├── requirements.txt
├── src/
│   ├── main.py
│   ├── database.py
│   └── utils.py
├── config/
│   └── settings.json
└── docs/
    └── architecture.md
```

The answer might be in `requirements.txt`, `database.py`, or `settings.json`. Reading all files and putting them into the context is inefficient and becomes impossible as the number of files grows.

This situation calls for exploration. First, understand the overall structure; then use filenames and folder names as clues to select seemingly relevant files and examine their contents. If necessary, explore additional files. This is how developers explore new codebases. Code projects are particularly suitable for this approach because they have systematic structures. Tools like Claude Code, Cursor, and Gemini CLI understand code in the same way. This search is structure based.

5.1.3 Large or extensive data

Structure-based exploration can't handle all situations, however. One example is when a single file is too large to fit in the context. Log files with tens of thousands of lines or large documents fall into this category. Another example is a case with too many documents. If a company wiki has thousands of documents, exploring them one by one isn't realistic. Also, information in company wikis isn't always systematically organized.

Therefore, we need a mechanism that selectively retrieves only the necessary parts. For a question about how to connect to the database, instead of putting tens of thousands of lines of code into the context, we should find only the database-related portions. This is why technologies such as vector search or keyword search are necessary.

5.2 Types of search methods

RAG is exactly what its name suggests: a method that searches (retrieval) for necessary information, adds (augmentation) it to the LLM input, and then generates (generation) a response. Rather than answering solely from its own knowledge, the LLM references information found in external data to formulate its response.

Figure 5.2 shows the overall flow of RAG. When a user question comes in, first the system searches for relevant information from the knowledge base. The search results are passed to the LLM along with the original question, and the LLM generates a response using this information as a reference. Through this process, RAG can overcome the LLM's knowledge limitations and use up-to-date information or private data.

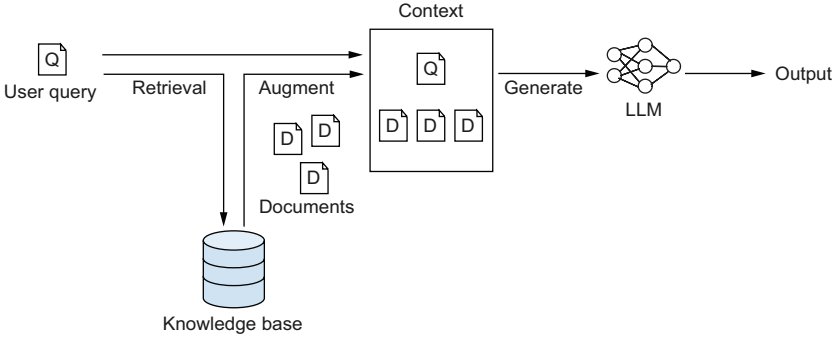

Figure 5.2 The overall flow of RAG. When a user question comes in, the system searches for relevant information from the knowledge base and then passes the search results and the question to the LLM, which generates a response.

We've experienced RAG through web search, in which the LLM resolved tasks by finding relevant information with search queries and adding those results to its context to generate answers. In this broad sense, RAG encompasses all forms of information retrieval and use. The difference in this chapter is that we'll build our own search tools rather than use prebuilt ones.

Figure 5.3 illustrates several types of search methods. The appropriate method varies depending on data characteristics and question types. Different approaches are effective when you need exact word matching or are searching for semantically similar content, traversing relationships, or examining file structure directly. This section explores the principles of four major search methods and discusses when each is most suitable.

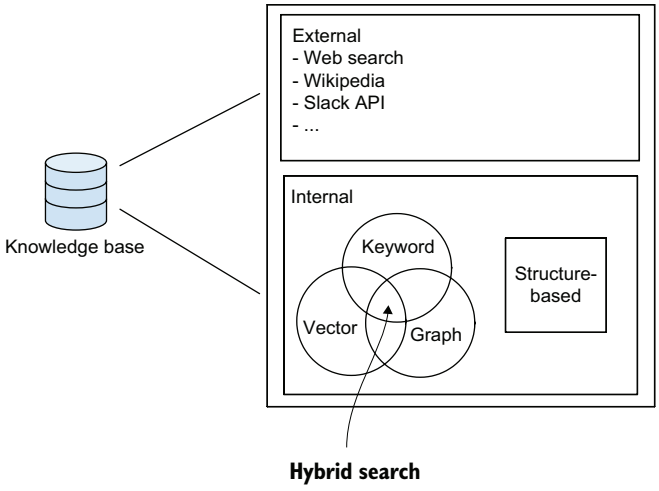

Figure 5.3 Types of search: external/internal repositories, keyword/vector/graph search, and structure-based search

5.2.1 Keyword search

Keyword search is the most traditional search method, evaluating relevance by calculating the frequency and importance of words contained in documents. Representative algorithms include BM25 and TF-IDF.

TF-IDF (Term Frequency-Inverse Document Frequency) combines two metrics. It multiplies how frequently a specific word appears within a document (TF) by how rare that word is across the entire document collection (IDF). Words like *the* that appear everywhere have low importance, and words that appear intensively only in specific documents have high importance. BM25 improves on TF-IDF by providing more accurate results through adjustments for document length and the saturation effect of word frequency.

Keyword search is most effective for finding exact words or phrases. No method is faster or more accurate than keyword search when searching for a function name like `get_user_by_id`, finding an error code like `404`, or checking a specific configuration value like `MAX_CONNECTIONS`. As a technology that's been proven over decades, it guarantees fast search speeds even on large-scale datasets.

But keyword search has fundamental limitations. One is that it can't handle synonyms or similar expressions. Figure 5.4 illustrates this problem. *Sofa* and *couch* refer to the same furniture, but keyword search doesn't know that these two terms are related. Searching

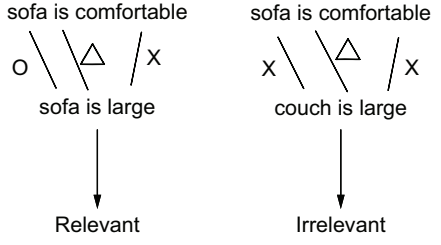

Figure 5.4 Although *sofa* and *couch* have similar meanings, a keyword search can't determine their similarity without separate processing. Common terms such as *is* that appear frequently aren't considered important (shown with triangles). The left example is judged relevant because it overlaps with *sofa*, but the right example is judged irrelevant despite being semantically relevant.

for *sofa* won't find documents containing *couch*. Without building a separate synonym dictionary, this semantic connection can't be recognized automatically.

5.2.2 Vector search

Vector search quantifies the meaning of text and calculates similarity. First, it uses an embedding model to transform text into a point in a high-dimensional vector space. In this process, texts with similar meanings are mapped to nearby positions, and texts with different meanings are mapped to distant positions.

An *embedding* represents words or sentences as arrays of hundreds to thousands of numbers. The word *cat*, for example, might become a vector like [0.2, –0.5, 0.8, ...]. The key point is that this transformation preserves meaning. *Cat* and *kitten* are different words but have similar meanings, so they're placed at nearby positions in vector space. By contrast, *cat* and *car* are similar in spelling but different in meaning, so they're placed far apart.

Figure 5.5 shows the principle of vector search. Documents are converted to vectors in advance and stored. When a search runs, the query is converted to a vector in the same way. Then the system calculates distances between the query vector and the stored document vectors to find and return the closest ones. Distance calculation typically uses cosine similarity or Euclidean distance. *Cosine similarity* is the cosine value of the angle between two vectors: values range from –1 to 1, with values closer to 1 indicating higher similarity. *Euclidean distance* is the straight-line distance between two points, with smaller values indicating more similarity.

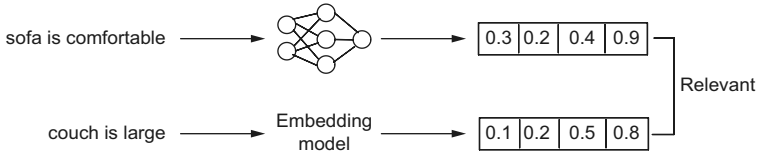

Figure 5.5 Vector search determines relevance by converting data to vectors. Using a well-trained embedding model, it can determine that *sofa* and *couch* are similar without synonym processing.

The biggest advantage of vector search is that it can handle natural-language questions directly. If you ask how to store information in a database, vector search can find semantically related code like "persist data to storage" even without the exact phrase. The synonym handling that is problematic in keyword search is naturally resolved because the embedding model learned during training that *sofa* and *couch* are used in similar contexts.

Modern embedding models also consider context. The same word is converted to different vectors depending on the situation. *Bank* is converted to different vectors when it refers to a financial institution versus a riverbank. This context-aware characteristic enables more accurate semantic search.

But vector search isn't always the best choice. When exact word matching is required, such as for finding specific function or variable names, keyword search is more effective. Keyword search is also advantageous in searches for special characters or code syntax. For these reasons, hybrid search, which combines keyword and vector search, is widely used. In section 5.3, we'll examine vector search in detail by implementing embeddings, chunking, and vector databases.

5.2.3 Graph search

Graph search represents and traverses information as a graph structure consisting of nodes (entities) and edges (relationships). Figure 5.6 shows a simple example. From the sentence "Alice writes Python code," *Alice* and *Python code* become nodes, and *writes* becomes an edge connecting them. When the graph is built, questions like "What does Alice write?" and "Who writes Python code?" can be answered by traversing the graph.

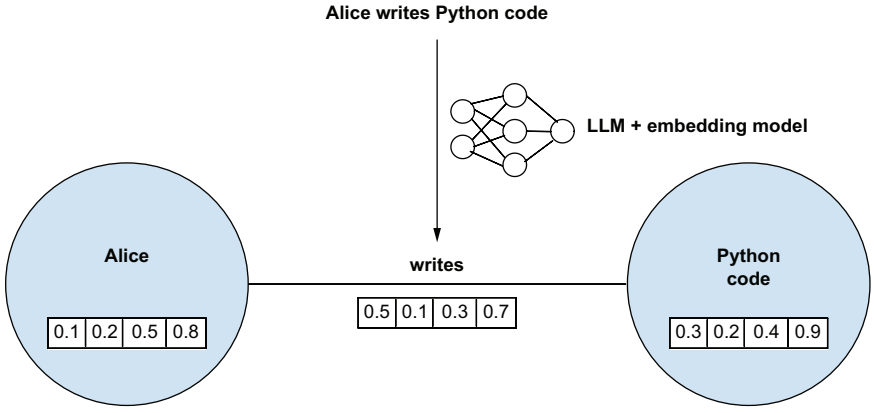

Figure 5.6 Graph search extracts entities and relationships between entities from sentences and represents them as a graph. Each entity and relationship is sometimes converted to embedding vectors so that vector search and graph search can be used together.

Graph search excels at complex relationship-based questions. Multistep relationship queries like "all code that calls this function," "other code written by developers who participated in this project," and "files recently modified by someone who has collaborated with A" can be handled effectively through graph traversal. Graph search is useful for code-dependency analysis, organizational-structure understanding, and knowledge-graph exploration.

The biggest challenge in graph search is extracting entities and relationships from unstructured text. In the past, complex rules or machine learning models were required, but recent advances in LLMs have greatly simplified this process. By instructing an LLM to "extract people, organizations, and places mentioned in this document

and identify relationships between them," you can construct a knowledge graph automatically. Thanks to this automation, graph search is receiving renewed attention.

Graph search has the disadvantages of high construction costs and a need for specialized knowledge in graph modeling. It requires careful decisions about which entities and relationships to extract and how to design the graph schema.

> **NOTE** This book doesn't cover graph search directly. Interested readers should refer to Manning's *Essential GraphRAG* (https://www.manning.com/books/essential-graphrag).

5.2.4 Structure-based search

Structure-based search uses the structure itself for exploration when data is already systematically organized. A representative example is a code repository. Code is systematically organized into folders and files, and each file has clear structures such as classes, functions, and variables.

Consider how a developer explores a new codebase. First, they use the `tree` command to grasp the overall folder structure. They check top-level directories like `src/`, `tests/`, and `docs/`; then they examine the substructure of modules of interest. Next, they use `grep` or IDE search functionality to find specific functions or classes. They can quickly find where the `get_user_by_id` function is defined and where it's called. Finally, they open the found files and read the code to understand the detailed implementation.

This exploration method is effective because the code structure itself contains information. Filenames (`user_service.py`), folder structures (`/services/auth/`), and function names (`authenticate_user`) are meaningful metadata. Without complex search algorithms, structure-based search can find the desired code by using just this structural information.

Modern coding tools such as Claude Code, Cursor, and Gemini CLI take this approach. Because LLMs are familiar with Bash commands such as `tree`, `grep`, and `cat`, agents can explore codebases the way human developers do. Like human developers, first they grasp the structure, then search for related files, read necessary parts, and gradually expand their understanding.

Structure-based search is also suitable for GAIA's zip file problems (section 5.1). When the archive is extracted, multiple files and folders emerge. The agent grasps the structure, gets hints from filenames, selects related files, and confirms their contents. We'll implement this approach directly in section 5.4.

These search methods are suited to different situations. Table 5.1 shows which method to choose for each situation.

In practice, using a single method alone is rare. Hybrid search is common. The most representative approach combines keyword search and vector search. Keyword search captures exact matches, and vector search captures semantic similarity. The scores from both search results are normalized, and a weighted average is calculated to determine the final ranking. This way, both exact names like the `get_user_by_id` function and semantic questions like "how to get user information" can be handled.

Table 5.1 Recommended search methods by situation

Situation	Recommended method	Reason
Exact word/code search	Keyword search	Exact matching needed for `get_user_by_id`, error code 404, etc.
Natural language questions	Vector search	Semantic search needed for queries like "how to store data"
Relationship-based questions	Graph search	Relationship traversal needed for "code that calls this function"
Folder/file structure exploration	Structure-based search	When structure itself is information, like zip files or codebases
Production environments	Hybrid search	Combines the advantages of multiple methods

In this book, we'll implement vector search and structure-based search directly. Vector search has received the most attention as a search method since the emergence of LLMs, and structure-based search is drawing significant interest centered on coding agents as LLM performance and context windows have grown larger. We've introduced only the concepts of keyword search and graph search, but they can be integrated into the system we build in this book.

5.3 Practicing vector search

The core of vector search is converting text to vectors and calculating distances. Let's convert text to vectors and observe how semantic similarity is expressed as distance. We'll implement embedding, chunking, and vector search in sequence. We'll conclude with a hands-on exercise that integrates these techniques to extract relevant information from web search results.

5.3.1 Embedding: Converting text to vectors

Embedding is the process of converting text to numerical arrays with hundreds to thousands of dimensions. The key insight is that semantically similar texts are placed close together in vector space. Embedding models learn to represent these semantic relationships as vector distances through training on massive amounts of text.

Let's implement a function that converts text to embedding vectors using OpenAI's text-embedding-3-small model. This model offers an excellent price-to-performance ratio, making it suitable for learning. The function in the following listing wraps single strings in a list for processing and extracts each embedding vector from the API response, returning them as a NumPy array.

Listing 5.1 Implementing the `get_embeddings` function

```
from openai import OpenAI
import numpy as np
```

```
client = OpenAI()

def get_embeddings(texts, model="text-embedding-3-small"):
    """Convert text to embedding vectors."""
    if isinstance(texts, str):
        texts = [texts]

    response = client.embeddings.create(input=texts, model=model)
    return np.array([item.embedding for item in response.data])
```

Wraps a single string in list for uniform processing

Calls the OpenAI embedding API

Extracts embedding vectors as a numpy array

Now let's verify that embeddings capture semantic similarity properly. We'll prepare three sentences, the first about a cat sleeping on a sofa, the second about a kitten playing with a toy, and the third about a dog running in a park. The `cosine_similarity` function calculates the similarity between the first sentence and the other two. Cosine similarity is the cosine of the angle between two vectors: values closer to 1 indicate high similarity, and values closer to 0 indicate low similarity.

Listing 5.2 Comparing embedding similarity

```
from sklearn.metrics.pairwise import cosine_similarity

sentences = [
    "The cat is sleeping on the couch",
    "A kitten is playing with a toy",
    "The dog is running in the park"
]
embeddings = get_embeddings(sentences)

cat_kitten = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]
cat_dog = cosine_similarity([embeddings[0]], [embeddings[2]])[0][0]

print(f"Cat vs Kitten: {cat_kitten:.3f}") # Cat vs Kitten: 0.385
print(f"Cat vs Dog: {cat_dog:.3f}") # Cat vs Dog: 0.200
```

Calculates the cosine similarity between embedding vectors

The results show that the similarity between *cat* and *kitten* sentences is higher than between *cat* and *dog* sentences. Although *cat* and *kitten* are different words, they're semantically related. Keyword search would have treated them as different documents. This is the solution to the *sofa* versus *couch* problem described earlier. Vector search can find semantically related content without exact word matching.

5.3.2 Chunking: Dividing long text into search units

Embedding an entire document as a single vector causes several problems:

- It may exceed the embedding model's token limit.
- Long documents contain multiple topics, and the vector becomes averaged out, reducing search accuracy. If a manual containing *installation instructions*, *troubleshooting*, and *product specifications* is converted to a single vector, searching for *installation instructions* might not surface that document among the top results.

- Returning entire documents as search results wastes context. The solution is to split documents into small units (*chunks*) and embed each chunk separately.

Chunk size involves tradeoffs. If a chunk is too small, context information is lost: you can't know what *this* refers to. If a chunk is too large, multiple topics mix together, reducing search accuracy. Generally, 200 to 1,000 tokens is appropriate—roughly 150 to 750 words in English. The optimal chunk size depends on document characteristics. Information-dense text like technical documentation benefits from smaller chunks; narrative writing benefits from larger chunks to maintain context.

An important concept in fixed-length chunking is overlap. Simply cutting text every 500 characters can break context at chunk boundaries. If the sentence "This function processes data. The processed result is" gets cut at a chunk boundary, the first chunk doesn't know what *the processed result* is, and the second chunk doesn't know what *this function* refers to. Overlap allows adjacent chunks to share some text, mitigating this problem. Typically, overlap is set to 10%–20% of chunk size.

The function in the next listing implements fixed-length chunking. The `while` loop cuts text from the `start` position by `chunk_size` to create chunks. The key part is calculating the next `start` position. If we haven't reached the end of the document, we set it to `end - overlap` to go back by the overlap amount. At the document's end, we use `end` as is to terminate the loop.

Listing 5.3 Fixed-length chunking function

```
def fixed_length_chunking(text, chunk_size=500, overlap=50):
    """Split text into fixed-length chunks."""
    chunks = []
    start = 0

    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        start = end - overlap if end < len(text) else end

    return chunks
```

Move back by the overlap amount to maintain context between chunks<code></code>.

A simple test demonstrates the effect of overlap. Splitting 283-character text with `chunk_size` 100 and `overlap` 20 produces more chunks (four) than without overlap (three). Thanks to overlap, each chunk shares 20 characters with adjacent chunks, maintaining context connections.

Listing 5.4 Testing fixed-length chunking

```
sample = "A" * 283
chunks = fixed_length_chunking(sample, chunk_size=100, overlap=20)
print(f"Original: {len(sample)} chars → {len(chunks)} chunks")
```

5.3.3 Implementing vector search

So far, we've learned how to convert text to embeddings and split long text into chunks. Now let's implement actual search. The goal is to find the chunks most similar to a query. The core logic is surprisingly simple.

The function in the following listing operates in four steps:

1. The function converts the query to a vector.
2. `cosine_similarity` calculates similarity between the query vector and all chunk vectors at the same time.
3. `argsort()[::-1]` sorts indices by descending similarity, selecting the `top_k`.
4. The function returns dictionaries pairing each chunk with its similarity score.

Listing 5.5 Implementing vector search to find chunks

```
def vector_search(query, chunks, chunk_embeddings, top_k=3):
    """Find the most similar chunks to the query."""
    query_embedding = get_embeddings(query)
    similarities = cosine_similarity(query_embedding, chunk_embeddings)[0]
    top_indices = similarities.argsort()[::-1][:top_k]

    results = []
    for idx in top_indices:
        results.append({
            'chunk': chunks[idx],
            'similarity': similarities[idx]
        })
    return results
```

Let's verify the operation with a simple test. We'll prepare four short documents, generate their embeddings, and search for *AI* and *coding*. We call `get_embeddings` to precompute all document embeddings and then pass the query along with them to `vector_search`.

Listing 5.6 Testing vector search

```
documents = [
    "Python is a programming language",
    "Machine learning uses Python extensively",
    "Cats are popular pets",
    "Deep learning is a subset of machine learning"
]

doc_embeddings = get_embeddings(documents)

results = vector_search("Artificial Intelligence", documents, doc_embeddings,
top_k=4)
for r in results:
    print(f"{r['similarity']:.3f}: {r['chunk']}")
```

The output is

```
0.353: Deep learning is a subset of machine learning
0.307: Machine learning uses Python extensively
0.162: Python is a programming language
0.048: Cats are popular pets
```

Searching for *artificial intelligence* surfaces the deep-learning and machine-learning documents at the top. A keyword search would have failed to find documents that lack the exact words *artificial* and *intelligence*. This is the essence of vector search: semantic search is possible with embeddings and similarity calculations, with no complex algorithms.

5.3.4 Why vector databases are necessary

The approach we've implemented has limitations. Because it calculates similarity across all chunks, the time complexity is O(n). With a million documents, every search requires a million calculations. Vector databases solve this problem. They support fast search through approximate nearest neighbor (ANN) algorithms, store embeddings efficiently, and provide metadata-based filtering. Popular vector databases include ChromaDB, Pinecone, Weaviate, Qdrant, and Milvus. For this exercise, our direct implementation suffices because we're handling small-scale data, and understanding the principles is our goal. Major vector databases and their features are

- *Chroma* (https://www.trychroma.com/)—A lightweight database that can run locally and is suitable for prototyping and small-scale projects. It requires no separate server setup.
- *Pinecone* (https://www.pinecone.io/)—A fully managed cloud service with excellent scalability. It requires payment. Choose Pinecone when you want to reduce the infrastructure management burden in large-scale production environments.
- *Weaviate* (https://weaviate.io/)—Open source with enterprise-grade features. It supports a GraphQL API, hybrid search, and integration with various embedding models.
- *Qdrant* (https://qdrant.tech/)—Written in Rust, with excellent performance and powerful filtering capabilities. It supports both on-premises and cloud deployment.
- *Milvus* (https://milvus.io)—Optimized for large-scale vector processing and supports GPU acceleration. It's suitable for large-scale systems handling billions of vectors.

5.3.5 Exercise: Finding relevant information from web search results

Let's bring together the embeddings, chunking, and vector search we've learned. The goal is to find only the parts related to a specific topic from actual web search results. This exercise lets you experience vector search's semantic-matching capability directly.

SCENARIO

Searching the web for *2025 Nobel Prize* returns mixed results across physics, chemistry, medicine, and other fields. What if we want to find only content related to quantum computing? Let's see whether we can find related content even when the exact phrase *quantum computing* doesn't appear in the search results.

STEP 1: PREPARE WEB SEARCH RESULTS

Search Tavily for *2025 Nobel Prize*, and collect the results. The key parameter is `include_raw_content=True`, which retrieves the full page content rather than only snippets. We iterate through results, keeping only those with `raw_content` available, and store the title, full content, and URL for each result.

Listing 5.7 Fetching web search results

```
from tavily import TavilyClient

tavily = TavilyClient()
response = tavily.search(
    "2025 Nobel Prize winners",
    max_results=10,
    include_raw_content=True
)

search_results = []
for result in response['results']:
    if result.get('raw_content'):
        search_results.append({
            'title': result['title'],
            'content': result['raw_content'],
            'url': result['url']
        })
```

STEP 2: CHECK TOKEN COUNT

How many tokens would it consume to feed web search results directly to an LLM? We use tiktoken's `encoding_for_model` to get an encoder for GPT-5; then we convert text to a token list with the `encode` method and measure its length.

Listing 5.8 Counting tokens in search results

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-5")
full_text = "\n\n".join([
    f"Title: {r['title']}\n{r['content']}"
    for r in search_results
])
total_tokens = len(enc.encode(full_text))
print(f"Total characters: {len(full_text)}")
print(f"Total tokens: {total_tokens}")
```

The output is

```
Total characters: 136448
Total tokens: 37312
```

If the token count reaches several thousand, putting it all in context every time is inefficient. Costs increase, and unnecessary information becomes noise that can degrade performance. Extracting only the relevant portions can reduce noise to a few hundred tokens.

STEP 3: PERFORM CHUNKING AND EMBEDDING

Split the full text into chunks, and generate embeddings for each chunk. Use `fixed_length_chunking` to create chunks of 500 characters with a 50-character overlap. Then pass the entire chunk list to `get_embeddings` to obtain all embeddings in a single API call.

Listing 5.9 Chunking and embedding search results

```
all_chunks = []
for result in search_results:
    text = f"Title: {result['title']}\n{result['content']}"
    chunks = fixed_length_chunking(text, chunk_size=500, overlap=50)
    for chunk in chunks:
        all_chunks.append({
            'text': chunk,
            'title': result['title'],
            'url': result['url']
        })

print(f"Total chunks: {len(all_chunks)}")

chunk_texts = [c['text'] for c in all_chunks]
chunk_embeddings = get_embeddings(chunk_texts)
```

STEP 4: EXECUTE VECTOR SEARCH

Search for *quantum computing* to find relevant chunks. Pass the query, chunk list, and chunk embeddings to the `vector_search` function we implemented earlier, requesting the top three results.

Listing 5.10 Executing vector search on chunks

```
query = "quantum computing"
results = vector_search(query, chunk_texts, chunk_embeddings, top_k=3)

print(f"Query: '{query}'\n")
print("=" * 60)
for i, r in enumerate(results, 1):
    print(f"\n[{i}] Similarity: {r['similarity']:.3f}")
    print(f"{r['chunk'][:300]}...")
```

The output shows the three chunks most semantically similar to *quantum computing*:

```
Query: 'quantum computing'

[1] Similarity: 0.596
Content about quantum mechanics as the foundation of digital technology...

[2] Similarity: 0.579
Content about quantum computing's economic potential reaching $2 trillion by
2035...

[3] Similarity: 0.566
Content about Physics laureates working on quantum experiments...
```

The results show that vector search successfully identified quantum-related content across different contexts: foundational physics, economic effects, and Nobel Prize winners, all with similarity scores above 0.5.

STEP 5: MEASURE TOKEN SAVINGS

Let's see how many tokens we save by using only the top three chunks. We concatenate the selected chunks, calculate the token count, and output the savings rate compared with the total number of tokens.

Listing 5.11 Calculating token savings

```
top_chunks = [r['chunk'] for r in results]
selected_text = "\n\n".join(top_chunks)
selected_tokens = len(enc.encode(selected_text))

print(f"Total tokens: {total_tokens}")
print(f"Selected tokens: {selected_tokens}")
print(f"Savings rate: {(1 - selected_tokens/total_tokens)*100:.1f}%")
The output shows a dramatic reduction in token usage:
Total tokens: 37312
Selected tokens: 304
Savings rate: 99.2%
```

By extracting only the three most relevant chunks, we reduced the context from more than 37,000 tokens to 304, achieving a 99.2% savings rate while preserving the information most relevant to *quantum computing*.

KEY INSIGHTS

This exercise confirms two key points:

- Vector search finds semantically related content without exact keyword matching.
- Extracting only the relevant portions instead of feeding entire web search results into the context saves a significant number of tokens.

In practice, LLM-optimized search services such as Tavily implement similar functionality internally. By using `search_depth="advanced"` and the `chunks_per_source`

parameter, you can achieve comparable results without building your own chunking and vector search pipeline:

```
response = tavily.search(
    "2025 Nobel Prize winners",
    search_depth="advanced",
    chunks_per_source=3
)
```

This code returns prechunked, relevance-filtered content directly from the API. But understanding the underlying mechanics, as in this exercise, helps you customize the approach for specific use cases and troubleshoot when built-in solutions fall short. The next section shows you how to apply this technique to callbacks to compress web search results automatically.

5.4 Structure-based search

Exploration is necessary when there are multiple files. Extracting a compressed file reveals multiple folders and files, and we can't know in advance which file contains the answer. In such situations, we can take an effective approach that doesn't require complex search algorithms: we use the structure of the filesystem itself.

We can approach GAIA benchmark problems with zip files in the same way. Extracting the archive reveals multiple files and folders. First, the agent examines the structure; then it picks relevant files based on hints from filenames and checks the contents. Let's implement this process directly. The tools we'll implement in this section are shown in table 5.2.

Table 5.2 Tools for structure-based search over files and folders

Tool	Purpose
`unzip_file`	Extract zip files
`list_files`	Examine folder structure
`read_file`	Read various file formats including text, CSV, and Microsoft Excel files
`read_media_file`	Analyze image, audio, and PDF files using LLM

We'll start by learning how to prepare the GAIA dataset with attached files.

5.4.1 Preparing the GAIA dataset

Some GAIA benchmark problems come with files attached alongside the question text. These files are in various formats, such as zip, PDF, XLSX, image, and audio. The agent must open and analyze the files to find the correct answer. In this section, we use problems with attachments to practice structure-based search.

In chapter 2, we loaded the GAIA dataset using `load_dataset()`. But this approach only retrieves the dataset and doesn't include attached files. To download the attached files, we have to use `snapshot_download()`.

The following code loads the GAIA dataset and downloads the attached files. `load_dataset()` provides metadata such as question text and answers as a `Dataset` object, and `snapshot_download()` saves the attached files locally. We set `local_dir` to the project root so we can reuse the cache across chapters.

Listing 5.12 Loading the GAIA dataset with attachments

```
from datasets import load_dataset
from huggingface_hub import snapshot_download
from pathlib import Path

# Load dataset (same as Chapter 2)
dataset = load_dataset("gaia-benchmark/GAIA", "2023_all", split="validation")

# Download attached files
NOTEBOOK_DIR = Path.cwd()
PROJECT_ROOT = NOTEBOOK_DIR.parent
CACHE_DIR = PROJECT_ROOT / "gaia_cache"

snapshot_download(
    repo_id="gaia-benchmark/GAIA",
    repo_type="dataset",
    allow_patterns="2023/validation/*",
    local_dir=CACHE_DIR
)
```

When an agent extracts or modifies files, restoring the original state becomes difficult. To solve this problem, we implement a function that copies files from the cache to a working folder. Calling this function before each problem ensures that we always start from the same initial state.

Listing 5.13 Resetting the `workspace` function

```
import shutil

WORK_DIR = NOTEBOOK_DIR / "gaia_workspace"

def reset_workspace():
    """Restore the workspace to its initial state."""
    shutil.rmtree(WORK_DIR, ignore_errors=True)
    shutil.copytree(CACHE_DIR / "2023/validation", WORK_DIR)
    print(f"Workspace reset: {WORK_DIR}")

reset_workspace()
```

Let's check which problems have attached files.

Listing 5.14 Finding problems with file attachments

```
problems_with_files = [p for p in dataset if p.get('file_name')]
problem_with_zip = [p for p in problems_with_files if p['file_name'].
endswith('.zip')]
```

```
print(f"Total problems: {len(dataset)}")
print(f"Problems with attachments: {len(problems_with_files)}")
print(f"Total problems with zip files: {len(problem_with_zip)}")

problem = problem_with_zip[0]
print(f"Question: {problem['Question'][:100]}...")
print(f"File name: {problem['file_name']}")

file_path = WORK_DIR / problem['file_name']
print(f"File exists: {file_path.exists()}")
```

The `file_name` field stores the attachment filename. Combining it with the working directory path gives us the file path. Next, we'll implement the tools that allow our agent to explore and read these files.

5.4.2 Implementing filesystem tools

Think about how developers explore a new codebase. First, they use the `tree` command to understand the overall structure. They pick relevant-looking files based on hints from folder and filenames. They open files to check the contents and explore additional files as necessary. Coding agents such as Claude Code, Cursor, and Gemini CLI operate in much the same way.

UNZIP_FILE: EXTRACTING ARCHIVES

When a GAIA problem includes a zip-file attachment, the first thing the agent must do is extract it. We can implement this step by using the standard library `zipfile`. If no extraction path is specified, the function creates a folder with the zip file's name and returns a list of extracted files so that the agent can plan its next steps.

Listing 5.15 Implementing the `unzip_file` tool

```
import zipfile
from pathlib import Path

def unzip_file(zip_path: str, extract_to: str = None) -> str:
    """Extract a zip file to the specified directory."""
    zip_path = Path(zip_path)

    if not zip_path.exists():
        return f"File not found: {zip_path}"

    # Default extraction path: create folder with zip filename
    if extract_to is None:
        extract_to = zip_path.parent / zip_path.stem
    else:
        extract_to = Path(extract_to)

    extract_to.mkdir(parents=True, exist_ok=True)

    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        file_list = zip_ref.namelist()
```

```
        zip_ref.extractall(extract_to)

    # Format results
    result = f"Extracted {len(file_list)} files to {extract_to}/\n\n"
    result += "Contents:\n"
    for f in file_list[:20]:
        result += f"  - {f}\n"
    if len(file_list) > 20:
        result += f"  ... and {len(file_list) - 20} more files\n"

    return result
```

LIST_FILES: EXAMINING FOLDER STRUCTURE

After extraction, we must see what files exist. The `list_files` tool returns a list of files and folders at the specified path. Directories are distinguished with a / suffix, and files include size information. Hidden files are excluded, and directories are sorted first to make the structure easier to understand.

Listing 5.16 Implementing the `list_files` tool

```
from pathlib import Path

def list_files(path: str = ".") -> str:
    """List files and directories in the given path."""
    path = Path(path)

    if not path.exists():
        return f"Path not found: {path}"

    if not path.is_dir():
        return f"Not a directory: {path}"

    items = []
    for item in sorted(path.iterdir()):
        if item.name.startswith('.'):
            continue

        if item.is_dir():
            items.append(f"{item.name}/")
        else:
            items.append(f"{item.name}")

    # Sort directories first
    dirs = [i for i in items if i.endswith('/')]
    files = [i for i in items if not i.endswith('/')]

    result = f"Directory: {path}\n"
    for item in dirs + files:
        result += f"  {item}\n"

    return result
```

READ_FILE: READING VARIOUS FILE FORMATS

After we understand the file structure, we have to read the contents. GAIA problems include attachments in various formats beyond text files, including CSV, Excel, and PDF. The `read_file` tool checks the extensions and reads the files appropriately, returning the content as text. The main function routes to the appropriate handler based on the file extension.

Listing 5.17 Implementing the `read_file` tool

```
from pathlib import Path

TEXT_EXTENSIONS = ['.txt', '.py', '.js', '.json', '.md', '.html',
                   '.css', '.xml', '.yaml', '.yml', '.log', '.sh']
SPREADSHEET_EXTENSIONS = ['.xlsx', '.xls', '.csv']

def read_file(file_path: str, start_line: int = 1, end_line: int = -1) -> str:
    """Read file content. Supports txt, py, json, md, csv, xlsx."""
    path = Path(file_path)

    if not path.exists():
        return f"File not found: {file_path}"

    ext = path.suffix.lower()

    if ext in TEXT_EXTENSIONS:
        return _read_text_file(file_path, start_line, end_line)
    elif ext == '.csv':
        return _read_csv(file_path)
    elif ext in SPREADSHEET_EXTENSIONS:
        return _read_excel(file_path)
    else:
        return _read_text_file(file_path, start_line, end_line)
```

Text files are returned with line numbers. The `start_line` and `end_line` parameters allow reading only specific ranges, which is useful for checking only the necessary parts of long files.

Listing 5.18 Reading text files with line numbers

```
def _read_text_file(file_path: str, start_line: int, end_line: int) -> str:
    with open(file_path, 'r', encoding='utf-8') as f:
        lines = f.readlines()

    # Adjust line numbers (1-indexed to 0-indexed)
    start_idx = max(0, start_line - 1)
    end_idx = len(lines) if end_line == -1 else min(end_line, len(lines))

    selected_lines = lines[start_idx:end_idx]

    result = []
```

```
    for i, line in enumerate(selected_lines, start=start_line):
        result.append(f"{i:4d} | {line.rstrip()}")
    return '\n'.join(result)
```

CSV and Excel files are read with pandas and converted to Markdown tables. The Markdown format makes it easy for LLMs to understand tabular structure.

Listing 5.19 Reading CSV and Excel files

```
import pandas as pd

def _read_csv(file_path: str) -> str:
    df = pd.read_csv(file_path)
    return df.to_markdown(index=False)

def _read_excel(file_path: str) -> str:
    df = pd.read_excel(file_path)
    return df.to_markdown(index=False)
```

READ_MEDIA_FILE: ANALYZING IMAGES AND AUDIO

Some GAIA problems include image, audio, or PDF file attachments. These files require LLM analysis rather than simple text extraction. We implement `read_media_file` as a tool separate from `read_file` because media analysis always requires a specific question: "What should I look for?"

The main function routes to the appropriate handler based on the file extension. We'll examine the PDF handler in detail because it combines text extraction with visual analysis.

Listing 5.20 Implementing the `read_media_file` tool

```
import fitz  # pymupdf
import base64
from pathlib import Path
from openai import OpenAI

IMAGE_EXTENSIONS = ['.png', '.jpg', '.jpeg', '.gif', '.webp', '.bmp']
AUDIO_EXTENSIONS = ['.mp3', '.wav', '.m4a', '.flac', '.ogg', '.webm']
PDF_EXTENSIONS = ['.pdf']

def read_media_file(file_path: str, query: str) -> str:
    """Analyze an image, audio, or PDF file using LLM."""
    ext = Path(file_path).suffix.lower()

    if ext in IMAGE_EXTENSIONS:
        return _analyze_image(file_path, query)
    elif ext in AUDIO_EXTENSIONS:
        return _analyze_audio(file_path, query)
    elif ext in PDF_EXTENSIONS:
        return _analyze_pdf(file_path, query)
    else:
```

```
        return f"Unsupported media format: {ext}"

def _analyze_image(file_path: str, query: str) -> str:
    with open(file_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode("utf-8")

    ext = Path(file_path).suffix.lower().lstrip('.')
    media_type = "image/jpeg" if ext == "jpg" else f"image/{ext}"

    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": query},
                {"type": "image_url", "image_url": {
                    "url": f"data:{media_type}; base64,{image_data}"
                }}
            ]
        }]
    )
    return response.choices[0].message.content

def _analyze_audio(file_path: str, query: str) -> str:
    with open(file_path, "rb") as f:
        audio_data = base64.b64encode(f.read()).decode("utf-8")

    ext = Path(file_path).suffix.lower().lstrip('.')

    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o-audio-preview",
        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": query},
                {"type": "input_audio", "input_audio": {
                    "data": audio_data,
                    "format": ext
                }}
            ]
        }]
    )
    return response.choices[0].message.content

def _analyze_pdf(file_path: str, query: str) -> str:
    doc = fitz.open(file_path)

    # Extract text for context
    text_content = ""
    for page in doc:
        text_content += page.get_text()

    # Convert pages to images
    images = []
```

```
    for page in doc[:5]:  # First 5 pages
        pix = page.get_pixmap(matrix=fitz.Matrix(2, 2))
        img_bytes = pix.tobytes("png")
        images.append(base64.b64encode(img_bytes).decode('utf-8'))

    # Build content with text and images
    content = [{
        "type": "text",
        "text": f"{query}\n\nExtracted text:\n{text_content[:3000]}"
    }]

    for img_b64 in images:
        content.append({
            "type": "image_url",
            "image_url": {"url": f"{img_b64}"}
        })

    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": content}]
    )
    return response.choices[0].message.content
```

The PDF handler combines two approaches: text extraction using `pymupdf` and page-to-image conversion for visual analysis. Text extraction alone can't handle tables, diagrams, or scanned documents. By sending both the extracted text and page images to the LLM, the agent can answer questions about any PDF content.

The two caps in this implementation, `doc[:5]` and `text_content[:3000]`, are simply heuristics for keeping token cost bounded; they're not principled thresholds. Vision input is the dominant cost (each high-detail page image uses roughly hundreds of input tokens), so we limit visual analysis to the first few pages and the extracted text to a few thousand characters. For longer documents, we'd replace these flat caps with chunking and retrieval, a per-document mini RAG using the building blocks from section 5.3.

Image analysis uses GPT-4o with Base64-encoded image data, whereas audio analysis uses `gpt-4o-audio-preview`, which supports native audio input. The implementations follow the same pattern as PDF analysis. See the complete code in the repository.

5.4.3 Connecting tools to the agent

Now let's connect the filesystem tools we implemented to the agent in chapter 4. We wrap each function with the `@tool` decorator and register them along with the web search tools from chapter 3.

The following code shows how to register the filesystem tools with the agent. Because file exploration requires multiple sequential steps, we set `max_steps` to 20.

Listing 5.21 Registering filesystem tools with the agent

```
from scratch_agents.tools import tool
from scratch_agents.agent import Agent
```

```
# File system tools
@tool
def unzip_file(zip_path: str, extract_to: str = None) -> str:
    """Extract a zip file to the specified directory."""
    # Implementation from Listing 5.15
(...)

# Configure with existing tools
tools = [
    search_web,
    unzip_file,
    list_files,
    read_file,
    read_media_file,
]

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=tools,
    instructions="You are a helpful assistant that can search the web and "
                 "explore files to answer questions.",
    max_steps=20
)
```

Now the agent can explore the filesystem like a human developer. When given a zip file, it extracts the archive, examines the folder structure, reads relevant files, and finds the answer.

5.4.4 Solving GAIA zip file problems

Let's solve a problem from the GAIA benchmark that includes a zip file attachment. We'll demonstrate the complete exploration process, from extraction to finding the answer. First, we reset the workspace and select a problem with a zip file attachment. We include the file path in the prompt so that the agent knows where the attachment is located.

Listing 5.22 Preparing the GAIA zip file problem

```
# Reset workspace to clean state
reset_workspace()

# Select a problem with zip file attachment
zip_problems = [p for p in problems_with_files if
p['file_name'].endswith('.zip')]
problem = zip_problems[1]

file_path = WORK_DIR / problem['file_name']

# Construct prompt including file location
prompt = f"""{problem['Question']}
```

```
The attached file is located at: {file_path}
"""
print(prompt)
```

Now let's run the agent and observe the file-structure exploration process.

Listing 5.23 Running the agent on the zip file problem

```
response = await agent.run(prompt)
print(response)
```

The agent's exploration process follows a systematic pattern. First, it extracts the zip file using `unzip_file`; then it calls `list_files` to examine the folder structure. Two files emerge: `Job Listing.pdf` and `Applicants.xlsx`. The filenames alone reveal their relationship clearly: one describes the job requirements, and the other lists candidates.

First, the agent analyzes the PDF using `read_media_file` to extract the qualification requirements. Then it reads the Excel file with `read_file` to review all applicants' data. Finally, it cross-references the requirements against each candidate to find the answer.

The key observation is that the agent navigates file structures without complex search algorithms. Filenames and extensions serve as natural hints. `Job Listing.pdf` suggests requirements to match against, and `Applicants.xlsx` indicates candidate data to filter. The agent uses these naming conventions and format cues to efficiently locate the information required to solve the problem.

5.5 Extending agents with callbacks

The agent we implemented in chapter 4 provides basic functionality for executing tools and receiving results. But in real-world deployments, sometimes we have to obtain user approval before executing a tool or compress search results to manage context efficiently. Do we have to modify the agent's core logic to address these requirements?

Fortunately, we don't. Using the callback pattern, we can extend the agent's behavior without touching its core code. In this section, we'll add a callback mechanism to the agent and learn how to use it through two practical examples.

5.5.1 The need for agent extension

Let's consider extension points for our agent from a context-engineering perspective. While the agent is calling the LLM and executing tools, we can intervene at several points.

Intervening before an LLM call allows us to inject additional information into the context, compress old messages, or check for cached responses. After the LLM responds, we can validate or postprocess the results. Before tool execution, we can request user approval for dangerous operations or return cached results. After tool

execution, we can compress results or mask sensitive information. Finally, after the entire execution completes, we can save the conversation or generate summaries.

All this is possible through *callbacks*—user-defined functions that are invoked at specific execution points. They enable us to add various capabilities without modifying the agent's core logic. We'll use the following types of callbacks (figure 5.7):

- `before_llm_callback` and `after_llm_callback` execute before and after LLM calls. They're used to manipulate context or intercept responses.
- `before_tool_callback` and `after_tool_callback` run before and after tool execution. They're used to approve tool execution or compress results. We focus on these callbacks in this section.
- `after_run_callback` executes after the entire agent run completes. It can save conversation content to memory, and we'll use it for that purpose in chapter 6.

In this section, we'll implement and use tool callbacks that are directly related to this chapter's topic. The remaining callbacks follow the same pattern. The complete code is available in the GitHub repository.

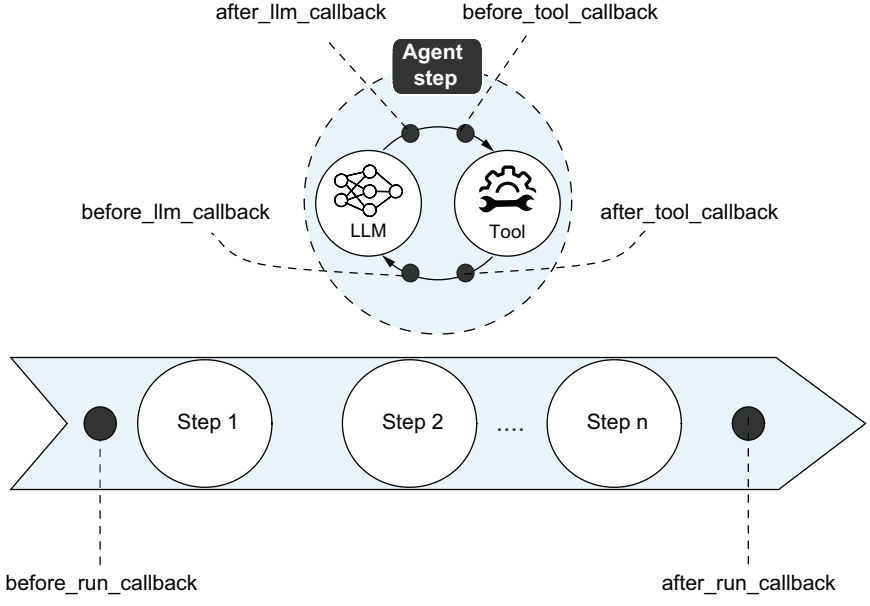

Figure 5.7 Points where callbacks are inserted into the agent execution flow. LLM callbacks are called before and after `think()`, and tool callbacks are called before and after each tool execution within `act()`.

5.5.2 Implementing tool callbacks

Now let's add tool callback support to the `Agent` class. First, we define how callbacks behave:

- `before_tool_callback(context, tool_call)` is called before a tool executes. If this callback returns a value, tool execution is skipped, and that value is used as the result. If the callback returns `None`, the tool executes.
- `after_tool_callback(context, tool_result)` is called after a tool executes. If this callback returns a value, that value replaces the original result. If the callback returns `None`, the original result is preserved.

When multiple callbacks are registered, execution stops at the first callback that returns a non-`None` value. This process is called the *Early Exit pattern*.

We modify the `Agent` constructor to add callback parameters. It accepts two lists: `before_tool_callbacks` and `after_tool_callbacks`. If callbacks aren't provided, they're initialized as empty lists so that subsequent code can iterate over them without `None` checks.

Listing 5.24 Adding callback parameters to `Agent`

```
class Agent:
    def __init__(
        self,
        model: LlmClient,
        tools: List[BaseTool] = None,
        instructions: str = "",
        max_steps: int = 10,
        output_type: Optional[Type[BaseModel]] = None,
        before_tool_callbacks: List[Callable] = None,
        after_tool_callbacks: List[Callable] = None,
    ):
        self.model = model
        self.instructions = instructions
        self.max_steps = max_steps
        self.output_type = output_type
        self.output_tool_name = None
        self.tools = self._setup_tools(tools or [])

        # Initialize callback lists
        self.before_tool_callbacks = before_tool_callbacks or []
        self.after_tool_callbacks = after_tool_callbacks or []
```

Next, we refactor the `act()` method. The original code executed tools directly for each `tool_call`, but now it's divided into three stages:

1. In the first stage, we iterate through `before_tool_callbacks` and execute each callback. If a callback returns a non-`None` value, we store that value in `tool_response` and break out of the loop. Also, we use `inspect.isawaitable()` to support asynchronous callbacks.
2. In the second stage, we execute the tool only if `tool_response` is still `None`. If a callback has already provided a result, this stage is skipped.
3. In the third stage, we iterate through `after_tool_callbacks`. If a callback returns a new `ToolResult`, that result replaces the original result.

Listing 5.25 Refactoring the `act()` method with callbacks

```
import inspect

async def act(
    self,
    context: ExecutionContext,
    tool_calls: List[ToolCall]
) -> List[ToolResult]:
    tools_dict = {tool.name: tool for tool in self.tools}
    results = []

    for tool_call in tool_calls:
        if tool_call.name not in tools_dict:
results.append(ToolResult(
    tool_call_id=tool_call.tool_call_id,
    name=tool_call.name,
    status="error",
    content=[f"Tool '{tool_call.name}' not found"],
))
continue

        tool = tools_dict[tool_call.name]
        tool_response = None
        status = "success"

        # Stage 1: Execute before_tool_callbacks
        for callback in self.before_tool_callbacks:
            result = callback(context, tool_call)
            if inspect.isawaitable(result):
                result = await result
            if result is not None:
                tool_response = result
                break

        # Stage 2: Execute actual tool only if callback didn't provide a result
        if tool_response is None:
            try:
                tool_response = await tool(context, **tool_call.arguments)
            except Exception as e:
                tool_response = str(e)
                status = "error"

        tool_result = ToolResult(
            tool_call_id=tool_call.tool_call_id,
            name=tool_call.name,
            status=status,
            content=[tool_response],
        )

        # Stage 3: Execute after_tool_callbacks
        for callback in self.after_tool_callbacks:
            result = callback(context, tool_result)
            if inspect.isawaitable(result):
                result = await result
            if result is not None:
```

```
                tool_result = result
                break

        results.append(tool_result)

    return results
```

5.5.3 Human-in-the-loop tool-execution approval

Although it's efficient for agents to perform tasks automatically, this can sometimes be risky. Sensitive operations such as file deletion, email sending, and database modifications should require user confirmation before execution. The human-in-the-loop (HITL) pattern is designed for these situations, as shown in figure 5.8.

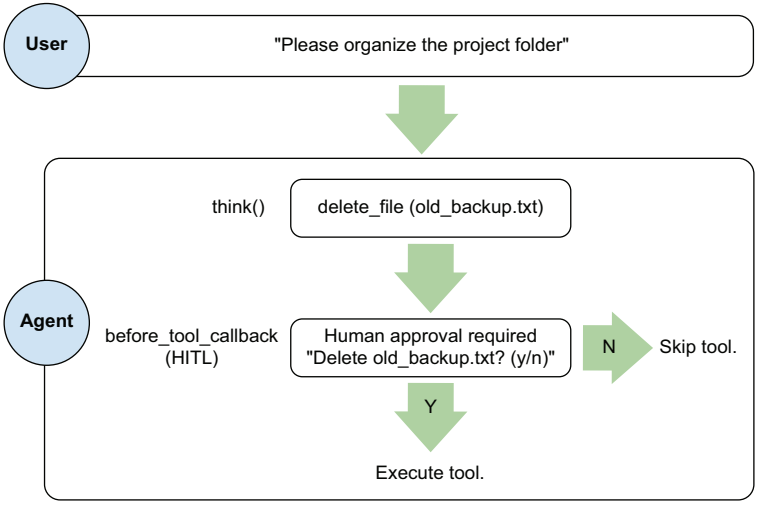

Figure 5.8 Tool-execution approval flow in the HITL pattern. When a user requests "Clean up the project folder," the agent pauses when it tries to call the `delete_file` tool and requests approval from the user.

Let's create a file-deletion tool as an example of a dangerous tool that requires approval. This dummy implementation returns a message without deleting files.

Listing 5.26 Implementing the `delete_file` tool

```
@tool
def delete_file(file_path: str) -> str:
    """Deletes a file. This action cannot be undone."""
    # Only returns message instead of actual deletion (for demo)
    return f"File {file_path} has been deleted."
```

Now let's implement a callback that requests approval only when dangerous tools like `delete_file` are called. First, the callback checks whether the tool's name is in the list

of tools that require approval. If the tool isn't listed, the callback returns `None` to allow immediate execution; if it is, the callback displays the tool information and waits for user input. Entering `y` returns `None` to allow the tool to execute; any other input returns a rejection message to skip execution.

Listing 5.27 Implementing approval callback

```
# List of dangerous tools requiring approval
DANGEROUS_TOOLS = ["delete_file", "send_email", "execute_sql"]

def approval_callback(context: ExecutionContext, tool_call: ToolCall):
    """Requests user approval before executing dangerous tools."""
    # Execute immediately if not a dangerous tool
    if tool_call.name not in DANGEROUS_TOOLS:
        return None

    print(f"\n⚠️ Dangerous tool execution requested")
    print(f"Tool: {tool_call.name}")
    print(f"Arguments: {tool_call.arguments}")

    response = input("Do you want to execute? (y/n): ").lower().strip()

    if response == 'y':
        print("✅ Approved. Executing...\n")
        return None  # Proceed with actual tool execution
    else:
        print("❌ Denied. Skipping execution.\n")
        return f"User denied execution of {tool_call.name}"
```

The callback's return value is key. Returning `None` causes the `act()` method to execute the tool; returning a string uses that string as the tool result, and the tool doesn't execute. The LLM can see this rejection message in the next step and try a different approach.

Let's apply this callback to an agent. Safe tools such as `list_files` and `read_file` execute without approval, whereas approval is requested when `delete_file` is called.

Listing 5.28 Creating an agent with approval callback

```
agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[list_files, read_file, delete_file],
    instructions="You are a helpful assistant that can explore and manage files.",
    max_steps=20,
    before_tool_callbacks=[approval_callback],
)
```

When you execute it, you'll see the following flow: `list_files` runs without approval because it's not in the dangerous-tools list. When `delete_file` is called, the approval prompt appears.

5.5.4 Compressing search results

In section 5.3.4, we confirmed that web search results can consume more than 37,000 tokens and that extracting only relevant portions through vector search can save more than 99%, reducing the count to 304 tokens. But performing this process manually every time is cumbersome. Let's automate it at the agent level using `after_tool_callback`.

The idea is simple. After the web search tool executes, if the result is too long, we use vector search to extract only the portions that are relevant to the query. This callback applies only to the search tool and passes through results from other tools unchanged.

The callback flow is as follows:

1. We check the tool name and return `None` if it's not `search_web`. If the tool is `search_web`, we check the result length and return it unchanged if it's less than 2,000 characters. For long results, we find the query for that search and perform the chunking and vector search we implemented in section 5.3.
2. We return a new `ToolResult` containing only the top three chunks.

Listing 5.29 Implementing search compressor callback

```
def search_compressor(context: ExecutionContext, tool_result: ToolResult):
    """Callback that compresses web search results."""
    # Pass through unchanged if not a search tool
    if tool_result.name != "search_web":
        return None

    original_content = tool_result.content[0]

    # No compression needed if result is short enough
    if len(original_content) < 2000:
        return None

    # Extract search query matching the tool_call_id
    query = _extract_search_query(context, tool_result.tool_call_id)
    if not query:
        return None

    # Use functions implemented in section 5.3
    chunks = fixed_length_chunking(original_content, chunk_size=500, overlap=50)
    embeddings = get_embeddings(chunks)
    results = vector_search(query, chunks, embeddings, top_k=3)

    # Create compressed result
    compressed = "\n\n".join([r['chunk'] for r in results])

    return ToolResult(
        tool_call_id=tool_result.tool_call_id,
        name=tool_result.name,
        status="success",
        content=[compressed]
    )
```

The `_extract_search_query` helper function finds the query for a specific search call from the `ExecutionContext`. Because the LLM can request multiple searches in parallel, it's safer to match by `tool_call_id` than find the most recent search. We iterate through events looking for items that are the `ToolCall` type, have the name `search_web`, and match the ID; then we return the query argument.

Listing 5.30 Extracting the search query from the context

```
def _extract_search_query(context: ExecutionContext, tool_call_id: str) -> str:
    """Extracts the search query for a specific tool_call_id from context."""
    for event in context.events:
        for item in event.content:
            if (isinstance(item, ToolCall)
                and item.name == "search_web"
                and item.tool_call_id == tool_call_id):
                return item.arguments.get("query", "")
    return ""
```

Now let's create an agent that applies both callbacks together. We register the approval callback in `before_tool_callbacks` and the compression callback in `after_tool_callbacks`.

Listing 5.31 Creating an agent with multiple callbacks

```
agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[search_web, list_files, read_file, delete_file],
    instructions="You are a helpful assistant that can search the web and "
                 "explore files to answer questions.",
    max_steps=20,
    before_tool_callbacks=[approval_callback],
    after_tool_callbacks=[search_compressor],
)
```

When web search executes, the results are automatically compressed. The work we performed manually in section 5.3.4 is applied automatically during agent execution. We achieve the same token savings and noise-reduction benefits without modifying the agent's core logic.

KEY INSIGHTS

In this section, we learned that we can extend agent behavior without modifying its core logic through the callback pattern.

`before_tool_callback` intervenes before tool execution. If it returns a value, execution is skipped, and that value is used as the result. You can use it for HITL approval, cache checking, input validation, and more.

`after_tool_callback` intervenes after tool execution. If it returns a value, that value replaces the original result. You can use it for search result compression, sensitive information masking, result format conversion, and more.

By combining multiple callbacks, you can handle complex requirements cleanly. Each callback has a single responsibility and can be added or removed as necessary.

The HITL implementation in this section operates synchronously. The entire program blocks while the `input()` function waits for user input. This is fine for testing in a local environment, but you can't use this approach in web services or API servers. In chapter 6, we'll implement asynchronous HITL, in which the agent doesn't pause in the middle but requests approval through a response to the user and resumes execution when the user's response arrives.

We'll also use `after_run_callback` in chapter 6 to save conversation content to memory after a conversation ends. You'll continue to see how the callback pattern is used as a core mechanism for extending agent functionality.

Summary

- Internal data requires approaches other than web search. Single files can be read directly into context, but multiple files need exploration, and large-scale data requires search mechanisms to extract only relevant portions.
- Four search methods serve different purposes: keyword search for exact matching, vector search for semantic similarity, graph search for relationship traversal, and structure-based search for navigating organized filesystems. Production systems often use multiple methods.
- Vector search converts text to numerical vectors in which semantic similarity becomes geometric distance. The pipeline involves chunking long documents, generating embeddings, and calculating cosine similarity to find the most relevant content.
- Structure-based search uses filesystem organization as information in its own right. Folder names, filenames, and directory hierarchies provide meaningful metadata that agents can navigate using simple tools such as `list_files` and `read_file`.
- The callback pattern extends agent behavior without modifying core logic. `before_tool_callback` enables HITL approval for dangerous operations, and `after_tool_callback` enables automatic compression of search results.
- Context engineering through search dramatically reduces token use. In our exercise, extracting only relevant chunks reduced more than 37,000 tokens to 304 tokens, achieving a 99% savings rate while preserving the information required to answer the query.

6 Adding memory to your agent

This chapter covers

- Understanding the role of memory in LLM agents
- Managing context growth with sliding window, compaction, and summarization
- Implementing sessions for multiturn conversations
- Building asynchronous human-in-the-loop workflows
- Creating long-term memory for cross-session knowledge retention

Memory is what separates a stateless tool from an intelligent assistant. Without memory, an agent can't recall previous events within the same task, continue conversations from earlier sessions, or learn from experiences. Each interaction starts from scratch, forcing users to repeat context and preventing the agent from improving over time.

This chapter (figure 6.1) addresses memory in three use patterns:

- We implement context optimization strategies to prevent context explosion, in which the context takes up a large share of the model's context window or grows past it during complex problem solving.
- We build `Session` and `SessionManager` to maintain conversation continuity across multiple interactions, extending this architecture to support asynchronous human-in-the-loop (HITL) workflows.
- We create a long-term memory system that extracts, stores, and retrieves knowledge across session boundaries using vector search.

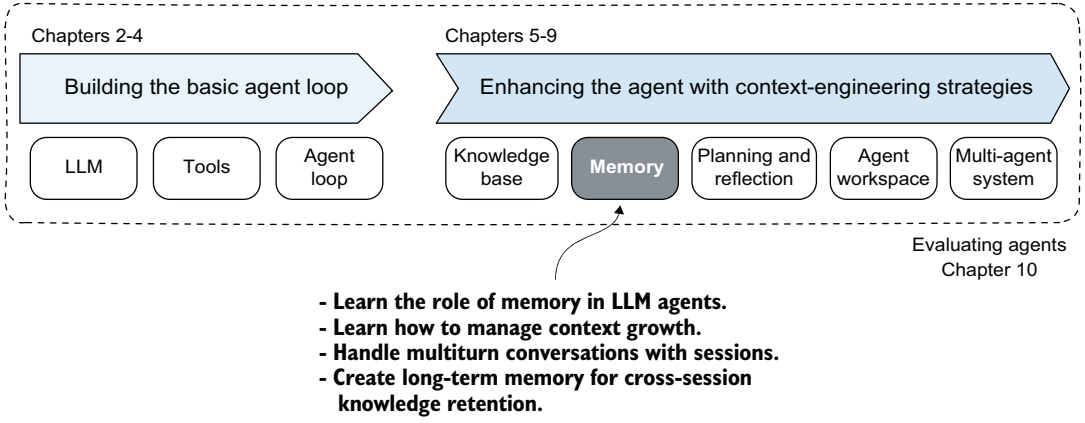

Figure 6.1 Book structure overview: chapter 6

6.1 The anatomy of agent memory

Memory transforms an agent from a simple tool to an intelligent assistant that learns and adapts. Without memory, even the most sophisticated agent starts from scratch every time, unable to reference previous conversations or learn from experiences.

In this section, we examine the limitations of our current agent, distinguish between short-term and long-term memory, and further divide short-term memory into two patterns based on agent use. Then we clarify how the context-engineering framework from chapter 1 connects to memory.

6.1.1 Limitations of the current memory architecture

The agent we've developed so far relies on a raw `ExecutionContext`. It operates simply, appending everything during execution and discarding everything after execution. Although this agent works for simple tasks, it lacks a mechanism to manage information effectively.

Consider two scenarios:

- *A problem of space*—When solving a complex General AI Assistants (GAIA) problem, tool-call results accumulate rapidly. The context can exceed the large language model's (LLM's) window limit, causing the agent to crash before it finishes.
- *A problem of time*—As a personal assistant across multiple conversations, the agent forgets everything between sessions. A user who said, "I'm James; I work as a marketer" yesterday must repeat this information today.

These limitations stem from the absence of memory management. Let's categorize these problems into three distinct challenges and outline our solutions.

6.1.2 Three challenges of the current memory architecture

Before diving into the challenges, let's establish three concepts that form the foundation for everything we'll build in this chapter:

- A *session* is a single, continuous conversation between a user and an agent. Think of it as opening a new chat window; everything discussed within that window belongs to the same session. Closing the window and starting a new conversation later creates a new session.
- *Short-term memory* stores the work done within a single session. That work includes conversation history, tool-call results, and intermediate reasoning, all stored in `ExecutionContext.events`.
- *Long-term memory* is information that persists after sessions end. When a user says "I'm a marketer" today, long-term memory ensures that the agent remembers this information next week, even in a new session.

The two memory stores differ in the way items enter them. Short-term memory automatically captures everything the agent does. Long-term memory is selective; at the end of each run, an extraction step decides which facts are worth saving (section 6.4).

Challenge 1: Context explosion

Many agents are used for one-off execution. When we ran the GAIA benchmark, we executed `run()` once and terminated. For such cases, our current implementation is sufficient. For complex problems, however, multiple tool calls can cause context to grow rapidly. The core challenge is how to compress context during current execution.

Challenge 2: Lack of conversation continuity

What happens when an agent receives multiple requests from users, as with a conversational chatbot, or has to pause because certain tool executions require user approval? How do we maintain state across multiple `run()` calls?

Without proper session management, each `run()` call operates independently, and the conversation breaks, as illustrated in figure 6.2. On the left, without a session,

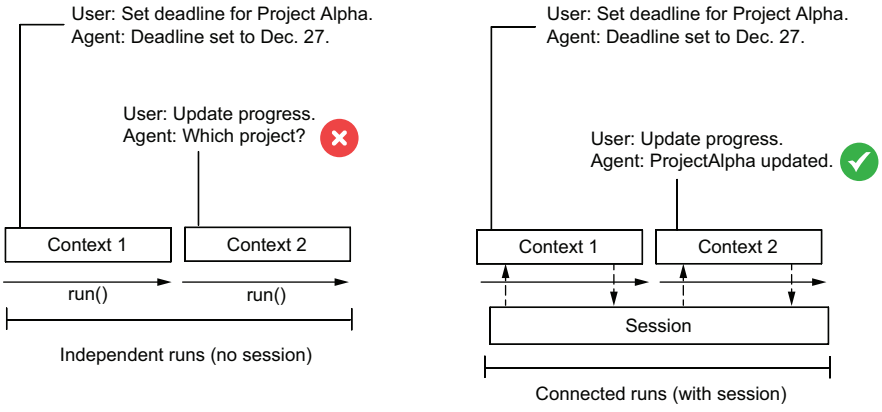

Figure 6.2 Without a session (left), each `run()` creates a new `ExecutionContext` that's discarded after completion. With a session (right), multiple `run()` calls share the session, maintaining conversation continuity.

a follow-up request after asking about Project Alpha leaves the agent unsure which project the user meant. On the right, with a session, context is preserved, and the agent understands the reference.

In web services, servers operate statelessly. Each HTTP request is processed independently, and server memory is released when the request ends. When a user sends another message 10 minutes later, that message is a new request.

Maintaining conversation context requires explicit work. You must assign a unique `session_id` to each conversation, save session state to external storage when `run()` ends, and load the previous state when the next `run()` starts.

CHALLENGE 3: LACK OF PERSISTENT KNOWLEDGE

Consider a personalized AI assistant. Information such as "I'm a marketer," shared today, should be remembered next week, even in a new conversation. What should survive after a session ends, and how?

This differs fundamentally from challenge 2. Conversation continuity maintains context within the same `session_id`; you can close and reopen the chat window, and context persists as long as the session is the same. Persistent knowledge shares information across different `session_ids`: the agent remembers the user when starting a new conversation.

Figure 6.3 shows how the process works. When session 1 ends, the system determines whether any information is worth preserving, and if so, that knowledge is stored in long-term memory. When a new session begins, the agent searches for and retrieves relevant information as necessary. Although the figure simplifies the process by showing storage at the session level, in practice, the system operates at each `run()` execution, storing and retrieving incrementally for finer control.

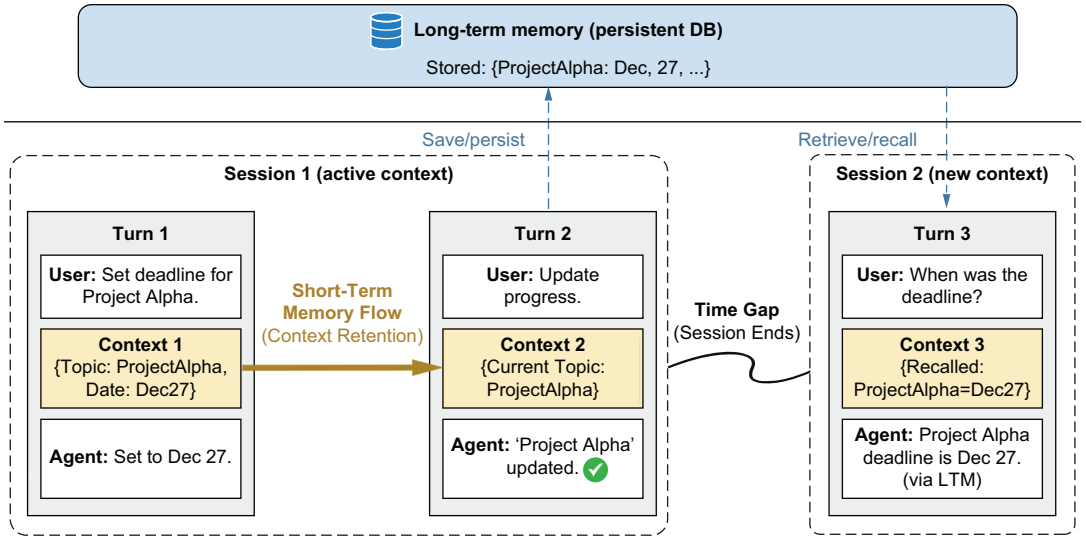

Figure 6.3 Long-term memory enables information sharing across sessions. Each `run()` creates an `ExecutionContext`, multiple `run()` calls form a session, and important information from sessions is extracted and stored for future use.

6.1.3 Context engineering and memory

Chapter 1 defined context engineering as the art of providing the right information at the right time and introduced five strategies (table 6.1). This chapter is a hub in which three of these strategies converge.

Table 6.1 Context-engineering strategies and related chapters

Strategy	Description	Related chapters
Generation	Constructing context with LLM-generated outputs	7
Retrieval	Bringing information from external sources into context	3, 5, and 6
Write	Saving from context to external storage	6 and 8
Reduce	Compressing and deleting context	6
Isolate	Separating tasks/tools into distinct environments	8 and 9

Each challenge defines a use pattern that this chapter addresses:

- *Pattern 1, in-run context management (challenge 1)*—Keeps context under control as it grows during a single `run()` call
- *Pattern 2, continuous execution (challenge 2)*—Maintains context across multiple `run()` calls within the same session

- *Pattern 3, long-term memory (challenge 3)*—Preserves and retrieves information after a session ends

These patterns map to the context-engineering strategies from chapter 1 as shown in table 6.2. Patterns 1 and 2 relate to reduce and write. To manage an expanding context, we shrink it using methods such as sliding window, compaction, and summarization, or we separate and store it externally. Pattern 3 saves important information to external storage (write) and retrieves it for use (retrieval).

Table 6.2 Mapping each pattern to strategies

Pattern	Primary strategy	Description
Patterns 1, 2	Reduce + write	Managing extensive context via sliding window, compaction, and summarization
Pattern 3	Write + retrieval	Extracting and storing memories (write) > Searching via tool in next session (retrieval)

We group patterns 1 and 2 because they both address the problem of managing the context being sent to the LLM now:

- *Pattern 1*—When context grows within a single `run()`
- *Pattern 2*—When conversation history accumulates across multiple `run()` calls

The solutions also use the same reduce strategy. Sliding window keeps only the most recent *n* messages. Compaction deletes tool calls and their results or saves them to a file. Summarization condenses old conversations.

The difference is where the data resides. In pattern 1, we work with `events` within `ExecutionContext`. In pattern 2, `SessionManager` loads `events` stored in a `Session` and processes them the same way.

6.2 Managing context during execution

The core challenge of pattern 1 is that context grows rapidly as tool calls and results accumulate while solving complex problems. Let's implement concrete strategies to address this problem.

6.2.1 Separating storage from presentation

The first principle of context management is separating storage from presentation. All events recorded in `ExecutionContext.events` remain immutable; only the `LlmRequest.contents` sent to the LLM are processed.

WHY THIS SEPARATION MATTERS

Think of `ExecutionContext` as your ground truth: a complete, structured record of everything that happened. `LlmRequest` is simply a computed projection that you can refine and optimize for each specific model call.

This separation provides several benefits. Without the original history, tracing the cause of agent failures becomes difficult. To answer the question "Why did it draw the wrong conclusion from these search results?", you need the original search results from that moment. Also, some tools can reference results from previous operations. If you delete the originals, these tools may not function properly.

Consider posting photos to social media: you keep the original photo and apply filters only when posting. If the original photo is preserved, you can process it differently later. Our context management works the same way. `ExecutionContext` is the repository that maintains the complete execution history, and `LlmRequest` processes that history for delivery to the LLM. You can change how the LLM sees the context without migrating or modifying your stored data.

Implementation via callbacks

Chapter 4 introduced `before_llm_callback`. This callback executes just before the LLM call, providing an opportunity to process the `LlmRequest`.

The callback receives both `ExecutionContext` and `LlmRequest`. It reads the current state from `ExecutionContext` and modifies the contents of `LlmRequest`. If the callback returns `None`, the LLM call proceeds with the modified request; if the callback returns an `LlmResponse`, the LLM call is skipped, and that response is used instead. The key point is that `ExecutionContext.events` is read-only; modifications occur only in `LlmRequest.contents`:

```
before_llm_callback(context: ExecutionContext, request: LlmRequest) ->
Optional[LlmResponse]
```

To apply context-optimization strategies flexibly, you adjust configuration values such as token thresholds. The following function uses the closure pattern. `create_optimizer_callback` takes an optimization function and a threshold as arguments to generate a callback function. The generated callback invokes the optimization function only when the token count exceeds the threshold.

Listing 6.1 Creating an optimizer callback with a threshold

```
def create_optimizer_callback(apply_optimization, threshold: int = 50000):
    """Factory function that creates a callback applying optimization
strategy"""

    async def callback(
        context: ExecutionContext,
        request: LlmRequest
    ) -> Optional[LlmResponse]:
        token_count = count_tokens(request)

        if token_count < threshold:
            return None

        # Support both sync and async functions
```

```
            result = apply_optimization(context, request)
            if inspect.isawaitable(result):
                await result
            return None

        return callback
```

Optimization functions follow the `(context, request) -> None` signature. They receive context because execution state (e.g., how far compression has progressed) must be recorded in `context.state`. Next, we'll look at specific optimization strategies.

6.2.2 Sliding-window strategy

The simplest approach is to keep only the most recent n messages. By discarding older messages and keeping only the newest, you limit the context size.

BASIC IMPLEMENTATION

The following function implements a sliding-window strategy with three steps:

1. Iterate through the contents to find the user's original question.
2. Unconditionally preserve everything up to the user's message, ensuring that the agent retains the problem context.
3. Keep only the most recent `window_size` items from subsequent items (assistant messages, tool calls, tool results).

Listing 6.2 Sliding-window strategy implementation

```
def apply_sliding_window(
    context: ExecutionContext,
    request: LlmRequest,
    window_size: int = 20
) -> None:
    """Sliding window that keeps only the most recent N messages"""

    contents = request.contents

    # Find user message position
    user_message_idx = None
    for i, item in enumerate(contents):
        if isinstance(item, Message) and item.role == "user":
            user_message_idx = i
            break

    if user_message_idx is None:
        return

    # Preserve up to user message
    preserved = contents[:user_message_idx + 1]

    # Keep only the most recent N from remaining items
    remaining = contents[user_message_idx + 1:]
```

```
    if len(remaining) > window_size:
        remaining = remaining[-window_size:]

    request.contents = preserved + remaining
```

To use the sliding window, pass it to `create_optimizer_callback`.

Listing 6.3 Using a sliding window with an optimizer callback

```
optimizer = create_optimizer_callback(
    apply_optimization=apply_sliding_window,
    threshold=30000
)

agent = Agent(
    model=LlmClient(model="gpt-5"),
    before_llm_callbacks=[optimizer]
)
```

LIMITATIONS

The sliding-window strategy is simple to implement and predictable, but it has several limitations. If n is small, you may lose important context; if n is large, it doesn't solve the context-explosion problem. More fundamentally, message count isn't a good indicator of context size. A file read result is one message, but it can occupy tens of thousands of tokens. Therefore, more precise management requires measuring token counts directly.

6.2.3 Token counting

The LLM's actual limit is determined by token count, not message count. A single file read result might take up 10,000 tokens, whereas 20 short conversations combined might take up only 500 tokens. Accurate context management requires measuring token counts directly.

INTRODUCING TIKTOKEN

tiktoken is a tokenizer library provided by OpenAI. Because it's identical to the tokenizer used by GPT models, you can calculate exact token counts. Even when you use other models (Claude, Gemini, and so on), it can provide a rough estimate.

The following function calculates the total token count of an `LlmRequest`. The key is to use `build_messages` to convert the request to the format sent to the API and then calculate the tokens. This function gives you an accurate token count that includes instructions, contents, and tool definitions.

Listing 6.4 Token counting with tiktoken

```
def count_tokens(request: LlmRequest) -> int:
    """Calculate total token count of LlmRequest"""
    import tiktoken
    from .llm import build_messages
```

```
    # Select encoding for model, use default on failure
    try:
        encoding = tiktoken.encoding_for_model(request.model_id or "gpt-5")
    except KeyError:
        encoding = tiktoken.get_encoding("o200k_base")

    # Convert to API message format then count tokens
    messages = build_messages(request)
    total_tokens = 0

    for message in messages:
        # Per-message overhead (role, separators, etc.)
        total_tokens += 4

        # Content tokens
        if message.get("content"):
            total_tokens += len(encoding.encode(message["content"]))

        # tool_calls tokens
        if message.get("tool_calls"):
            for tool_call in message["tool_calls"]:
                func = tool_call.get("function", {})
                if func.get("name"):
                    total_tokens += len(encoding.encode(func["name"]))
                if func.get("arguments"):
                    total_tokens += len(encoding.encode(func["arguments"]))

    # Tool definition tokens
    if request.tools:
        for tool in request.tools:
            tool_def = tool.tool_definition
            total_tokens += len(encoding.encode(json.dumps(tool_def)))

    return total_tokens
```

WHY OPTIMIZATION MATTERS EVEN BELOW THE LIMIT

Most modern models support context windows of 128,000 to more than 1 million tokens. You may think optimization is necessary only when a model approaches these limits, but context bloat creates problems well before the maximum.

Context optimization isn't just about avoiding errors. Even within the limit, a bloated context creates three problems:

- Cost and latency grow with token count. Model pricing is per-token, and time to first token increases with context size.
- Irrelevant information distracts the model from the immediate task. Research shows that models struggle to find relevant information buried in long contexts—a phenomenon known as "lost in the middle."
- You have to reserve space for the model's response. If the prompt consumes most of the context window, the model may not have enough room to generate a complete response.

Recent research and production experience show that performance degradation begins well below the advertised context limit. Chroma's "Context Rot" study (https://www.trychroma.com/research/context-rot; 2025) found that model performance becomes increasingly unreliable as input length grows, even on simple tasks. Manus's engineering team notes that "model performance tends to degrade beyond a certain context length, even if the window technically supports it."

The threshold varies by model, task complexity, and use case, but the consensus is clear: the optimal working context is much smaller than the maximum window size. As a general rule, minimize context to what's required for the task. In this book's examples, we use a small threshold value (around 20,000 tokens) to clearly observe compaction and context-management behaviors.

6.2.4 Compaction strategy

The core idea of compaction is to replace already processed data with lightweight references. If you've read a file and completed the analysis, there's no need to keep the entire file contents in context. Instead, you leave a hint saying that the file was read and can be reread if necessary.

This approach follows a "scope by default" principle: give the agent the minimum context required and let it explicitly request more when necessary. The agent sees a reference like "Search results processed. Re-search if needed." If it needs that data again, it calls the tool. Compaction is divided into two types based on the compression target: tool call and tool result.

TOOLCALL COMPRESSION: WRITE OPERATIONS

Tools like `create_file`, which store data externally, include large data in the `ToolCall`'s arguments. Saving a 10,000-character report to a file results in a `ToolCall` like this:

```
ToolCall(
    name="create_file",
    arguments={"path": "report.md", "content": "... 10000 characters of
report content ..."}
)
```

You don't have to maintain content that's already saved to a file in the context. Remove the content from the `ToolCall`'s arguments, and replace it with a reference message.

TOOLRESULT COMPRESSION: READ OPERATIONS

Tools like `read_file` and `search_web`, which fetch data from external sources, include large amounts of data in the `ToolResult` content. File contents or search results can occupy tens of thousands of tokens.

If the agent has already processed the information, leave only what information was retrieved instead of the original data. If the agent has to reference that content again, it can call the tool again.

> **NOTE** Calling the tool again isn't free. Compaction trades LLM tokens for an extra tool invocation, which can mean API fees, added latency, and (for tools

like `search_web`) slightly different results on the second call. The strategy pays off when the discarded blob is large relative to those refetch costs.

IMPLEMENTATION

The code in the following listing implements compaction. The key is to process both `ToolCalls` and `ToolResults` while iterating through contents with a single `for` loop. Because `ToolCalls` always appear before their corresponding `ToolResults`, when you encounter a `ToolCall`, you can save its arguments for compressing the `ToolResult` later. `TOOLCALL_COMPACTION_RULES` defines tools whose `ToolCall` arguments should be compressed, and `TOOLRESULT_COMPACTION_RULES` defines tools whose `ToolResult` content should be compressed.

Listing 6.5 Compaction-strategy implementation

```
# Tools to compress ToolCall arguments
TOOLCALL_COMPACTION_RULES = {
    "create_file": "[Content saved to file]",
}

# Tools to compress ToolResult content
TOOLRESULT_COMPACTION_RULES = {
    "read_file": "File content from {file_path}. Re-read if needed.",
    "search_web": "Search results processed. Query: {query}. Re-search if
needed.",
    "tavily_search": "Search results processed. Query: {query}. Re-search if
needed.",
}

def apply_compaction(context: ExecutionContext, request: LlmRequest) -> None:
    """Compress tool calls and results into reference messages"""

    tool_call_args: Dict[str, Dict] = {}
    compacted = []

    for item in request.contents:
        if isinstance(item, ToolCall):
            # Save arguments (for use when compressing ToolResult later)
            tool_call_args[item.tool_call_id] = item.arguments

            # If the ToolCall itself is a compression target (create_file,
etc.)
            if item.name in TOOLCALL_COMPACTION_RULES:
                compressed_args = {
                    k: TOOLCALL_COMPACTION_RULES[item.name] if k == "content"
else v
                    for k, v in item.arguments.items()
                }
                compacted.append(ToolCall(
                    tool_call_id=item.tool_call_id,
                    name=item.name,
                    arguments=compressed_args
```

```
                ))
            else:
                compacted.append(item)

        elif isinstance(item, ToolResult):
            # If ToolResult is a compression target (read_file, search_web,
etc.)
            if item.name in TOOLRESULT_COMPACTION_RULES:
                args = tool_call_args.get(item.tool_call_id, {})
                template = TOOLRESULT_COMPACTION_RULES[item.name]
                compressed_content = template.format(
                    file_path=args.get("file_path", args.get("path",
"unknown")),
                    query=args.get("query", "unknown")
                )
                compacted.append(ToolResult(
                    tool_call_id=item.tool_call_id,
                    name=item.name,
                    status=item.status,
                    content=[compressed_content]
                ))
            else:
                compacted.append(item)
        else:
            compacted.append(item)

    request.contents = compacted
```

This code turns 100,000 tokens of noise in every prompt into a precise, on-demand resource. The data can be huge, but the context window remains lean.

TRADEOFFS WITH PROMPT CACHING

It's important to consider prompt caching when applying compaction. *Prompt caching* is a cost-optimization feature provided by LLM APIs. When the initial portion of a prompt is identical across consecutive requests, the API reuses previously computed results, which can reduce costs by up to 90% and speed response times. OpenAI, Anthropic, and Google support this feature; it's applied automatically without special configuration.

The problem is that compaction modifies past messages. This matters because prompt caching reuses computation for identical prefixes. Consider what happens across two consecutive steps:

```
Step 5: [system][user][msg1][msg2][msg3][msg4][msg5]
Step 6: [system][user][msg1][msg2][msg3][msg4][msg5][msg6]
                                                    ↑ only this needs new
computation
```

When you modify messages in the middle of context (such as compressing old tool results), everything after that point becomes different:

```
Step 6 (Compacted): [system][user][msg1][compressed msg2][msg3]...[msg6]
                                                ↑ changed → everything after is
cache miss
```

The solution is threshold-based optimization. Below a certain threshold, keep the context stable to preserve caching benefits. Modify only when the cost of context overflow exceeds the cost of cache misses. The `create_optimizer_callback` you've already implemented handles this role:

```
# Below 50000 tokens, don't apply Compaction (maintain caching benefits)
# At 50000 tokens or above, apply Compaction (prevent degradation)
optimizer = create_optimizer_callback(
    apply_optimization=apply_compaction,
    threshold=50000
)
```

6.2.5 Summarization strategy

Compaction targets only specific tool calls and results. If the general conversation or the agent's reasoning becomes lengthy, compaction alone can't solve the problem. In these cases, summarization is a last resort.

SELECTING SUMMARIZATION TARGETS

Summarization selects the oldest messages first as summarization targets. Still, the user's original question must be preserved. The agent must not forget what it's currently trying to solve. Summarization targets only the intermediate process: which tools were called, what information was discovered, and what progress was made. This information is summarized and added to the system prompt.

DESIGNING THE SUMMARY PROMPT

The following prompt summarizes the agent's intermediate process. Requesting structured output ensures that summaries are consistent:

```
SUMMARIZATION_PROMPT = """You are summarizing an AI agent's work progress.

Given the following execution history, extract:
1. Key findings: Important information discovered
2. Tools used: List of tools that were called
3. Current status: What has been accomplished and what remains

Be concise. Focus on information that will help the agent continue its work.

Execution History:
{history}

Provide a structured summary."""
```

IMPLEMENTATION

The function in listing 6.6 implements summarization. The core logic consists of four steps:

1. Find the user-message position to mark the starting point of the preservation range.
2. Keep the most recent `keep_recent` messages in their original form.
3. Summarize the intermediate process between them by using the LLM.
4. Reconstruct the result as "user question + summary message + recent messages."

Recording the last summarization position in `context.state` prevents resummarizing already summarized portions in later summarization calls.

Listing 6.6 Summarization strategy implementation

```
async def apply_summarization(
    context: ExecutionContext,
    request: LlmRequest,
    llm_client: LlmClient,
    keep_recent: int = 5
) -> None:
    """Replace old messages with a summary"""

    contents = request.contents

    # Find user message position
    user_idx = None
    for i, item in enumerate(contents):
        if isinstance(item, Message) and item.role == "user":
            user_idx = i
            break

    if user_idx is None:
        return

    # Check previous summary position (skip already-summarized portions)
    last_summary_idx = context.state.get("last_summary_idx", user_idx)

    # Calculate summarization target range
    summary_start = last_summary_idx + 1
    summary_end = len(contents) - keep_recent

    # Overlap prevention: exit if nothing to summarize or range is invalid
    if summary_end <= summary_start:
        return

    # Determine portions to preserve (no overlap)
    preserved_start = contents[:last_summary_idx + 1]
    preserved_end = contents[summary_end:]
    to_summarize = contents[summary_start:summary_end]

    # Generate summary
    history_text = format_history_for_summary(to_summarize)
    summary = await generate_summary(llm_client, history_text)

    # Add summary to instructions
    request.append_instructions(f"[Previous work summary]\n{summary}")
```

```
    # Keep only preserved portions in contents
    request.contents = preserved_start + preserved_end

    # Record summary position
    context.state["last_summary_idx"] = len(preserved_start) - 1


def format_history_for_summary(items: List[ContentItem]) -> str:
    """Convert ContentItem list to text for summarization"""
    lines = []
    for item in items:
        if isinstance(item, Message):
            lines.append(f"[{item.role}]: {item.content[:500]}...")
        elif isinstance(item, ToolCall):
            lines.append(f"[Tool Call]: {item.name}({item.arguments})")
        elif isinstance(item, ToolResult):
            content_preview = str(item.content[0])[:200] if item.content else ""
            lines.append(f"[Tool Result]: {item.name} -> {content_preview}...")
    return "\n".join(lines)


async def generate_summary(llm_client: LlmClient, history: str) -> str:
    """Generate history summary using LLM"""

    request = LlmRequest(
        instructions=[SUMMARIZATION_PROMPT.format(history=history)],
        contents=[Message(role="user", content="Please summarize.")]
    )

    response = await llm_client.generate(request)

    for item in response.content:
        if isinstance(item, Message):
            return item.content

    return ""
```

COMPACTION VS. SUMMARIZATION

These two strategies differ fundamentally. Compaction is *deterministic*: we can predict exactly what will be compressed based on tool names. Summarization is *semantic*: an LLM decides what's important. Deterministic operations are cheaper, faster, and more predictable, so we apply them first.

Another important difference is reversibility. With compaction, the agent can restore the original data by calling the tool again. With summarization, the original reasoning is permanently lost. Therefore, try compaction first and apply summarization only when compaction is insufficient.

6.2.6 *Hierarchical context management*

In what order should we apply these strategies? Considering cost and effectiveness, the hierarchical approach illustrated in figure 6.4 is reasonable.

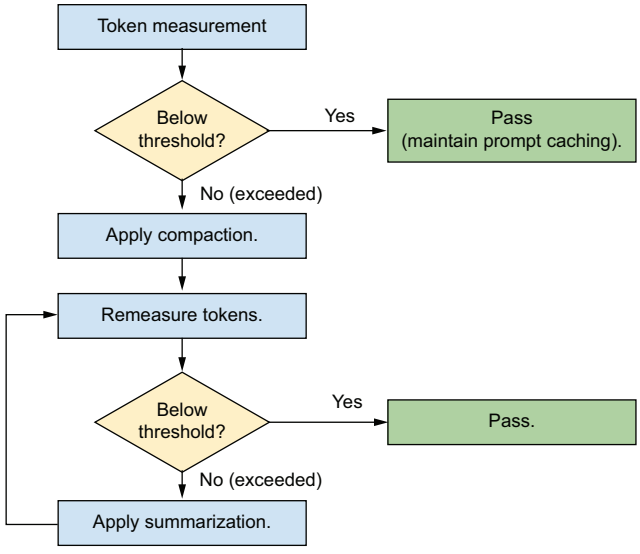

Figure 6.4 Context-management strategy decision flow

First, measure the token count. If it's below the threshold, do nothing. This maintains prompt-caching benefits and eliminates unnecessary processing. If the threshold is exceeded, apply compaction. Compaction is free and fast. In most cases, file reads or search results are the main culprits in context growth, so compaction alone is often sufficient. If the threshold is still exceeded after compaction, apply summarization as a resort. Summarization incurs additional LLM call costs, but it's better than failure due to context errors.

INTEGRATED IMPLEMENTATION

To apply multiple strategies sequentially, implement them as a class. This process is cleaner. The following class integrates all the strategies. The `__call__` method is registered as `before_llm_callback` and executes the three stages sequentially. After each stage, tokens are measured again, and if the total is below the threshold, the method returns immediately.

Listing 6.7 Hierarchical `ContextOptimizer` class

```
class ContextOptimizer:
    """Hierarchical context optimization strategy"""

    def __init__(
        self,
        llm_client: LlmClient,
        token_threshold: int = 50000,
        enable_compaction: bool = True,
        enable_summarization: bool = True,
        keep_recent: int = 5
    ):
```

```
        self.llm_client = llm_client
        self.token_threshold = token_threshold
        self.enable_compaction = enable_compaction
        self.enable_summarization = enable_summarization
        self.keep_recent = keep_recent

    async def __call__(
        self,
        context: ExecutionContext,
        request: LlmRequest
    ) -> Optional[LlmResponse]:
        """Register as before_llm_callback"""

        # Step 1: Measure tokens
        if count_tokens(request) < self.token_threshold:
            return None

        # Step 2: Apply Compaction
        if self.enable_compaction:
            apply_compaction(context, request)

            if count_tokens(request) < self.token_threshold:
                return None

        # Step 3: Apply Summarization
        if self.enable_summarization:
            await apply_summarization(
                context,
                request,
                self.llm_client,
                self.keep_recent
            )

        return None
```

REGISTERING WITH THE AGENT

The following code shows how to register `ContextOptimizer` as the agent's `before_llm_callback`. The optimization LLM uses a less expensive model (GPT-5 mini) to reduce costs; the agent's main model uses a more powerful model (GPT-5):

```
optimizer = ContextOptimizer(
    llm_client=LlmClient(model="gpt-5-mini"),
    token_threshold=50000,
    enable_compaction=True,
    enable_summarization=True
)

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[search_web, read_file, create_file],
    instructions="You are a helpful research assistant.",
    max_steps=30,
    before_llm_callbacks=[optimizer]
)
```

6.3 Continuous execution: Session and state management

In section 6.1.1, we identified the core challenge of pattern 2 (continuous execution): how to maintain conversation context across multiple `run()` calls. Currently, our agent creates a new `ExecutionContext` with each `run()` call, causing all information to be lost when execution ends. It remembers neither yesterday's conversation nor the one from moments ago.

The `Session` and `SessionManager` we'll implement in this section address this problem. By storing conversation history in a `Session` and accessing it again with the same `session_id`, we can seamlessly continue previous conversations. The following example demonstrates this capability:

```
# Maintaining conversation continuity with the same session_id
result1 = await agent.run("My name is Alice", session_id="session_1")
# → "Nice to meet you, Alice!"

result2 = await agent.run("What's my name?", session_id="session_1")
# → "Your name is Alice."
```

But the `Session`'s role extends beyond simple chat history storage. A `Session` can store not only conversation history but also intermediate execution state. By using this capability, we can overcome the fundamental limitations of the synchronous HITL implementation.

In chapter 5, we implemented user approval using the `input()` function. Before executing a dangerous tool, the agent would prompt "Do you approve? (Y/N)" in the terminal and wait for the user's response. Although this approach works during local development, it's unusable in production services.

The solution is to pause the agent's execution when it encounters a tool that requires approval, save the state, and resume later. When the agent encounters a tool call that requires approval, it saves the current execution state to the session and returns immediately. Then the user can click a button on a web page, respond to a notification on a mobile app, or even approve via email days later. When approval arrives, the saved state is restored and execution continues. In this section, we'll achieve two goals:

- `Session` *fundamentals*—We'll explore the structure of the `Session` class and `SessionManager`; then we'll integrate them with our agent to support multiturn conversations.
- *HITL*—We'll use the session's state-persistence capabilities to implement an asynchronous tool-approval workflow. Through this workflow, we'll see firsthand that `Session` offers value far beyond simple chat history storage.

6.3.1 The Session class

A *session* is a container that holds all information that must persist across multiple `run()` calls. It serves two core purposes: it stores conversation history so that previous

conversations can be continued, and it stores an arbitrary execution state so that interrupted tasks can be resumed.

Before designing the session's structure, we must clarify its relationship with the `ExecutionContext` we implemented in chapter 4. An `ExecutionContext` holds all events and states that occur during a single `run()` execution. When execution ends, that `ExecutionContext` is no longer used. By contrast, a session persists across multiple `run()` executions. Therefore, a session has a longer lifecycle than an `ExecutionContext`. The pattern works as follows: when each `run()` execution completes, necessary information is saved to the session; when the next `run()` execution begins, information is restored from the session. Figure 6.5 shows this relationship.

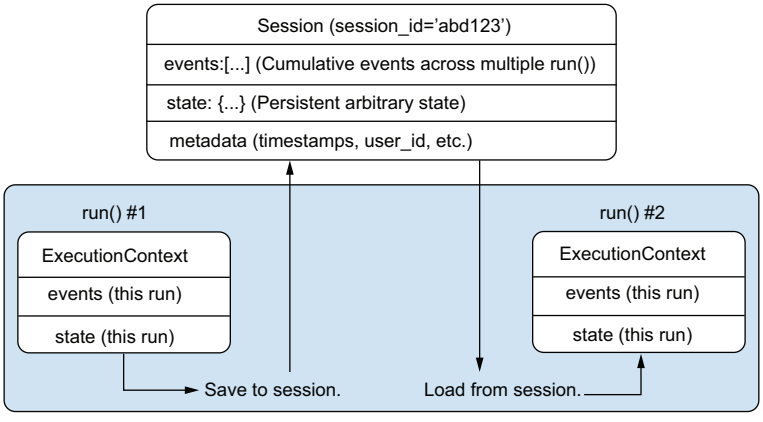

Figure 6.5 Relationship between session and `ExecutionContext`

Now, let's implement the `Session` class. `session_id` uniquely identifies the session, and `user_id` optionally identifies the user. The `events` list stores all events accumulated across multiple `run()` calls, and the `state` dictionary stores the arbitrary execution state. The timestamps track when the session was created and last updated.

Listing 6.8 `Session` class design

```
class Session(BaseModel):
    """Container for persistent conversation state across multiple run()
calls."""

    session_id: str
    user_id: str | None = None
    events: list[Event] = Field(default_factory=list)
    state: dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=datetime.now)
    updated_at: datetime = Field(default_factory=datetime.now)
```

A notable aspect of the `Session` class's design is the versatility of the `state` field. With its `dict[str, Any]` type, the `state` field can store any kind of data. It might hold a pending tool call awaiting approval in an HITL scenario, or it might store the progress of a long-running task. We adopted this generic design so that it can support diverse execution patterns without modifying the `Session` class itself.

6.3.2 Managing sessions with SessionManager

The `Session` class is merely a data container; it doesn't know how to create or persist itself. We need a separate manager that's responsible for creating, retrieving, and saving sessions. This role belongs to `SessionManager`. The core responsibilities of `SessionManager` are

- *Create*—Make a new session and store it
- *Get*—Find and return an existing session by `session_id`
- *Get or create*—Return an existing session if one is found; otherwise, create one
- *Save*—Persist changes to a session back to storage

First, we define these responsibilities as an `Abstract` class; then we provide concrete implementations depending on the storage type. `InMemorySessionManager`, for example, stores sessions in a Python dictionary, whereas production environments might use a database-backed implementation instead.

Defining the Abstract class

First, we define the interface that all `SessionManager` implementations must follow. The `create` method generates a new session, `get` retrieves an existing one, and `save` persists changes to storage. Because `get_or_create` is a combination of `get` and `create`, we implement it as a regular method rather than an abstract one. Subclasses implement only `get` and `create`, and `get_or_create` works automatically.

All methods are declared as `async`. For simple implementations like `InMemorySessionManager`, asynchronous operations aren't necessary, but implementations that use databases or external APIs require them. By defining an asynchronous interface in the `Abstract` class, we can swap implementations without modifying the calling code.

Listing 6.9 `BaseSessionManager Abstract` class

```
from abc import ABC, abstractmethod

class BaseSessionManager(ABC):
    """Abstract base class for session management."""

    @abstractmethod
    async def create(
        self,
        session_id: str,
        user_id: str | None = None
    ) -> Session:
        """Create a new session."""
```

```
        pass

    @abstractmethod
    async def get(self, session_id: str) -> Session | None:
        """Retrieve a session by ID. Returns None if not found."""
        pass

    @abstractmethod
    async def save(self, session: Session) -> None:
        """Persist session changes to storage."""
        pass

    async def get_or_create(
        self,
        session_id: str,
        user_id: str | None = None
    ) -> Session:
        """Get existing session or create new one."""
        session = await self.get(session_id)
        if session is None:
            session = await self.create(session_id, user_id)
        return session
```

IN-MEMORY IMPLEMENTATION

Next, we implement `InMemorySessionManager` for development and testing environments. It uses a Python dictionary as storage, meaning that all sessions are lost when the program terminates. In production, you would replace this with a database-backed implementation.

The `create` method instantiates a new `Session` object and stores it in the dictionary. If a session with the same `session_id` already exists, `create` raises a `ValueError` to prevent accidental overwrites. The `get` method looks up a session in the dictionary and returns `None` if it finds none. The `save` method stores the session in the dictionary.

Listing 6.10 `InMemorySessionManager` implementation

```
class InMemorySessionManager(BaseSessionManager):
    """In-memory session storage for development and testing."""

    def __init__(self):
        self._sessions: dict[str, Session] = {}

    async def create(
        self,
        session_id: str,
        user_id: str | None = None
    ) -> Session:
        """Create a new session."""
        if session_id in self._sessions:
            raise ValueError(f"Session {session_id} already exists")

        session = Session(
```

```
            session_id=session_id,
            user_id=user_id
        )
        self._sessions[session_id] = session
        return session

    async def get(self, session_id: str) -> Session | None:
        """Retrieve a session by ID."""
        return self._sessions.get(session_id)

    async def save(self, session: Session) -> None:
        """Save session to storage."""
        self._sessions[session.session_id] = session
```

6.3.3 Integrating sessions into the agent

With `Session` and `SessionManager` in place, let's integrate them into the agent to enable multiturn conversations. The integration involves two parts: extending `ExecutionContext` to reference a session and modifying the agent's `run()` method to load and save sessions.

Extending ExecutionContext

The `ExecutionContext` we implemented in chapter 4 served as a container for all state within a single execution. It stored execution history in the `events` list and arbitrary runtime state in the `state` dictionary. Now that sessions have been introduced, these responsibilities shift. Events and state now live inside the `Session`, and `ExecutionContext` holds a reference to the current session.

The key design principle here is backward compatibility. Existing code uses `context.events` and `context.state` extensively. We use properties to provide transparent access, allowing this code to work without modification.

The following code shows the extended `ExecutionContext`. The new fields are `session` and `session_manager`. The `session` field references the session to which the current execution belongs, and `session_manager` handles session persistence and retrieval. The `events` and `state` properties return `session.events` and `session.state` internally, so existing code continues to work unchanged.

Listing 6.11 `ExecutionContext` with session support

```
@dataclass
class ExecutionContext:
    # Existing attributes...

    # Session support
    session: Session | None = None
    session_manager: BaseSessionManager | None = None

    @property
    def events(self) -> list[Event]:
        if self.session:
```

```
            return self.session.events

    @property
    def state(self) -> dict[str, Any]:
        if self.session:
            return self.session.state

    def add_event(self, event: Event):
        """Append an event to the execution history."""
        self.events.append(event)
        # Update session timestamp if available
        if self.session:
            self.session.updated_at = datetime.now()
```

MODIFYING AGENT INITIALIZATION

Next, we modify the `Agent` class constructor to accept a `SessionManager`. When a `SessionManager` is provided, the agent stores it; otherwise, the agent defaults to `InMemorySessionManager`.

Listing 6.12 Agent initialization with SessionManager

```
class Agent:
    """Tool-calling agent with session support."""

    def __init__(
        self,
        model: LlmClient,
        tools: list[BaseTool] | None = None,
        instructions: str = "",
        max_steps: int = 10,
        session_manager: BaseSessionManager | None = None,
    ):
        self.model = model
        self.instructions = instructions
        self.max_steps = max_steps
        self.tools = self._setup_tools(tools or [])

        # Use provided session manager or create default
        self.session_manager = session_manager or InMemorySessionManager()
```

MODIFYING THE RUN() METHOD

The most important changes occur in the `run()` method. Session support requires three additions:

- We accept a `session_id` parameter to specify which session to execute within.
- We load or create the session at the start of execution.
- We save the session when execution completes.

Note the key changes in the next listing. When `session_id` is provided, `get_or_create()` retrieves the session. If the session already exists, the previous conversation history is

contained in `session.events`. When creating `ExecutionContext`, we inject both the `session` and `session_manager`. When execution completes, `save()` persists the session.

Listing 6.13 `run()` method with session support

```
async def run(
    self,
    user_input: str,
    session_id: str | None = None,
    context: ExecutionContext | None = None,
) -> AgentResult:
    """Execute the agent with optional session support."""

    # Load or create session if session_id is provided
    session = None
    if session_id:
        session = await self.session_manager.get_or_create(session_id)

    # Create context with session support
    if context is None:
        context = ExecutionContext(
            session=session,
            session_manager=self.session_manager,
        )

    # Existing run() logic...

    # Save session if available
    if session:
        await self.session_manager.save(session)

    return AgentResult(output=context.final_result, context=context)
```

6.3.4 Basic example: Multiturn conversation

Now that we've integrated `Session` and `SessionManager` into the agent, let's verify that everything works. We create an agent with `InMemorySessionManager`, which stores sessions in a dictionary in memory. This implementation is suitable for testing and single-process applications.

Listing 6.14 Creating an agent with session support

```
agent = Agent(
    model=model,
    instructions="You are a helpful assistant.",
    session_manager=InMemorySessionManager()
)
```

WITHOUT SESSION_ID

What happens when we don't specify a `session_id` or use different values for each call? Each `run()` invocation is treated as an independent conversation, with no memory of previous interactions:

```
result1 = await agent.run("Hi! My name is Alice and I'm a software engineer.")
print(f"Response 1: {result1.output}")

result2 = await agent.run("What's my name and what do I do?")
print(f"Response 2: {result2.output}")
```

The output is similar to the following:

```
Response 1: Hello Alice! Nice to meet you. It's great to hear that
you're a software engineer. How can I help you today?

Response 2: I don't have any previous context about you.
Could you please tell me your name and what you do?
```

The agent has no memory of the first conversation in the second call.

WITH THE SAME SESSION_ID

Now let's use the same `session_id` for both calls. The two `run()` invocations share the same `Session`, preserving the conversational context:

```
result1 = await agent.run(
    "Hi! My name is Alice and I'm a software engineer.",
    session_id="alice_session"
)

result2 = await agent.run(
    "What's my name and what do I do?",
    session_id="alice_session"
)
```

The output is similar to the following:

```
Response 1: Hello Alice! Nice to meet you. It's great to hear that
you're a software engineer. How can I help you today?

Response 2: Your name is Alice, and you're a software engineer.
```

This time, the agent correctly recalls information from the previous interaction.

UNDERSTANDING THE INTERNAL FLOW

Let's examine what happens internally to understand the difference between these two scenarios. When we use the same `session_id`,

1. First, `run()`– `get_or_create("alice_session")` creates a new session.
2. The conversation is stored in `session.events`, and `save()` persists it.
3. Second, `run()`– `get_or_create("alice_session")` returns the existing session.
4. `ExecutionContext` references this `Session`, so `context.events` includes the previous conversation.
5. The LLM generates its response with full access to the previous context.

When `session_id` is not specified or differs,

1. Each `run()` call creates a new session.
2. The new `Session`'s `events` list is empty, so the LLM responds without previous context.

In practice, you'll use sessions when building conversational applications in which users interact across multiple messages. Without sessions, each `run()` call is independent, which is appropriate for one-off tasks such as GAIA benchmark problems.

6.3.5 Data structures for tool confirmation

Now let's implement an asynchronous HITL workflow. This workflow will show that session provides value far beyond simple chat history storage: it also enables pausing and resuming execution at arbitrary points, which are essential for workflows that require human approval.

PENDING TOOL CALL

To implement the workflow, first we have to define data structures. We need types that represent which tool calls are awaiting approval and what decisions the user has made. `PendingToolCall` represents a tool call waiting to be executed. It's similar to the `ToolCall` we defined in chapter 4, but it includes additional information required for approval requests. The `tool_call` field preserves the original `ToolCall` object as is, and the `confirmation_message` field contains the message to display to the user, such as "This action will delete database.db. Do you want to continue?"

Listing 6.15 `PendingToolCall` class

```
class PendingToolCall(BaseModel):
    """A tool call awaiting user confirmation."""

    tool_call: ToolCall
    confirmation_message: str
```

TOOL CONFIRMATION

`ToolConfirmation` represents the user's approval decision. The `tool_call_id` identifies the tool call this response corresponds to, and the `approved` field indicates whether it was approved. `modified_arguments` is an optional field containing modified values if the user changed the arguments. A user might make this request: "I approve the file deletion, but change the target file to temp.txt."

Listing 6.16 `ToolConfirmation` class

```
class ToolConfirmation(BaseModel):
    """User's decision on a pending tool call."""

    tool_call_id: str
    reason: str | None = None
    approved: bool
    modified_arguments: dict | None = None
```

EXTENDING AGENTRESULT

The `AgentResult` we implemented in chapter 4 is a container for agent execution results. The `output` field stores the final result, and the `context` field stores the `ExecutionContext`. To support HITL workflows, we must distinguish whether execution is complete or paused while waiting for approval.

Add a `status` field to express this. `complete` means normal completion, `pending_confirmation` means waiting for approval, and `error` means an error occurred. When the status is `pending`, the `pending_tool_calls` field contains the list of tool calls awaiting approval.

Listing 6.17 `AgentResult` with pending support

```
from typing import Literal

class AgentResult(BaseModel):
    """Result of agent execution."""

    output: str | BaseModel | None = None
    context: ExecutionContext
    status: Literal["complete", "pending_confirmation", "error"] = "complete"
    pending_tool_calls: list[PendingToolCall] = Field(default_factory=list)
```

SERIALIZING STATE WITH PYDANTIC

The session's `state` field is of type `dict[str, Any]`, which can store arbitrary data. But Python dictionaries can't store Pydantic `model` objects directly. Problems arise when you save the session to a database or serialize it to JSON.

Pydantic's `model_dump()` and `model_validate()` methods solve this problem. `model_dump()` converts a Pydantic object to a dictionary, and `model_validate()` reverts a dictionary to a Pydantic object. Using this pattern, you can safely store and restore complex objects in the session's state.

The following code shows the pattern for storing and restoring pending tool calls. When the `act()` method discovers a tool that requires approval, it serializes the call with `model_dump()` and saves it to `state`. On the next `run()` call, it restores the call with `model_validate()` and resumes execution:

```
# Saving: when pending occurs in act()
pending_calls = [PendingToolCall(tool_call=tc, confirmation_message=msg)]
context.state["pending_tool_calls"] = [p.model_dump() for p in pending_calls]

# Restoring: when resuming run()
raw_pending = context.state.get("pending_tool_calls", [])
pending_calls = [PendingToolCall.model_validate(d) for d in raw_pending]
```

This serialization pattern allows you to store various execution states without modifying the session class. Beyond HITL scenarios, you can store the progress of long-running tasks, checkpoint data, intermediate results collected by the agent, and more in the same way.

Now that the data structures are ready, let's add the approval requirement feature to the tools themselves and implement the agent's pause and resume logic.

6.3.6 *Extending tools for confirmation*

How can we determine whether a tool requires approval? The most intuitive approach is to include this information in the tool itself. By extending the `BaseTool` class we implemented in chapter 4 with confirmation-related attributes, the agent can check these attributes before executing a tool and pause execution when necessary.

EXTENDING BASETOOL

Let's add two attributes to the `BaseTool` class: `requires_confirmation`, a Boolean indicating whether the tool requires user approval before execution, and `confirmation_message_template`, a template for the approval request message shown to the user. The template uses `{name}` and `{arguments}` placeholders, which are dynamically filled with tool-call information.

Listing 6.18 `BaseTool` with confirmation support

```
class BaseTool(ABC):
    # Existing attributes: name, description, parameters, execute()...

    # Confirmation support
    requires_confirmation: bool = False
    confirmation_message_template: str = (
        "The agent wants to execute '{name}' with arguments: {arguments}. "
        "Do you approve?"
    )

    def get_confirmation_message(self, arguments: dict[str, Any]) -> str:
        """Generate a confirmation message for this tool call."""
        return self.confirmation_message_template.format(
            name=self.name,
            arguments=arguments
        )
```

The `get_confirmation_message()` method fills the template with arguments to generate the final message. When the agent pauses tool execution, it includes this message in the `PendingToolCall` to present to the user.

DEFINING TOOLS THAT REQUIRE CONFIRMATION

We also modify the `@tool` decorator implemented in chapter 4 to accept `requires_confirmation` and `confirmation_message` parameters. The complete modified code is available in the GitHub repository.

Here's an example of defining tools that require confirmation using the updated decorator. `delete_file` is configured with `requires_confirmation=True`, requiring user approval before execution. By contrast, `get_weather` is a read-only operation and executes immediately without approval. Using the `{arguments[key]}` format in custom

messages includes specific argument values in the message, helping users understand what action will be performed before deciding whether to approve.

Listing 6.19 Tools with confirmation requirements

```
@tool(
    name="delete_file",
    description="Delete a file from the filesystem",
    requires_confirmation=True,
    confirmation_message="⚠️ The agent wants to delete
     '{arguments[filename]}'. "
                         "This action cannot be undone. Do you approve?"
)
def delete_file(filename: str) -> str:
    """Delete the specified file."""
    import os
    os.remove(filename)
    return f"Successfully deleted {filename}"


@tool(
    name="get_weather",
    description="Get current weather for a location"
)
def get_weather(location: str) -> str:
    """Get weather information. No confirmation needed."""
    return f"Weather in {location}: Sunny, 22°C"
```

6.3.7 *Implementing pause and resume in the agent*

Now that we've added the confirmation requirement attribute to tools, it's time to implement the logic for the agent to check this attribute and pause or resume execution. We'll modify the `act()` and `run()` methods to complete the HITL workflow.

INTERNAL FEATURES VS. EXTERNAL EXTENSIONS

Before diving into implementation, let's address an important design decision. We've explored how to customize agent behavior using callbacks, modifying prompts with `before_llm_callback` and postprocessing tool results with `after_tool_callback`. These callbacks are external extension points injected by users.

Tool confirmation is different. This feature isn't one that users optionally add; it's a core feature that the framework provides. The logic of stopping execution when encountering a tool that requires approval and resuming after receiving approval is part of the agent's internal behavior.

For this reason, tool confirmation handling is implemented inside the `_prepare_llm_request()` method rather than as a callback. This method is responsible for the internal logic of constructing requests to send to the LLM, and it converts approval results to `ToolResults` to include them naturally in the conversation flow, as shown in figure 6.6.

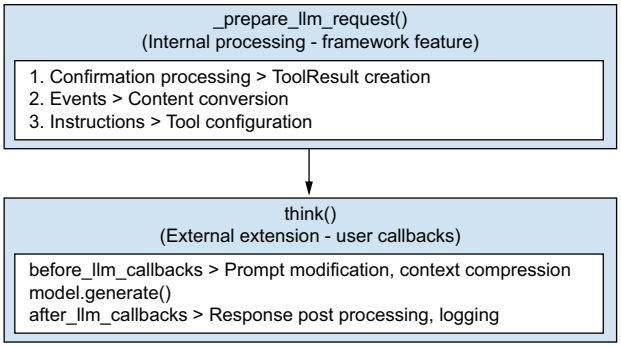

Figure 6.6 Internal features vs. external extensions in agent architecture

In this structure, `_prepare_llm_request()` handles the framework's essential logic, and callbacks handle user customization. With the two layers clearly separated, each layer can focus on its own responsibilities.

Design principles

Before implementation, let's establish two core design principles:

- *Pre-execution blocking*—Tools that require approval must receive user approval before execution. It's meaningless to execute a tool first, show the results, and then ask, "I performed this action. Is that OK?" Actions like file deletion and payment processing can't be undone.
- *Whitelist approval*—Users can approve only some of the pending tool calls. When the agent requests three tool calls, the user should be able to approve two and reject one. Only tools included in the approval list are executed, and tools not in the list are rejected automatically.

Modifying the act() method

The `act()` method that we implemented in chapter 4 immediately executed all tool calls returned by the LLM. Here, we modify it to check the `requires_confirmation` attribute and store tools that require approval in `context.state` instead of executing them.

Listing 6.20 Modified `act()` method with confirmation check

```
async def act(
    self,
    context: ExecutionContext,
    tool_calls: List[ToolCall]
) -> List[ToolResult]:
    tools_dict = {tool.name: tool for tool in self.tools}
    pending_calls = []
    results = []

    for tool_call in tool_calls:
        tool = tools_dict.get(tool_call.name)
```

```
            # Check if confirmation is required
            if tool.requires_confirmation:
                pending = PendingToolCall(
                    tool_call=tool_call,
                    confirmation_message=tool.get_confirmation_message(
                        tool_call.arguments
                    )
                )
                pending_calls.append(pending)
                continue  # Do not execute, do not create ToolResult

            # Execute tools that don't require confirmation
            # ... existing tool execution logic ...

        # Store pending calls in session state
        if pending_calls:
            context.state["pending_tool_calls"] = [
                p.model_dump() for p in pending_calls
            ]

        return results
```

Let's examine the key changes. As we iterate through tool calls, we check each tool's `requires_confirmation` attribute. For tools that require approval, we create a `PendingToolCall` object, add it to the `pending_calls` list, and move to the next tool without creating a `ToolResult`. Only tools that don't require approval are executed immediately and return results.

The return type of `act()` remains the same: `List[ToolResult]`. Because pending tool-call information is stored in `context.state`, callers can check `context.state` to determine pending status. This approach adds new functionality while maintaining compatibility with the existing interface.

What about dependent tool calls? LLMs typically batch only independent operations and chain dependent ones across separate rounds, so the `act()` loop rarely encounters a tool that's supposed to consume the output of a pending one. If a batch does mix pending and executed tools, both the executed results and the pending status flow back to the LLM, and the pending tools' results join the conversation when approval arrives.

MODIFYING THE RUN() METHOD

The `run()` method must handle two scenarios:

- When the method encounters a tool that requires approval during a new execution, it pauses execution and returns with a pending status.
- When the method resumes a previously paused execution, it registers the user's approval decisions in `context.state` and continues execution.

Listing 6.21 Modified `run()` method with pause and resume

```
async def run(
    self,
    user_input: str | None = None,
```

```
    session_id: str | None = None,
    tool_confirmations: List[ToolConfirmation] | None = None,
    context: ExecutionContext | None = None,
) -> AgentResult:

    # Register confirmations for processing in _prepare_llm_request
    if tool_confirmations:
        context.state["tool_confirmations"] = [
            c.model_dump() for c in tool_confirmations
        ]

    # Main agent loop
    while not context.final_result and context.current_step < self.max_steps:
        await self.step(context)

        # Check for pending confirmations after each step
        if context.state.get("pending_tool_calls"):
            pending_calls = [
                PendingToolCall.model_validate(p)
                for p in context.state["pending_tool_calls"]
            ]
            return AgentResult(
                status="pending_confirmation",
                context=context,
                pending_tool_calls=pending_calls,
            )
```

Let's review the key changes:

- `tool_confirmations` *parameter*—`run()` accepts a list of user-approval decisions. When resuming from a previously pending state, this list is registered in `context.state`. The processing is performed in `_prepare_llm_request()` during the next `step()` call.
- *Pending check*—After executing `step()` in the agent loop, `run()` checks for pending tool calls in `context.state`. If tool calls are present, it immediately returns with `pending_confirmation` status to pause execution.
- *Optional* `user_input` *parameter*—When resuming a paused execution, only `tool_confirmations` is required; now `user_input` is optional.

MODIFYING _PREPARE_LLM_REQUEST()

The `_prepare_llm_request()` method processes approval decisions registered in `context.state`. It executes approved tools and records results as events while logging rejection messages for rejected tools. This method is async because `await` is required for executing approved tools.

Listing 6.22 Modified `_prepare_llm_request()` for confirmation processing

```
async def _prepare_llm_request(
    self,
    context: ExecutionContext
```

```
) -> LlmRequest:
    """Package context into an LlmRequest, processing confirmations first."""

    # Process pending confirmations if both are present
    if ("pending_tool_calls" in context.state and
        "tool_confirmations" in context.state):

        confirmation_results = await self._process_confirmations(context)

        # Add results as an event so they appear in contents
        if confirmation_results:
            confirmation_event = Event(
                execution_id=context.execution_id,
                author=self.name,
                content=confirmation_results,
            )
            context.add_event(confirmation_event)

        # Clear processed state
        del context.state["pending_tool_calls"]
        del context.state["tool_confirmations"]

    # Flatten events into content items
    flat_contents = []
    for event in context.events:
        flat_contents.extend(event.content)

    # ... existing LlmRequest construction logic ...
```

The processing order is important. First, process approval decisions to generate `ToolResults` and record them as events; then construct `contents` including those events. This order ensures that the LLM can see the approval/rejection results in the next step.

This ordering is guaranteed because `_prepare_llm_request()` is an internal method. If we implemented this logic in `before_llm_callback`, a timing problem would occur: newly added events wouldn't be included in `contents` because callbacks receive an already constructed `LlmRequest`. This is another reason why frameworks' internal features should be handled in internal methods rather than callbacks.

IMPLEMENTING CONFIRMATION-PROCESSING LOGIC

The `_process_confirmations()` method performs the approval processing. For each pending tool call, it checks the approval status, executes approved tools, and records rejection messages for unapproved cases (either explicit rejection or absence from the approval list).

Listing 6.23 `_process_confirmations()` implementation

```
async def _process_confirmations(
    self,
    context: ExecutionContext
) -> List[ToolResult]:
```

```
tools_dict = {tool.name: tool for tool in self.tools}
results = []

# Restore pending tool calls from state
pending_map = {
    p["tool_call"]["tool_call_id"]: PendingToolCall.model_validate(p)
    for p in context.state["pending_tool_calls"]
}

# Build confirmation lookup by tool_call_id
confirmation_map = {
    c["tool_call_id"]: ToolConfirmation.model_validate(c)
    for c in context.state["tool_confirmations"]
}

# Process ALL pending tool calls
for tool_call_id, pending in pending_map.items():
    tool = tools_dict.get(pending.tool_call.name)
    confirmation = confirmation_map.get(tool_call_id)

    if confirmation and confirmation.approved:
        # Merge original arguments with modifications
        arguments = {
            **pending.tool_call.arguments,
            **(confirmation.modified_arguments or {})
        }

        # Execute the approved tool
        try:
            output = await tool(context, **arguments)
            results.append(ToolResult(
                tool_call_id=tool_call_id,
                name=pending.tool_call.name,
                status="success",
                content=[output],
            ))
        except Exception as e:
            results.append(ToolResult(
                tool_call_id=tool_call_id,
                name=pending.tool_call.name,
                status="error",
                content=[str(e)],
            ))
    else:
        # Rejected: either explicitly or not in confirmation list
        if confirmation:
            reason = confirmation.reason or "Tool execution was rejected
 by user."
        else:
            reason = "Tool execution was not approved."

        results.append(ToolResult(
            tool_call_id=tool_call_id,
            name=pending.tool_call.name,
            status="error",
```

```
                content=[reason],
            ))

    return results
```

This implementation has three key steps:

- *Whitelist approval*—We iterate through all pending tool calls and execute only those that are in the approval list with `approved=True`. Any tool calls that are not in the approval list or show `approved=False` are treated as rejections.
- *Argument merging*—When `modified_arguments` is provided, we merge it with the original arguments using `{**original, **modified}`. Modified arguments overwrite the original arguments, allowing users to modify specific arguments selectively, such as updating only the file path.
- *Clear feedback*—Rejected tools are also recorded as `ToolResults`. Because the LLM expects results for tool calls, we must explicitly inform it when a tool is rejected so that it can respond appropriately. If a file deletion is rejected, for example, the LLM can suggest alternative approaches or ask the user for clarification.

COMPLETE FLOW SUMMARY

Figure 6.7 illustrates the complete flow of the pause-and-resume mechanism we've implemented.

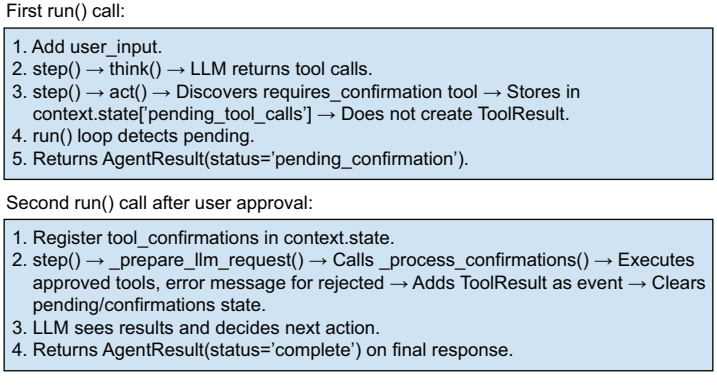

Figure 6.7 Overall flow of the pause-and-resume mechanism

6.3.8 Complete example: HITL workflow

Now let's combine all the components we've implemented to demonstrate a complete HITL workflow. We'll create a simple scenario in which the agent attempts to delete a file, pauses for user approval, and then resumes execution based on the user's decision.

SETTING UP THE EXAMPLE

First, we define a tool that requires confirmation and create an agent with session support. The `delete_file` tool is configured with `requires_confirmation=True`, so the agent pauses and waits for user approval before executing.

Listing 6.24 Setting up the tools and agent for HITL

```
from agent import Agent, InMemorySessionManager, ToolConfirmation
from tools import tool

@tool(
    name="delete_file",
    description="Delete a file from the filesystem",
    requires_confirmation=True,
    confirmation_message="⚠️ Delete '{arguments[filename]}'? This cannot be
     undone."
)
def delete_file(filename: str) -> str:
    import os
    os.remove(filename)
    return f"Deleted {filename}"

@tool(name="list_files", description="List files in a directory")
def list_files(directory: str = ".") -> str:
    import os
    return "\n".join(os.listdir(directory))

agent = Agent(
    model=model,
    tools=[delete_file, list_files],
    instructions="You are a helpful file management assistant.",
    session_manager=InMemorySessionManager()
)
```

STEP 1: INITIAL REQUEST

The user asks the agent to delete a temporary file. When the agent attempts to call the `delete_file` tool, execution pauses because confirmation is required.

Listing 6.25 Initial request triggering pending confirmation

```
result = await agent.run(
    "Please delete the file named temp.txt",
    session_id="file_session"
)

print(f"Status: {result.status}")
# Status: pending_confirmation

print(f"Pending tools: {len(result.pending_tool_calls)}")
# Pending tools: 1

for pending in result.pending_tool_calls:
```

```
    print(f"Tool: {pending.tool_call.name}")
    print(f"Arguments: {pending.tool_call.arguments}")
    print(f"Message: {pending.confirmation_message}")
# Tool: delete_file
# Arguments: {'filename': 'temp.txt'}
# Message: ⚠️ Delete 'temp.txt'? This cannot be undone.
```

The agent immediately returns `status="pending_confirmation"`. The `pending_tool_calls` list includes details about the tool that's awaiting approval, such as the confirmation message shown to the user.

STEP 2: USER APPROVAL

In a real application, the user might click a button on a web page or respond to a mobile notification. Here, we simulate approval by creating a `ToolConfirmation` object and resuming execution.

Listing 6.26 Resuming execution with user approval

```
# User approves the deletion
confirmation = ToolConfirmation(
    tool_call_id=result.pending_tool_calls[0].tool_call.tool_call_id,
    approved=True
)

result = await agent.run(
    session_id="file_session",
    tool_confirmations=[confirmation]
)

print(f"Status: {result.status}")
# Status: complete

print(f"Output: {result.output}")
# Output: I've deleted the file temp.txt as requested.
```

When we call `run()` again with the same `session_id` and the approval decision, the agent resumes execution. It executes the approved tool, receives the result, and generates a final response.

6.4 Long-term memory: Accumulating knowledge across sessions

`Session` and `SessionManager` allowed us to continue conversations within the same `session_id`. But when a session ends, its information is lost. In this section, we implement long-term memory to accumulate and use knowledge beyond session boundaries.

Long-term memory is a prime example of combining context engineering's write (storage) and retrieval (search) strategies. After a session ends, important information is saved to external storage (write). In new sessions, this information is retrieved and added to the context (retrieval).

6.4.1 The structure of long-term memory

The `SessionManager` we implemented earlier and the `MemoryManager` introduced here address different problems:

- `SessionManager` maintains conversation continuity within a session. When you call `run()` multiple times with the same `session_id`, the previous conversation content is preserved. But starting a new conversation with a different `session_id` prevents access to the previous session content.
- `MemoryManager` (figure 6.8) stores important information even after a session ends, allowing retrieval and use in future sessions. Examples include "the user mentioned that they're a marketer" or "the approach used when solving this problem before."

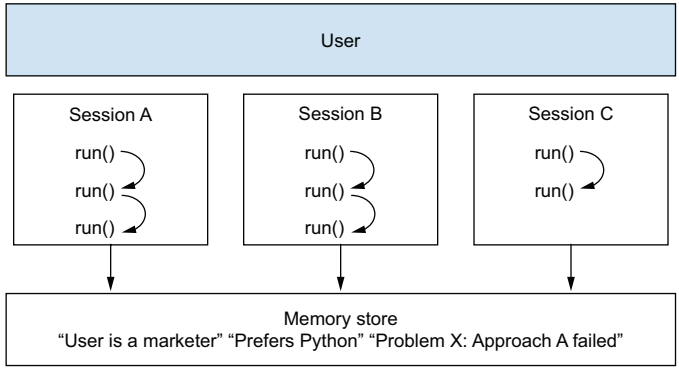

Figure 6.8 The role of `MemoryManager`

`SessionManager` handles horizontal continuity within sessions (conversation flow), and `MemoryManager` handles vertical continuity across sessions (knowledge accumulation).

Long-term memory systems have two core operations: storage (Write) and retrieval. The simplest approach is to convert text directly into vector embeddings for storage and then find and return vectors similar to the query text during retrieval. But storing entire conversations introduces noise, and important information can get buried.

Recent memory libraries such as Mem0 and Graphiti improve on this by using LLMs to extract only information worth preserving. When storing new information, these libraries perform one of four operations to manage conflicts with existing memories:

- `ADD` stores new information that doesn't already exist.
- `UPDATE` modifies existing memories.
- `DELETE` removes contradictory memories.
- `SKIP` ignores information that's already stored.

In this chapter, we implement only `ADD` and `SKIP`. User profile information (name, occupation, preferences, and so on) can change over time and require `UPDATE` and

DELETE, but problem-solving records are different. Even for the same problem, different approaches or results are worth storing as separate records (ADD), whereas identical attempts can be ignored (SKIP). We don't have to modify or delete past failure records; keeping them helps us avoid repeating the same mistakes. This immutability of memory simplifies our implementation.

6.4.2 Information extraction: Structured output

If the information to extract from memory consists of simple facts, a text format is sufficient. Information that fits in a single sentence, such as "the user is a marketer" or "prefers Python," can be stored and searched as is. Sometimes, however, the information to store is more complex. Consider GAIA problem-solving records: instead of storing "solved this problem," we have to store what the problem was, what approach was used, and what the result was. This information must be structured so that the agent can reference past successful approaches and avoid failed ones when encountering similar problems.

EXTENDING LLMCLIENT

We add two methods to `LlmClient` for information extraction and vector embeddings. The `generate()` method from chapter 4 takes `LlmRequest` as input and is specialized for agent execution. Simpler interfaces are required for basic text generation.

The `ask()` method takes a prompt and returns text or JSON. When a `response_format` parameter is provided, it returns output in the requested format:

```
# Simple text generation
answer = await llm_client.ask("What is 2 + 2?")

# JSON structured output
data = await llm_client.ask(prompt, response_format=GaiaOutput)
```

DEFINING THE MEMORY SCHEMA

We use Pydantic to define a schema for GAIA problem-solving records. The schema captures the essential elements of a problem-solving attempt. `task_summary` describes what the problem asked, `approach` records the methods and tools used to solve it, `final_answer` stores the agent's submitted answer, `is_correct` indicates whether the answer was right, and `error_analysis` captures why the attempt failed when it did.

Listing 6.27 `TaskMemory` schema definition

```
class TaskMemory(BaseModel):
    """Structured memory for GAIA problem-solving records"""

    task_summary: str = Field(description="What the problem asked")
    approach: str = Field(description="Methods and tools used to solve it")
    final_answer: str = Field(description="The agent's submitted answer")
    is_correct: bool = Field(description="Whether the answer was correct")
    error_analysis: str | None = Field(
```

```
        default=None,
        description="Why the attempt failed, if it did"
    )

    def to_embedding_text(self) -> str:
        """Generate text for vector search"""
        return f"Task: {self.task_summary}"
```

The `to_embedding_text()` method generates text for vector search. Here, it includes only the problem summary so that similar problems can be found during retrieval. The approach and answer are stored as metadata and used after retrieval.

EXTRACTION PROMPT DESIGN

We design a prompt that extracts structured memories from the agent's execution history. The prompt instructs the LLM to analyze the complete execution trace and identify the key elements: what problem was being solved, what approach was taken, and whether the approach succeeded:

```
TASK_MEMORY_EXTRACTION_PROMPT = """You are analyzing an agent's problem-
solving record to extract a learning record.

Analyze the following execution history:

<execution_history>
{execution_history}
</execution_history>
"""
```

The execution history contains the agent's reasoning process, tool calls, intermediate results, and feedback added by the user, indicating whether the answer was correct. The LLM synthesizes all this information in structured memory.

DUPLICATE CHECK PROMPT

Before storing a new memory, we check whether similar information already exists. This check prevents the memory store from filling with redundant entries about the same problem.

Listing 6.28 `DuplicateCheckResult` **schema and prompt**

```
class DuplicateCheckResult(BaseModel):
    """Result of duplicate check"""
    decision: str = Field(description="ADD (new information) or SKIP
(duplicate)")
    reason: str = Field(description="Explanation for the decision")

DUPLICATE_CHECK_PROMPT = """Compare the new memory against existing memories
to determine if it's a duplicate.

Existing memories:
```

```
{existing_memories}

New memory:
{new_memory}

Respond with one of:
- ADD: This is new information that should be stored
- SKIP: Similar information already exists, no need to store

Judgment criteria:
- Same problem with different approach or different result counts as new
information
- Same problem with same approach and same result is a duplicate
"""
```

The prompt provides existing memories as context and asks the LLM to make a binary decision. The `DuplicateCheckResult` schema captures both the decision and the reasoning behind it.

6.4.3 Building a vector store with ChromaDB

In chapter 5, we learned the principles of vector search through hands-on implementation. We covered the process of calculating cosine similarity and returning top results. In this section, we'll implement storage and retrieval using a vector database. For practice, we use ChromaDB, which is lightweight and simple to configure.

INSTALLING CHROMADB AND CORE CONCEPTS

ChromaDB is a lightweight vector database that runs locally. You can use it immediately by installing the Python package, with the following command. No separate server setup is required, making it ideal for prototyping and learning:

```
uv add chromadb
```

The core concept in ChromaDB is the collection. A *collection* groups related documents, similar to a table in a relational database. Each collection stores document text, embedding vectors, and metadata together.

BASIC USE

The following code demonstrates ChromaDB's basic storage and retrieval flow. We create an in-memory client with `chromadb.Client()` and a collection with `get_or_create_collection()`. When we specify an embedding function in the `embedding_function` parameter, ChromaDB automatically converts text to vectors during both storage and retrieval.

Listing 6.29 ChromaDB basic use

```
import chromadb
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

# Create client (in-memory)
```

```
client = chromadb.Client()

# Set up embedding function
embedding_fn = OpenAIEmbeddingFunction(
    model_name="text-embedding-3-small"
)

# Create collection
collection = client.get_or_create_collection(
    name="my_memories",
    embedding_function=embedding_fn
)

# Store documents - just pass text, embeddings are generated automatically
collection.add(
    ids=["mem_001", "mem_002"],
    documents=["The cat is sleeping on the sofa", "The dog is playing in the
     park"],
    metadatas=[{"category": "cat"}, {"category": "dog"}]
)

# Search - just pass query text, embeddings are generated automatically
results = collection.query(
    query_texts=["Where is the cat resting"],
    n_results=1
)

print(results["documents"])  # [['The cat is sleeping on the sofa']]
print(results["metadatas"])  # [[{'category': 'cat'}]]
```

When we specify an `embedding_function`, the document text is embedded automatically when we call `add()`, and `query_texts` are also embedded automatically when we call `query()` to perform similarity search. The embedding generation and cosine similarity calculation are handled internally.

USING METADATA

The `metadatas` parameter stores additional information for each document. You can use this metadata in search results to check supplementary information such as document category, creation time, and source. In the memory store we implement in the next section, we store information such as approach and correctness as metadata.

6.4.4 *Implementing TaskMemoryManager*

In this section, we implement `TaskMemoryManager`, which stores and retrieves GAIA problem-solving records using ChromaDB. This class handles the pipeline for extracting structured memories from execution history, checking for duplicates, and storing them in ChromaDB.

Table 6.3 lists the methods we'll build and shows how they fit together. The first three (`_format_execution_history`, `_extract_memory`, and `_is_duplicate`) are helpers, `save()` orchestrates them, and `search()` is the read-side counterpart.

Table 6.3 `TaskMemoryManager` methods

Method	Purpose	Listing
`__init__`	Wires up the LLM client and Chroma DB collection	6.30
`_format_execution_history`	Flattens the event list into a text block that the LLM can read	6.31
`_extract_memory`	Asks the LLM to produce a structured `TaskMemory` from that text	6.31
`_is_duplicate`	Asks the LLM whether a new memory duplicates the retrieved neighbors	6.32
`save`	Orchestrates formatting, extraction, duplicate checking, and storage	6.33
`search`	Embeds a query and returns the top-*k* `TaskMemory` neighbors	6.34

CLASS INITIALIZATION

The `TaskMemoryManager` constructor receives an `LlmClient` through injection and uses it for memory extraction and duplicate checking. The ChromaDB client and collection are also initialized here.

Listing 6.30 `TaskMemoryManager` initialization

```
import uuid
import chromadb
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction


class TaskMemoryManager:
    """Memory manager for GAIA problem-solving learning"""

    def __init__(
        self,
        llm_client: LlmClient,
        collection_name: str = "task_memories",
    ):
        self.llm_client = llm_client

        # ChromaDB setup
        self.client = chromadb.Client()
        embedding_fn = OpenAIEmbeddingFunction(
            model_name="text-embedding-3-small"
        )
        self.collection = self.client.get_or_create_collection(
            name=collection_name,
            embedding_function=embedding_fn
        )
```

Memory extraction methods

We implement methods to extract structured memories from execution history. The `_format_execution_history()` converts the event list to a text format suitable for the LLM. It iterates through each event's content and formats messages, tool calls, and tool results separately.

In `_extract_memory()`, we pass `response_format=TaskMemory` so that the LLM generates JSON that matches the `TaskMemory` schema. Because `LlmClient` parses this and returns a `TaskMemory` instance, no separate JSON parsing logic is required.

Listing 6.31 Implementing memory extraction

```
class TaskMemoryManager:
    # ... previous code ...

    async def _extract_memory(self, execution_history: str) -> TaskMemory | None:
        """Extract structured memory from execution history"""
        prompt = TASK_MEMORY_EXTRACTION_PROMPT.format(
            execution_history=execution_history
        )

        try:
            return await self.llm_client.ask(
                prompt=prompt,
                response_format=TaskMemory
            )
        except Exception as e:
            print(f"Memory extraction failed: {e}")
            return None

    def _format_execution_history(self, events: list[Event]) -> str:
        """Convert event list to text"""
        lines = []
        for event in events:
            for item in event.content:
                if isinstance(item, Message):
                    lines.append(f"[{item.role}]: {item.content}")
                elif isinstance(item, ToolCall):
                    lines.append(f"[Tool Call]: {item.name}({item.
arguments})")
                elif isinstance(item, ToolResult):
                    content_preview = str(item.content[0])[:500] if item.
content else ""
                    lines.append(f"[Tool Result]: {item.name} -> {content_
preview}")
        return "\n".join(lines)
```

Duplicate check method

This method checks whether a new memory duplicates an existing one. If there are no existing search results, it returns `False`. It converts existing memories to text and includes them, along with the new memory, in the prompt. When a response is received

with the `DuplicateCheckResult` schema, the decision can be accessed directly through `result.decision`. If the decision is `SKIP`, the memory is a duplicate and shouldn't be stored, so the method returns `True` to prevent storage.

Listing 6.32 Implementing duplicate check

```
class TaskMemoryManager:
    # ... previous code ...

    async def _is_duplicate(
        self,
        new_memory: TaskMemory,
        existing_results: dict
    ) -> bool:
        """Determine if a new memory duplicates an existing one"""
        if not existing_results["metadatas"] or not existing_
results["metadatas"][0]:
            return False

        # Convert existing memories to text
        existing_texts = []
        for meta in existing_results["metadatas"][0]:
            existing_texts.append(
                f"task_summary: {meta.get('task_summary')}, "
                f"- approach: {meta.get('approach')}, "
                f"is_correct: {meta.get('is_correct')}"
            )

        prompt = DUPLICATE_CHECK_PROMPT.format(
            existing_memories="\n".join(existing_texts),
            new_memory=f"task_summary: {new_memory.task_summary}, "
                       f"approach: {new_memory.approach}, "
                       f"is_correct: {new_memory.is_correct}"
        )

        try:
            result = await self.llm_client.ask(
                prompt=prompt,
                response_format=DuplicateCheckResult
            )
            return result.decision == "SKIP"
        except Exception:
            return False  # Proceed with storage if parsing fails
```

SAVE METHOD

The `save()` method brings all components together. It converts the execution history to text, extracts structured memory using the LLM, searches for similar existing memories to check for duplicates, and stores the memory in ChromaDB if it's not a duplicate. For documents, it generates searchable text using `to_embedding_text()`. For metadata, it converts all `TaskMemory` fields to a dictionary using `model_dump()` for storage.

Listing 6.33 `TaskMemoryManager` `save()` method

```
class TaskMemoryManager:
    # ... previous code ...

    async def save(self, context: ExecutionContext) -> str | None:
        """
        Extract and save memory from execution context.

        Returns:
            memory_id if saved, None if ignored as duplicate
        """
        # 1. Convert execution history to text
        execution_history = self._format_execution_history(context.events)

        # 2. Extract structured memory using LLM
        memory = await self._extract_memory(execution_history)
        if memory is None:
            return None

        # 3. Duplicate check - search with the same text used for storage
        existing = self.collection.query(
            query_texts=[memory.to_embedding_text()],
            n_results=3
        )
        if await self._is_duplicate(memory, existing):
            return None

        # 4. Store in ChromaDB
        memory_id = str(uuid.uuid4())
        metadata = memory.model_dump()
        metadata = {k: ("" if v is None else v) for k, v in metadata.items()}
        self.collection.add(
            ids=[memory_id],
            documents=[memory.to_embedding_text()],
            metadatas=[metadata]
        )
        return memory_id
```

SEARCH METHOD

The `search()` method retrieves memories related to a query and returns them as a list of `TaskMemory` objects. When we call `collection.query()`, ChromaDB handles embedding generation, similarity calculation, and sorting. By converting the result metadata to `TaskMemory`, we can access the `task_summary`, `approach`, `final_answer`, `is_correct`, and `error_analysis` fields in a type safe way.

Listing 6.34 `TaskMemoryManager` `search()` method

```
class TaskMemoryManager:
    # ... previous code ...
```

```
    async def search(self, query: str, top_k: int = 5) -> list[TaskMemory]:
        """Search for memories related to the query."""
        results = self.collection.query(
            query_texts=[query],
            n_results=top_k
        )

        if not results["metadatas"] or not results["metadatas"][0]:
            return []

        return [TaskMemory(**meta) for meta in results["metadatas"][0]]
```

INTEGRATING MEMORYMANAGER INTO THE AGENT

We add `memory_manager` to the `Agent` class. The `memory_manager` is optional; for simple tasks that don't require long-term memory, it can be left as `None`. In the `run()` method, we pass the `memory_manager` to `ExecutionContext` so that tools can access it via `context.memory_manager`.

Listing 6.35 Adding `memory_manager` to the `Agent`

```
class Agent:
    def __init__(
        ...
        memory_manager: TaskMemoryManager | None = None,  # Added
    ):
        ...
        self.memory_manager = memory_manager  # Added
```

6.4.5 *Retrieving memories*

The most intuitive way to use stored memories is to implement retrieval as a tool the agent can call when necessary. But what if certain memories should always be retrieved before the LLM sees the request? You might retrieve user profile information at the start of every conversation, for example, or check whether a similar problem has been solved before. You could implement a separate callback for this purpose, but then memory-related logic would be split between a tool and a callback. A cleaner approach is to let the tool itself inject context into the LLM request.

TOOL-BASED RETRIEVAL

We start with a `MemoryTool` that retrieves past problem-solving records. The agent can call this tool to check for similar problems. The `execute()` method searches the memory store and formats results so that the LLM can understand which approaches worked and which failed.

Listing 6.36 `MemoryTool` for explicit retrieval

```
class MemoryTool(BaseTool):
    """Tool for retrieving past problem-solving records"""
```

```
    def __init__(self):
        super().__init__(
            name="recall_memory",
            description=(
                "Search for past problem-solving records. "
                "Use this to check if similar problems were solved before."
            ),
        )

    async def execute(self, context: ExecutionContext, query: str) -> str:
        """Search memories and return formatted results"""
        memories = await context.memory_manager.search(query, top_k=3)

        if not memories:
            return ""

        return self._format_memories(memories)

    def _format_memories(self, memories: list[TaskMemory]) -> str:
        """Format memories for display"""
        results = []
        for i, mem in enumerate(memories, 1):
            status = "Correct" if mem.is_correct else "Incorrect"
            text = f"""[Record {i}]
- Problem: {mem.task_summary}
- Approach: {mem.approach}
- Answer: {mem.final_answer}
- Result: {status}"""
            if not mem.is_correct and mem.error_analysis:
                text += f"\n- Error analysis: {mem.error_analysis}"
            results.append(text)
        return "\n\n".join(results)
```

With this tool registered, the agent can decide when to search for relevant experiences.

AUTOMATIC INJECTION WITH PROCESS_LLM_REQUEST

Tool-based retrieval works well when searching is optional. But for some use cases, you want memories retrieved automatically before every LLM call. Instead of creating a separate callback, override the `process_llm_request()` method in `BaseTool`. This method runs before each LLM request and can modify the request directly.

The key insight is that you can reuse the `execute()` logic inside `process_llm_request()`. Rather than duplicate the search and formatting code, `process_llm_request()` calls `execute()` and appends its result to the LLM's request.

Whether the tool is callable by the LLM or used only for automatic injection depends on `tool_definition`. If `tool_definition` is a dict, the LLM can call it explicitly. If `tool_definition` is `None`, the tool won't appear in the LLM's tool list, but `process_llm_request()` still runs before every LLM call. For simplicity, we use automatic injection here by setting `tool_definition=None`.

Listing 6.37 `MemoryTool` with automatic injection

```
class MemoryTool(BaseTool):
    # ... execute, _format_memories same as above ...

    def __init__(self):
        super().__init__(
            name="recall_memory",
            description="...",
            tool_definition=None,  # Automatic injection only
        )

    async def process_llm_request(
        self,
        context: ExecutionContext,
        request: LlmRequest,
    ) -> None:
        """Inject relevant memories before LLM call"""
        user_msgs = [c for c in request.contents if isinstance(c, Message)
and c.role == "user"]
        if not user_msgs:
            return

        result = await self.execute(context, user_msgs[-1].content)
        if not result:
            return

        request.append_instructions(
            f"""The following are records from similar problems solved in the
past:

<PAST_EXPERIENCES>
{result}
</PAST_EXPERIENCES>

Reference successful approaches and avoid approaches that led to failures."""
        )
```

The only change from the previous version is adding `tool_definition=None` in `init`. This makes the tool invisible to the LLM, but its `process_llm_request()` method still runs before every LLM call, injecting relevant memories automatically. If you want the LLM to call this tool explicitly instead, remove the `tool_definition=None` line.

INTEGRATING INTO THE AGENT

To make `process_llm_request()` work, we modify the `Agent`'s `_prepare_llm_request()` method. In chapter 4, we built the tools list with a simple comprehension:

```
tools = [t.tool_definition for t in request.tools]
```

Now we need two changes:

- Filter out tools in which `tool_definition` is `None` because some tools may provide only automatic injection without being callable by the LLM.

- Call each tool's `process_llm_request()` method. Note that the method becomes async because `process_llm_request()` is asynchronous.

Listing 6.38 Modified `_prepare_llm_request()` for tool injection

```
class Agent:
    # ... existing code ...

    async def _prepare_llm_request(
        self, context: ExecutionContext
    ) -> LlmRequest:
        flat_contents = []
        for event in context.events:
            flat_contents.extend(event.content)

        # Filter tools that should be exposed to the LLM
        llm_tools = [t for t in self.tools if t.tool_definition is not None]

        request = LlmRequest(
            instructions=[self.instructions] if self.instructions else [],
            contents=flat_contents,
            tools=llm_tools,
            tool_choice="auto" if llm_tools else None,
        )

        # Let tools modify the request
        for tool in self.tools:
            await tool.process_llm_request(context, request)

        return request
```

The filtering logic allows a tool to participate in `process_llm_request()` without appearing in the LLM's tool list. `MemoryTool` uses this pattern: it sets `tool_definition=None` so that the LLM can't call it, but its `process_llm_request()` still runs to inject memories automatically.

Summary

- Memory transforms an agent from a stateless tool to an intelligent assistant that learns and adapts across interactions.
- The three use patterns address different challenges. Pattern 1 (one-off execution) manages context within a single run, pattern 2 (continuous execution) maintains conversation across multiple runs, and pattern 3 (long-term memory) preserves knowledge across sessions.
- Context-management strategies follow a hierarchical approach: measure tokens, apply compaction for tool results, and use summarization as a last resort. The key principle is separating storage from presentation: `ExecutionContext` remains immutable, whereas `LlmRequest` is optimized for each call.

- `Session` and `SessionManager` enable multiturn conversations by persisting events and state across `run()` calls. The same architecture supports asynchronous HITL workflows, in which execution pauses for user approval and resumes when confirmation arrives.
- Long-term memory combines write and retrieval strategies. Structured information is extracted from execution history, checked for duplicates, stored in a vector database, and retrieved when it's relevant to new tasks.
- Memory retrieval can be implemented as an explicit tool the agent calls on demand or as automatic injection via `process_llm_request()` that adds relevant context before every LLM call.

7 Planning and reflection for complex tasks

This chapter covers

- The limitations of reactive agents in performing complex tasks
- Planning tools that decompose problems into task lists
- Reflection tools that enable progress checking and failure recovery
- The complementary cycle of planning and reflection

We've built an agent that observes the current situation, decides which tool to call, and repeats this cycle. This reactive approach works well for simple tasks but struggles with complex problems that require multiple steps. Even with the memory systems from chapter 6, the agent can lose direction, fail to use what it has already gathered, or get stuck repeating failed attempts. Memory helps the agent remember what happened. Planning and reflection address these limitations by giving agents time to think. Planning breaks complex problems into manageable tasks before execution. Reflection pauses to check progress and adjust direction when things go wrong.

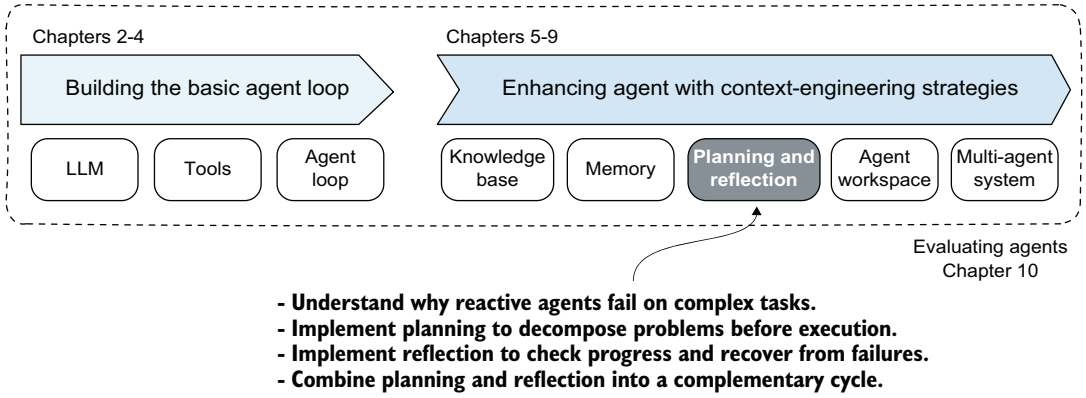

Figure 7.1 The roles of planning and reflection in AI agents

In this chapter (figure 7.1), we'll examine why reactive agents fail and how human experts approach complex tasks differently. Then we'll implement a simple planning tool that records task lists in the context, followed by a reflection tool that enables progress checking and failure recovery. Finally, we combine both strategies into a complementary cycle to see how they resolve the failure modes of reactive agents.

7.1 Giving agents time to think

Planning and reflection are two core strategies that give agents time to think. First, we'll examine the limitations of the ReAct approach; then we'll look for solutions in the way human experts work. We'll also explore how the time-to-think principle, validated in large language model (LLM) research, applies to agents and what roles planning and reflection play from a context-engineering perspective.

7.1.1 The limitations of ReAct

The agent we've implemented operates using the ReAct approach. It observes the current situation, decides which tool to call, observes the result, and decides on the next action. This approach excels at adaptability. If search results differ from expectations, the agent tries different search terms; if a tool fails, it looks for alternatives because it selects whatever action seems most appropriate at each moment.

But this approach is similar to driving without GPS. At every intersection, you go in whichever direction looks best from what you can see right now. This works when the destination is close, but when you have to take a complex route, you can easily get lost.

In complex problems, ReAct agents exhibit several typical failure modes:

- They lose direction by trying to do too much at the same time.
- They execute multiple web searches to quickly retrieve the information needed for a task and then fail to use the results properly.

- They forget what they did in the middle, fail to use information found previously, and repeat searches.
- They hastily declare "I found the answer" but attempt to answer with only partial information.
- They fail to recover from failures.
- When tool calls fail or go in the wrong direction, instead of analyzing the cause and finding alternatives, they repeat the same attempts until reaching `max_steps`.

What do these problems have in common? The agent thinks only about what to do right now; it doesn't consider how to approach this problem overall or whether things are progressing well so far.

7.1.2 How human experts work

Consider how a skilled researcher handles a complex question. When asked "How many hours would it take to reach the Moon at Eliud Kipchoge's marathon pace?", they don't start searching immediately. Instead, they follow these steps:

1. They decompose the question. ("What's Kipchoge's marathon record? I can calculate the pace from that. What's the distance to the Moon? I should use the closest-approach distance.")
2. They investigate one thing at a time, organizing along the way. ("Found the marathon record; converting to kilometers per hour is about 21km/h.")
3. When unexpected results appear, they adjust their approach. ("The Moon distance varies across sources. Should I use the average distance or the perigee? The question asked about closest approach, so I'll use the perigee.")
4. Before the final answer, they check for anything that's missing. ("Marathon record, pace calculation, Moon distance, time calculation—all done.")

The same pattern appears in software development. Experienced developers don't start writing code immediately. First, they write a product requirements document (PRD) or spec document; then they proceed with implementation and testing while referencing that document. When problems arise during implementation, they revisit the spec document and adjust their approach. Coding agents like Claude Code and Cursor follow this method too. When implementing complex features, first they create an implementation plan; then they check progress against the plan at each stage. This workflow of creating a plan and checking progress against it is planning (writing specs) and reflection (checking against specs). Anthropic's agent development guide emphasizes this point:

> *Inspiration for these practices came from knowing what effective software engineers do every day.*

The message is to design agents inspired by how human experts work.

7.1.3 Why time to think matters

A pattern repeatedly confirmed in LLM research is that giving models time to think improves performance compared with requiring immediate answers. Chain-of-thought prompting significantly improves accuracy in solving math problems merely by adding the phrase "Let's think step by step." This works because it makes the model generate intermediate reasoning steps explicitly. Reasoning models took this principle further, generating thinking tokens before answers and greatly enhancing complex problem-solving capabilities.

Planning and reflection apply this principle to agents. *Planning* is time to think before acting ("What should I do? In what order should I proceed?"). *Reflection* is time to look back after acting ("What have I learned so far? Am I on the right track? Should I change direction?"). This grants agents a form of metacognition: the ability to examine their own thinking process.

From a context-engineering perspective, planning and reflection have an interesting commonality: neither performs tasks directly. Neither searches the web, performs calculations, or reads files; instead, they generate text signals that control the flow of work. Planning adds "what to do next" to the context. When text like "1. Research Kipchoge's record, 2. Research Moon distance, 3. Calculate time" enters the context, the LLM references this plan in subsequent steps to maintain direction. Reflection adds "evaluation and direction so far" to the context. Text like "Marathon record acquired. Moon distance research: Wikipedia failed, need to switch to web search" influences the LLM's next decision.

Among the five context-engineering strategies introduced in chapter 1, planning and reflection belong to the generation strategy. Unlike retrieval, which brings in information from external sources, and writing, which stores information externally, this approach has the LLM generate text within the context itself to control the workflow. We'll examine how to implement these two strategies.

7.2 Planning: Setting direction

The core of planning is decomposing complex problems into manageable units. Even the to-do lists we use in daily life are small forms of decomposition. Breaking a large task like "write a report" into "gather materials > write draft > review and revise" makes the task much easier to handle. Let's use the Kipchoge problem as an example:

- *Original question*—"If marathon runner Eliud Kipchoge could maintain his world record marathon pace indefinitely, how many thousand hours would it take him to reach the Moon?"

  What happens if an agent receiving this question starts searching immediately? It searches "Kipchoge moon hours," and when the desired results don't appear, it tries different search terms, easily losing direction in the process. Instead, decomposing the question yields the following.

- *Decomposed tasks*
  - Research Kipchoge's marathon record > calculate pace.

  - Research the distance to the Moon (perigee).
  - Calculate time = distance ÷ speed.
  - Round to thousands of hours.

With this decomposition, each step becomes clear, and the completion criteria are obvious. You can easily judge the success of each step with questions like "Did I find the marathon record?" and "Did I confirm the Moon distance?"

7.2.1 When planning is necessary

Not every problem requires planning. Because planning requires LLM calls, using it adds unnecessary overhead:

- *When planning is necessary*
  - Problems that require multiple steps of research (such as the Kipchoge problem)
  - Problems that require combining multiple pieces of information (e.g., comparing the stock prices of two companies)
- *When planning is unnecessary*
  - Simple questions that can be solved with a single search (e.g., "What's the weather in Seoul today?")
  - Tasks that already have clear procedures (e.g., "Translate this text for me")

Creating a plan for "What's the weather today?" is wasteful. For a problem that can be solved with one search, creating a plan such as "1. Confirm location, 2. Search weather, 3. Organize results" wastes tokens and time.

To have agents make this distinction themselves, the planning tool's description must clearly state when to use it and when not to use it, as we'll see in our implementation.

7.2.2 Implementing the planning tool

Let's create the simplest form of a planning tool. Before we implement it, we need to make a few design decisions:

- *Should we manage only a single plan or multiple plans simultaneously?* In complex systems, we may need to separate main plans from subplans or track multiple goals simultaneously. But in most cases, one plan per question is sufficient. Managing multiple plans requires additional logic to decide which plan to reference. Let's start simple.
- *When modifying a plan, should we update only the changed parts or regenerate the entire thing?* Supporting partial modifications like "change task 3's status to completed" can save tokens. But this approach requires logic to track the current plan state accurately, and bugs resulting from referencing the wrong indices or losing synchronization are common. Instead, regenerating the entire plan each time simplifies implementation. The LLM rewrites the entire plan, saying "the current state is this." For plans with 5–10 tasks, the additional token cost is minimal.

- *How should we represent each task's progress status?* Distinguishing whether a task hasn't started yet (`pending`), is in progress (`in_progress`), or has been `completed` (completed) makes it easy for the LLM to determine what to do next.

TASK DATA STRUCTURE

The `Task` class is a simple data structure that stores task content and status. The `__str__` method applies different checkbox formats for each status, allowing the LLM to grasp progress quickly.

Listing 7.1 Task data structure definition

```
from typing import Literal, List
from pydantic import BaseModel

class Task(BaseModel):
    content: str
    status: Literal["pending", "in_progress", "completed"]

    def __str__(self):
        if self.status == "pending":
            return f"[ ] {self.content}"
        elif self.status == "in_progress":
            return f"[>] **{self.content}**"
        elif self.status == "completed":
            return f"[x] ~~{self.content}~~"
        return self.content
```

Pending tasks display as empty checkboxes `[ ]`, `in_progress` tasks as progress indicators `[>]` with bold text, and completed tasks as completion marks `[x]` with strikethrough. This format is a common pattern in to-do lists, so the LLM understands it easily.

THE CREATE_TASKS TOOL

Now let's implement the planning tool. We can create it using the `@tool` decorator we learned in chapter 4. The tool receives a task list and returns it as a string, and the result is recorded in the context as a `ToolResult`.

The tool's description is the key information the LLM uses to judge "when to use this tool." The description clearly states the following:

- *When to use*—Complex questions that require multiple research steps or combine information from different sources
- *When not to use*—Simple questions that can be solved with a single search or tasks with obvious procedures
- *How to use*—Mark completed tasks as completed and set the next task to `in_progress`.

Writing a detailed description prevents the LLM from calling the planning tool unnecessarily.

Listing 7.2 Planning-tool implementation

```
from typing import List
from scratch_agents.tools import tool

@tool
def create_tasks(tasks: List[Task]) -> str:
    """Create or update a task plan.

    WHEN TO USE:
    - Complex queries requiring multiple steps of research
    - Questions that need to combine information from different sources

    WHEN NOT TO USE:
    - Simple questions answerable with a single search
    - Tasks with obvious, straightforward procedures

    HOW TO USE:
    - Regenerate the entire task list with updated statuses
    - Mark completed tasks as 'completed'
    - Mark the next task to work on as 'in_progress'
    - Keep future tasks as 'pending'
    """
    result = []
    for task in tasks:
        result.append(str(task))
    return "\n".join(result)
```

The tool implementation itself is simple: it receives a task list and converts it to a string to return. The returned string is recorded in the context as a `ToolResult`, allowing the LLM to reference the plan in subsequent steps.

Why implement it so simply? LLMs already have powerful capabilities to understand and generate language. Instead of implementing complex logic, using the LLM's capabilities is more effective. The tool records the plan in the context; the LLM handles planning and state management.

7.2.3 Planning-tool example

Let's add the planning tool to an agent to see how it works.

Listing 7.3 Running an agent with the planning tool

```
from scratch_agents import Agent, LlmClient
from scratch_agents.tools import search_web

agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[create_tasks, search_web],
    instructions="You are a helpful assistant that plans before acting on
     complex queries.",
    max_steps=10,
)
```

```
result = await agent.run(
    "If Eliud Kipchoge could maintain his marathon pace indefinitely, "
    "How many thousand hours would it take him to reach the Moon?"
)
```

When the agent receives this question, it calls `create_tasks` to make a plan. The tool's return value is recorded in the context as a `ToolResult`:

```
Step 1: create_tasks call
Result:
[ ] Find Kipchoge marathon world record time
[ ] Calculate pace in km/h
[ ] Find Earth-Moon distance at perigee
[ ] Calculate time and convert to thousands of hours
```

Then it updates the status as it progresses through each step. When the first task is complete, the agent marks it as `completed` and sets the next task to `in_progress`:

```
Step 2: search_web("Kipchoge marathon world record") call

Step 3: create_tasks call (state update)
Result:
[x] ~~Find Kipchoge marathon world record time~~
[>] **Calculate pace in km/h**
[ ] Find Earth-Moon distance at perigee
[ ] Calculate time and convert to thousands of hours
```

When the plan is recorded in the context, the LLM references it in subsequent steps, like a person looking at a to-do list to decide what to do next. Thanks to the checkbox format, you can see at a glance which tasks are complete, which task is currently in progress, and which tasks remain.

If tasks are too granular, `create_tasks` calls for status updates can become excessive, increasing execution time. Rather than separate every simple action, it's more efficient to group tasks in meaningful units. The LLM decides granularity each time it calls `create_tasks`, but you steer it through the tool's docstring (listing 7.2). The `WHEN TO USE` and `WHEN NOT TO USE` sections set the threshold (multistep research versus single-search questions), and you can bias toward coarser plans by adding a hint such as "Group related lookups into a single task" to the docstring or to the agent's instructions parameter.

7.2.4 Extension directions

The planning tool we've implemented so far is intentionally simple. This tool is sufficient for most cases, but if you have to handle complex tasks, you can extend it in the following directions:

- *Completion criteria*—Explicitly define when each task is complete. Instead of "Research Kipchoge's record," use specific completion conditions such as

"Confirm Kipchoge's marathon record time and distance" so that the LLM can more accurately judge whether a task is complete.

- *Hierarchical structure (task/action)*—Decompose large tasks (tasks) into smaller actions (actions). In a market research task, you might have "Search competitor A," "Search competitor B," and "Compare results" actions.
- *Dependency management (deps)*—Express order relationships between tasks. "Research Kipchoge's record" and "Research Moon distance" are independent and can proceed simultaneously, but "Calculate time" is possible only after both tasks are complete. Making such dependencies explicit enables parallel execution.

Still, these extensions significantly increase complexity. Code is needed to manage dependency graphs, track hierarchical structures, and coordinate parallel execution. It's best to add this code incrementally only when simple planning tools don't solve the problem.

The key is questions, not structure. More important than complex dependency graphs is guiding the LLM to ask itself the right questions. Including questions like the following in the tool's description can be effective:

- What do I need to know to solve this problem?
- What order would be efficient to proceed in?
- How will I know when each step is complete?

7.3 Reflection: Checking and correcting

If planning is making a plan before acting, reflection is stopping in the middle to check. A skilled researcher naturally pauses when search results differ from expectations, when synthesizing multiple sources, or when checking for anything that's missing before the final answer. Researchers don't search, immediately search again, and promptly answer. They ask themselves along the way, "What have I found so far?" and "Am I on the right track?"

ReAct agents lack this checking habit. At each moment, they think only about what to do next and don't ask, "Is this going well so far?" Even when search results are strange, they move on to the next step; even when information conflicts, they proceed. Reflection makes agents ask themselves these questions:

- What have I learned so far?
- How close am I to the original goal?
- Is the current approach effective?
- Does anything have to change?

When the answers to these questions are recorded in the context, the LLM references this information in subsequent decisions. If planning adds "what to do next" to the context, reflection adds "evaluation and direction so far" to the context.

7.3.1 When reflection is necessary

Think about the moments when people naturally pause during work. These moments are exactly when agents need reflection too:

- *When a meaningful step has been completed*—People naturally pause when a step ends. ("First step done. Let me organize before moving on.") Agents can also check progress and prepare for the next step when completing a stage of the plan. This checkpoint is the most predictable one.
- *When an error occurs in a tool*—When a search fails or an API doesn't respond, people think, "Why isn't this working? I should find another way." Without reflection, agents can easily repeat the same failures until reaching `max_steps`. Errors are clear signals, making this moment particularly important for checking.
- *When synthesizing multiple pieces of information*—When a lot of material has been gathered, people pause, saying, "Wait—let me organize this first." Checking is especially important when information is contradictory. When searching for the Moon's distance in the Kipchoge problem, you might get different numbers, such as "average distance 384,400 km" and "perigee 356,500 km, apogee 406,700 km." To judge which value to use, stop and think. ("The problem asked about closest approach, so I should use the perigee.")
- *When a review is needed before the final answer*—Just before answering, people check, "Let me make sure nothing is missing." To prevent agents from hastily declaring "I found the answer," a self-verification step is required before completion. This checkpoint is the most important one but is easily overlooked.

WHEN REFLECTION IS UNNECESSARY

Not every moment requires checking. Doing reflection after every tool call only increases overhead. During simple, straightforward operations, reflection creates an unnecessary delay. If everything is proceeding as expected, continuing is more efficient.

Summarization (covered in chapter 6) is different from reflection. Summarization is compression that triggers automatically when context length exceeds a threshold. Its purpose is to reduce the token count. By contrast, the LLM selectively executes reflection based on the judgment that checking is necessary. Its purpose is to check direction and make corrections if necessary.

7.3.2 Implementing the reflection tool

As with the planning tool, we implement the reflection tool simply. Instead of creating complex logic in code, we use the LLM's judgment capabilities.

The `reflection` parameter is free-form text. Because different content is required depending on the situation (error analysis, information synthesis, progress checking, and so on), forcing structure would reduce flexibility.

The `need_replan` flag is a coordination signal with the planning tool. When the current plan is judged to be no longer valid, setting this flag to `True` encourages modifying the plan in the next step.

The tool's description specifies four use situations: `PROGRESS REVIEW` (when completing a step), `ERROR ANALYSIS` (when a tool fails), `RESULT SYNTHESIS` (when synthesizing information), and `SELF CHECK` (before the final answer). Including specific examples of each situation guides the LLM to call the tool at appropriate times.

Listing 7.4 Reflection tool implementation

```
from scratch_agents.tools import tool

@tool
def reflection(analysis: str, need_replan: bool = False) -> str:
    """Pause and analyze progress before continuing.

    WHEN TO USE:
    1. PROGRESS REVIEW - After completing a meaningful step
       "Kipchoge's record found: 2:01:09. Moving to moon distance research."

    2. ERROR ANALYSIS - When a tool fails or returns unexpected results
       "Wikipedia tool failed. Cause: service unavailable. Alternative: use
     web search."

    3. RESULT SYNTHESIS - When combining information from multiple sources
       "Two different moon distances found. Problem asks for closest
     approach, so using perigee: 356,500km."

    4. SELF CHECK - Before providing final answer
       "Have all required data: marathon pace 20.81km/h, moon distance
     356,500km. Ready to calculate."

    WHEN NOT TO USE:
    - After every single tool call (excessive overhead)
    - During simple, straightforward operations
    - When everything is proceeding as expected

    Args:
        analysis: Your assessment of current situation and next direction
        need_replan: Set True if the current plan needs modification
    """
    if need_replan:
        return f"Reflection recorded (REPLAN NEEDED): {analysis}"
    return f"Reflection recorded: {analysis}"
```

The tool implementation itself is simple. It receives the analysis content and returns it as a string. This result is recorded in the context as a `ToolResult`, which the LLM references in subsequent decisions.

If the planning tool records "what to do next" in the context, the reflection tool records "evaluation so far" in the context. Both generation strategies add LLM-generated text to the context to influence subsequent decisions.

7.3.3 The real value of reflection: Failure recovery

Reflection's value becomes apparent not when everything is going smoothly, but when problems arise. Let's intentionally create a failure to see reflection's effect.

Create a Wikipedia tool that always fails. In reality, external APIs can fail for various reasons: server overload, network issues, API changes, and so on.

Listing 7.5 A Wikipedia tool that always fails

```
@tool
def get_wikipedia_page(title: str) -> str:
    """Get a Wikipedia page by title."""
    raise RuntimeError("Wikipedia service is temporarily unavailable")
```

Add the reflection and Wikipedia tools to an agent and run them.

Listing 7.6 Running an agent with reflection for error recovery

```
agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[get_wikipedia_page, search_web, reflection],
    instructions="Always try Wikipedia first when researching topics.",
    max_steps=5,
)

result = await agent.run(
    "What is the distance from Earth to Moon at perigee?"
)
```

When run, the agent calls `get_wikipedia_page`, receives the `RuntimeError`, and then calls reflection with an analysis like "Wikipedia tool failed. Cause: service unavailable. Alternative: use web search." This analysis is the `ERROR ANALYSIS` pattern from listing 7.4's docstring. That recorded analysis steers the next step toward `search_web` instead of retrying Wikipedia. Without reflection or without precise error information for the LLM to analyze the cause, the agent would likely repeat the failing call until reaching `max_steps`.

7.3.4 Running an agent that uses reflection for research synthesis

Reflection is useful not only for recovering from failure but also for synthesizing multiple pieces of information. The following listing is an example of organizing search results.

Listing 7.7 Running an agent with reflection for research synthesis

```
agent = Agent(
    model=LlmClient(model="gpt-5-mini"),
    tools=[search_web, reflection],
    instructions="You are a research assistant.",
```

```
)

result = await agent.run(
    "Research recent developments in quantum computing."
)
```

After the agent performs multiple searches, it synthesizes the results through reflection. Without reflection, the agent might have kept adding searches or answered without organizing the collected information properly. Through reflection, the judgment "I've gathered enough information" is explicitly recorded in the context, guiding the next step to generate an answer.

7.4 Integrating planning and reflection

Planning and reflection don't operate separately; they form a single cycle, as shown in figure 7.2. Let's see how these two strategies connect and how they resolve ReAct's limitations.

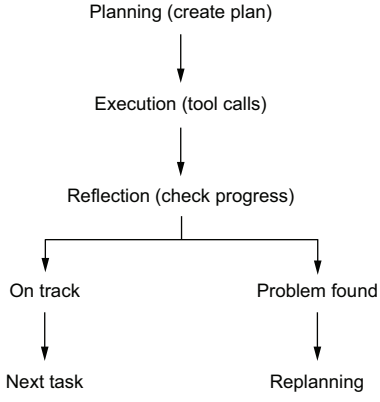

Figure 7.2 The planning-reflection cycle in agent execution

First, the agent decomposes tasks and establishes order through planning. Then it executes work by calling tools according to the plan. After execution, it checks progress through reflection. If the check shows that things are going well, it moves to the next task; if a problem has occurred, it modifies the plan.

The `need_replan` parameter of the implemented reflection tool serves as this connecting link. When reflection determines that the current plan is no longer valid, it sets `need_replan=True`. When this signal is recorded in the context, the LLM calls `create_tasks` again in the next step to modify the plan.

Consider the Wikipedia tool's failure in the Kipchoge problem. On reflection, you might analyze "Wikipedia service unavailable, need to switch to web search" but set `need_replan=False`. The plan itself—research Moon distance—is still valid. Only the method has to change. On the other hand, if search results revealed "Kipchoge holds the half-marathon record, not the marathon record," you should set `need_replan=True`. Because the premise of the question itself has changed, the entire plan must be reconsidered.

In this way, planning and reflection form a complementary cycle. Planning provides direction; reflection checks that direction and, when necessary, returns to planning.

Section 7.1.1 identified four failure modes of the reactive ReAct loop: losing direction, hastily declaring an answer, forgetting work in progress, and failing to recover from errors. With both planning and reflection on hand, we can revisit each one and see which capability addresses it.

The first failure mode was "losing direction by trying to do too much at the same time." This happens when agents execute multiple searches simultaneously and fail to use the results properly. Planning solves this problem. It decomposes tasks into clear units and provides a structure for proceeding one unit at a time. When tasks are decomposed as "1. Research Kipchoge's record, 2. Research Moon distance, 3. Calculate," the agent moves to the second task only after completing the first.

The second failure mode is "hastily declaring that an answer has been found." The agent attempts to answer with partial information before gathering all the necessary details. Planning solves this problem by specifying completion criteria, and the `SELF CHECK` step of reflection asks, "Have I secured all the information?" The "Before providing final answer" timing, included in the reflection tool's description, serves this role.

The third failure mode is "forgetting what was done in the middle," when the agent fails to use previously found information and repeats the same searches while going through multiple steps. Planning mitigates this problem by recording the status of each task in the plan to track how far we've gotten. Reflection explicitly records what we've learned so far in the context through `PROGRESS REVIEW`. When this text remains in the context, the LLM references it in subsequent decisions.

The fourth failure mode is "failing to recover from failures." The agent repeats the same attempts until reaching `max_steps` instead of analyzing causes and finding alternatives when tool calls fail or go in the wrong direction. This is reflection's core value: `ERROR ANALYSIS` analyzes failure causes and suggests alternatives. When an analysis like "Wikipedia service unavailable. Cause: server overload. Alternative: use web search" is recorded in the context, the LLM tries alternatives instead of repeating the same failure. Table 7.1 summarizes this content.

Table 7.1 ReAct failure modes and solutions

Failure mode	Planning solution	Reflection solution
Trying too much at the same time	Decompose tasks; proceed one at a time.	
Premature completion declaration	Specify completion criteria.	Self-verification (`SELF CHECK`)
Losing direction/forgetting	Record status in plan.	Progress check (`PROGRESS REVIEW`)
Unable to recover from failure		Cause analysis; suggest alternatives (`ERROR ANALYSIS`)

Planning solves mainly "looking ahead" problems, and reflection solves "looking back" problems. Some problems are solved through collaboration. A skilled researcher doesn't search immediately. They decompose the question (planning), investigating one thing at a time while organizing along the way (reflection), and adjusting their approach when unexpected results appear (replanning). The integration of planning and reflection implements this natural working style in agents.

In chapter 8, we'll explore how to give agents code-execution capabilities through a sandbox environment. Code expresses procedures for performing tasks in a programming language, which inherently shares characteristics with the planning covered in this chapter. The sandbox becomes a new workspace for the agent, much as computers are for humans.

Summary

- Planning and reflection give agents time to think. Instead of reacting moment by moment, agents plan before acting and check after acting. This grants metacognition: the ability to examine their own process.
- Planning decomposes complex problems into clear, manageable units. When "1. Research Kipchoge's record, 2. Research Moon distance, 3. Calculate time" is recorded in the context, the LLM references this plan to maintain direction across multiple steps.
- Reflection is valuable when problems arise, not when everything goes smoothly. When tools fail or results are unexpected, reflection enables cause analysis and alternative strategies instead of repeating the same failures.
- Planning and reflection form a complementary cycle. Planning provides direction; reflection checks that direction and, when necessary, triggers replanning. Neither works in isolation.
- From a context-engineering perspective, both strategies are Generation strategies. Planning adds "what to do next" to the context, while reflection adds "evaluation and direction so far." These texts influence the LLM's subsequent decisions.

8 Empowering agents with code execution

This chapter covers

- Understanding the limitations of predefined tool approaches and the benefits of code execution
- Connecting code-execution environments to agents
- Porting tools to sandboxes for efficient execution
- Providing complete workspaces with filesystems and the CLI
- Using agent skills for hierarchical tool management

So far, we've built agents that register tools and have the large language model (LLM) select and call appropriate tools. This approach is predictable and easy to control, but it reveals clear limitations when applied to complex real-world problems. Let's explore a new approach. Instead of creating individual functions as tools, we'll provide an execution environment in which the LLM can write and run code directly. This is what it means to give an agent a computer.

As shown in figure 8.1, we're ready to enhance agents with context-engineering strategies. We'll focus on agent workspace, giving agents the same working environment that human developers use.

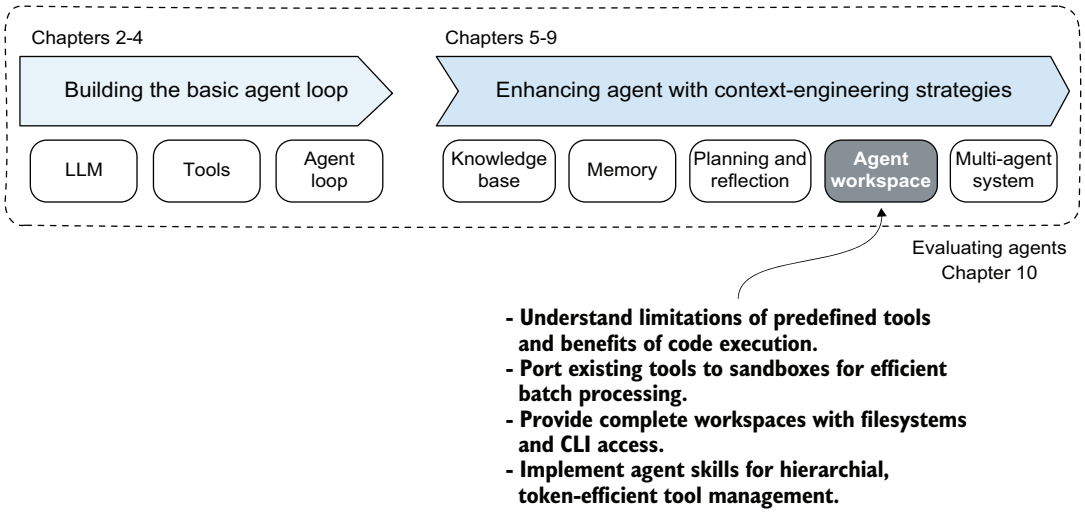

Figure 8.1 Chapter 8: providing complete workspaces

We'll begin by examining the limitations of predefined-tool approaches and exploring the powerful capabilities of code execution. Then we'll see how to connect sandboxed code execution environments to agents safely using E2B and how to port existing tools to sandboxes for efficient batch processing. We'll go beyond code execution to provide complete workspaces with filesystems and access to a command-line interface (CLI). Finally, we'll learn about agent skills, a hierarchical approach to tool management that maximizes token efficiency.

8.1 Giving agents a computer

The tool-calling approach limits agent behavior to predefined functions, making control easy but revealing clear limitations in solving complex real-world problems. The action space is narrow, and task execution is inefficient. To solve this problem, we'll have the LLM generate and execute code directly instead of calling tools one by one. We'll explore how code enables handling complex logic and data, examine the performance improvements provided by CodeAct, review the security concerns of this approach, and discuss the new architectural challenge of tool portability.

8.1.1 Limitations of the predefined-tool approach

Let's reflect on the agents we've built so far. In chapter 3, we defined tools that perform specific functions, such as `search_web`, `read_file`, and `calculator`. In chapter 4,

we implemented an `Agent` that has the LLM select and call appropriate tools based on the situation. We can call this approach the predefined-tool approach. The developer defines tools in advance, and the agent operates only within that list.

Now let's consider a fundamentally different approach. Instead of creating individual functions as tools one by one, we provide an execution environment in which the LLM can write and execute code directly. Technically, this approach still involves calling a tool like `execute_python`, but it's fundamentally different because the LLM can do anything within that environment.

Why does this difference matter? Let's examine the limitations of the predefined-tool approach.

LIMITED ACTION SPACE

The first limitation of the predefined-tool approach is that the agent's action space is strictly limited to the tool list defined by the developer. The agent can't perform any operation that doesn't exist in the list.

Suppose that an agent is given only `read_file` and `write_file` tools. What happens when a user requests "Check what files are in the current folder"? The agent can't fulfill this request because there's no `list_files` tool. When user's demands become even slightly complex or unexpected situations arise, the agent gets stuck and can't solve the problem on its own.

It would be nice, of course, if developers could anticipate all edge cases and necessary functions and create them as tools in advance, but this is practically impossible. User demands are infinitely diverse, and new situations can arise at any time.

INEFFICIENT TASK EXECUTION

The second limitation is that this approach can be inefficient, depending on the nature of the task. The predefined-tool approach operates on the basic unit "one LLM call, one tool execution, one observation." Although recent LLMs increasingly support parallel tool calling, which provides some improvement, typically, only five or six tool calls occur per LLM invocation.

Consider the approach of our current agent. If it were tasked with analyzing 10 files, at each step it would call the LLM, check the result, and decide on the next action. Even reading and processing 10 files requires multiple LLM calls, creating significant latency:

```
Action: read_file("file_1.txt")
Observation: File contents returned.

Action: read_file("file_2.txt")
Observation: File contents returned.

... (repeat 8 more times)

Action: analyze_data(...)
Observation: Analysis results returned.
```

Because all tool results go directly into the context, they can increase costs and reduce speed. Suppose that you don't need the entire contents of a text file; you only need to check the links it contains. Unless you create a separate tool that extracts only links from files, you must add all the file contents to the context and have the agent check for links.

TOOL PROLIFERATION AND CONTEXT COSTS

The third limitation stems from the fact that doing more things requires registering more tools. The emergence of Model Context Protocol (MCP), covered in chapter 3, has made it much easier to reuse and integrate tools, but it creates a new problem: a single MCP server contains multiple tools. The GitHub MCP server, for example, provides dozens of tools for repository search, issue creation, PR (pull request) checking, and more. Connecting only a few servers rapidly increases the number of tools available to the agent. In practice, adding a few MCP servers to products like Claude Code can add thousands to tens of thousands of tokens to the prompt.

When the number of tools and their definitions occupy a significant portion of the prompt, two side effects occur:

- Longer prompts increase LLM API call costs.
- Giving the LLM too many options at the same time degrades its ability to accurately select the most appropriate tool for the situation. The probability that the LLM will make mistakes is higher with 100 tools than with 10.

8.1.2 What code execution brings

What if we wrote the same task as Python code? If the LLM generates code like the following in a single call, all the work can be completed with one LLM call and one code execution. This scenario is the core reason why agents need code-execution capability. Instead of having the LLM call tools one by one, we have it write a program that performs the entire task. In the earlier example that requires only links from a file, we can filter the file contents using regular expressions without creating a separate tool.

Listing 8.1 Processing multiple files with a single code execution

```
results = []
for i in range(1, 11):
    content = read_file(f"file_{i}.txt")
    results.append(content)

analysis = analyze_data(results)
print(f"Analysis result: {analysis}")
```

Beyond efficiency, code inherently has rich expressive power. Three core capabilities in particular increase the possibility of solving more complex problems: control flow, data flow, and tool composition. We cover these capabilities in the following sections.

CONTROL FLOW

The most basic yet powerful capability of programs is control flow. You can select different paths based on conditions, repeat specific tasks, and handle exceptional situations.

Consider a scenario of adding only high-rated products within budget to a shopping cart in an online store. Assume `search_products`, `get_price`, and `add_to_cart` tools are available. With the predefined-tool approach, you would have to check each product one by one. But if you can use code, you can handle complex logic all at once.

Listing 8.2 Processing complex business logic with control flow

```
budget_remaining = 5000
purchased_items = []

for product in search_products("laptop"):
    price = get_price(product)

    if price <= budget_remaining and product.rating >= 4.5:
        add_to_cart(product)
        budget_remaining -= price
        purchased_items.append(product)
```

Conditionals and loops combine to express complex business logic naturally. Budget checking, rating filtering, and cart addition happen within a single code block.

DATA FLOW: STORING INTERMEDIATE RESULTS

The second core capability is storing intermediate results. If you can store values in variables to transform and pass along, you don't have to make fresh calculations every time, which makes things much more efficient. Also, the output of one tool becomes the input of the next, and multiple results can be combined to create new insights.

Listing 8.3 Efficient data transformation through an intermediate variable

```
raw_data = fetch_sales_data("2024-Q1")

cleaned_data = raw_data[raw_data['amount'] > 0]
grouped = cleaned_data.groupby('category').agg({
    'amount': 'sum',
    'quantity': 'count',
    'customer_id': 'nunique'
})
```

In the preceding code, `raw_data`, `cleaned_data`, and `grouped` hold meaningful intermediate results that are reused in later steps. With the predefined-tool approach, you'd have to pass the entire context to the LLM at each step and then decide on the next action.

TOOL COMPOSITION

The third and most powerful capability is combining tools to create new functionality. Like assembling LEGO blocks, you can combine small functions to create more complex tools. Python's powerful ecosystem provides an excellent foundation for implementing new functionality. You can analyze data with pandas, visualize it with Matplotlib, and build simple models with scikit-learn. Using this composition capability, you can combine existing tools (such as `search_news`, `get_stock_price`, `analyze_sentiment`, and `create_report`) to create more complex company-analysis tools (such as `analyze_company`).

Listing 8.4 Creating new tools through tool composition

```
def analyze_company(company_name):
    """Simple company analysis combining news and stock data"""

    news = search_news(company_name, limit=10)
    stock_data = get_stock_price(company_name, days=30)

    summary = {
        'price_change': stock_data['change_percent'],
        'recent_news': [n['title'] for n in news[:3]],
        'sentiment': analyze_sentiment(news)
    }

    report = create_report(data=summary)

    return report
```

Figure 8.2 summarizes the four approaches to performing the same task. Each approach builds on the preceding one, progressively expanding the agent's action space:

- *Simple tool call*—Executes each tool independently. This approach is straightforward and easy to debug but limited to one action at a time, with no data flow between tools.
- *Tool call + code (*`loop` *and* `if/else`)—Adds control flow, letting the agent repeat actions or branch based on conditions. Use this approach when tasks require conditional logic or iteration, such as retrying a search until the results are satisfactory.
- *Tool call + code (tool composition)*—Goes further by combining tools into a pipeline, with the output of one tool feeding directly into the next. This approach is useful when multiple tools have to work together in sequence, such as fetching and then analyzing data.
- *Tool creation by code*—Gives the agent full flexibility to connect tools freely and create new tools through code. Choose this approach for complex workflows when predefined tools are insufficient and the agent has to build custom solutions on the fly.

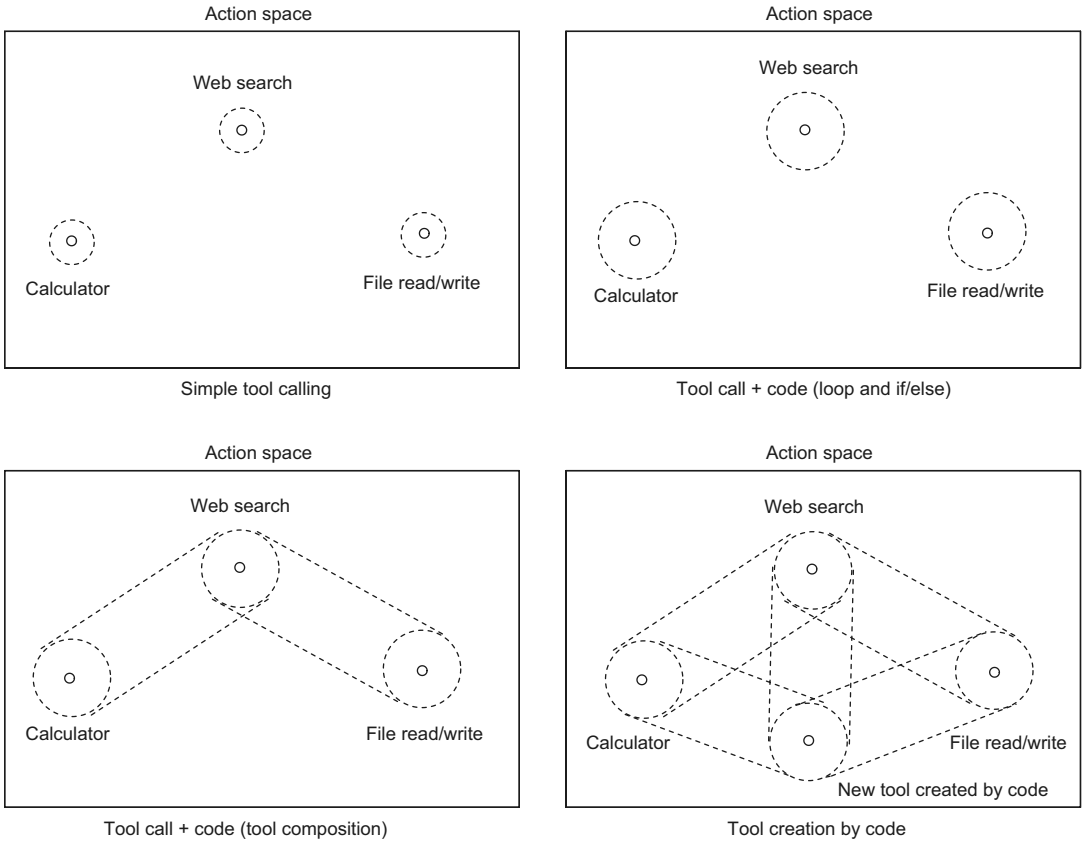

Figure 8.2 Comparison of tool-execution approaches, from simple tool calling to tool creation through code

8.1.3 The effectiveness of code-based actions

The CodeAct research published in 2024 (https://arxiv.org/abs/2402.01030) posed an important question about how LLM agents express their actions: "What difference would there be if we had the same LLM solve the same problem, but only varied how actions are expressed?" The researchers had the models call tools in three formats:

- *JSON format*—This approach, adopted by many commercial APIs, expresses actions as structured data.
- *Text format*—This approach, close to natural language, is easy for humans to read but difficult to parse.
- *Code format (CodeAct)*—This approach, which expresses actions as Python code, uses the full expressiveness of programming languages.

Figure 8.3 compares the three approaches. Having the model take actions through code resulted in the highest success rate and required the fewest LLM calls.

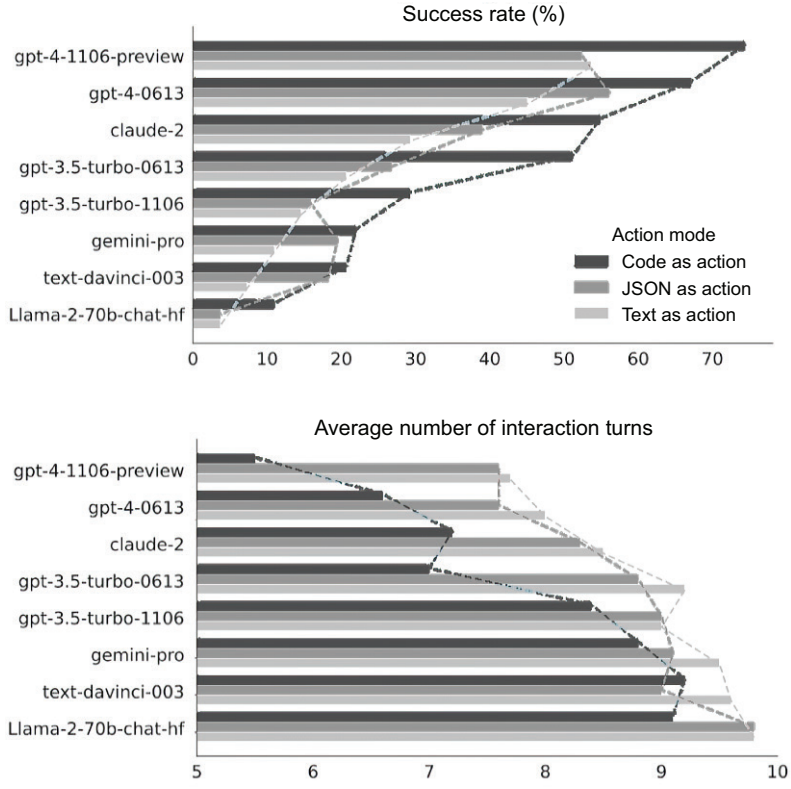

Figure 8.3 CodeAct achieved higher success rates with fewer interactions than JSON and text formats.

M3ToolEval, the benchmark used in this study, consists of 82 complex real-world tasks. It includes domains such as web browsing, financial analysis, travel planning, scientific computation, and information processing, with each task requiring combinations of multiple tools.

Also, 12 of 17 LLMs showed the highest success rates when using CodeAct. Following are the results for the GPT-4 model (gpt-4-1106-preview):

- *CodeAct*—74.4% success rate, average 5.5 interactions
- *JSON*—52.4% success rate, average 7.6 interactions
- *Text*—53.7% success rate, average 7.7 interactions

These results mean that CodeAct achieved performance improvement of more than 20% and reduced interactions by 30%. An interesting pattern was discovered: more powerful models tended to use CodeAct's benefits better. This pattern suggests that code-writing ability is closely related to the model's overall reasoning capabilities.

The researchers analyzed failure cases to identify the limitations of the JSON and text formats. Frequent problems included failure to use intermediate results properly,

difficulty expressing conditional logic like "if A then B, otherwise C," and having to handle each iteration individually when repeating the same task multiple times. By contrast, CodeAct reused values naturally by storing them in variables, handled branching with conditionals, and solved repetition with loops.

The key insight from CodeAct is that LLMs can directly leverage programming languages they already know, allowing the approach to be applied immediately without additional training. Because the code itself represents the plan and procedure for actions, fewer LLM calls are required to complete the task. Although CodeAct uses Python, the code-execution approach itself isn't limited to any single language. Python is the practical choice because LLMs are most proficient at generating Python code, and its rich ecosystem of libraries makes it ideal for data processing, API calls, and automation tasks.

8.2 Connecting code environments

Now let's add code-execution functionality to the agent we've built. First, we'll examine why sandbox environments are necessary for safe code execution, learn the basics of the E2B service, and connect code execution to our `Agent` class. Then we'll verify that our implementation works correctly through simple tests.

8.2.1 Why sandboxes are necessary

Code execution greatly expands an agent's capabilities, but this power comes with risk. Executing code generated by an LLM means allowing unpredictable behavior. LLMs can unintentionally generate dangerous code. An `os.remove()` command might target the wrong path, an infinite loop could exhaust system resources, and memory-intensive code might freeze the entire system. We also can't rule out the possibility that malicious users will deliberately try to execute harmful code through prompt injection.

THE NEED FOR ISOLATION

The most effective way to mitigate these risks is to isolate the code-execution environment. Running code in an isolated environment, or *sandbox*, provides important protections. A sandbox is isolated from external systems so that code can be executed safely. A sandbox provides several layers of protection that reduce the risks of executing LLM-generated code:

- *Security:* Sandboxes block access to sensitive files on the host system. Authentication keys in the `.ssh/` directory, system files like `/etc/passwd`, and API keys stored in environment variables are protected. We can enhance security by restricting which domains the sandbox environment can access.
- *Resource management:* Sandboxes can limit CPU use, memory allocation, and execution time. Even if an infinite loop occurs, it will be forcibly terminated by a timeout, and code that uses excessive memory will stop at the limit. This prevents the entire host system from being affected.

- *Recoverability*: If something goes wrong in a sandbox environment, you can easily restore it to its initial state. Even if files are deleted or the environment becomes corrupted, creating a new sandbox gives you a chance to start fresh.

SANDBOX OPTIONS

There are several ways to build a sandbox environment. You can manage Docker containers directly, but doing so requires complex setup and manual configuration of resource limits, network isolation, and more. Cloud sandbox services such as E2B and Modal abstract away this complexity, allowing you to create isolated environments and execute code through simple API calls.

This book uses E2B for state persistence, along with its simple API. E2B sandboxes maintain variables and functions defined once throughout the session. Just as when we run Jupyter Notebook cells in sequence, we can use results from preceding code executions immediately in subsequent executions. This means we can use the data flow advantages emphasized in section 8.1.2. With the Python SDK, you can create a sandbox, execute code, and retrieve results in a few lines of code.

8.2.2 Introduction to E2B

To use E2B, you must sign up for the service and get an API key. After signing up at https://e2b.dev/sign-up, generate a new key from the API Key section of the dashboard. Add the generated key to your project's `.env` file:

```
E2B_API_KEY=your_api_key_here
```

Next, install the E2B package:

```
uv add e2b-code-interpreter
```

BASIC FLOW

The E2B flow is straightforward: create a sandbox, execute code, check results, and terminate the sandbox. The following listing demonstrates this process. `Sandbox.create()` creates an isolated Python environment in the cloud, and `run_code()` executes code. Execution results are returned in an `execution` object, and when work is complete, `kill()` terminates the sandbox.

Listing 8.5 Basic E2B sandbox flow

```
from e2b_code_interpreter import Sandbox

# Create sandbox
sandbox = Sandbox.create()
print(f"Sandbox ID: {sandbox.sandbox_id}")

# Execute code
code = """
```

```
result = 2 + 3
print(f"Calculation: 2 + 3 = {result}")
result
"""
execution = sandbox.run_code(code)

# Check results
print("Output:", execution.logs.stdout)
print("Return value:", execution.results[0].text)

# Terminate sandbox
sandbox.kill()
```

Execution results from `run_code()` can be checked through three properties. `execution.logs.stdout` contains content printed by `print()` statements, and `execution.results` returns a list of the evaluated results of the code's last expression. In the preceding listing, the last line `result` is evaluated, and `"5"` is stored in `results[0].text`. The `execution.error` property contains error information if an error occurred during execution and is `None` on successful execution.

TIMEOUT AND ENVIRONMENT VARIABLE SETTINGS

Sandboxes terminate automatically after a certain period. You can set this period with the `timeout` parameter `Sandbox.create()`. The unit is seconds, and this time doesn't reset even when code is executing. If necessary, you can extend the timeout with `sandbox.set_timeout()`.

When you have to call external APIs from within the sandbox, you can pass sensitive information such as API keys as environment variables. Setting environment variables in dictionary form with the `envs` parameter makes them accessible through `os.environ` in code running inside the sandbox.

Listing 8.6 Configuring timeout and environment variables

```
import os

# 5-minute timeout, environment variable settings
sandbox = Sandbox.create(
    timeout=300,
    envs={
        'TAVILY_API_KEY': os.getenv('TAVILY_API_KEY'),
        'MY_CONFIG': 'some_value',
    }
)

# Extend timeout if needed
sandbox.set_timeout(600)  # Extend to 10 minutes
```

STATE PERSISTENCE

One key advantage of the E2B sandbox is the fact that state is maintained. When you call `run_code()` multiple times within the same sandbox, variables and functions

defined in previous executions can be used directly in subsequent executions. In the next listing, the first execution defines the `data` variable, and the second execution reuses it. If state weren't maintained, the second execution would produce an error message saying that `data` can't be found.

Listing 8.7 State persistence across multiple code executions

```
# First execution: define variable
sandbox.run_code("data = [1, 2, 3]; print('Data preparation complete')")

# Second execution: reuse variable
execution = sandbox.run_code("sum(data)")
print("Sum:", execution.results[0].text)  # 6
```

This behavior is identical to running Jupyter Notebook cells in sequence. This characteristic is extremely useful when an agent processes data across multiple steps or performs additional analysis based on previous calculation results.

Error handling

When an error occurs during code execution, error information is stored in `execution.error`. Even if an error occurs, the sandbox remains usable, and you can execute subsequent code without any problem. You can check the error type with `error.name` and the error message with `error.value`.

Listing 8.8 Handling execution errors

```
execution = sandbox.run_code("x = 1 / 0")
if execution.error:
    print("Error type:", execution.error.name)    # ZeroDivisionError
    print("Error message:", execution.error.value)  # division by zero
```

From an agent's perspective, this error information is crucial. When code written by the LLM contains bugs, passing the error message to the LLM allows it to generate corrected code. This is how the ReAct pattern's observation > retry cycle works in code-execution environments.

8.2.3 Connecting the sandbox to the agent

Now that we've learned how to use E2B, let's add code-execution functionality to the `Agent` class. The goal is to integrate code execution naturally while maintaining the existing `Agent` interface as much as possible. We want developers to set `code_execution="e2b"` and have the agent create a sandbox, execute code, and clean up when it's finished. This task requires three modifications:

- Add a field to `ExecutionContext` to store the sandbox instance.
- Add a parameter to the `Agent` constructor to accept code-execution options.
- Manage the sandbox lifecycle in the `Agent.run()` method.

EXTENDING EXECUTIONCONTEXT

The `ExecutionContext` implemented in chapter 4 is the central repository holding all the state of agent execution. We add a `code_env` field here to store the sandbox instance. When a tool executes, it can access the sandbox through `ExecutionContext.code_env`. This field is declared as an `Optional` type, remaining `None` for agents for which code execution isn't enabled.

Listing 8.9 Adding the `code_env` field to the `ExecutionContext`

```
@dataclass
class ExecutionContext:
    ...
    code_env: Optional["Sandbox"] = None  # Added
```

MODIFYING THE AGENT CLASS

Add a `code_execution` parameter to the `Agent` class constructor. Currently, only `"e2b"` is supported, but we accept it as a string to allow adding other sandbox services in the future.

Listing 8.10 Adding the `code_execution` parameter to the `Agent` constructor

```
class Agent:
    def __init__(
        ...
        code_execution: Optional[str] = None,  # Added
    ):
        self.code_execution = code_execution
        self.tools = self._setup_tools(tools or [])
```

Modify the `_setup_tools()` method to automatically add the `execute_python` tool when `code_execution="e2b"`. Then code execution is available without developers having to register the tool explicitly. We'll implement `execute_python` shortly.

Listing 8.11 Autoregistering the `execute_python` tool in `_setup_tools`

```
def _setup_tools(self, tools: List[BaseTool]) -> List[BaseTool]:
    tools = list(tools)

    if self.code_execution == "e2b":
        from .tools.code_execution import execute_python
        tools.append(execute_python)

    return tools
```

Manage the sandbox life cycle in the `run()` method. Create the sandbox at the start of execution, and clean up the sandbox when execution ends (whether successful or failed). Using a `try-finally` construct ensures that resources aren't leaked even if an exception occurs.

Listing 8.12 Managing the sandbox lifecycle in the `run()` method

```
async def run(
    self,
    user_input: str,
    context: Optional[ExecutionContext] = None
) -> AgentResult:
    if context is None:
        context = ExecutionContext()

    # Initialize sandbox
    if self.code_execution == "e2b" and context.code_env is None:
        from e2b_code_interpreter import Sandbox
        context.code_env = Sandbox.create(timeout=300)

    try:
        # Existing agent execution logic
        ...
    finally:
        # Clean up sandbox
        if context.code_env is not None:
            context.code_env.kill()
```

IMPLEMENTING THE EXECUTE_PYTHON TOOL

Finally, implement the `execute_python` tool that executes code. This tool retrieves the sandbox instance from `ExecutionContext`, executes the code, and returns the results in JSON format. Returning the results in JSON format allows the LLM to distinguish among and understand stdout, return values, and error information.

Listing 8.13 Implementing the `execute_python` tool

```
@tool(
    name="execute_python",
    description="Execute Python code in a sandboxed environment. "
                "Use this to perform calculations, data processing, or any
Python operations."
)
def execute_python(context: ExecutionContext, code: str) -> str:
    if context.code_env is None:
        raise RuntimeError("No code execution environment available.")

    sandbox: Sandbox = context.code_env
    execution = sandbox.run_code(code)

    return json.dumps(execution.to_json(), indent=2, ensure_ascii=False)
```

MAKING DESIGN DECISIONS

Why automatically add the tool in `_setup_tools()`? We do that to improve the developer experience. With `code_execution="e2b"` set, code execution is immediately available without separate imports or tool registration. Because `execute_python` is included in the tool list, the LLM recognizes that the tool exists and can call it when necessary.

Why pass the sandbox through `ExecutionContext`? We do that to reuse the pattern established in chapter 4. Having tools access execution state through `ExecutionContext` is the consistent approach in our framework. Also, because the sandbox is stored in `ExecutionContext`, even when a tool is called multiple times within a single agent execution, the calls use the same sandbox, and state is maintained.

8.2.4 Testing the code-execution agent

With implementation complete, let's verify that it works with a simple test. We ask the agent to find the 100th Fibonacci number. This problem is difficult for an LLM to answer accurately on its own because the 100th Fibonacci number is a large number with 21 digits.

Listing 8.14 Testing the code-execution agent

```
from scratch_agents import Agent, LlmClient

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[],  # No other tools
    instructions="You can execute Python code to solve problems.",
    code_execution="e2b"
)

result = await agent.run("What is the 100th Fibonacci number?", verbose=True)
print(result.output)
```

When the agent receives this request, it writes and executes Python code to calculate the Fibonacci sequence. It might generate code like this:

```
def fibonacci(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

result = fibonacci(100)
print(result)
```

The execution produces the exact value `354224848179261915075`. The LLM's reasoning ability (deciding what code to write), combined with the code's precise calculation capability, solved a problem that would be difficult to solve with either the LLM or code execution alone.

8.3 Porting tools to sandboxes

Now the agent can write and execute Python code, but there's one limitation: it can't directly call the tools we built in previous chapters from code inside the sandbox.

Let's explore how to make existing tools executable within the sandbox. The goal is to achieve both the flexibility of tool calling and the efficiency of code execution.

8.3.1 The need for tool portability

Consider the following General AI Assistants (GAIA) benchmark problem: "How many times was a Twitter/X post cited as a reference on the English Wikipedia pages for each day from August 1 through August 31 in the last June 2023 versions of the pages?"

This problem contains an interesting requirement: we have to check the "last version from June 2023," not just the current Wikipedia page. This problem can't be solved with a regular web search or the current Wikipedia API. We need a custom tool that uses Wikipedia's revision API to query page versions from before a specific point in time. Let's compare two methods for solving this problem.

The first approach is to register `get_wikipedia_revisions` as a regular agent tool and have the agent call it 31 times—once for each day from August 1 to August 31. This works, but its limitations are immediate. As shown in the top part of figure 8.4, each daily query runs through the full loop (LLM call > tool calling > tool execution > result) and then the process repeats. In other words, you end up invoking the LLM 31 separate times for 31 dates. Further, each tool call brings back the full Wikipedia page content and adds it to the agent's context. Because a typical Wikipedia page can be thousands to tens of thousands of tokens, accumulating content from 31 runs makes token usage explode.

The second approach is to move the logic into sandbox-executed code. Instead of having the agent repeatedly decide and call the tool day by day, we invoke the LLM once to generate a program that performs the iteration. As the bottom part of figure 8.4 illustrates, the LLM produces code that contains the loop (e.g., `for i in range(0, 31): execute tool`), and the sandbox executes that loop internally. The key difference is that the 31 iterations happen inside the sandbox, not through 31 separate LLM calls.

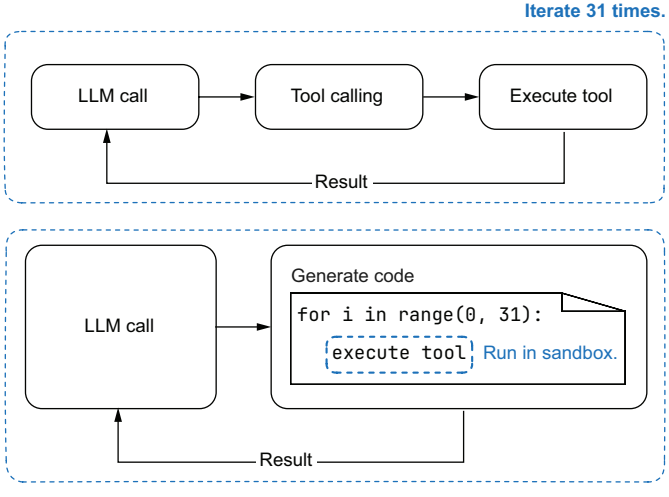

Figure 8.4 Tool calling versus code execution. Repeated tool calls require 31 LLM iterations, whereas code execution handles the same task in a single LLM call.

Also, rather than returning entire page contents to the model each time, the sandbox processes the pages locally and returns only the small extracted outputs, such as the Twitter/X link counts, to the LLM. This keeps the context small and makes token use far more efficient.

A core question arises: how can we execute a Python function defined in our local environment in a remote sandbox? The E2B sandbox runs as an isolated environment in the cloud. The functions we define exist only in local memory. Connecting these two is the problem we'll solve in this section.

8.3.2 Extending FunctionTool

Porting all tools to the sandbox is complex. We'll limit this implementation to a practical scope.

We'll limit tools to `FunctionTool`. The `FunctionTool` implemented in chapter 4 wraps pure Python functions. Python's `inspect.getsource()` function can extract a function's source code as a string. By sending this source code to the sandbox, we can define the same function in the remote environment. MCP tools and complex class-based tools are difficult to port this way, but because most custom tools are defined as `FunctionTool`, this choice is practical.

Also, we limit to tools that don't need `ExecutionContext`. The sandbox is an independent environment that can't access the agent's `ExecutionContext`. In chapter 4, some tools received a context parameter to access the execution state, but such tools can't run in the sandbox. Only pure functions for which the output is determined solely by input parameters can be ported. This constraint is verified explicitly to prevent developer mistakes.

MODIFYING FUNCTIONTOOL

Add a `sandbox_executable` flag to the `FunctionTool` class. Tools with this flag set to `True` can be called directly from within the sandbox. At initialization time, we verify that there's no conflict with the context parameter and immediately raise an error for invalid combinations.

Listing 8.15 Adding the `sandbox_executable` flag to `FunctionTool`

```
class FunctionTool(BaseTool):
    def __init__(
        ...
        sandbox_executable: bool = False,
    ):
        ...
        self.sandbox_executable = sandbox_executable

        # sandbox_executable tools cannot have context
        if sandbox_executable and self.needs_context:
            raise ValueError(
                f"Tool '{func.__name__}' cannot be sandbox_executable "
                "because it requires 'context' parameter."
            )
```

SOURCE CODE EXTRACTION METHOD

To register a tool in the sandbox, we need the function's source code. The `get_source_code()` method extracts the source code using `inspect.getsource()` and removes the `@tool` decorator. The decorator is framework-specific and not necessary in the sandbox. If the tool decorator remains, an error will occur because it's not defined in the sandbox environment.

Listing 8.16 Extracting the function source code for sandbox registration

```
def get_source_code(self) -> str:
    """Extract function source code for sandbox registration."""
    if not self.sandbox_executable:
        raise ValueError(f"Tool '{self.name}' is not marked as sandbox_
executable")

    source = inspect.getsource(self.func)

    # Remove @tool decorator
    lines = source.split('\n')
    filtered_lines = []
    skip_decorator = False

    for line in lines:
        stripped = line.strip()
        if stripped.startswith('@tool'):
            skip_decorator = True
            # Check if single-line decorator
            if '(' not in stripped or ')' in stripped:
                skip_decorator = False
            continue
        if skip_decorator:
            if ')' in stripped:
                skip_decorator = False
            continue
        filtered_lines.append(line)

    return '\n'.join(filtered_lines)
```

8.3.3 Implementing the Wikipedia revision tool

Wikipedia provides all edit history through an API. By combining the `action=query`, `prop=revisions`, and `rvstart` parameters, you can query page versions from before a specific point in time. Setting `rvstart=2023-07-01T00:00:00Z`, for example, retrieves the version from before July 1, 2023, meaning the "last version from June 2023."

TOOL IMPLEMENTATION

We implement a tool marked with `sandbox_executable=True`. This tool is a pure function that operates only on input, without a context parameter. Note that the `import` statement is inside the function. Because the tool must run independently in the sandbox, the function imports the required modules internally.

Listing 8.17 Wikipedia revision tool with the `sandbox_executable` flag

```
@tool(sandbox_executable=True)
def get_wikipedia_revisions(
    title: str,
    before_date: str,
    limit: int = 10,
    language: str = "en",
    output: str = "full",
) -> list:
    import requests

    api_url = f"https://{language}.wikipedia.org/w/api.php"
    params = {
        "action": "query",
        "titles": title,
        "prop": "revisions",
        "rvprop": "ids|timestamp|user|content",
        "rvlimit": limit,
        "rvstart": before_date,
        "rvdir": "older",
        "format": "json",
    }

    response = requests.get(api_url, params=params)
    data = response.json()

    pages = data.get("query", {}).get("pages", {})
    for page_id, page_data in pages.items():
        if page_id == "-1":
            return []
        revisions = page_data.get("revisions", [])
        if output == "content":
            return [rev.get("*", "") for rev in revisions]
        return revisions

    return []
```

CHARACTERISTICS OF SANDBOX_EXECUTABLE TOOLS

This tool differs from regular tools in several ways:

- *There's no context parameter.* A pure function doesn't depend on agent state.
- *The import statement is inside the function.* Because the statement must run independently in the sandbox environment, external dependencies are resolved within the function.
- *The return value is clear.* It returns structured data that can be used immediately in LLM-generated code.

8.3.4 Handling sandbox tools in the Agent

Add a separate list to the `Agent` class to manage sandbox tools. These tools run as code inside the sandbox instead of as regular tool calls.

Listing 8.18 Adding a sandbox tools list to the `Agent`

```
def __init__(self, ...):
    self._sandbox_tools: List[FunctionTool] = []
    self.tools = self._setup_tools(tools or [])
```

TOOL CLASSIFICATION LOGIC

Modify the `_setup_tools()` method to identify and separate tools with `sandbox_executable=True`. If a `sandbox_executable` tool exists but code execution isn't enabled, raise an error. This check helps developers discover configuration mistakes quickly.

Listing 8.19 Classifying `sandbox_executable` tools in `_setup_tools`

```
def _setup_tools(self, tools: List[BaseTool]) -> List[BaseTool]:
    tools = list(tools)

    # Identify sandbox_executable tools and stack any errors
    invalid_sandbox_tools = []
    for t in tools:
        if not isinstance(t, FunctionTool) or not t.sandbox_executable:
            continue
        if self.code_execution != "e2b":
            invalid_sandbox_tools.append(t.name)
            continue
        self._sandbox_tools.append(t)

    if invalid_sandbox_tools:
        raise ValueError(
            f"Tools {invalid_sandbox_tools} are marked as sandbox_executable "
            "but code_execution is not enabled."
        )

    # Add code execution tool
    if self.code_execution == "e2b":
        from .tools.code_execution import execute_python
        tools.append(execute_python)

    return tools
```

REGISTERING TOOLS IN THE SANDBOX

When the sandbox is created, immediately execute the source code of the `sandbox_executable` tools to define the functions. Then LLM-generated code can call these functions directly.

Listing 8.20 Registering sandbox tools during agent execution

```
async def run(self, user_input: str, ...):
    # Initialize sandbox
    if self.code_execution == "e2b" and context.code_env is None:
```

```
            from e2b_code_interpreter import Sandbox
            context.code_env = Sandbox.create(timeout=300)

            # Register sandbox_executable tools to sandbox
            if self._sandbox_tools:
                self._register_sandbox_tools(context.code_env)

    def _register_sandbox_tools(self, sandbox) -> None:
        """Register source code of sandbox_executable tools to sandbox."""
        tool_sources = []
        for t in self._sandbox_tools:
            source = t.get_source_code()
            tool_sources.append(source)

        combined_source = "\n\n".join(tool_sources)
        result = sandbox.run_code(combined_source)

        if result.error:
            raise RuntimeError(f"Failed to register sandbox tools: {result.error}")
```

INFORMING THE SYSTEM PROMPT ABOUT TOOLS

Add guidance to the system prompt so that the LLM knows about functions available in the sandbox. Providing tool signatures and descriptions in JSON format allows the LLM to use these functions naturally when generating code. This prompt tells the LLM, "These functions can be called directly in the sandbox." The LLM can use them directly in code without separate tool calls.

Listing 8.21 Generating the system prompt for sandbox-available tools

```
def _get_sandbox_tools_prompt(self) -> str:
    """Generate prompt describing tools available in sandbox."""
    if not self._sandbox_tools:
        return ""

    tool_definitions = [t.tool_definition for t in self._sandbox_tools]
    tools_json = json.dumps(tool_definitions, indent=2, ensure_ascii=False)

    return (
        "\n\n## Sandbox-Executable Tools\n"
        "The following functions are pre-registered in the sandbox "
        "and can be called directly in your Python code:\n"
        f"{tools_json}"
    )
```

8.3.5 *Practical testing*

Let's apply our implementation to the representative problem. Register `get_wikipedia_revisions` as a `sandbox_executable` tool, and create an agent with code execution enabled.

Listing 8.22 Setting up the agent with `sandbox_executable` tools

```
from scratch_agents import Agent, LlmClient

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[get_wikipedia_revisions],  # sandbox_executable tool
    instructions="""You can execute Python code to solve problems.
    Use the get_wikipedia_revisions function to fetch Wikipedia page content
    from specific dates.""",
    code_execution="e2b"
)
question = """How many times was a Twitter/X post cited as a reference
on the English Wikipedia pages for each day of August in the last
June 2023 versions of the pages?"""

result = await agent.run(question, verbose=True)
```

PROBLEM-SOLVING PROCESS

When the problem is passed to the agent, the LLM generates Python code that uses loops. The generated code will be similar to the following:

```
import re

twitter_count = 0

for day in range(1, 32):
    title = f"August_{day}"
    revisions = get_wikipedia_revisions(
        title=title,
        before_date="2023-07-01T00:00:00Z",
        limit=1,
        output="content"
    )

    if revisions:
        content = revisions[0]
        # Extract only Twitter/X reference patterns
        twitter_refs = re.findall(
            r'twitter\.com|x\.com',
            content,
            re.IGNORECASE
        )
        twitter_count += len(twitter_refs)
        print(f"{title}: {len(twitter_refs)} Twitter references")

print(f"\nTotal Twitter references: {twitter_count}")
```

The entire page content is processed only inside the sandbox. One number per page and the final total are returned to the context.

WHEN THIS APPROACH IS USEFUL

This approach is suitable in the following cases:

- *Processing large amounts of data when only aggregations or summaries are needed*—The example of extracting only link counts from 31 Wikipedia pages is a perfect case.
- *Multistep workflows requiring three or more dependent tool calls*—In suitable cases, tool results must be filtered, sorted, or transformed; intermediate data shouldn't influence LLM reasoning; and parallel work is required on many items, such as checking 50 endpoints.

Conversely, there are cases where this approach is not suitable:

- *Single tool calls that return simple responses*—In this case, regular tool calling is fine. Tools that require immediate user feedback aren't suitable because they must wait for code execution to complete.
- *Very fast operations in which code execution overhead outweighs the benefits*—For these tasks, the overhead of starting code execution may exceed the actual work being done. Keeping these criteria in mind when designing tools can greatly increase agent efficiency.

8.4 Workspace: Using filesystems and the CLI

If we think about how a real developer works, code execution alone isn't enough. Developers save files, execute commands in the terminal, and save intermediate results to files. In this section, we explore how to give agents this complete working environment: the workspace.

8.4.1 The workspace concept

Workspace is a complete environment that combines code execution, a filesystem, and a CLI. With a workspace, agents can go beyond simply executing code to manage files and use various CLI tools.

Even without a `list_files` tool, you can generate `os.listdir()` code. Using the rich functionality of the CLI, you can use `grep`, `find`, `curl`, and more without separate implementation. Developers don't have to anticipate all edge cases; the LLM generates the required functionality directly.

If a user requests "Find all log files containing the word 'error' in this folder," the agent can generate and execute code like the following.

Listing 8.23 Generating file-search functionality dynamically with code

```
error_files = []
for log_file in glob.glob("**/*.log", recursive=True):
    with open(log_file, 'r') as f:
        if 'error' in f.read().lower():
            error_files.append(log_file)

print(f"Found {len(error_files)} files containing 'error': {error_files}")
```

You could use the CLI to do the same thing more simply:

```
grep -r -l "error" --include="*.log" .
```

In the E2B sandbox, these workspace capabilities appear concretely as CLI access and filesystem operations. First, you can execute CLI commands with `commands.run()`. Any command you use in the terminal (including `ls`, `cat`, `grep`, and `pip`) can be executed inside the sandbox, and the results can be retrieved:

```
result = sandbox.commands.run("ls -la")
print(result.stdout)
```

Then you can upload local files to the sandbox with `files.write()`. You can transfer data files the agent needs to analyze or scripts it needs to execute to the sandbox:

```
with open("data.xlsx", "rb") as f:
    sandbox.files.write("/home/user/data.xlsx", f)
```

8.4.2 Implementing workspace tools

Figure 8.5 shows how the agent uses bash and file tools to interact with the sandbox's CLI and filesystem.

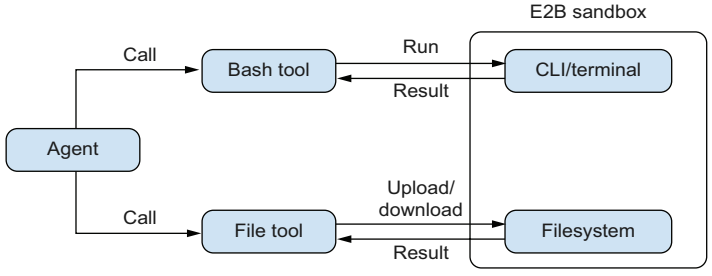

Figure 8.5 Workspace tools. The agent calls bash and file tools to interact with the CLI and filesystem inside the E2B sandbox.

We implement `bash_tool` to execute CLI commands. It calls E2B's `commands.run()` method and formats the execution results into a readable format. The `_format_command_result` function separates stdout, stderr, error messages, and exit codes so that the LLM can clearly understand the results.

Listing 8.24 Implementing `bash_tool` for CLI command execution

```
from e2b_code_interpreter import Sandbox

@tool(name="bash_tool", description="Execute a bash command in a sandboxed
environment.")
def bash_tool(context: ExecutionContext, command: str) -> str:
```

```
    """Execute a bash command in E2B sandbox."""
    if context.code_env is None:
        raise RuntimeError("No code execution environment available.")

    sandbox: Sandbox = context.code_env
    result = sandbox.commands.run(command)

    return _format_command_result(result)

def _format_command_result(result) -> str:
    """Format E2B CommandResult to readable text."""
    parts = []

    if result.stdout:
        parts.append(f"[stdout]\n{result.stdout}")

    if result.stderr:
        parts.append(f"[stderr]\n{result.stderr}")

    if result.error:
        parts.append(f"[error]\n{result.error}")

    parts.append(f"[exit_code] {result.exit_code}")

    return "\n\n".join(parts) if parts else "[No output]"
```

IMPLEMENTING UPLOAD_FILE

Next, we implement the `upload_file` tool, which uploads local files to the sandbox. It takes a local file path and a destination path within the sandbox; then it transfers the file. If no sandbox path is specified, the file is saved by default in `/home/user/` with the original filename. If the file doesn't exist or the path isn't a file, an error message is returned.

Listing 8.25 Implementing `upload_file` for file transfer to the sandbox

```
import os

@tool(name="upload_file", description="Upload a local file to the E2B sandbox
environment.")
def upload_file(
    context: ExecutionContext,
    local_path: str,
    sandbox_path: str | None = None
) -> str:
    """Upload a local file to the E2B sandbox."""
    if context.code_env is None:
        raise RuntimeError("No code execution environment available.")

    sandbox: Sandbox = context.code_env

    if not os.path.exists(local_path):
        return f"Error: Local file not found: {local_path}"
```

```
    if not os.path.isfile(local_path):
        return f"Error: Path is not a file: {local_path}"

    if sandbox_path is None:
        filename = os.path.basename(local_path)
        sandbox_path = f"/home/user/{filename}"

    try:
        with open(local_path, "rb") as file:
            sandbox.files.write(sandbox_path, file)

        return f"Successfully uploaded '{local_path}' to '{sandbox_path}'"

    except Exception as e:
        return f"Error uploading file: {str(e)}"
```

MODIFYING THE AGENT CLASS

Finally, we modify the `Agent` class's `_setup_tools()` method to automatically register the `execute_python` tool when `code_execution="e2b"` is set. Workspace tools such as `bash_tool` and `upload_file` are registered explicitly when the agent needs CLI and file-transfer capabilities. In section 8.2, we added the `execute_python` tool in a similar way, so we won't examine the code directly here.

8.4.3 Practical example: Analyzing an Excel file

Let's solve a GAIA benchmark problem using the workspace tools we've implemented. Here's a problem that requires analyzing a Microsoft Excel file: "The attached spreadsheet contains the sales of menu items for a regional fast-food chain. Which city had the greater total sales: Wharvton or Algrimand?" To solve this problem, we have to read the Excel file, aggregate sales by city, and compare the total sales of the two cities.

We create an agent with workspace functionality enabled. Setting `code_execution="e2b"` automatically registers `execute_python`. To use workspace capabilities for CLI commands and file transfer, register `bash_tool` and `upload_file` explicitly.

Listing 8.26 Setting up an agent with workspace for Excel analysis

```
from scratch_agents import Agent, LlmClient
from scratch_agents.tools.code_execution import bash_tool, upload_file

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[bash_tool, upload_file],
    instructions="""You have access to a sandboxed Linux environment.
    You can upload files, run bash commands, and execute Python code.
    Use these capabilities to analyze data and solve problems.""",
    code_execution="e2b"
)

prompt = f"""The attached spreadsheet contains the sales of menu items
for a regional fast-food chain. Which city had the greater total sales:
```

```
Wharvton or Algrimand?

The file is located at: ./gaia_workspace/7cc4acfa-63fd-4acc-a1a1-e8e529e0a97f.xlsx
"""

result = await agent.run(prompt, verbose=True)
print(result.output)
```

During the problem-solving process, the agent uploads the local Excel file, installs the necessary libraries, and analyzes the data using pandas. Based on this analysis, it calculates each city's sales.

In chapter 5, we implemented filesystem tools directly in the local environment. We wrote Python functions such as `list_files` and `read_file` and registered them with the agent. This approach works but has several constraints. Supporting new file formats requires adding tools each time. In chapter 5, reading Excel files required implementing a separate `_read_excel` function. Because the agent executes directly in the local environment, this approach introduces security risks. The agent could accidentally modify or delete important files.

The workspace approach is different. Inside the sandbox, you can install packages immediately with `pip install`. You can use all of pandas' functionality without a separate tool implementation. Because the sandbox is isolated from the local system, installing packages and running code doesn't affect the host environment. State is maintained during the session, so results from previous steps can be used immediately in the next step.

8.5 Agent skills

Agents can now write and execute Python code, upload files, and run bash commands, but the problem of tool proliferation and context costs mentioned in section 8.1.1 remains.

We explained that although MCP's emergence has made it much easier to reuse and integrate tools, it creates a new problem: a single MCP server contains multiple tools, and connecting a few servers rapidly increases the number of tools available to the agent. When the number of tools and their definitions occupy a significant portion of the prompt, LLM API call costs increase, and the LLM's ability to accurately select appropriate tools degrades.

A fundamental question arises: if an agent can use 100 tools, should detailed definitions for all 100 tools always be in the prompt? Most requests require only one or two of those tools, so having the remaining 98 definitions consume tokens every time is inefficient.

To solve this problem, Anthropic introduced agent skills in 2025. Agent skills maximizes token efficiency by progressively disclosing tool information and solves tool-portability problems through a folder-based structure.

8.5.1 The agent skills concept

Think about a new employee joining a company. They don't read all the company's work manuals on the first day. Instead, they find and study documents related to their

assigned tasks. Documents are usually organized in folders by topic, with each folder containing an overview document that helps the new employee grasp the big picture before diving into the details.

Agent skills follows this approach. Instead of delivering detailed use instructions for all tools to the LLM from the start, first we provide only "these tools exist" information. Then we have the LLM read the detailed documentation for a tool when it determines that the specific tool is required.

FOLDER-BASED STRUCTURE

In agent skills, each skill is organized as a single folder. Each folder must contain a `SKILL.md` file, which provides the overall description of the skill. If code is required to execute the skill, scripts are added within the folder, and their use is documented in `SKILL.md`.

Because all content is stored in a single folder this way, skills can be easily copied and used in other environments or with other agents. This structure naturally solves the tool-portability problem.

PROVIDING SUMMARIES WITH YAML FRONT MATTER

Each `SKILL.md` file begins with YAML front matter (figure 8.6). The skill's name and short description are written here. Only the front matter's `name` and `description` are included in the prompt. The LLM receives only information like "there's a skill called PDF, used when processing PDF files." The actual use is learned when the LLM determines that a skill is required and reads the entire `SKILL.md` file.

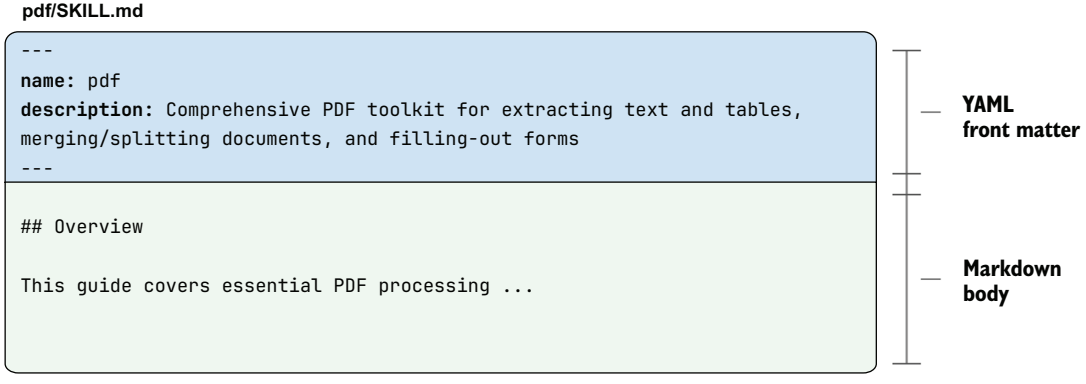

Figure 8.6 YAML front matter in SKILL.md providing name and description for prompt inclusion

8.5.2 Progressive tool-information disclosure

The core mechanism of agent skills is progressively disclosing tool information. This allows token use to be divided into three levels.

LEVEL 1: SKILL LIST (HUNDREDS OF TOKENS)

Only skill names and one-line descriptions are included in the system prompt. Even with 10 skills, only a few hundred tokens are used. With this information alone, the LLM can understand what skills exist and what each is for. For most requests, the LLM looks at this list (figure 8.7) and determines whether a required skill exists.

Level	File	Context window	# Tokens
1	SKILL.md Metadata (YAML)	Always loaded	~100
2	SKILL.md Body (markdown)	Loaded when skill triggers	<5k
3+	Bundled files (text files, scripts, data)	Loaded as needed by Claude	unlimited*

Figure 8.7 Level 1: Skill summary list in system prompt showing minimal token use (Source: https://mng.bz/Ow5n)

LEVEL 2: SKILL DETAILED DOCUMENTATION (THOUSANDS OF TOKENS)

When the LLM determines that a specific skill is required, it uses `bash_tool` to read that skill's `SKILL.md`. Then the LLM learns the complete use of that skill. It obtains detailed information, including code examples, option explanations, and caveats. Thousands of tokens may be added at this stage, but this approach is efficient because only the necessary skills are read.

LEVEL 3: ADDITIONAL DOCUMENTS AND EXECUTION (TENS OF THOUSANDS OF TOKENS)

Complex skills can include additional documents or scripts beyond `SKILL.md`. If `SKILL.md` states "See ADVANCED.md for advanced usage," the LLM reads additional documents only when necessary. Also, the actual work is performed by executing Python scripts included in the skill.

Let's compare token use in the traditional approach and the agent skills approach. In the traditional approach, if there are 100 tools and each tool definition averages 500 tokens, 50,000 tokens are included in the prompt for every request. This is true even if only one or two tools are used.

In the agent skills approach, only the names and descriptions of 100 skills (50 tokens each) are included in the prompt, using 5,000 tokens. Reading the detailed documentation for the one or two skills needed (2,000 tokens each) brings the total to about 9,000 tokens. This saves more than 80% of tokens compared with the traditional approach. The core approach of agent skills is that many tokens are used only when tools are truly needed and detailed use information is required.

8.5.3 Implementation

Let's integrate agent skills into our agent framework. Figure 8.8 shows the runtime architecture: equipped skills are listed in the agent configuration, while the full skill directories live in the agent computer's filesystem.

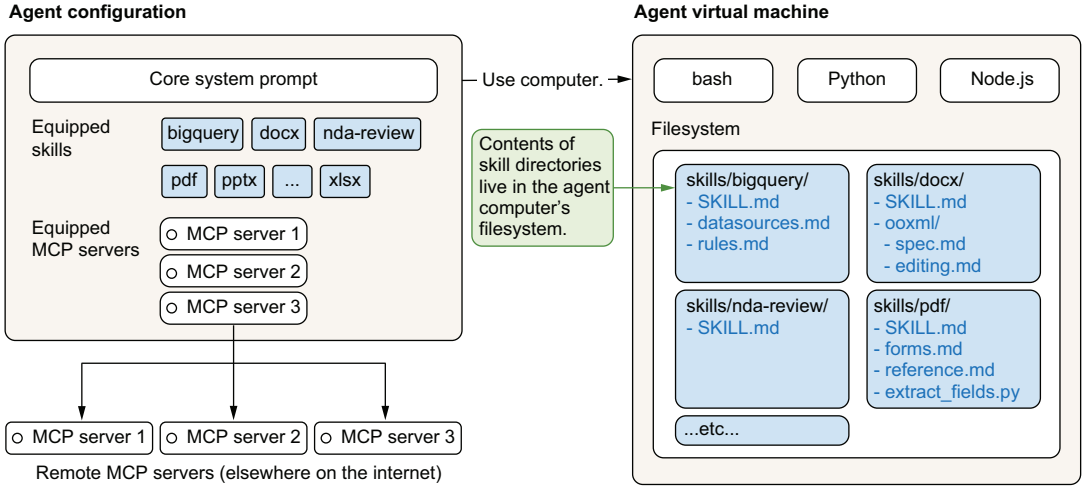

Figure 8.8 Agent skills implementation architecture showing `SkillInfo`, loader, and `Agent` integration

SKILLINFO DATA CLASS

We define a simple data class to hold the core skill information. `name` and `description` are summary information included in the prompt, and `path` is the skill folder's path.

Listing 8.27 `SkillInfo` data class for skill metadata

```
from dataclasses import dataclass
from pathlib import Path

@dataclass
class SkillInfo:
    """Class to hold skill information."""
    name: str
    description: str
    path: Path
```

SKILL LOADER

We implement functions to parse YAML front matter from `SKILL.md` files and create `SkillInfo` objects. The `parse_frontmatter` function extracts the YAML block surrounded by `---` at the beginning of the Markdown file, and the `load_skill` function reads `SKILL.md` from the skill directory and returns a `SkillInfo`.

Listing 8.28 Parsing YAML front matter and loading skill information

```
import re
from typing import Optional

def parse_frontmatter(content: str) -> dict:
    """Extract YAML frontmatter from markdown file."""
    pattern = r'^---\s*\n(.*?)\n---'
    match = re.match(pattern, content, re.DOTALL)

    if not match:
        return {}

    result = {}
    for line in match.group(1).split('\n'):
        if ':' in line:
            key, value = line.split(':', 1)
            result[key.strip()] = value.strip().strip('"\'')

    return result

def load_skill(skill_dir: Path) -> Optional[SkillInfo]:
    """Read SKILL.md from skill directory and return SkillInfo."""
    skill_md = skill_dir / "SKILL.md"

    if not skill_md.exists():
        return None

    content = skill_md.read_text(encoding='utf-8')
    frontmatter = parse_frontmatter(content)

    name = frontmatter.get('name')
    description = frontmatter.get('description')

    if not name or not description:
        return None

    return SkillInfo(name=name, description=description, path=skill_dir)
```

The `discover_skills` function explores all skills within a specified directory and returns them as a list.

Listing 8.29 Discovering all skills in a directory

```
from typing import List

def discover_skills(skills_path: str | Path) -> List[SkillInfo]:
    """Discover all skills in the skills directory."""
    skills_path = Path(skills_path)

    if not skills_path.exists():
        return []

    skills = []
```

```
    for item in skills_path.iterdir():
        if item.is_dir():
            skill = load_skill(item)
            if skill:
                skills.append(skill)

    return sorted(skills, key=lambda s: s.name)
```

SYSTEM-PROMPT GENERATION

We implement a function to add the skill list to the system prompt. It includes only each skill's name and description, along with guidance on how to use them.

Listing 8.30 Generating skills section for system prompt

```
def generate_skills_prompt(
    skills: List[SkillInfo],
    sandbox_path: str = "/home/user/skills"
) -> str:
    """Generate skill guidance to include in system prompt."""
    if not skills:
        return ""

    lines = [
        "## Available Skills",
        f"Skills are installed at `{sandbox_path}`. "
        "Read the SKILL.md file before using a skill.",
        ""
    ]

    for skill in skills:
        lines.append(f"- **{skill.name}**: {skill.description}")

    lines.append("")
    lines.append("To use a skill:")
    lines.append(f"```bash")
    lines.append(f"cat {sandbox_path}/<skill-name>/SKILL.md")
    lines.append(f"```")

    return "\n".join(lines)
```

AGENT CLASS INTEGRATION

We add a `skills_path` parameter to the `Agent` class and upload skill folders when creating the sandbox. The skill list is included when generating the system prompt.

Listing 8.31 Adding the `skills_path` parameter to the `Agent` constructor

```
class Agent:
    def __init__(
        ...
        skills_path: str | None = None,  # Added
```

```
    ):
        ...
        self.skills_path = skills_path
```

In the `run()` method, skill folders are uploaded after the sandbox is created. The repository implementation uses `discover_skills()` to find skill folders and uploads their files to the sandbox.

Listing 8.32 Uploading skills to the sandbox during agent execution

```
async def run(self, user_input: str, ...):
    if self.code_execution == "e2b" and context.code_env is None:
        context.code_env = Sandbox.create(timeout=300)

        # Upload skill folders to sandbox
        if self.skills_path:
            self._upload_skills_to_sandbox(context.code_env)
```

8.5.4 Practical example: PDF merge skill

Let's use the agent skills we implemented. We'll create a simple skill that merges PDF files into one and registers it with the agent.

SKILL STRUCTURE

The skills folder contains a `pdf-merge` skill with the following structure:

```
skills/
└── pdf-merge/
    └── SKILL.md
```

The contents of `SKILL.md` are as follows. Following a summary of the `pdf-merge` skill, it explains how to use the `pypdf` library to merge multiple input PDF files into one. We assume that `pypdf` is already available in the sandbox image (E2B's default Python image ships with it); the `SKILL.md` file documents only how the agent should call it, not how to install it.

Listing 8.33 `SKILL.md` file for the `pdf-merge` skill

```
---
name: pdf-merge
description: Merge multiple PDF files into a single PDF document. Use when
the user wants to combine, merge, concatenate, or join PDF files together
into one file.
---

# PDF Merge

Merge multiple PDFs into a single document using pypdf.

## Usage
```

```python
from pypdf import PdfWriter, PdfReader

writer = PdfWriter()
pdf_files = ["file1.pdf", "file2.pdf", "file3.pdf"]

for pdf_file in pdf_files:
    reader = PdfReader(pdf_file)
    for page in reader.pages:
        writer.add_page(page)

with open("merged.pdf", "wb") as output:
    writer.write(output)
```

We create an agent by specifying the skill folder path:

```
from scratch_agents import Agent, LlmClient

agent = Agent(
    model=LlmClient(model="gpt-5"),
    tools=[],
    instructions="You have access to a sandboxed Linux environment.",
    code_execution="e2b",
    skills_path="./skills"
)
```

EXECUTION FLOW

When a user requests "Merge report_q1.pdf, report_q2.pdf, and report_q3.pdf into one," the agent operates as follows:

1. Determines from the skill list in the system prompt that `pdf-merge` is appropriate
2. Executes `cat /home/user/skills/pdf-merge/SKILL.md` with `bash_tool`
3. Reads `SKILL.md` content and learns `pypdf` use
4. Writes and executes `pdf-merge` code with `execute_python`
5. Confirms that the result file `merged.pdf` is created and responds to the user

Detailed documentation for the `pdf-merge` skill is read only when that functionality is needed. For other requests, the agent determines from the one-line description that "this skill isn't needed now" and moves on.

8.5.5 Hierarchical tool structure

Let's review what we've learned and organize the tool structure we've built. Agent tools can be divided into three layers.

LEVEL 1: CORE TOOLS

A small number of basic tools are always included in the prompt when workspace functionality is enabled. `execute_python`, `bash_tool`, and `upload_file` belong here. These

tools have fixed schemas, making them cache friendly, and they serve as gateways that extend to all other functionality:

- `execute_python`—Executes Python code to perform level-3 functionality
- `bash_tool`—Reads level-2 skill documents via CLI commands or uses system tools
- `upload_file`—Transfers local files to the sandbox

LEVEL 2: INSTALLABLE TOOLS (AGENT SKILLS)

The agent skills belong here. Only tool summaries (name and description) are included in the prompt; detailed definitions are stored in the workspace filesystem. The LLM uses level 1's `bash_tool` to read skill documents and learn how to use them when necessary.

LEVEL 3: GENERATED TOOLS

One-time functionality isn't defined in advance. The LLM writes code as needed on the spot and executes it with level 1's `execute_python`. Because the Python ecosystem can be used to implement various functionalities, countless functions can be implemented with a single tool.

The core of this hierarchical structure is offloading tool definitions from the prompt to a workspace. In the traditional approach, detailed definitions of all tools have to be included in the prompt. In the hierarchical structure, only level-1 core tools remain in the prompt; the rest move to a workspace. From a context-engineering perspective, the workspace is a powerful method for removing tool information and tool-execution results that don't require LLM judgment from the context.

The interface that the LLM calls directly (level 1) is simple and fixed. But through the two gateways of `bash_tool` and `execute_python`, the LLM can use the filesystem (level 2) and code generation (level 3) to perform virtually unlimited work.

Yet as systems grow in complexity, a single agent, even a powerful one, can struggle with too many responsibilities. When one agent has to manage dozens of tools, maintain context across multiple domains, and make decisions across varied tasks, its performance begins to degrade. In chapter 9, we'll see how multiple specialized agents can work together, each focused on what it does best and collaborating through different patterns to tackle larger challenges.

Summary

- The predefined-tool approach has three limitations: the agent's action space is restricted to the developer-defined tool list; executing one tool per LLM call is inefficient; and as tools proliferate, prompt token costs increase while the LLM's tool-selection accuracy decreases.
- Code execution provides three core capabilities: control flow through conditionals and loops, data flow by storing intermediate results in variables, and tool composition by combining small functions to create new functionality.

- Sandboxes are isolated environments for executing LLM-generated code safely. They provide host-system security protection, resource management for CPU and memory, and recoverability when problems occur.
- Tool composition can be implemented by extracting the source code of user-defined tools and registering them in the sandbox. This approach maximizes token efficiency in processing large amounts of data when you need only aggregation or summaries.
- A workspace is a complete working environment that combines code execution, a filesystem, and a CLI. Just as human developers write code, save files, and execute commands in the terminal, agents can work in the same environment.
- Agent skills is an approach that progressively discloses tool information. Only skill names and one-line descriptions are included in the prompt; actual use is learned when the LLM determines that a skill is required and reads the `SKILL.md` file.
- Agent tools can be divided into three layers: core tools, which are always included in the prompt (`execute_python`, `bash_tool`, and `upload_file`); installable tools, which have only summaries in the prompt and detailed definitions in the filesystem (agent skills); and generated tools, for which the LLM writes code on the spot as needed.
- The essence of the hierarchical tool structure is to offload tool definitions from the prompt to a workspace. From a context-engineering perspective, the workspace is a powerful method for removing tool information and tool-execution results that don't require LLM judgment from the context.

9 Orchestrating multi-agent systems

This chapter covers

- Using three collaboration patterns: Workflow, Agent as Tool, and Transfer
- Implementing composable workflows with Sequential, Parallel, and Loop
- Wrapping agents as tools with context isolation
- Transferring control between agents using a dynamically created transfer tool
- Connecting agents across networks with the A2A protocol

Throughout this book, we've explored how a single agent can perform diverse tasks. By assigning roles through system prompts and providing tools, we enabled capabilities such as web search, calculation, and code execution. A single agent could solve many problems in the General AI Assistants (GAIA) benchmark. But real-world problems are more complex. As an agent's responsibilities grow, system prompts become longer, and the number of required tools increases. It's like expecting one person to be a developer, product manager, designer, and operations specialist.

No matter how capable that person is, performance suffers when roles multiply. We'll address this challenge by implementing a multi-agent system in which multiple agents collaborate. We'll assign each agent a specialized role and dedicated tools and then orchestrate the agents to work together.

The multi-agent system we'll build in this chapter (figure 9.1) directly extends the `Agent` class, tool system, and `ExecutionContext` we've developed throughout the book. Each specialist agent is the same `Agent` you already know, complete with a system prompt, tools, and a `run()` method. The collaboration patterns we introduce are built on top of this existing foundation.

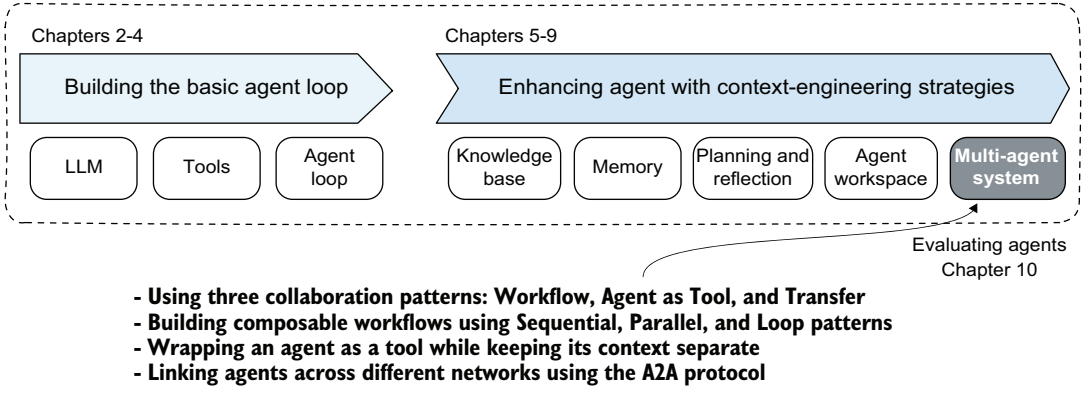

Figure 9.1 Book structure overview: chapter 9

9.1 Why multi-agent?

The agent we've built so far defines its role through a system prompt (instructions) and interacts with the external world through tools. With just a web search tool, it could answer many questions, and adding calculator and code-execution tools enabled it to solve more complex problems. But what happens when we demand more roles from a single agent? Consider building a technical blog writing assistant. This agent needs to research the latest technology trends, write example code, draft the content, and review it. What happens when we pack all the instructions and tools for each role into a single agent?

QS
```
instructions = """
You are a technical blog writing assistant.
[Research Role]
- Search for the latest information on the given topic
- Prioritize reliable sources
- Summarize and organize the search results
```

```
[Coding Role]
- Write example code needed for explanations
- Code must be executable
- Include error handling and comments
[Writing Role]
- Write a blog post based on research results and code
- Explain at the appropriate level for the reader
- Structure with introduction, body, and conclusion
[Review Role]
- Verify technical accuracy
- Check grammar and readability
- Suggest improvements
Available tools: search_web, execute_python, ...
"""
```

The longer the system prompt becomes, the easier it is for the large language model (LLM) to miss what's essential. When it should be writing, it might focus on review guidelines. When it should be coding, it might worry about writing style. Adding more tools makes things even more complicated. With more than 10 tools to choose among, it becomes harder to judge which tool to use at any given moment, and incorrect tool selections become more frequent. An LLM generates the next token based on information within its context window. The more irrelevant information fills that context, the harder it becomes to focus on what's needed.

The most natural way to solve complex problems is to assemble a team of specialists. Multi-agent systems work on the same principle. Let's divide the blog writing assistant from our earlier example into four specialists, as shown in figure 9.2.

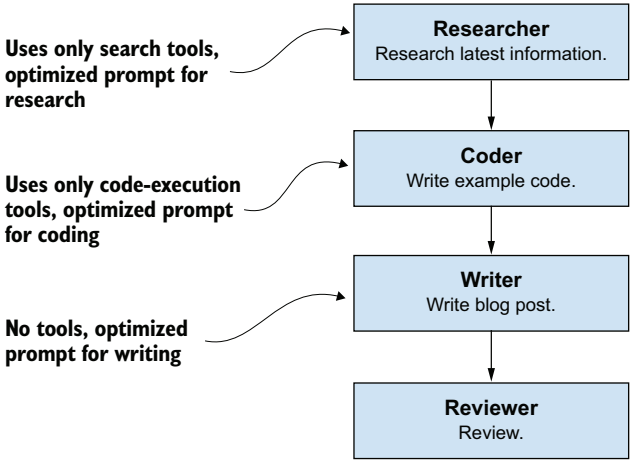

Figure 9.2 Four specialist agents for technical blog writing. Each agent focuses on a single role with optimized prompts and dedicated tools.

Each agent focuses on a single role. System prompts become short and clear, and each agent receives only the tools it needs. The Researcher focuses on searching, the Coder on writing code, the Writer on drafting content, and the Reviewer on evaluation. Consider these benefits of multi-agent systems:

- *Separation of roles*—Because each agent focuses on one role, system prompts become short and clear. The LLM can concentrate on its assigned task without confusion.
- *Specialization*—You can configure optimized prompts and tools for each domain. The search specialist gets search tools and strategies; the coding specialist gets code-execution tools and guidelines.
- *Reusability*—After you build a specialist agent, you can use it across multiple systems. A Researcher agent can be reused not just for blog writing but also for report generation, question-answering systems, and more.
- *Scalability*—When you need a new role, just add a new agent. You can extend the system without modifying existing agents.

9.2 Three collaboration patterns

When designing a multi-agent system, there are two key questions:

- *Who decides the next agent?* The developer can define the execution order in code, or the LLM can decide based on the situation. If the developer decides, the system is predictable and stable. If the LLM decides, the system is flexible, but the results may vary.
- *Is the context shared?* Agents can work while viewing the same conversation history, or they can work independently in isolated environments. Sharing allows them to inherit the previous context. Isolation lets them hide intermediate steps and pass only results. These two criteria distinguish three collaboration patterns.

How do you choose which pattern to use? Start by asking whether the execution order is known in advance. If you can define the steps ahead of time (such as "research, then write, then review"), use the Workflow pattern. If the steps depend on the user's request, ask whether the main agent needs to stay in control. If the orchestrator should coordinate multiple specialists and synthesize their results, use Agent as Tool. If the request clearly belongs to one specialist who should take over entirely, use Transfer. A good rule of thumb: use Workflow for predictable pipelines, Agent as Tool when you need a coordinator who delegates, and Transfer when you want to route the user to the right expert.

In Workflow, the developer decides not only the execution order but also whether to share context. You can pass the same context sequentially or create new contexts for each agent to isolate them. By contrast, patterns where the LLM decides transitions are split into Transfer and Agent as Tool based on how context is handled (table 9.1).

Table 9.1 Comparison of three collaboration patterns

Pattern	Order decision	Context sharing	Best use case
Workflow	Developer	Developer's choice	When the workflow is predictable
Agent as Tool	LLM	Isolated	When synthesizing multiple expert results
Transfer	LLM	Shared	When domains are clearly separated

We'll implement Workflow (section 9.3), Agent as Tool (section 9.4), and Transfer (section 9.5) in order, building up the components needed for a complete multi-agent system. By the end of the chapter, you'll be able to combine all three patterns to create agents that collaborate in flexible, powerful ways.

WORKFLOW

In the Workflow pattern, the developer defines the execution flow in code. The blog writing example we saw earlier follows this pattern. The developer explicitly specifies the order: Researcher > Coder > Writer > Reviewer.

Listing 9.1 Workflow: Sequential execution example

```
# Workflow pattern: Developer defines the order
workflow = run_sequential(agents=[
    researcher,  # 1. Research
    coder,       # 2. Write code
    writer,      # 3. Write content
    reviewer     # 4. Review
])
```

How to handle context is also the developer's choice. If you pass the previous agent's context as is, the Writer can see the Researcher's search results. If you create a new context for each agent, they execute independently. Beyond sequential execution, you can compose various flows, including parallel execution and loops.

AGENT AS TOOL

The Agent as Tool pattern wraps other agents and calls them like tools. The main agent maintains overall control while calling specialist agents when needed.

Listing 9.2 Agent as Tool: Agent registration

```
# Agent as Tool pattern: Wrap agents as tools
assistant = Agent(
    instructions="Answer user questions",
    tools=[
        AgentTool(researcher),   # Call when research is needed
        AgentTool(calculator),   # Call when calculation is needed
    ]
)
```

The key to this pattern is isolation. The called agent works independently in a separate context and returns only the result to the main agent. The main agent uses only the final result without needing to know the process the specialist went through. It's like calling a function but with an agent.

TRANSFER

The Transfer pattern lets the LLM choose which agent to execute next. Unlike Workflow, where the developer sets the order, the LLM determines which agent is most appropriate. To implement this pattern, you register `sub_agents` with an agent, as shown in the following listing. The agent examines the registered `sub_agents` list and transfers control to a more suitable agent if one exists for the current request.

Listing 9.3 Transfer: LLM-selected agent routing

```
# Transfer pattern: LLM selects the next agent
router = Agent(
    name="router",
    instructions="Analyze the request and route to the appropriate expert",
    sub_agents=[code_expert, math_expert, writing_expert]
)
```

Internally, `sub_agents` are converted to a transfer tool called `transfer_to_agent`. When the LLM calls this tool, the system switches execution to that agent. If a user says, "Write a Fibonacci function in Python," the router recognizes this as a coding request and transfers to `code_expert`.

The key difference from Agent as Tool is control. In Agent as Tool, the main agent "calls" a specialist and receives results to decide the next action. In Transfer, control is "handed over" to the specialist, which handles the task to completion.

Think of Agent as Tool as being like a manager delegating a task to a team member and waiting for a status report. The manager stays in charge. Transfer is like forwarding a phone call to another department. The new person takes over the conversation, and the caller may never return to the original agent.

COMBINING PATTERNS

The three patterns are not mutually exclusive. Real systems often combine multiple patterns. You might define the overall flow with Workflow, call specialists with Agent as Tool at certain stages, and use Transfer to switch agents based on the type of user request.

9.3 Agent workflow: Defining order in code

The Workflow pattern is an approach in which the developer explicitly defines the execution order of agents. As in the blog-writing example, you specify the order Researcher > Coder > Writer > Reviewer in code. Instead of the LLM deciding the next step, execution follows the flow the developer has predetermined. Now it's time to implement the three workflow patterns. Sequential runs agents in order, passing

context from one to the next. Use it when tasks have a natural step-by-step progression (e.g., research > write > review). Parallel runs agents simultaneously on the same input. Use it when you need multiple independent perspectives or analyses at once. Loop repeats agents until a condition is met. Use it when output quality improves through iteration, such as writing and reviewing cycles.

Before examining each pattern's implementation, let's understand the core principles of workflow design. We start with Workflow because it is the simplest and most predictable of the three collaboration patterns. Because the developer controls the execution order, the behavior is deterministic and easy to debug, making it the natural starting point before introducing LLM-driven routing.

9.3.1 Workflow design principles

How should we implement workflows? The simplest approach is to call agents sequentially in a `for` loop. But this approach makes it difficult to compose workflows. Nesting becomes impossible, such as putting a Parallel inside a Sequential or a Sequential inside a Loop.

The solution is to make workflows behave like agents. If all workflows inherit from `Agent` and implement the same `run()` method, you can freely compose agents and workflows together.

Listing 9.4 Workflow design: `Agent` inheritance

```
class SequentialWorkflow(Agent):
    async def run(self, user_input=None, context=None, verbose=False) ->
AgentResult:
        ...

class ParallelWorkflow(Agent):
    async def run(self, user_input=None, context=None, verbose=False) ->
AgentResult:
        ...

class LoopWorkflow(Agent):
    async def run(self, user_input=None, context=None, verbose=False) ->
AgentResult:
        ...
```

This design enables nested compositions like the following. You can put a `ParallelWorkflow` inside a `SequentialWorkflow` and have another agent process the results:

```
workflow = SequentialWorkflow(
    agents=[
        researcher,
        ParallelWorkflow(agents=[optimist, pessimist, realist]),
# Three perspective agents
        editor,
    ]
)
result = await workflow.run("Analyze AI trends for me")
```

The researcher runs first; then three perspective agents (optimist, pessimist, and realist) analyze simultaneously based on those results. The analyses from all three perspectives are consolidated and passed to the editor. From the outside, this entire structure behaves like a single agent.

To be clear, it is still a multi-agent system internally. Multiple agents collaborate behind the scenes. The point is that the external interface is identical to a single `Agent`: you call `run()` with input and get a result back. This design means any workflow can be used wherever a single agent is expected, making the system fully composable.

These agents evaluate the same input from different viewpoints: optimistic (focusing on opportunities), pessimistic (focusing on risks), and realistic (focusing on practical feasibility). We'll define them fully in section 9.3.3.

9.3.2 Sequential: Step-by-step execution

Now let's implement each workflow pattern one by one. The Sequential workflow is the most intuitive pattern. It executes agents in order and passes the previous agent's context to the next. The structure is like a pipeline where data flows through one stage at a time (figure 9.3.).

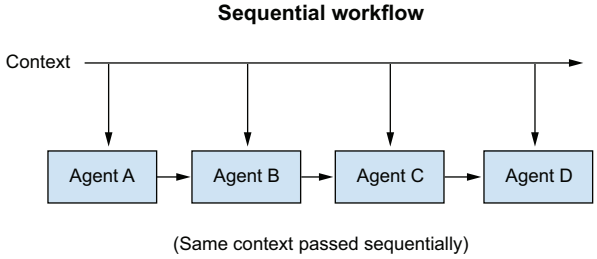

Figure 9.3 Sequential Workflow: same context is passed through agents one at a time.

There are three key implementation points:

- Only the first agent receives `user_input`. Subsequent agents already have the user input in their context, so we must avoid adding it again.
- Reset `final_result` and `current_step` before each agent execution. This prevents the previous agent's termination state from affecting the next agent.
- Extract the context from each agent's result and pass it to the next agent.

Listing 9.5 `SequentialWorkflow` implementation

```
class SequentialWorkflow(Agent):

    def __init__(self, agents: List[Agent], name: str = "sequential_workflow"):
        self.agents = agents
        self.name = name
```

```
    async def run(
        self,
        user_input: Optional[str] = None,
        context: Optional[ExecutionContext] = None,
        verbose: bool = False,
    ) -> AgentResult:
        result = None

        for i, agent in enumerate(self.agents):
            if context is not None:
                # Reset state for next agent execution
                context.final_result = None
                context.current_step = 0

            if i == 0:
                # First agent: pass both user_input and context
                result = await agent.run(user_input, context=context,
     verbose=verbose)
            else:
                # Subsequent agents: pass only context (no user_input)
                result = await agent.run(context=context, verbose=verbose)

            context = result.context

        return result
```

Let's examine the `run()` method logic. It iterates through the agent list and executes each agent. The first agent (`i == 0`) receives both `user_input` and `context`, while subsequent agents receive only `context`. After each execution, `result.context` becomes the input for the next agent. The last agent's result becomes the overall workflow result.

Let's look at a practical example. This scenario has three agents collaborating to write a blog post. The writer can see the researcher's findings in the context, and the editor can see the writer's draft. Each agent naturally picks up where the previous one left off:

```
researcher = Agent(
    model=model,
    name="researcher",
    instructions="Research the topic and organize key information."
)
writer = Agent(
    model=model,
    name="writer",
    instructions="Write a blog post based on the research results."
)
editor = Agent(
    model=model,
    name="editor",
    instructions="Review the writing and improve grammar and readability."
)

workflow = SequentialWorkflow(agents=[researcher, writer, editor])
result = await workflow.run("Write a post about the latest trends in AI agents")
```

9.3.3 Parallel: Concurrent execution

Sometimes, it's useful to analyze multiple perspectives simultaneously. You analyze the same input from multiple viewpoints and then synthesize the results. The Parallel workflow implements this pattern.

When evaluating a business idea, you might want optimistic, pessimistic, and realistic viewpoints at the same time. When reviewing code, you could run security, performance, and readability checks in parallel. When writing a report, multiple agents can research different subtopics concurrently. The Parallel workflow is ideal whenever tasks are independent and can produce richer results by combining diverse analyses (figure 9.4).

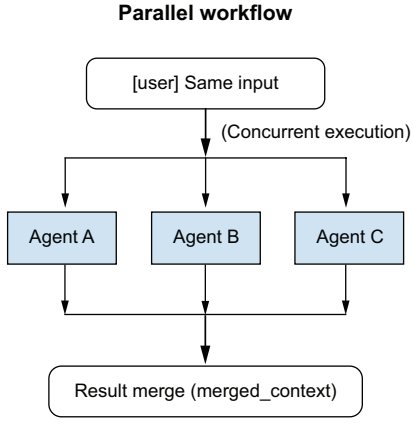

Figure 9.4 Parallel Workflow: same input distributed to multiple agents with results merged

The trickiest part of parallel implementation is merging results. When three agents run simultaneously, each generates separate events. These events must be combined into a single context, with several considerations:

- *Prevent duplicate user events.* Because the same `user_input` is passed to all agents, the same user message is added to each agent's context. This duplication must be removed when merging results.
- *Preserve the existing context if present.* When Parallel runs inside a Sequential, events from previous stages are already in the context. Keep these existing events, and add only newly generated events.
- *Distinguish each agent's results.* The `event.author` field identifies which agent generated each event.

Listing 9.6 `ParallelWorkflow` implementation and merge

```
class ParallelWorkflow(Agent):

    def __init__(self, agents: List[Agent], name: str = "parallel_workflow"):
        self.agents = agents
        self.name = name

    async def run(
        self,
        user_input: Optional[str] = None,
        context: Optional[ExecutionContext] = None,
        verbose: bool = False,
    ) -> AgentResult:
        # Count existing events (to distinguish newly added ones)
        existing_event_count = len(context.events) if context else 0
```

```
        # Run all agents concurrently (pass same context)
        results: List[AgentResult] = await asyncio.gather(
            *[agent.run(user_input, context=context, verbose=verbose)
              for agent in self.agents]
        )

        # Create merged context
        merged_context = ExecutionContext()

        # 1. Copy existing context events
        if context is not None:
            for event in context.events:
                merged_context.add_event(event)

        # 2. Add only new events from each agent (after existing_event_count)
        seen_user_event = (context is not None)

        for result in results:
            new_events = result.context.events[existing_event_count:]

            for event in new_events:
                if event.author == "user":
                    # Add user event only once
                    if not seen_user_event:
                        merged_context.add_event(event)
                        seen_user_event = True
                else:
                    # Add all agent-generated events
                    merged_context.add_event(event)

        # Combine all outputs and return
        combined_output = "\n\n".join(
            f"[{agent.name}]\n{result.output}"
            for agent, result in zip(self.agents, results)
        )

        return AgentResult(output=combined_output, context=merged_context)
```

Let's examine the core logic of the `run()` method. First, `existing_event_count` records the number of existing events. Then `asyncio.gather()` runs all agents concurrently. When merging results, existing events are copied first, and then only new events, those after `existing_event_count`, from each agent are added. The `seen_user_event` flag prevents duplicate user events. The final output is a string that combines each agent's result with an `[agent_name]` header.

Let's look at an example. This scenario analyzes a business idea from three perspectives simultaneously:

```
optimist = Agent(
    model=model,
    name="optimist",
    instructions="Analyze from a positive perspective, focusing on
     opportunities and possibilities."
```

```
)
pessimist = Agent(
    model=model,
    name="pessimist",
    instructions="Analyze focusing on risks and potential problems."
)
realist = Agent(
    model=model,
    name="realist",
    instructions="Analyze practical feasibility from a balanced perspective."
)

workflow = ParallelWorkflow(agents=[optimist, pessimist, realist])
result = await workflow.run("Evaluate my AI-based education platform startup
idea")

print(result.output)
```

9.3.4 Loop: Iterative execution

Some tasks don't finish perfectly in one pass. Sometimes, you need to write, review, incorporate feedback, and rewrite repeatedly. The Loop workflow runs agents repeatedly until a specific condition is met.

Loop workflow

Iteration 1:
[user] Write marketing copy
[writer] Drafting ...
[reviewer] score: 60, feedback: "Make it more impactful"
→ stop_condition(60) = False, continue

Iteration 2:
[writer] Revising based on feedback ...
[reviewer] score: 85, feedback: "Looks good!"
→ stop_condition(85) = True, stop

Figure 9.5 Loop Workflow: iterative execution with writer and reviewer until quality threshold is reached

The Loop pattern (figure 9.5) is appropriate when you can measure the quality of the output (e.g., a score, pass/fail criteria, or keyword presence) and when iterative refinement can improve the result. Common examples include a writer drafting content and a reviewer providing feedback until quality is satisfactory; a code generator producing code and a tester running tests until all tests pass; and a translator producing a translation and a reviewer checking accuracy until the translation meets standards.

The key to the Loop implementation is the stop condition (`stop_condition`). This function receives the current result and iteration count and then returns whether to terminate. We also set `max_iterations` to prevent infinite loops.

As in Sequential, only the first agent receives `user_input`; all subsequent executions receive only the context. But in Loop, the first agent exists only once across all iterations. Only the first agent in the first iteration receives `user_input`; everything after inherits the context.

Listing 9.7 `LoopWorkflow` implementation and stop condition

```
# Stop condition type: (current result, iteration count) -> whether to stop
StopCondition = Callable[[AgentResult, int], bool]
```

```
class LoopWorkflow(Agent):

    def __init__(
        self,
        agents: List[Agent],
        stop_condition: Optional[StopCondition] = None,
        max_iterations: int = 10,
        name: str = "loop_workflow",
    ):
        self.agents = agents
        self.stop_condition = stop_condition
        self.max_iterations = max_iterations
        self.name = name

    async def run(
        self,
        user_input: Optional[str] = None,
        context: Optional[ExecutionContext] = None,
        verbose: bool = False,
    ) -> AgentResult:
        result = None
        is_first_agent = True  # Whether this is the first agent overall

        for iteration in range(1, self.max_iterations + 1):
            # One iteration: run all agents sequentially
            for agent in self.agents:
                if context is not None:
                    context.final_result = None
                    context.current_step = 0

                if is_first_agent:
                    # Very first: pass user_input
                    result = await agent.run(user_input, context=context,
verbose=verbose)
                    is_first_agent = False
                else:
                    # After: pass only context
                    result = await agent.run(context=context, verbose=verbose)

                context = result.context

            # Check stop condition
            if self.stop_condition and self.stop_condition(result, iteration):
                break

        return result
```

Let's examine the `run()` method. The outer loop manages iterations, and the inner loop runs agents sequentially within each iteration. The `is_first_agent` flag tracks whether this is the first execution in the entire workflow. At the end of each iteration, the `stop_condition` is checked, and the loop terminates if it returns `True`.

Let's look at a practical example. In this scenario, a writer agent drafts marketing copy, and a reviewer agent evaluates it according to criteria such as clarity, persuasiveness,

and grammar, assigning a score from 0 to 100. The threshold of 80 represents a "good enough" quality bar for this use case. You would adjust this threshold based on your project's requirements. The loop iterates through writing and reviewing to improve quality until the score reaches 80 or higher:

```
from pydantic import BaseModel

class Review(BaseModel):
    content: str
    score: int  # 0-100
    feedback: str

writer = Agent(
    model=model,
    name="writer",
    instructions="Write content or improve it based on feedback."
)
reviewer = Agent(
    model=model,
    name="reviewer",
    instructions="Evaluate the writing and provide a score (0-100) with
feedback.",
    output_type=Review
)

def quality_reached(result: AgentResult, iteration: int) -> bool:
    """Stop if score is 80 or higher"""
    if isinstance(result.output, Review):
        return result.output.score >= 80
    return False

workflow = LoopWorkflow(
    agents=[writer, reviewer],
    stop_condition=quality_reached,
    max_iterations=5,
)
result = await workflow.run("Write marketing copy")
```

This workflow operates as follows:

1. *Iteration 1*—The writer creates a draft, and the reviewer evaluates it. If the score is 60, continue.
2. *Iteration 2*—The writer revises based on feedback, and the reviewer evaluates again. If the score is 75, continue.
3. *Iteration 3*—The writer makes final revisions, and the reviewer evaluates. If the score is 85, `quality_reached` returns `True`, and the loop terminates.

The `stop_condition` function can be defined in various ways to fit your situation. It can stop generation when a specific keyword is found, for example, or when the result is long enough:

```
# Example: Stop when specific keyword is found
def keyword_found(result: AgentResult, iteration: int) -> bool:
    return "APPROVED" in str(result.output)

# Example: Stop after at least 3 iterations when content is long enough
def sufficient_content(result: AgentResult, iteration: int) -> bool:
    return iteration >= 3 and len(str(result.output)) > 1000
```

9.3.5 Composing workflows

All three workflow patterns (`SequentialWorkflow`, `ParallelWorkflow`, and `LoopWorkflow`) inherit from `Agent`. This means each workflow implements the same `run()` interface as any regular agent, so a workflow can be used anywhere an agent is expected. This design enables composability: you can nest a `ParallelWorkflow` inside a `SequentialWorkflow` or put a `LoopWorkflow` as one stage of a larger pipeline, and everything works. The following example composes research > multiperspective analysis > write/review iteration > final editing into a single workflow:

```
workflow = SequentialWorkflow(
    agents=[
        researcher,
        ParallelWorkflow(agents=[optimist, pessimist, realist]),
        LoopWorkflow(
            agents=[writer, reviewer],
            stop_condition=quality_reached,
            max_iterations=3,
        ),
        editor,
    ]
)

result = await workflow.run("Write an AI market analysis report")
```

Even though Parallel and Loop are nested inside Sequential, from the outside, they behave like a single agent. Context flows naturally through the entire pipeline, allowing each stage to reference the results of previous work.

The key characteristic of this pattern is that the execution flow is explicitly defined in code, making the flow predictable. Because the developer sets the order, debugging is easier, and execution follows the same flow every time. Now let's implement the Agent as Tool pattern, where the main agent calls other agents like tools when needed.

9.4 Agent as Tool: Calling agents as tools

The Workflow pattern excels when the execution order is known in advance. Once a developer sets a Researcher > Coder > Writer sequence, it always runs in that order. It is predictable, debuggable, and reliable. But some requests require dynamic routing: which specialists are needed depends on what the user asks.

A question like "Tell me about AI trends" needs only research. A request like "Calculate the average of this data" needs only code execution. A request like "Analyze the latest AI papers and visualize keyword frequency" needs both research and code execution. The specialists needed vary by request.

The most natural way to solve this problem is to make agents into tools. LLMs can already decide when to call tools. If `web_search` is needed, they call `web_search`; if a calculator is needed, they call the calculator. If we register agents as tools, the LLM can decide which specialist is needed on its own.

9.4.1 Making agents into tools

From an LLM's perspective, what is a tool? It has a name, description, and parameters, and it returns a result when called. Whether it's `web_search`, a calculator, or another agent, the structure is identical. The LLM doesn't need to know whether the researcher is an agent or a simple function. It just calls it when needed. This is the essence of the Agent as Tool pattern, as shown in figure 9.6.

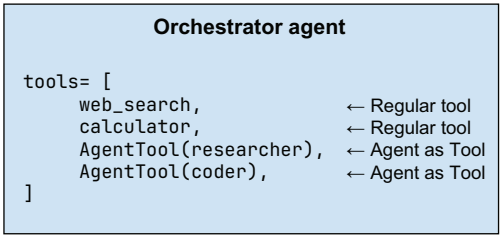

Figure 9.6 Orchestrator agent: Regular tools and agent tools in the same tool list

In this structure, the tool list the LLM sees looks like this:

```
{
  "tools": [
    {"name": "web_search", "description": "Search the web", ...},
    {"name": "calculator", "description": "Calculate expressions", ...},
    {"name": "researcher", "description": "Research a topic", ...},
    {"name": "coder", "description": "Write and execute code", ...}
  ]
}
```

9.4.2 Implementing AgentTool

`AgentTool` is an adapter that wraps an `Agent` with the `BaseTool` interface. Because it's fully compatible with the existing tool system, you can mix regular tools and agent tools.

The implementation has two core parts: `__init__` converts the agent's name and description into a tool definition, and `execute` takes input, runs the agent, and returns only the result.

`AgentTool` takes only a single string parameter called `request` by default. The `request` argument lets the main agent specify what task the tool agent should perform. But if more precise control is needed, you can specify a Pydantic model with `input_schema` to receive structured input. This is useful when you want to explicitly pass research depth or maximum source count to a research agent, for example.

Listing 9.8 `AgentTool`: Agent wrapper adapter

```
class AgentTool(BaseTool):
    """Adapter that wraps an Agent as a tool."""

    def __init__(
        self,
        agent: Agent,
        input_schema: Optional[Type[BaseModel]] = None,
    ):
        self.agent = agent
        self.input_schema = input_schema

        # Create parameters from input schema
        if input_schema:
            # Extract JSON Schema from Pydantic model
            parameters = input_schema.model_json_schema()
            parameters.pop("$defs", None)
            parameters.pop("title", None)
        else:
            # Default: request string parameter
            parameters = {
                "type": "object",
                "properties": {
                    "request": {
                        "type": "string",
                        "description": "The task or question to delegate"
                    }
                },
                "required": ["request"]
            }

        # Create tool definition from agent's name and description
        tool_def = format_tool_definition(
            agent.name,
            agent.description,
            parameters
        )

        super().__init__(
            name=agent.name,
            description=agent.description,
            tool_definition=tool_def,
        )

    async def execute(self, context: ExecutionContext, **kwargs) -> Any:
        if self.input_schema:
            # Structured input: validate with Pydantic, convert to JSON string
            validated = self.input_schema.model_validate(kwargs)
            request = validated.model_dump_json(exclude_none=True)
        else:
            # Default: use request parameter
            request = kwargs.get("request", str(kwargs))

        # Run agent (in new context)
```

```
    result = await self.agent.run(request)

    # Return only the result (intermediate steps not included)
    return result.output
```

In `__init__`, if `input_schema` is provided, the Pydantic model's JSON schema is used as the tool parameters; otherwise, the default `request` parameter is used.

In the `execute` method, input processing differs. If `input_schema` exists, it validates with Pydantic and then converts to a JSON string; otherwise, it uses the `request` parameter as is. Either way, the child agent receives a string.

In the Agent as Tool pattern, the parent agent (also called the *orchestrator*) is the agent that holds other agents as tools and coordinates the overall task. A child agent is a specialist agent that the parent calls to handle a specific subtask. The parent delegates work; the child executes it and returns a result.

Note that the execute method receives the parent agent's `context` but doesn't pass it to the child agent. Instead, it passes only the `request` string, and the child agent runs in a new context. This is context isolation.

9.4.3 Context isolation

The biggest difference between the Agent as Tool pattern and the Workflow pattern is how context is shared. In Sequential Workflow (figure 9.7), agents use the same context. Agent A's web search results, agent B's analysis process, and agent C's reasoning accumulate in the context. Each agent can see in detail what previous agents did.

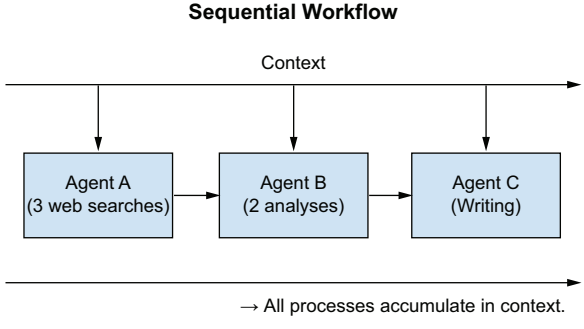

Figure 9.7 Sequential Workflow: All processes accumulate in a shared context.

In Agent as Tool (figure 9.8), child agents run in isolated environments. The orchestrator requests "do research" but can't know how many web searches the researcher did or which sites it visited. Only the final result returns.

This isolation has several advantages:

- *Abstraction*—The orchestrator needs only the research result. How many web searches happened in between and which URLs were visited are not its concern. It's like calling a function without needing to know its internal implementation.

- *Context efficiency*—Even if a child agent goes through 10 tool calls and five reasoning steps, only one line of the final result is added to the parent context. This saves context window space.
- *Independence*—The child agent's tool use or failures don't directly affect the parent agent. Regardless of what tools the researcher uses internally, from the orchestrator's perspective, only the result matters.

There are disadvantages too, of course. When debugging, you need a separate inspection to see what happened inside a child agent. When an intermediate state needs to be shared, a workflow is more suitable.

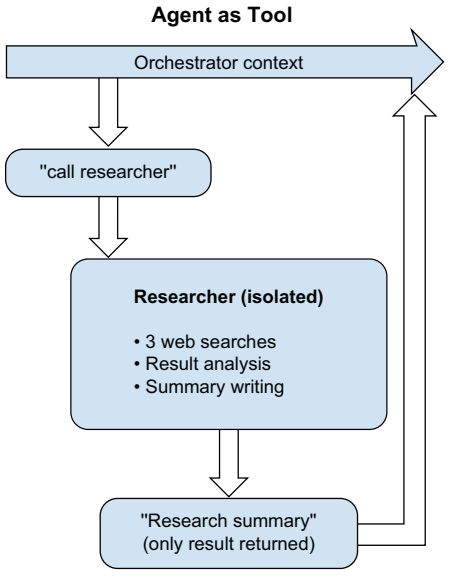

Figure 9.8 Agent as Tool: the child agent runs in an isolated environment and returns only the final result.

9.4.4 Practical example: Researcher and coder collaboration

Let's examine how Agent as Tool works through a concrete example. We'll create two specialist agents and have an orchestrator use them as tools.

Suppose that a user asks, "Research the latest AI agent frameworks and write a Python script that compares their GitHub stars." The orchestrator delegates the research task to the researcher, which uses a web search to find frameworks and their details, and the coding task to the coder, which writes and executes a Python script for the comparison. The orchestrator combines both results into a coherent response.

First, define the specialist agents. The researcher has a web search tool, and the coder has code-execution capability.

Listing 9.9 Agent as Tool example: Researcher and coder

```
from scratch_agents import Agent, LlmClient
from scratch_agents.tools import AgentTool, search_web

llm = LlmClient(model="gpt-5")

# Research specialist: has web search tool
researcher = Agent(
    model=llm,
    name="researcher",
    description="Search the web and summarize findings on any topic",
```

```
    instructions="""You are a research specialist.
    Use web_search to find relevant information.
    Summarize your findings concisely.""",
    tools=[search_web],
)

# Coding specialist: has code execution capability
coder = Agent(
    model=llm,
    name="coder",
    description="Write and execute Python code for data analysis and
visualization",
    instructions="""You are a Python expert.
    Write clean, working code.
    Execute code to verify results.""",
    code_execution="e2b",
)
```

Now, create the orchestrator. Wrap both specialists with `AgentTool` to register them as tools:

```
# Orchestrator: utilizes specialists as tools
orchestrator = Agent(
    model=llm,
    name="coordinator",
    instructions="""You are a project coordinator.
    Analyze user requests and delegate to appropriate specialists:
    - Use 'researcher' for information gathering
    - Use 'coder' for data analysis and visualization
    Combine their results to fulfill the user's request.""",
    tools=[
        AgentTool(researcher),
        AgentTool(coder),
    ],
)
```

9.5 Agent Transfer pattern

In the Agent as Tool pattern, the main agent calls specialists. The orchestrator invokes the Researcher like a tool, receives the result, and uses it for the next task. Control always stays with the orchestrator.

The Transfer pattern is different. Rather than one agent calling another, control itself is handed over. It's like when a customer service call says, "I'll connect you to technical support for this inquiry," and the representative changes. The previous conversation remains intact, but a different person is responding.

9.5.1 Differences between Agent as Tool and Transfer

Let's compare the two patterns using a customer service scenario. In the Agent as Tool approach, a general support agent handles all inquiries. When a technical question comes in, the agent internally consults a technical expert and relays the answer to the

customer. From the customer's perspective, they're continuously talking to the same representative.

```
[Agent as Tool]
Customer: "I can't connect to the server"
General Agent: (checks with tech team) "Please check your network settings"
Customer: "How do I do that?"
General Agent: (checks with tech team again) "Go to Settings > Network..."
In the Transfer approach, when a technical question comes in, the customer
is directly connected to a technical expert. The customer now talks to the
technical expert. The expert also knows what was discussed before.
[Transfer]
Customer: "I can't connect to the server"
General Agent: "I'll connect you to technical support"
Tech Expert: "Let me check your network settings. What error are you seeing?"
Customer: "It says connection timed out"
Tech Expert: "Please check your firewall settings..."
```

There are three key differences between the two approaches:

- *Location of control*—In Agent as Tool, the main agent always maintains control. In Transfer, control is handed over to another agent.
- *Method of context sharing*—In Agent as Tool, the child agent runs in an isolated environment, hiding intermediate processes. In Transfer, the same context is inherited, maintaining the previous conversation.
- *Conversation continuity*—In Agent as Tool, only the result is returned, so the main agent responds to follow-up questions. In Transfer, the transferred agent takes over the conversation and responds directly to follow-up questions (figure 9.9).

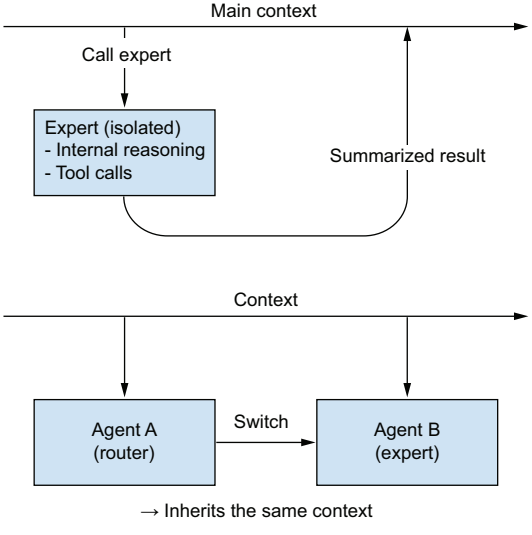

Figure 9.9 Context-handling comparison: Agent as Tool isolates the child agent and returns only the summarized result, while Transfer hands over control with shared context.

9.5.2 Transfer is also a tool

How can we implement the Transfer pattern (figure 9.10)? The core idea is simple: make the agent transfer into a tool. When the agent decides that a different specialist is needed, it calls the `transfer_to_agent` tool.

The advantage of this approach is that it uses the existing tool system as is. Instead of creating a new mechanism, you simply add one tool called `transfer_to_agent`. From the LLM's perspective, it's no different from any other tool.

To make the Transfer pattern work, we implement four main things one by one:

- *Agent tree composition*—Define which agent can transfer to which agent.
- *Transfer tool creation*—Dynamically create the `transfer_to_agent` tool.
- *Transfer detection and execution*—Hand over control when the tool is called.
- *Context preservation*—Ensure that the previous conversation continues after the transfer.

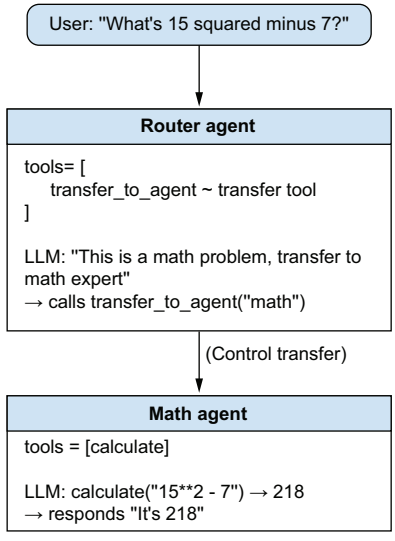

Figure 9.10 Transfer pattern flow: the router agent calls the transfer tool to hand over control to the math agent, which inherits the same context.

9.5.3 Agent tree

Before diving into implementation details, here's how the entire transfer process works end to end:

1. A router agent receives the user's request.
2. It examines its registered sub-agents and calls the `transfer_to_agent` tool to select the most appropriate specialist.
3. Control is handed over to that specialist, who inherits the full conversation context.
4. The specialist handles the task to completion and stores the result. If the specialist can't handle a follow-up request, the request can be transferred back to the router or to a sibling agent. The agent tree defines which transfers are allowed. Now, let's build this step by step.

First, we extend `ExecutionContext` to track transfer state. We add two fields;

- `transfer_to` stores the name of the target agent. When the `transfer_to_agent` tool is called, the target name is set in this field, and the `run()` method detects this to perform the actual transfer.

- `transfer_tools` caches the transfer tool for each agent. Since each agent has different transfer targets, we store the transfer tool for each agent using the agent name as the key:

```
@dataclass
class ExecutionContext:
    ...

    # Multi-agent fields
    transfer_to: Optional[str] = None
    transfer_tools: Dict[str, "FunctionTool"] = field(default_factory=dict)
```

COMPOSING THE AGENT TREE

Next, we need to define relationships between agents. To determine which agent can transfer to which agent, agents must be connected in a tree structure.

We add three things to the `Agent` class:

- `sub_agents` is a list of sub-agents managed by this agent.
- `parent` is the agent that manages this agent, automatically assigned when `sub_agents` is set.
- `description` is a string describing what this agent does, which other agents reference when deciding whether to transfer.

Listing 9.10 `Agent` class: Transfer attributes

```
class Agent:
    def __init__(
        # ... existing parameters ...

        # Multi-agent
        sub_agents: Optional[List["Agent"]] = None,
        disallow_transfer_to_peers: bool = False,
    ):
        self.disallow_transfer_to_peers = disallow_transfer_to_peers

        # Compose agent tree
        self.sub_agents = sub_agents or []
        self.parent: Optional["Agent"] = None

        # Validate and set parent for sub_agents
        self._validate_and_set_sub_agents()
```

In the agent tree, each agent is identified by its `name`. This is because the transfer tool receives a name like `transfer_to_agent("math")` and finds the corresponding agent. Therefore, names cannot be duplicated within the same tree.

Also, a single agent cannot have multiple parents. To maintain a tree structure, each agent's parent must be unique. If the same agent is registered as `sub_agents` in multiple places, it becomes unclear which is the real parent.

The `_validate_and_set_sub_agents` method validates these two constraints. It iterates through `sub_agents` to check for name duplicates and raises an error if an agent already has a different parent. If validation passes, it sets the parent of each sub-agent to the current agent.

Listing 9.11 Agent tree validation: Name and parent

```
def _validate_and_set_sub_agents(self) -> None:
    """Validate name and parent duplicates in sub_agents and set parent."""
    seen_names = set()

    for sub in self.sub_agents:
        # Check for name duplicates
        if sub.name in seen_names:
            raise ValueError(f"Duplicate sub-agent name: '{sub.name}'")
        seen_names.add(sub.name)

        # Check for parent duplicates
        if sub.parent is not None:
            raise ValueError(
                f"Agent '{sub.name}' already has parent '{sub.parent.name}'"
            )
        sub.parent = self
```

DETERMINING TRANSFER TARGETS

Once the agent tree is composed, we can determine which agents each agent can transfer to. The `_get_transfer_targets` method returns the list of targets the current agent can transfer to.

There are three types of transfer targets:

- *Child agents*—A parent agent can delegate work to its sub-agents.
- *Parent agent*—A sub-agent can return to the parent when work is complete or cannot be handled.
- *Sibling agents*—Agents with the same parent can transfer directly to each other. However, sibling transfer is disabled if `disallow_transfer_to_peers` is `True`.

Listing 9.12 Transfer targets retrieval

```
def _get_transfer_targets(self) -> List["Agent"]:
    """List of targets the current agent can transfer to."""
    targets = []

    # 1. Children
    targets.extend(self.sub_agents)

    # 2. Parent and siblings
    if self.parent:
        targets.append(self.parent)
```

```
        # 3. Siblings (optional)
        if not self.disallow_transfer_to_peers:
            for sibling in self.parent.sub_agents:
                if sibling.name != self.name:
                    targets.append(sibling)
    return targets
```

To verify that the agent tree is properly composed and transfer targets are correctly extracted, let's create an agent group where the router has `math_agent` and `search_agent` as sub-agents:

```
math_agent = Agent(
    model=llm,
    name="math",
    description="Math calculation specialist",
    instructions="Performs mathematical calculations.",
    tools=[calculate],
)

search_agent = Agent(
    model=llm,
    name="search",
    description="Information search specialist",
    instructions="Searches for information on the web.",
    tools=[web_search],
)

router = Agent(
    model=llm,
    name="router",
    description="Routes user requests to appropriate specialists",
    instructions="Analyze requests and route to the appropriate specialist.",
    sub_agents=[math_agent, search_agent],
)
```

When we check the available agents with `math_agent._get_transfer_targets()`, we can confirm it returns router (the parent) and `search_agent` (which shares the same parent). If we set `disallow_transfer_to_peers` to `True` when initializing `math_agent` to prevent transfer to sibling nodes sharing the same parent, `search_agent` will be excluded from the available transfer targets:

```
math_agent._get_transfer_targets() # router, search_agent

math_agent = Agent(
    name="math",
    disallow_transfer_to_peers=True
)
math_agent._get_transfer_targets() # router
```

The strength of the Transfer pattern is that you can create various multi-agent structures simply by changing the `sub_agents` configuration and the scope of selectable

target agents (e.g., `disallow_transfer_to_peers` setting). Supervisory patterns where one router manages multiple specialists, hierarchical patterns where agents delegate step by step, and network patterns where agents can call each other are all possible.

Use the Supervisory pattern when a single router can handle all routing decisions. This is the simplest structure and works for most use cases. Choose the Hierarchical pattern when tasks break down into multiple levels of delegation, such as a project manager routing to team leads who then route to individual specialists. The Network pattern fits scenarios where agents need to call each other directly, such as a debugging workflow where a tester and developer iterate back and forth (figure 9.11).

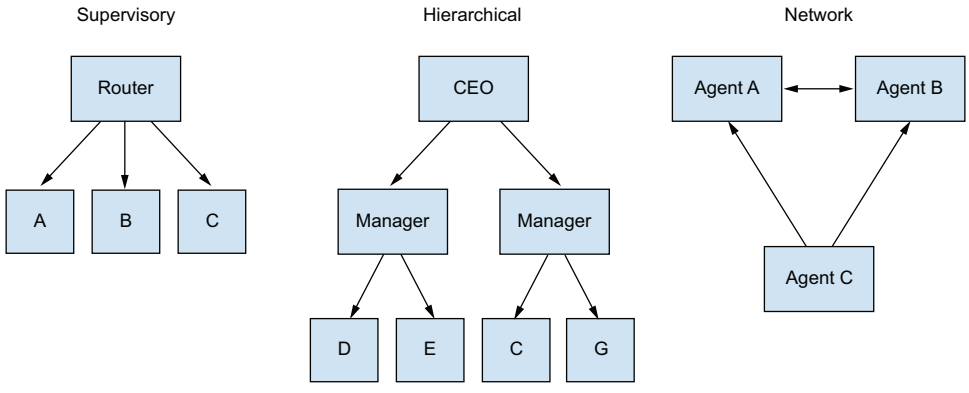

Figure 9.11 Agent tree structures: Supervisory pattern with single router, Hierarchical pattern with delegation levels, and Network pattern with peer-to-peer transfers

We also need a method to find an agent by name. When the transfer tool receives an agent name, it needs to find the actual agent object. The `_find_agent` method handles this.

The search must cover the entire tree. Regardless of where the current agent is in the tree, it should be able to find any agent given just the name. To accomplish this, we first traverse up to the root by following parent references, then search the entire tree from the root. `_find_in_subtree` works recursively. It first checks if its own name matches, and if not, performs the same search on each `sub_agent`. If found somewhere in the subtree, it returns that agent; if not found after traversing everything, it returns `None`.

Listing 9.13 Agent tree search by name

```
def _find_agent(self, name: str) -> Optional["Agent"]:
    """Search by name in the agent tree."""
    # Find root
    root = self
    while root.parent:
        root = root.parent
```

```
    # Search entire tree from root
    return root._find_in_subtree(name)

def _find_in_subtree(self, name: str) -> Optional["Agent"]:
    """Search in current agent and subtree."""
    if self.name == name:
        return self
    for sub in self.sub_agents:
        if found := sub._find_in_subtree(name):
            return found
    return None
```

9.5.4 *Implementing the transfer tool*

Now it's time to create the core transfer tool. The `create_transfer_tool` function receives a list of transferable agents and creates the `transfer_to_agent` tool.

There are three key aspects of this function:

- It includes the list of transferable agents and each agent's role in the tool description. This helps the LLM decide which agent to transfer to.
- It adds an enum constraint to the `agent_name` parameter. This forces the LLM to select only valid agent names.
- It doesn't perform the actual transfer; it only sets the target in `context.transfer_to`. The actual transfer is handled in the `run()` method.

Listing 9.14 Transfer tool creation

```
def create_transfer_tool(target_agents: List["Agent"]) -> FunctionTool:
    """Create transfer tool from list of transferable agents."""

    # Compose agent info
    target_names = [agent.name for agent in target_agents]
    agent_descriptions = []
    for agent in target_agents:
        desc = agent.description or agent.instructions[:100].replace('\n', '
     ')
        if len(desc) > 100:
            desc = desc[:100] + "..."
        agent_descriptions.append(f"  - {agent.name}: {desc}")

    agent_info = "\n".join(agent_descriptions)

    @tool(
        name="transfer_to_agent",
        description=f"""Transfers work to another agent.

Use this tool when the current question belongs to another agent's specialty.

Available agents:
{agent_info}
"""
```

```
    )
    def transfer_to_agent(context: ExecutionContext, agent_name: str) -> str:
        """Agent transfer tool."""
        if agent_name not in target_names:
            return f"Error: '{agent_name}' is not valid. Available: {target_
names}"

        # Only apply first transfer request
        if context.transfer_to is None:
            context.transfer_to = agent_name
            return f"Transferring to {agent_name}..."
        else:
            return f"Transfer already requested to {context.transfer_to}"

    # Add enum constraint
    transfer_to_agent.tool_definition["function"]["parameters"]["properties"]
["agent_name"]["enum"] = target_names

    return transfer_to_agent
```

The generated tool definition looks like this:

```
{
    "name": "transfer_to_agent",
    "description": "Transfers work to another agent...",
    "parameters": {
        "type": "object",
        "properties": {
            "agent_name": {
                "type": "string",
                "enum": ["math", "search"]  # Only valid targets
            }
        },
        "required": ["agent_name"]
    }
}
```

9.5.5 Implementing agent transfer

All preparations are complete. Now we need to integrate everything across several methods in the `Agent` class. Let's implement the flow in order: registering the transfer tool, passing it to the LLM, executing it, and detecting the transfer.

REGISTERING THE TRANSFER TOOL

First, register the transfer tool at the start of `run()`. Calculate the current agent's transfer targets, create the `transfer_to_agent tool`, and store it in `context.transfer_tools`. Skip if already registered.

Listing 9.15 Registering the transfer tool in `run()`

```
async def run(
    self,
```

```
    user_input: Optional[str] = None,
    context: Optional[ExecutionContext] = None,
    verbose: bool = False,
) -> AgentResult:

    context = context or ExecutionContext()

    # Register transfer tool (if not already)
    if self.name not in context.transfer_tools:
        targets = self._get_transfer_targets()
        if targets:
            from .transfer import create_transfer_tool
            context.transfer_tools[self.name] = create_transfer_tool(targets)

    # ... rest of execution logic ...
```

INCLUDING THE TRANSFER TOOL IN THE LLM REQUEST

The registered transfer tool must be included in the request so the LLM can select it. In `_prepare_llm_request()`, retrieve the current agent's transfer tool from `context.transfer_tools` and add it to the tool list.

Listing 9.16 Including the transfer tool in the LLM request

```
async def _prepare_llm_request(self, context: ExecutionContext) ->
LlmRequest:
    # Existing tool list
    llm_tools = [t for t in self.tools if t.tool_definition is not None]

    # Add transfer tool
    if self.name in context.transfer_tools:
        llm_tools.append(context.transfer_tools[self.name])

    # ... create LlmRequest ...
```

EXECUTING THE TRANSFER TOOL

When the LLM calls `transfer_to_agent`, it must be executed in `act()`. Add the transfer tool to `tools_dict` when searching for tools.

Listing 9.17 Adding transfer tool to `act()`

```
async def act(
    self,
    context: ExecutionContext,
    tool_calls: List[ToolCall],
) -> List[ToolResult]:

    tools_dict = {tool.name: tool for tool in self.tools}

    # Include transfer tool
    if self.name in context.transfer_tools:
```

```
            transfer_tool = context.transfer_tools[self.name]
            tools_dict[transfer_tool.name] = transfer_tool

    # ... tool execution logic ...
```

When the transfer tool executes, the target agent name is set in `context.transfer_to`. The actual transfer hasn't happened yet.

DETECTING AND EXECUTING TRANSFER

Back in `run()`, detect and handle transfer in the execution loop. After `step()` execution, if `context.transfer_to` is set, find the corresponding agent and call `run()`.

Listing 9.18 Detecting and executing the transfer

```
# Execution loop inside run()
while not context.final_result and context.current_step < self.max_steps:
    await self.step(context)

    # Check for transfer
    if context.transfer_to:
        next_agent = self._find_agent(context.transfer_to)
        context.transfer_to = None  # Reset flag

        if next_agent:

            # Delegate (pass only context, no user_input)
            await next_agent.run(None, context, verbose)
            break  # Exit current agent's loop

    # Check for final response
```

When a transfer occurs, `break` exits the current agent's execution loop. The transferred agent takes over the work and leaves the result in context, so the current agent has nothing more to do. The final result is stored in `context.final_result`, so we return it at the end of `run()`.

9.6 A2A: Collaborating across networks

All the multi-agent systems we've implemented so far operate within the same process. Whether running agents sequentially through Workflows, calling them as tools via `AgentTool`, or transferring control with Transfer, all agent objects must exist in the same codebase.

But different situations arise in practice:

- You want to use an agent that another team already operates.
- Security requirements mandate running certain agents on separate servers.
- You need to connect with the agent services provided by partner companies.
- Agents are developed in different languages or frameworks.

In these cases, you cannot directly import agent objects. You need a standardized way to communicate over the network. The A2A (Agent-to-Agent) protocol solves exactly this problem. As long as agents follow the A2A protocol, they can collaborate regardless of language or framework.

A2A is an open protocol introduced by Google that defines how agents built with different frameworks and languages can discover each other and collaborate over the network through standardized HTTP-based communication.

Let's go over A2A's core concepts and explore how to expose our agents to the network or call remote agents. The code presented here is not a complete, executable implementation but rather examples to help you understand A2A's key concepts. Actual implementations should follow the detailed specifications of the A2A library. For detailed specifications and advanced features, refer to the official A2A documentation (https://a2a-protocol.org/latest/).

9.6.1 The core of A2A: Agent card and task request/response

A2A (figure 9.12) follows an HTTP-based client-server model. The side exposing an agent to the network is the server, and the side calling a remote agent is the client. A2A communication centers on two things: the *agent card* (agent information) and *task request/response*.

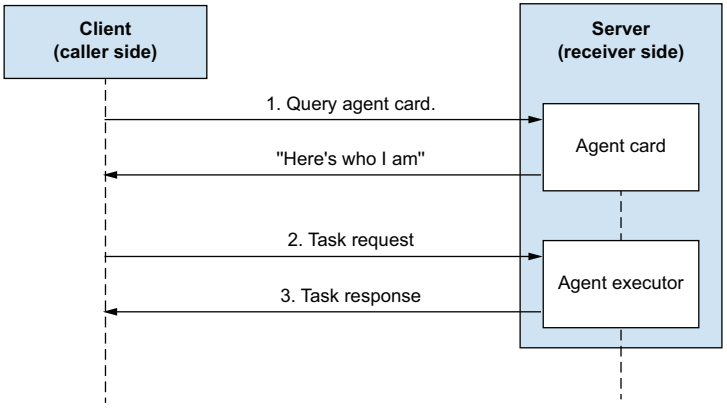

Figure 9.12 A2A client-server model: client queries agent card and sends task requests, server provides agent information and executes tasks.

AGENT CARD: THE AGENT'S BUSINESS CARD

An agent card is a JSON document where an agent introduces itself. The server provides this at the `/.well-known/agent.json` path. Clients use this information to understand "what kind of agent this is" and "where to send requests":

```
from a2a.types import AgentCard, AgentCapabilities

agent_card = AgentCard(
```

```
    name="math_expert",
    description="Specializes in mathematical calculations and equation solving",
    url="http://localhost:9999/",        # Address to receive requests
    version="1.0.0",
    capabilities=AgentCapabilities(streaming=True),
)
```

The `name` field serves as the agent identifier, while `description` explains what the agent does and is used by LLMs when making routing decisions. The `url` specifies the endpoint where clients should send their requests, and `capabilities` indicate features like streaming support.

Remember when we created the `transfer_to_agent` tool in section 9.5 and included the `name` and `description` of available agents in the tool description? Agent card information serves the same purpose. It helps clients decide which agent to send requests to.

TASK REQUEST AND RESPONSE

Clients send task requests to the `url` from the agent card, and servers execute the agent and return results. A2A standardizes the format of these requests and responses. Ultimately, A2A is HTTP communication. Servers provide agent cards and process task requests. Clients query agent cards and send task requests. That's all there is to it.

9.6.2 Server: Exposing an agent to the network

To make our agent callable from other systems, we need to expose it as an A2A server. There are only two things to implement:

1. *Provide agent card*—Return agent information at `/.well-known/agent.json`.
2. *Process task requests*—Receive requests, run the agent, and return responses.

AGENTEXECUTOR: THE CORE TASK REQUEST HANDLER

`AgentExecutor` is an adapter that receives A2A-formatted requests, runs our agent, and returns results in A2A format.

In the following code, the `execute` method is the key. It extracts user input from A2A's `RequestContext`, calls our `Agent`'s `run()` method, then converts and returns the result in A2A format. In other words, it's a translation layer between the A2A protocol and our agent:

```
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.utils import new_agent_text_message

class MathAgentExecutor(AgentExecutor):
    """Adapter connecting A2A requests to our Agent."""

    def __init__(self, agent: Agent):
        self.agent = agent

    async def execute(
        self,
```

```
        context: RequestContext,  # A2A-formatted input
        event_queue: EventQueue,
    ) -> None:
        # 1. Extract user input from A2A format
        user_input = context.get_user_input()

        # 2. Run our Agent
        result = await self.agent.run(user_input)

        # 3. Convert result to A2A format and respond
        await event_queue.enqueue_event(
            new_agent_text_message(result.output)
        )
```

RUNNING THE SERVER

Combine the agent card and `AgentExecutor` to run an A2A server. Most of the following code is configuration as defined by the A2A library. The key elements are `agent_card` (agent information) and `agent_executor` (request handler). The A2A library handles the rest, including HTTP endpoint configuration and request routing:

```
import uvicorn
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import InMemoryTaskStore

# Key 1: Define Agent Card
agent_card = AgentCard(
    name="math_expert",
    description="Specializes in mathematical calculations and equation solving",
    url="http://localhost:9999/",
    version="1.0.0",
    capabilities=AgentCapabilities(streaming=True),
)

# Key 2: Request handler (AgentExecutor)
request_handler = DefaultRequestHandler(
    agent_executor=MathAgentExecutor(math_agent),
    task_store=InMemoryTaskStore(),
)

# Create and run A2A server (handled by library)
server = A2AStarletteApplication(
    agent_card=agent_card,
    http_handler=request_handler,
)

uvicorn.run(server.build(), host="0.0.0.0", port=9999)
```

Now the agent card can be queried at `http://localhost:9999/.well-known/agent.json`, and task requests to that URL will be processed by `MathAgentExecutor`.

9.6.3 Client: Calling remote agents

The client is the opposite of the server. Its core job is to check remote agent information through the agent card and send task requests to that agent.

RemoteAgent: Using remote agents like local ones

`RemoteAgent` hides HTTP communication behind the agent interface. During initialization, it queries the agent card to set `name` and `description`, and when `run()` is called, it sends task requests to the remote server. From the caller's perspective, there's no way to distinguish it from a local agent.

The `__init__` calls `_load_agent_info()` to fetch agent card information in advance. This ensures that when `RemoteAgent` is registered in `sub_agents` or wrapped with `AgentTool`, the `name` and `description` are already set:

```
# Create and run A2A server (handled by library)
import httpx

class RemoteAgent(Agent):
    """Wraps a remote A2A agent to be used like a local Agent."""

    def __init__(self, base_url: str):
        self.base_url = base_url
        self._agent_card = None

        # Load Agent Card information during initialization
        self._load_agent_info()

    def _load_agent_info(self):
        """Query Agent Card and set metadata."""
        with httpx.Client() as client:
            response = client.get(
                f"{self.base_url}/.well-known/agent.json"
            )
            self._agent_card = response.json()

        # Extract metadata from Agent Card
        self.name = self._agent_card["name"]
        self.description = self._agent_card["description"]

    async def run(
        self,
        user_input: str,
        context: ExecutionContext = None,
        verbose: bool = False,
    ) -> AgentResult:
        # Send task request to remote agent
        async with httpx.AsyncClient() as client:
            response = await client.post(
                self._agent_card["url"],
                json={"input": user_input},
            )

        # Process response
```

```
        result = response.json()
        output = result.get("output", "")

        return AgentResult(output=output, context=context)
```

INTEGRATING WITH EXISTING PATTERNS

By implementing `RemoteAgent` to inherit from the agent, we can integrate remote agents using the `AgentTool` and the Transfer approaches we explored earlier. As shown in the following code examples, you can include `RemoteAgent` in `sub_agents` to enable transfers, or wrap it with `AgentTool` to call it like a tool:

```
# Transfer approach: Register in sub_agents to enable transfers
router = Agent(
    name="router",
    sub_agents=[
        local_search_agent,
        RemoteAgent(base_url="http://partner.com"),  # Remote agent
    ],
)

# Agent as Tool approach: Call like a tool
orchestrator = Agent(
    tools=[
        search_web,
        AgentTool(RemoteAgent(base_url="http://math-service.internal")),
    ],
)
```

Whether local or remote, the way you work with agents is identical. This is the core value of A2A. Even across network boundaries, even with different languages or frameworks, you can compose agents using the same patterns. A2A includes various features for network environments, such as streaming and asynchronous task processing. These advanced features can be explored in detail in the official A2A documentation.

When deploying agents across networks, security is a critical concern. You should consider authentication between agents (e.g., API keys or OAuth tokens to verify that a calling agent is authorized), communication encryption (HTTPS/TLS for all inter-agent traffic), and access control (defining which agents can request which operations). The A2A specification addresses these needs with built-in support for authentication schemes and push notification security. Always treat inter-agent communication with the same rigor as any public API.

With the multi-agent patterns from this chapter, you can build systems where specialized agents collaborate on complex tasks. In the next chapter, we'll shift focus from building agents to deploying and operating them in production, covering testing strategies, evaluation frameworks, and observability practices that ensure your agents perform reliably in the real world.

Summary

- Single agents face limitations as responsibilities grow. Longer system prompts and more tools make it harder for the LLM to focus on what's relevant, leading to confusion and incorrect tool selection.
- Multi-agent systems address this by assigning each agent a specialized role with focused prompts and dedicated tools. Benefits include separation of concerns, specialization, reusability, and scalability.
- Three collaboration patterns exist, distinguished by two criteria: who decides the next agent (developer or LLM) and whether context is shared or isolated.
- The Workflow pattern has the developer define execution order in code. By making workflows inherit from `Agent`, you can compose Sequential, Parallel, and Loop patterns freely. Sequential passes context from one agent to the next, Parallel runs agents concurrently, merges results, and Loop repeats until a stop condition is met.
- The Agent as Tool pattern wraps agents with the `BaseTool` interface using `AgentTool`. The orchestrator calls specialists like functions, receiving only final results while intermediate processes remain hidden. This provides abstraction and context efficiency through isolation.
- The Transfer pattern lets the LLM decide which agent should handle a request by calling a `transfer_to_agent` tool. Unlike Agent as Tool, control is fully handed over, and context is shared, so the transferred agent continues the conversation directly.
- The agent tree structure defines transfer relationships through `sub_agents` and parent references. Each agent can transfer to children, parent, and optionally siblings, with name uniqueness enforced across the tree.
- A2A (Agent-to-Agent) protocol enables agents to collaborate across network boundaries. Servers expose agents through agent cards and process task requests, while clients use `RemoteAgent` to call remote agents with the same interface as local ones.
- These patterns are not mutually exclusive. Real systems often combine Workflow for overall structure, Agent as Tool for specialist consultation, and Transfer for domain-based routing, with A2A enabling cross-system integration.

10 Evaluating agents

This chapter covers

- Why agent evaluation must be automated
- Concrete procedures for automating evaluation
- How to put automated evaluation to work in practice

An agent goes beyond merely suggesting an answer. It decides for itself what information to seek, which tools to use, in what sequence to execute tasks, and when to stop. This autonomy is a powerful advantage, but a single small mistake can cascade into real-world costs and risks: payments, procurement, permission changes, and external communications. That is why rigorous evaluation is essential: to prevent agents from causing unexpected harm and to protect users from potentially significant losses.

Yet evaluating agents is truly difficult. Agent evaluation must be automated because the evaluation space is enormous. An agent's output is not the result of a single operation; it is formed through a chain of multistep reasoning and actions that interlock and accumulate. If you evaluate only the final output, it becomes difficult

to distinguish a correct result that happened by luck from one produced by consistently sound reasoning. Proper evaluation requires examining intermediate reasoning, tool selection, and action decisions. As autonomy increases, the number of possible action paths expands, and with it, the number of failure points and evaluation dimensions grows exponentially. Without automation, evaluation becomes the bottleneck that blocks development and improvement.

But even defining good performance is not straightforward. For tasks with clear ground truth, such as fruit classification, evaluation is relatively simple. For open-ended tasks such as writing or problem solving, the first challenge is establishing an explicit rubric: what counts as success and by what criteria quality should be judged. In other words, automation is essential, yet building an automated evaluation is itself extremely difficult.

We address this dilemma in this chapter (figure 10.1) by walking step by step through the procedures required to automate evaluation. We begin by briefly surveying what aspects of an agent should be evaluated; then, we discuss how to collect the data needed for evaluation, how to define evaluation criteria, and how to perform evaluation using LLMs. Finally, we connect evaluation to continuous integration/continuous delivery (CI/CD) to present a system for continuously checking and improving quality after deployment.

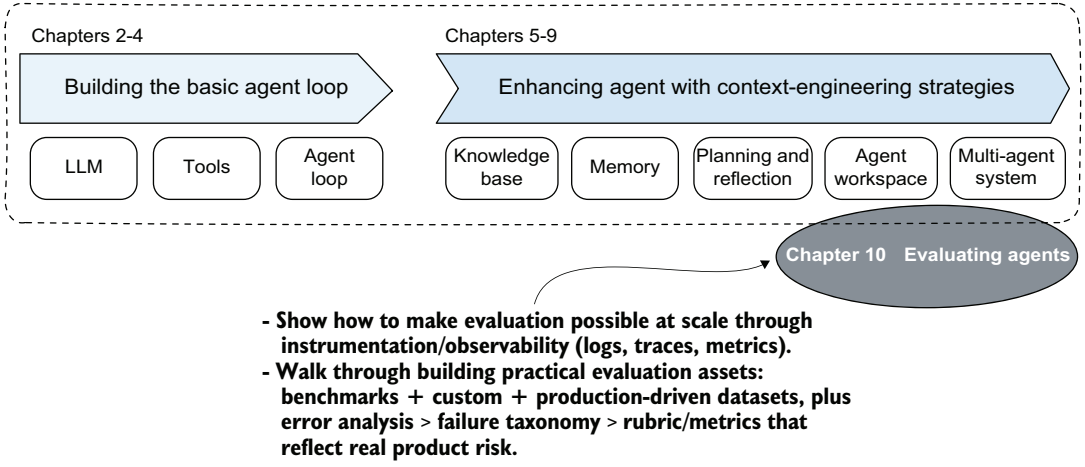

Figure 10.1 Overview of chapter 10

10.1 Observing an agent

Observability is tightly coupled to evaluation because the data you collect during system operation becomes the input for evaluation. Observability means making every layer of an agent visible, from input handling to reasoning, tool use, and final output.

By capturing internal signals, developers can analyze agent behavior, detect failures, and evaluate performance over time.

Observability simplifies debugging during development, preserves detailed logs for performance evaluation, and helps you understand and control the behavior of large language models (LLMs), which can be difficult to predict. Even before the rise of agents, observability was important in software engineering. But although traditional monitoring focuses on infrastructure-level signals such as CPU use, memory, and request counts, agent observability must also address LLM-specific challenges: nondeterministic behavior, hallucinations, and the effectiveness of prompt engineering.

To support this, systems collect *telemetry*: the signals and operational data generated by a system and transmitted for monitoring and analysis. In the context of agents, telemetry can include latency metrics, traces of reasoning steps, logs of tool invocations, token-use statistics, and error reports.

Telemetry is the raw data layer that enables observability. When the data is collected, it can be used for evaluation, allowing developers to measure an agent's reliability, cost, performance, and behavioral quality.

With observability in place, we can understand what happened inside the agent. We can also monitor these signals, detect threshold violations or anomalies, and respond, such as by identifying LLM call latency spikes or increased tool-call failure rates.

10.1.1 Pillars of observability: Metrics, traces, and logs

Observability systems typically rely on three complementary types of telemetry: metrics, traces, and logs. Metrics tell you that something is happening, logs tell you what happened, and traces show you how it happened.

METRICS

Metrics are numerical measurements of a system's or component's state at a point in time. Metrics are an effective way to understand how smoothly the system is operating. Because metric payloads are typically small, they can be collected efficiently even at a large scale. Metrics are particularly useful for dashboards, alerting systems, and performance tracking. Examples include

- Latency per `ToolCall`
- Total task-completion time
- API call frequency
- Token use per interaction
- Cost per task completion
- Context window use
- LLM call error rate

TRACES

Traces capture the path a request takes through the system. In other words, a trace shows the full journey of a single request. If a user says, "Plan a round-trip KTX itinerary to

Busan," the entire workflow, from interpreting the request to invoking tools and generating the final response, constitutes a single trace. Because traces can become large, they are subdivided into spans and events.

A *span* is a unit of work within a trace. An LLM call, a tool invocation, a memory read, or a planning step can be a span. An *event* is a notable instantaneous occurrence within a span: an exception, the start of a retry, an external response, user feedback, and so on. Events have no duration—only timestamps. Put differently, a trace is the story of the entire request, a span is a scene in that story, and an event is an incident within a scene. Using traces, spans, and events, you can transform a heap of logs into a coherent flow, making clear when, where, and why things happened.

Logs

Logs capture information such as authentication failures, service start and stop events, error messages from API calls, and tool invocation inputs and outputs. Logs provide rich contextual information that is essential for debugging. But logging everything can cause data volume to explode, making analysis harder rather than easier. For this reason, logs are typically used selectively alongside metrics and traces.

10.1.2 Generating, collecting, and exporting telemetry: OpenTelemetry

To implement observability, systems must generate, collect, and export telemetry data. This process is often referred to as *instrumentation*. Instrumentation involves inserting code into the system so that important events, such as LLM calls, tool invocations, and workflow steps, emit telemetry data. That data must be collected, processed, and exported to an observability backend where it can be visualized and analyzed.

One of the most widely adopted frameworks for this purpose is OpenTelemetry (OTel). OpenTelemetry is an open source, vendor-neutral standard for collecting and exporting telemetry data. It provides SDKs, APIs, and instrumentation libraries that allow developers to capture metrics, traces, and logs in a consistent way.

Its biggest advantage is vendor neutrality. If you instrument your service using a proprietary monitoring solution, switching platforms later often requires rewriting all instrumentation code. OpenTelemetry eliminates this lock-in by defining standardized telemetry formats and export mechanisms.

This interoperability is enabled by the OpenTelemetry Protocol (OTLP), which specifies how telemetry data is structured and transmitted. Using OTLP, telemetry collected from an agent can be exported to many observability platforms.

In practice, developers use OpenTelemetry to instrument key parts of an agent system, such as LLM requests and responses, tool calls, planning, and reasoning steps. These signals allow developers to trace the internal execution of an agent and analyze performance, reliability, and cost.

Although OpenTelemetry provides the instrumentation layer, the collected data is often visualized and analyzed using specialized observability or evaluation platforms. Commercial tools such as Langfuse, LangSmith, Arize AI, and Weights & Biases provide

dashboards and evaluation tools for analyzing LLM behavior, inspecting traces, tracking token use, and running systematic evaluations.

In many production systems, OpenTelemetry is used to generate and export telemetry, while these platforms serve as the analysis and evaluation layer on top of the collected data (table 10.1).

Table 10.1 What to instrument

What to record	Description
Input prompt	The user's request given to the agent
Intermediate reasoning	The agent's internal reasoning process (e.g., chain of thought)
Tool-call logs	Which tools were called and in what order
Tool outputs	The results returned by each tool
Final output	The response delivered to the user
Runtime/cost	Latency, number of API calls, token use, and so on

10.2 Building datasets and establishing evaluation criteria

Now that we've seen how to observe an agent, the next step is establishing the criteria for deciding whether the agent behaves correctly. Agent quality is not merely about whether an answer sounds plausible. Because agents can call tools that change real-world state, schedules can be modified, tickets can be created, and messages can be sent. Evaluation must include not only the agent's words but also the consequences of its actions.

The evaluation design process typically follows this flow: first, build a dataset containing representative user requests. Next, run the agent on this dataset and collect full execution traces, including tool calls, intermediate reasoning steps (if available), outputs, and errors. These traces provide the raw material for evaluation.

Then you perform error analysis by reviewing these traces and identifying recurring failure patterns. The goal at this stage is not to assign scores but to understand how and why the system fails. Humans read real execution traces and write free-form notes about what went wrong. It is usually best to focus on the first error that occurred. If the agent claims that a tool call succeeded when it failed, no matter how plausible the subsequent steps may appear, the core error is tool-result hallucination. Based on these notes, similar errors are grouped into a failure taxonomy. Next, you analyze their frequency and severity—that is, which problems occur most often and which ones cause the greatest damage.

Finally, using this failure taxonomy, you derive evaluation criteria for what good agent behavior looks like and organize them into a rubric. A *rubric* is a scoring guide that gives humans and models a common standard for evaluating agent behavior. Based

on this rubric, you define metrics that translate the evaluation criteria into measurable, trackable numbers.

Through this process, evaluation becomes not a vague impression but a practical quality-management tool grounded in real user behavior and system risk. Let's walk through each step (figure 10.2).

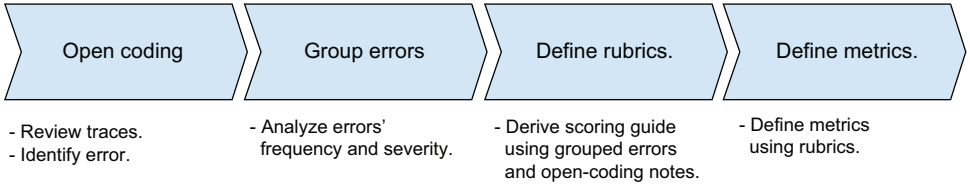

Figure 10.2 Workflow for defining agent evaluation criteria, from open coding to defining metrics

10.2.1 What should we evaluate?

Agent evaluation is not simply about whether the agent got the right answer. An agent selects and executes tools through multiple stages of reasoning and produces a final artifact based on those outcomes.

The problem is that this process is nondeterministic. Even with the same input, an agent can reach an answer through different paths each time. Sometimes, it searches first; at other times, it calculates first. Even if the final result looks good, you can't tell whether it was systematically derived or correct by coincidence. An agent that got lucky cannot guarantee the same quality next time.

Therefore, to evaluate an agent properly, you must examine not only the final result but also how the agent arrived there. In practice, agent evaluation typically considers three perspectives (figure 10.3):

- *Final output*—Does the delivered result satisfy the requirements?
  Example: For a summarization request, does it accurately capture the key points without omissions? Does it follow the required format (JSON, table, and so on)?
- *Component*—Did the individual modules that make up the agent perform their roles correctly?
  Example: Did the search tool return highly relevant documents? Did the LLM interpret the retrieved results correctly? Did the code executor run without errors?
- *Trajectory*—Was the path the agent took to reach its goal (reasoning steps, tool-call sequence, and so on) reasonable and efficient?
  Example: Did it reach the answer via a minimal path without unnecessary tool calls? Did it wander in the wrong direction and then backtrack?

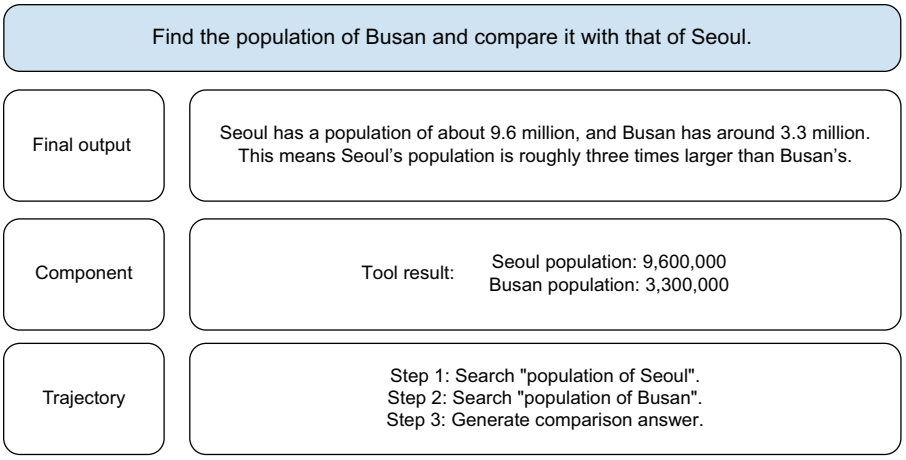

Figure 10.3 Three aspects to consider in agent evaluation

Evaluating these aspects requires data that allows the agent to be tested under realistic conditions. In other words, we need a collection of representative user requests that can be repeatedly executed and analyzed.

This is the role of an evaluation dataset. By running the agent on a well-designed dataset, we can observe its outputs, component behavior, and trajectories across many scenarios.

10.2.2 Creating a dataset

An evaluation dataset can be created in several ways, depending on the stage of development. In practice, teams typically construct datasets using three main sources:

- *Benchmarks*—Publicly available evaluation datasets
- *Custom datasets*—Internally designed scenarios reflecting product use cases
- *Production data*—Real user interactions collected after deployment

Each source requires a different creation process. The following sections explain not only what these datasets are but also how they are typically constructed and used (table 10.2).

Table 10.2 Types of evaluation datasets

Type	Description	Pros	Cons
Benchmark	Standardized public evaluation sets (e.g., HotpotQA, GAIA, and SWE-bench)	Comparable across systems; validated difficulty and quality	May diverge from real use; risk of overfitting to the benchmark
Custom	Internally constructed evaluation sets designed to reflect real scenarios	Tailored to product needs; domain-specific evaluation	High build cost

Table 10.2 Types of evaluation datasets (*continued*)

Type	Description	Pros	Cons
Production	Real user requests and feedback collected in production	Most realistic evaluation; reflects real use patterns	Often lacks ground-truth labels

Benchmarks are typically the starting point in evaluating an agent. Because they are publicly available and standardized, they make it easy to verify baseline capability and compare results across systems. Still, benchmark performance should be interpreted carefully. These datasets are created in controlled environments and may not reflect the kinds of questions users ask in real applications. As a result, strong benchmark scores do not necessarily translate to strong product performance.

Several widely used benchmarks are available for evaluating LLM-based agents:

- *General AI Assistants (GAIA)*—Evaluates general AI assistants on complex multi-step reasoning and tool-use tasks
- *SWE-bench*—Evaluates an agent's ability to resolve real GitHub issues by modifying code in large software repositories
- *HotpotQA*—Focuses on multihop question answering that requires reasoning across multiple documents

These benchmarks are typically distributed through public repositories such as GitHub and dataset platforms like Hugging Face, where both the dataset and evaluation scripts can be downloaded.

Custom datasets are critical in evaluating agents for real deployment scenarios. They are constructed internally to reflect the kinds of tasks users will perform with the system. When preparing a custom dataset, it is useful to include examples across multiple difficulty levels, domains, and edge cases. Because building such datasets can be expensive, teams often begin with a small set of core scenarios, sometimes only a few dozen examples, and expand the dataset over time.

Production data becomes especially valuable after deployment. Logs from real user interactions reveal what users truly ask and which responses they find satisfactory or unsatisfactory. This makes production data one of the most realistic evaluation sources. But it often lacks ground-truth labels, which means additional human annotation may be required before the data can be used in formal evaluations. If you build your own test set, it typically falls into one of three categories:

- *Representative core use cases*—Includes the key scenarios most users execute most frequently. The purpose is to validate overall performance and stability—specifically, "Does the agent consistently produce expected outcomes in normal situations?" This category usually occupies the largest portion of the test set.
- *Complex and challenging edge cases*—Includes scenarios in which inputs are ambiguous, multiple constraints apply simultaneously, or the agent is likely to

misjudge. Examples include cases in which incomplete information is provided, multiple tool calls must be chained, and a small error early in the process has a large downstream impact. These cases are important for evaluating robustness and error-handling capabilities.

- *Adversarial or stress-testing inputs*—Includes malicious or abnormal inputs, attempts to bypass policies, or prompts designed to exploit agent weaknesses. Examples are cases in which unclear instructions are changed repeatedly, contradictory demands are made simultaneously, and inputs try to induce the misuse of tools. This category is essential for validating safety, policy compliance, and exception handling.

A test set that balances these three types allows you to evaluate how well the agent performs in everyday scenarios, how reliably it handles difficult conditions, and how safely it behaves under unexpected inputs. Practically, it helps to begin with representative core scenarios, expand to harder cases, and finally add adversarial/abnormal inputs so you can improve quality and stability step by step without taking on unnecessary complexity too early.

10.2.3 Analyzing errors

After creating a dataset, the next steps are running the agent on that dataset and collecting traces. These traces provide the data needed for error analysis. Error analysis determines what should become evaluation criteria. You could track countless metrics in an agent system. Error analysis grounds you in actual data: you identify failure patterns, clarify what to fix first, and decide what tests you need. Let's walk through an example error analysis for a research agent.

STEP 1: OPEN CODING

Open coding is a qualitative analysis method in which evaluators read traces and describe problems in their own words rather than choose them from predefined categories. This work is usually performed by people who understand the system and its domain, such as

- Product engineers building the agent
- Domain specialists familiar with the task (e.g., support experts for a customer-support agent)
- Evaluation or QA engineers responsible for model quality

Evaluators review the full trace of an interaction—including the user request, the agent's intermediate steps, tool calls, and the final output—and record short notes describing what went wrong. These notes are often collected in a spreadsheet or annotation tool with fields such as those shown in table 10.3.

Table 10.3 Example of open coding result

Input	Agent output	Tool trace	Evaluation note
"Summarize paper X."	Plausible summary	Tool call failed	"Tool failed, but agent claims success."

STEP 2: BUILD A FAILURE TAXONOMY

After open coding, similar observations are grouped into broader categories called a *failure taxonomy*. Open coding notes such as "Citations do not match the source," "Sources unclear," and "Answer sounds plausible but doesn't answer the question" can be grouped in categories such as evidence/citation failures, reasoning/synthesis failures, and format/requirement compliance failures.

This taxonomy provides the foundation for evaluation design. Each category can later be translated into rubric items and measurable metrics that can be applied automatically or semiautomatically during evaluation.

STEP 3: REPEAT

If you no longer see new failure types emerging in new traces, the first taxonomy pass is complete. Repeat the process whenever there are major changes. For each error analysis cycle, we recommend reviewing 50–100 samples.

10.2.4 Designing rubrics and metrics

Through error analysis, we turned "what went wrong" into a shared language. Now we connect those findings to consistent evaluation standards, rubrics, and quantitative indicators, metrics, so we can reliably catch those failures.

RUBRIC: DESIGNING EVALUATION CRITERIA

A rubric stabilizes evaluation by making the criteria for good/bad explicit. It plays a crucial role in ensuring that evaluators grade cases consistently. Rubrics can be designed using Likert scales or binary decisions. A Likert scale asks evaluators to rate an item on a graded scale (for example, 1–5 from "very poor" to "excellent"), whereas a binary decision requires choosing between two options, such as `PASS` or `FAIL`. Binary-first designs are generally more stable. Explaining the difference between a 3 and a 4 is harder than explaining why something is `PASS` or FAIL:

- *Answer relevancy*
  - `PASS`—Directly answers the user's core question (does not evade or ramble with only background context).
  - `FAIL`—Provides a plausible summary but does not answer the core question or answers a different question.
- *Citation reliability (evidence/citation alignment)*
  - `PASS`—Key claims are supported by sources, and citations connect to the claims they support.
  - `FAIL`—There are no sources, citations differ from the original text, or sources are unrelated to the claims.
- *Requirement compliance*
  - `PASS`—Follows the requested format (table/bullets/comparison criteria/length/language).

  - `FAIL`—Breaks the format, omits required criteria, or goes beyond the requested scope.

METRIC: BUILDING TRACKABLE INDICATORS

If a rubric defines how to judge each case, a metric aggregates those judgments into numbers that can be tracked over time:

- *Overall pass rate*—The fraction of cases that PASS on answer relevancy, citation reliability, and requirement compliance
  Calculation—`OPR = (# all-pass) / (total evaluations)`
- *Answer relevancy rate*—`PASS` rate for the answer relevancy rubric item
- *Citation reliability rate*—`PASS` rate for the citation reliability rubric item
- *Requirement compliance pass rate*—`PASS` rate for the requirement compliance rubric item

10.3 Evaluating with LLM-as-a-judge

LLM-as-a-judge is a method that uses an LLM to evaluate the outputs of another system. In this approach, you define an evaluation prompt aligned with your rubric and ask the judge model to assign scores or make pass/fail decisions.

The main motivation for LLM-as-a-judge is scalability. Traditional metrics such as NLP metrics are fast and inexpensive, but they have two major limitations: they require reference answers, and they often fail to capture semantic nuance. As a result, they provide limited coverage for diverse natural-language outputs. Human evaluators, on the other hand, understand context and nuance well, but they are slow, expensive, and difficult to scale.

This scalability problem becomes especially important for agent systems. Agents are constantly evolving: prompts change, models are replaced, and data is updated. Each change requires validation. If evaluation relies solely on humans, the development cycle quickly becomes a bottleneck. LLM-as-a-judge addresses this challenge by using an LLM to grade agent responses according to a rubric, allowing large numbers of cases to be evaluated quickly and consistently. But LLM judges are only one type of evaluator. In practice, agent evaluation often combines several evaluation approaches.

When you have a dataset, you need someone—or something—to grade the agent's responses. Evaluators generally fall into four categories: code-based evaluators, LLM judges, human evaluators, and end-user feedback.

CODE-BASED EVALUATORS

A code-based evaluator automatically grades an agent's response using predefined rules or programs. The simplest version checks for exact string matches to the answer. More advanced versions use regular expressions, parse-tree comparisons, and function execution results. Typical examples include

- *Strict format validation*—Does the output satisfy a JSON schema? Are the required fields present?

- *Answer correctness*—Use for tasks with unambiguous answers (such as math and coding problems).
- *Execution-based evaluation*—Run generated code or SQL and check whether it passes test cases.
- *Constraint checks*—Use to find disallowed words, length limits, policy violations, and so on.

The main advantages of code-based evaluation are objectivity and reproducibility. The same input always yields the same result, and the evaluation criteria are explicit. Because code-based evaluation is inexpensive and fast, it is often used in CI pipelines to detect regressions after model or prompt changes.

But its limitations are equally clear. Code-based evaluation works only when the criteria can be precisely expressed in code. Qualities such as writing quality, logical flow, persuasiveness, and user friendliness are difficult to evaluate programmatically. For this reason, code-based evaluation is often insufficient for judging natural-language outputs.

LLM JUDGES

An LLM judge uses another language model to evaluate the agent's output. You provide the evaluation rubric as part of the prompt, and the judge model assigns scores or pass/fail decisions.

QS
```
Evaluate the agent response on a 1–5 scale based on:
Accuracy: Is the information factually correct?
Completeness: Does it fully answer the user's question?
Relevance: Does it focus on the core points without unnecessary content?
User question: {query}
Agent response: {response}
Reference answer: {reference}
Provide scores and justifications for each criterion.
```

Using an LLM to evaluate another LLM may seem recursive, but it offers a powerful advantage: scalability. LLM judges can evaluate thousands of cases quickly and often at a lower cost than human graders.

In general, use a stronger model as the judge than the one being evaluated. A common heuristic is to select a model with stronger reasoning capabilities or a larger parameter scale. If an agent was built with Claude Haiku, for example, developers may choose Claude Opus as the judge.

Judging with LLMs also incurs additional cost, so researchers are actively exploring whether smaller models can serve as effective evaluators. Studies such as "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment" (Yang Liu et al., 2023) and "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" (Lianmin Zheng et al., 2023) investigate whether smaller models can approximate the judgments of stronger models or be trained through distillation to replicate them.

HUMAN EVALUATION

Human evaluation involves professional graders or domain experts directly reviewing and assessing the agent's outputs. In many cases, this remains the most trustworthy evaluation method. Humans outperform LLMs in complex contextual understanding, subtle error detection, and subjective quality judgments. This is particularly important for qualitative criteria such as safety, ethics, brand tone, and communication style—areas in which automated methods still struggle.

But human evaluation is expensive and slow. Inter-rater agreement can be difficult to maintain, and evaluation quality may vary depending on evaluator fatigue. For this reason, human evaluation is rarely applied to every case. A common approach is a hybrid pipeline. First, an LLM judge evaluates large volumes of outputs; then, a subset of cases is routed to human evaluators for verification or deeper analysis. Evaluator training and calibration sessions are also important for ensuring consistent judgments.

END-USER FEEDBACK

After a system is deployed in production, another valuable signal becomes available: end-user feedback. This includes signals such as thumbs-up/-down ratings, star scores, and follow-up questions from users. End-user feedback reflects real user satisfaction and can scale easily across large numbers of interactions. It is particularly useful for monitoring quality trends over time.

Still, this signal is noisy. Most satisfied users leave no feedback, and dissatisfied users are more likely to respond. The same response may be rated differently depending on user expectations or mood. For these reasons, end-user feedback is often more useful for detecting relative changes than for measuring absolute quality.

If the thumbs-down rate suddenly increases after a new deployment, for example, it may indicate a regression. Collecting qualitative feedback alongside numerical ratings can also help identify improvement opportunities.

10.3.1 Types of LLM-as-a-judge

Among the evaluation approaches described earlier, LLM-as-a-judge has become especially popular for large-scale evaluation. There are several ways to implement LLM-based judging. Two of the most common methods are single-output grading and pairwise comparison.

SINGLE-OUTPUT GRADING

In single-output grading, the judge evaluates one output at a time and assigns a score according to the rubric. If an expected output exists, it can be provided alongside the agent response. Providing reference answers often improves scoring consistency, although preparing such references requires additional effort.

QS You are evaluating an AI-generated answer.

User request:

Summarize the key findings of this research paper.

Agent answer:

"The study finds that remote work increases productivity and reduces employee turnover."

Rubric:

1 Does the answer capture the key findings?

2 Is the summary concise and accurate?

Respond PASS or FAIL and explain briefly.

PAIRWISE

Pairwise evaluation compares two candidate outputs for the same input and asks the judge to decide which one better satisfies the evaluation criteria.

Because it relies on relative comparison rather than absolute scoring, pairwise evaluation is often simpler and more reliable for tasks such as A/B testing.

It is particularly useful for subjective judgments that are difficult to express with binary criteria, such as evaluating fluency, clarity, or helpfulness. In this setup, the judge selects A, B, or Tie, and the results are aggregated into statistics such as win rates.

QS You are comparing two answers to the same user question.

Question:

What are the benefits of renewable energy?

Answer A:

Renewable energy reduces carbon emissions and helps mitigate climate change.

Answer B:

Renewable energy sources such as solar and wind reduce greenhouse gas emissions, decrease dependence on fossil fuels, and improve long-term energy sustainability.

Which answer better satisfies the user's request?

Respond with:

A

B

Tie

and briefly explain your reasoning.

10.3.2 Building a rubric-based evaluation system

Now we build an evaluation system based on our rubric. Earlier, we chose to check answer relevancy, citation reliability, and requirement compliance. Therefore, at minimum, the judge needs the user query, the agent's final response, and the materials and metadata the agent saw. We'll provide the following:

- The user prompt
- The output to evaluate

- The rubric (evaluation criteria)
- If available, a reference (expected answer)

Based on this data, the judge will assess the thread.

QS

{
"query": "The user's question/request to the research agent",
"final_answer": "The agent's final response to be evaluated",
"evidence_pack": [
{
"source_id": "S1",
"title": "...",
"url": "...",
"excerpt": "Verbatim excerpt(s) the agent relied on (keep only the relevant parts)."
}
],
}

When the input format is standardized, you write the evaluator prompt:

QS

[System]

You are a "Research Agent Evaluator." Your job is to grade a research agent's final answer strictly against the rubric.

CRITICAL RULES:

- You MUST base your judgment ONLY on the provided evidence_pack (excerpts + source metadata). Do NOT use outside knowledge or general world knowledge to infer correctness.
- If a key claim is not supported by evidence_pack, it is a FAIL even if it sounds plausible.
- Do not reward verbosity, style, or friendliness. Evaluate only: relevance, evidence/citation alignment, and requirement compliance.
- If citations exist but the excerpt does not actually support the claim, mark it as "citation mismatch" and FAIL.
- Output MUST be a single JSON object only. No extra text.

[User]

Evaluate the following.

[INPUT]

query:

{query}

final_answer:

{final_answer}

evidence_pack (array of sources with source_id, title, url, excerpt):

{evidence_pack_json}

[RUBRIC]

Score EACH rubric item as PASS or FAIL (no partial credit).

1. Answer Relevancy
   - PASS: Directly answers the core intent of the query and provides a clear takeaway/conclusion.
   - FAIL: Does not answer the core question, reframes the question into something else, is mostly background/rambling, or lacks a conclusion.
2. Citation Reliability (Evidence/Citation Alignment)
   - PASS: Every key claim (conclusions, numbers, comparisons, causal statements) is supported by evidence_pack, and any citations used in the answer match the claim they support.
   - FAIL if ANY of the following occurs:
     a. A key claim lacks evidence/citation support
     b. Citations exist, but excerpts are irrelevant/insufficient ("citation mismatch")
     c. The answer asserts content not present in evidence_pack as fact ("unsupported insertion")
     d. The answer claims tools/sources said something when they did not ("source/tool hallucination")
3. Requirement Compliance
   - PASS: Fully satisfies ALL explicit requirements (format, language, scope, count, comparison criteria, etc.).
   - FAIL: Violates ANY explicit requirement or omits any required element.

[OUTPUT JSON SCHEMA]

{

"overall_pass": "PASS" | "FAIL",

"rubric": {

"answer_relevancy": "PASS" | "FAIL",

"citation_reliability": "PASS" | "FAIL",

"requirement_compliance": "PASS" | "FAIL"

},

"failure_reasons": [

{

"rubric": "answer_relevancy|citation_reliability|requirement_compliance",

"problematic_span_in_final_answer": "verbatim quote from final_answer",

"why": "one paragraph explanation grounded in the rubric",

"evidence_support": ["S1", "S2"] // source_ids used for this judgment; empty array if none

}

]

```
}
[DECISION RULE]
  – overall_pass is PASS only if ALL three rubric items are PASS. Otherwise, overall_pass is FAIL.
Now grade it.
```

10.4 Operations: CI/CD

In the previous sections, we built a method to automate evaluation. Now let's connect that automated evaluation to a CI/CD pipeline so we can enforce and maintain agent quality.

In production systems, agent behavior can change even with small updates such as prompt edits, tool integrations, and model upgrades. Without systematic checks, these changes can silently degrade answer quality, break requirements, or introduce new failure modes. For this reason, evaluation should not be treated as a one-time validation step during development. Instead, evaluation must become part of the operational pipeline. By integrating automated evaluation into CI/CD, teams can detect regressions early, prevent unverified changes from reaching production, and continuously maintain agent reliability as the system evolves.

10.4.1 Evaluation-gated deployment

The core principle is simple: without quality verification, there is no production deployment. You can create this gate through manual evaluation processes or automated gates inside the deployment pipeline.

Manual evaluation enforces predeployment verification through a process, and it can be suitable for teams just starting. Before opening a PR, you run the evaluation locally and include the results as required PR artifacts. Reviewers examine these artifacts alongside the code.

Automated gates inside the deployment pipeline integrate the evaluation harness directly into CI/CD. If evaluation fails, deployment is automatically blocked, preventing unverified changes from reaching production and enforcing quality standards consistently. You can programmatically block deployment if the overall pass rate falls below a threshold, for example. Concretely, building an automated gate in the deployment pipeline looks like the following.

STAGE 1: PREMERGE INTEGRATION (CI)

This stage runs automatically when a PR is opened, serving as the first line of defense to keep the main branch clean. It runs fast, repeatable checks such as unit tests, linting, and dependency and vulnerability scans. Whenever prompts, policies, or flows change, the system executes tests automatically and uses a small test set (around 20–50 cases, including core scenarios) to confirm that the branch is healthy and performance has not regressed relative to the previous version.

STAGE 2: STAGING VALIDATION (CD)

After a change passes CI and is merged, you must verify that it can run as an integrated system. The Ops/platform team packages the agent and deploys it to staging (a production-like replica environment); then it runs heavier, more realistic validation that reflects operational concerns. This helps reproduce production failures in advance, such as the following:

- Load testing and long-run stability testing (memory leaks, rate limits, and retry storms)
- Remote-service integration tests (permissions/auth/quota/network-failure scenarios)
- Evaluation using a larger test set than the CI stage

10.4.2 Improving the test set and evaluator: Agent quality flywheel

If evaluation does not earn the trust of product developers, it will feel less like a helpful process and more like a gatekeeping obstacle. This perception typically stems from the sense that evaluation is disconnected from product value.

To avoid this, you must accept one fact clearly: evaluation is also a product, and it must continuously improve after deployment. In particular, an agent's failure patterns evolve as use environments, user behavior, and external tools/APIs change. A single test set and rubric are not sufficient to maintain quality in the long term.

As shown in figure 10.4, we must periodically review operational data, repeatedly funnel it through error analysis > taxonomy > rubric/metric, and then continuously incorporate those results into the evaluation dataset so that the evaluation keeps pace with reality.

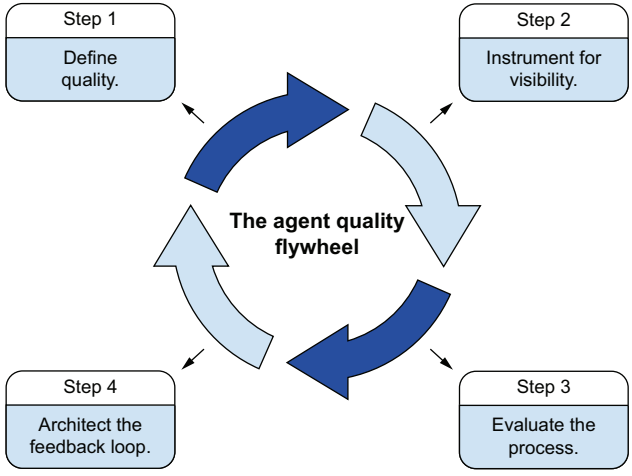

Figure 10.4 Agent quality flywheel is a continuous four-step loop for improving agent quality. The process begins by defining quality criteria (1), followed by instrumenting the system to gain visibility into real-world behavior (2). The collected data is used to evaluate the agent's performance and identify failure patterns (3). Finally, feedback loops are architected to update datasets, rubrics, and metrics (4), feeding insights back to step 1 to refine how quality is defined.

Over the course of this book, we've built an AI agent from the ground up. We started by understanding what an agent is and how LLMs serve as its brain (chapters 1–2); then we gave it the ability to act on the world through tools and the ReAct loop (chapters 3–4). From there, we expanded what the agent could know and remember by adding retrieval-augmented generation and memory (chapters 5–6), taught it to plan and reflect on complex tasks (chapter 7), and handed it the power to write and execute code (chapter 8). We moved beyond a single agent to orchestrate multiple agents working together (chapter 9). In this chapter, we confronted the question every production system must answer: how do you know whether any of this works?

That question, how to evaluate an autonomous system whose behavior is nondeterministic and whose failure modes evolve, is arguably the hardest problem in applied AI engineering today. No single metric captures agent quality, and no evaluation framework stays valid forever. What we've built instead is a process: observe, analyze, codify, automate, and repeat. If you take one lesson from this book, let it be that building an agent is not a one-time act of engineering but a continuous cycle of improvement, in which every deployment teaches you something new about what your agent can and cannot do.

The field of AI agents is moving fast. New models, tools, and architectures will emerge long after this book goes to print. But the fundamentals we've covered—the think–act loop, structured memory, planning, multi-agent coordination, and rigorous evaluation—are the building blocks that remain stable even as the surface-level technology shifts. Now you have the knowledge to build agents that are not just impressive demos but also reliable systems that earn user trust in production. Go build something worth evaluating.

Summary

- As agent autonomy increases, failures shift from minor bugs to real-world effects with meaningful cost.
- Evaluation must cover not only the final output but also the components and trajectory to separate luck from reproducible success.
- Instrumentation and observability are required because evaluation is impossible without execution records.
- Strong evaluation starts by converting failures to a shared language via trace review, open-coded error logging, and a failure taxonomy.
- Convert the most frequent and severe failures into a `PASS/FAIL`-first rubric. Metrics are a dashboard that turns rubrics into numbers to catch regressions and guide improvement.
- Embed automated evaluation in the deployment pipeline.
- Treat evaluation as a continuous flywheel because environments, users, and tools/APIs change: operational data > error analysis > taxonomy > rubric/metrics > test set updates.

appendix
OpenAI API Key

1. Create an OpenAI account. Visit https://platform.openai.com, and click Sign up. You can register using your email address or sign in with a Google or Microsoft account. After signing up, you'll need to verify your email address.
2. Add a payment method. Navigate to the billing section by clicking your profile icon and selecting Billing. Click Add Payment Method to register a credit or debit card. OpenAI requires a payment method on file even if you plan to use free credits.
3. Check credits and add funds. New accounts previously received $5–$18 in free credits, but this policy has changed. Check your current balance in the Billing section. If you have no credits, click Add to Credit Balance and add at least $5 to get started. The minimum charge is $5, and credits expire after one year.
4. Generate your API key. Go to the API Keys section (https://platform.openai.com/api-keys), and click Create New Secret Key." Give your key a descriptive name like agent-development, and click Create Secret Key.

WARNING Copy the key immediately; you won't be able to see it again. If you lose it, you'll have to create a new one. The key shouldn't be hardcoded into your application. Instead, store it securely in an `.env` file, and use the `python-dotenv` library to load it as an environment variable (figure A.1).

Figure A.1 Example of storing the OpenAI API key in a `.env` file

index

Numbers

404 error code 119

A

A2A (Agent-to-Agent) protocol 282
Abstract class, defining 169
abstraction 269
A/B testing 301
acompletion 29–31, 39, 110
Agent as Tool 266–271
  context isolation 269
  implementing AgentTool 267
  making agents into tools 267
  practical example 270
  researcher and coder collaboration 270
Agent as Tool pattern 256
agent card 282
Agent class 99, 100, 141, 227, 245, 253
AgentExecutor 283
agent loop 7
AgentResult class 101
  extending 176
agents 8–10
  adding memory to 167, 178
  code execution 224–230
  combining with workflows 10
  connecting tools to 138
  empowering with code execution 217–224
  evaluating 288, 290–292, 304–306
  extending with callbacks 140–148
  integrating sessions into 171
  LLM-directed flow 9–10
  memory 154–155, 168
  modifying initialization 172
  orchestrating multi-agent systems 258–266
  ReAct (reasoning and acting) agents 99–105
  tool creation and use with multistep loop 10
AI agents 3
  context engineering 13–18
  overview of 4
  prerequisites for reading book 18
  tasks that require agents 11–13
  workflow vs. agent 8–11
ANN (approximate nearest neighbor) 127
answer correctness 299
APIs (application programming interfaces), unfamiliar 66
Arize AI 292
ask() method 188
assistant role 28
async/await syntax 29
asyncio library 29

async with 73
autonomous control flow 6
autonomy, elements that realize 7
await keyword 30

B

BaseModel 28
BaseTool 88–99, 100, 177, 267

C

calculator function 54, 6, 88
callbacks 141
  implementing separation of storage from presentation via 155
chains, defined 9
child agents 275
ChromaDB, building vector store with 190–191
chunking 124, 131
CI/CD (continuous integration/continuous deployment) 289, 304–306
  evaluation-gated deployment 304–305
CLI (command-line interface) 217
closed LLMs (large language models) 21–22
code execution 216
  agent skills 242–250
  connecting code environments 224–230
  empowering agents with 217–224
    benefits of code execution 219–221
    effectiveness of code-based actions 222
    limitations of predefined-tool approach 218–219
  porting tools to sandboxes 231–238
  Workspace 238–242
code format (CodeAct) 222
collaborating across networks 281–286
  agent card 282
  remote agents 285–286
  servers 283–284
  task request/response 282
collaboration patterns 255–257
collecting telemetry 291
compaction strategy 159–161
complete status 176
completion function 27
compressing search results 146–147
confirmation, extending tools for 177
confirmation-processing logic 182
constraint checks 299
context, managing during execution 154
context efficiency 270
context engineering 13–18
  agent failures 14
  bigger context isn't always better 15
  five strategies 15
  journey of this book 17
  memory and 153
  prompt engineering to context engineering 14
continuous execution, session and state management 167
control flow 220
conversations, multiturn 173–174
core tools 250
coroutines, defined 30
cosine similarity 120, 124
cost optimization 22
customer-facing agents 4

D

data flow 220
Dataset object 132
data structures, for tool confirmation 175–176
debugging, with execution traces 111
deps (dependency management) 209
DuplicateCheckResult schema 190

E

E2B platform 225–227
Early Exit pattern 142
embedding 120, 123, 190–191
enum constraint 50
ERROR ANALYSIS situation 211
error cost 12
Euclidean distance 120
evaluating agents
  analyzing errors 296–297
  aspects to consider 293
  building datasets and establishing evaluation criteria 292–298
  creating datasets 294–296
  designing rubrics and metrics 297–298
  LLM-as-a-judge 298–304
  observing agents 290–292

Event class 101, 102
events 88, 291
ExecutionContext 85–88, 99, 101, 103, 104, 111, 147, 253
  agent execution 85–87
  extending 171, 228
  implementation 87
execution.error 227
execution object 225
exporting telemetry 291
external extensions 178
ExtractedInfo 28

F

FastMCP instance 75
folder-based structure 243
function calling 23
FunctionTool 90
  extending 232
  modifying 232
  source-code extraction method 233

G

GAIA benchmark 21
  testing with ReAct agents 109–112
GAIA (General AI Assistant) 4, 13, 116, 151, 253, 295
  preparing dataset 131
  solving zip-file problems 139
GaiaOutput 109
  Pydantic model 105
generated tools 250
generating telemetry 291
graph search 121

H

HF_TOKEN (Hugging Face token) 25, 36
hierarchical context management 164–166
hierarchical tool structure 249–250
HITL (human-in-the-loop) 116, 144
  workflows, complete example 184–186
HotpotQA 295

I

independence, defined 270
information extraction 188–189
inspect module 61–63
installable tools 250
instrumentation, defined 291
internal features 178
IoT (Internet of Things) 47
isolation, defined 224

J

JSON format 222

K

keyword search 119
knowledge bases
  building with RAG 115, 131, 138
  extending agents with callbacks 140–148
  search methods 118–123
  solving GAIA zip-file problems 139
  structure-based search 131
  using internal data 116–117

L

Langfuse 292
LangSmith 292
LiteLLM 26
LLM-as-a-judge 298–304
  building rubric-based evaluation system 301–304
  code-based evaluators 298
  end-user feedback 300
  human evaluation 300
  LLM judges 299
  types of 300–301
LlmClient class 95
  building messages 96
  extending 188
  parsing responses 97
LlmRequest class 94, 104
LlmResponse class 94, 95
LLMs (large language models) 4–5, 20, 45, 115, 151, 216, 290
  agents 5–8
  API basics for building agents 24–31
  building tools and tool definitions for 58–67
  capabilities for agents 22–24
  choosing for agents 21–24
  closed 21–22

LLMs (large language models) *(continued)*
communication layer 93–98
from to agent 109
limitations of 35–44
LLM call 9
multimodal LLMs 11
open 22
prompt engineering 31–35
tasks that require 11
tools and 47–58
logs, defined 291
long context support 23
long-term memory 151, 186
structure of 187
loop workflows 263–265

M

MAX_CONNECTIONS 119
MCP Inspector 71
MCP (Model Context Protocol) 17, 45, 219
client 72–74
server-client architecture 68–77
MCP (multi-channel processing) tools, integrating 92
memory 149
anatomy of 150–154
building vector store with ChromaDB 190
compaction strategy 159–161
continuous execution, session and state management 167
data structures for tool confirmation 175–176
extending tools for confirmation 177
hierarchical context management 164–166
implementing pause and resume in agent 178–182
information extraction 188–189
integrating sessions into agents 171–172
long-term memory 186–187
managing context during execution 154
managing sessions with SessionManager 169–170
multiturn conversations 173–174
retrieving 196–198
separating storage from presentation 154–155
session class 168
sliding-window strategy 156–157
summarization 162, 164
TaskMemoryManager 191–196
token counting 157–158
MemoryTool 196
messages
building 96
list 28
metadata 191
metrics 290, 297
building trackable indicators 298
designing rubrics 297
multi-agent systems
benefits of 253
collaborating across networks 281–286
collaboration patterns 255–257
orchestrating 258–266
multimodal LLMs 11
multiply operator 51
multistep loops 10

N

npm packages 70

O

OOP (object-oriented programming) 18
OpenAI API 24, 25
key 307
open coding 296
orchestrating multi-agent systems 252
Agent as Tool 266–271
transfer pattern 271–280
orchestrator 269
OTel (OpenTelemetry) 291

P

parallel workflows 261
parameters section 50
parent agent 275
pause, implementing 178
design principles 179
implementing confirmation-processing logic 182
internal features vs. external extensions 178
modifying act() method 179
modifying _prepare_llm_request() method 181
modifying run() method 180
PDF merge skill 248–249

pending status 176, 181
pending tool call 175
personal AI agents 4
planning
  extension directions 208
  implementing tool 205–207
  integrating with reflection 213–215
  setting direction 204–209
  tool example 207
  when necessary 205
planning and reflection 201
  reflection 209–213
  time to think 202–204
PRD (product requirements document) 203
predefined-tool approach 218–219
pre-execution blocking 179
problem of time 151
PROGRESS REVIEW situation 211
prompt, defined 13
prompt caching 161
prompt engineering 31–35
  guidelines for agent prompts 33–35
  system prompts 31–33
Pydantic 28
  serializing state with 176
pymupdf 138
python-dotenv library 25, 307

R

RAG (retrieval-augmented generation) 115
  building knowledge bases with 115, 138
  extending agents with callbacks 140–148
  knowledge bases 116–117, 131, 139
  preparing GAIA dataset 131
  search methods 118–123
  vector search 120, 123–131
RAG (retrieval-augmented generation) pipeline, implementing filesystem tools 133–136
ReAct (reasoning and acting) agents 79
  adding structured output 105–109
  architecture overview 82–85
  implementing 99–105
  LLM communication layer 93–98
  overview 80–82
  testing with GAIA benchmark 109–112
  text parsing to tool calling 81
  think-act cycle 80
  tool abstraction 88–93
ReAct approach, limitations of 202
  ExecutionContext 85–88
read operations 135–136, 140, 159
reasoning models 24
recoverability, defined 225
reflection 209–213
  failure recovery 212
  implementing tool 210
  integrating with planning 213–215
  running agent that uses reflection for research synthesis 212
  when necessary/unnecessary 210
remote agents 285–286
  integrating with existing patterns 286
resource management 224
responses, parsing 97
RESULT SYNTHESIS situation 211
resume, implementing 178–181
routers, defined 9
rubrics 293, 297
  building rubric-based evaluation system 301–304
  building trackable indicators 298
  designing evaluation criteria 297

S

sandboxes
  connecting to agents 227–230
  necessity of 224
  options for 225
  porting tools to 231–238
search methods 118–123
  graph search 121
  keyword search 119
  structure-based search 122
  vector search 120
security, defined 224
SELF CHECK situation 211
SentimentAnalysis object 109
sequential workflows 259
serializing state with Pydantic 176
server-client architecture 68
  connecting to MCP server 72
  implementing MCP server 75–77

server-client architecture *(continued)*
running MCP server 70–72
standardized interfaces 69
transport mechanisms 69
servers 283–284
session, defined 151
SessionManager 150, 169, 173
defining Abstract class 169
in-memory implementation 170
sessions, integrating into agents 171–172
short-term memory 151
sibling agents 275
SkillInfo data class 245
skill loader 245
sliding-window strategy 156–157
spans, defined 291
specialized agents 5
src/ directory 122
stability and control 22
state, defined 88
stateless APIs 27
stdio (standard I/O) 70
strict format validation 298
structure-based search 122
structured output 23, 28
summarization 162, 164
SWE-bench 295
system-prompt generation 247
system prompts 13, 31–33

T

Task class 206
task complexity 12
task data structure 206
TaskMemoryManager 191
class initialization 192
duplicate check method 194
integrating into agent 196
memory extraction methods 193
save method 194
search method 195
task request/response 282
task value 12
TavilyClient instance 58
tavily tools 77
telemetry 290
generating, collecting, and exporting 291
temperature parameter 26, 96
text format 222
text parsing 81
think-act cycle 80
tiktoken, defined 157
time to think 202–204
human experts 203
importance of 204
limitations of ReAct 202
token counting 157–158
tokens, defined 5
tool abstraction 88–93
BaseTool 89
FunctionTool 90
integrating MCP tools 92
unified tool interface 88
tool-based retrieval 196
toolbox 64
tool call 48
tool callbacks 141
ToolCall compression 159
tool calling 23, 49–54, 81
executing 51
feeding results back to LLMs 53
guidelines for effective 55–58
running tools 52
setting up tools 51
specifying tool definitions 50
tool composition 221
tool confirmation 175, 181
tool definition 48, 89
converting web search tool to 61–64
tool discovery and execution 69, 144
tool-information disclosure 243–244
ToolResult compression 159
tools 45
as output formatters 105
building 58–67
creating 10
LLMs and 54, 58
LLM tools 47–48
standardizing 68–77
tool calling 49–54
traces, defined 290–291
transfer pattern 271–281
agent tree 273–277

as tool 273
differences between Agent as Tool and Transfer 272
implementing agent transfer 279–281
implementing transfer tool 278
Transfer pattern 257

U

unified tool interface 88
user prompts 13, 31
user role 28

V

vector search 120, 123–131
chunking 124
embedding 123
exercise 127–131
implementing 126
vector databases 127

W

web search tools 58, 111
adding search options 60
challenges of custom tools 66–67
converting to tool definitions 61–64
end-to-end tool execution 64–66
handling errors gracefully 60
starting simple 58–61
Weights & Biases 292
whitelist approval 179
Wikipedia revision tool 212, 233–234
Wikipedia tool 212
Workflow pattern 256
workflows 8–11, 258–266
combining with agents 10
composing 266
design principles 258
developer-defined flow 9
loop 263–265
parallel 261
sequential 259
workspace 238–242
analyzing Excel file 241
concept of 238
implementing tools 239–241
write operations 159

Y

YAML front matter 243

Z

zipfile library 133

(continued from back cover)

After you build the basic agent, you'll progressively expand its capabilities by integrating custom web-search, local file system exploration, and vector-based RAG to navigate complex data. You'll also enable the agent to recall past failures and adapt state persistence through hierarchical context optimization and long-term memory. Along the way, you'll see how to use the Model Context Protocol (MCP) to seamlessly connect your agent to external tools and servers.

Later chapters push into advanced territory. To prevent the agent from hallucinating or looping during complex tasks, you'll implement explicit planning and procedural reflection. Then, you'll explore the CodeAct paradigm, empowering the agent with sandboxed cloud environments to securely write scripts, run CLI commands, and compose tools dynamically. Finally, you'll up-scale your project from a single agent into a collaborative, multi-agent architecture using the A2A protocol, where specialized tasks are routed to isolated sub-agents to maximize token efficiency.

Because highly autonomous agents carry real-world risks, the book culminates in a rigorous observability and evaluation pipeline. You'll implement an automated LLM-as-a-judge system—powered by Open Telemetry—to trace, grade, and continuously refine the agent's trajectory and outputs. By building every layer from the ground up, the book ensures that readers are left with a deep, practical understanding of AI mechanics.

"Delivers the essence of agent behavior instead of temporary usage techniques."

—Dongseok Hyun, NAVER

"Gives you X-ray vision into the anatomy of autonomous agents, so the tools you use every day stop being black boxes."

—Toby Lee, author of *Toby's Spring Framework Bible*

"Structured progression from fundamentals to advanced topics makes the book a strong long-term reference."

—Balaji Salem Balasundram, Amazon

"An absolute gold. The essential book for the agentic AI era."

—James Liu, Mediaocean

"Clear, well structured, and hands-on."

—Maksim Nechaev, Snoonu

"A practical guide for anyone who wants to move beyond demos and create agentic systems with confidence."
—Elliot Kim, Coupang Inc.

Build an AI Agent (From Scratch) is a fascinating step-by-step journey through design, development, and deployment of a system of autonomous AI agents. Authors Jungjun Hur and Younghee Song guide you concept-by-concept as you build a sophisticated research agent in Python, designed to solve complex, multi-step tasks from the rigorous GAIA (General AI Assistants) benchmark. By seeing the construction of the system end to end, you'll understand how the underlying mechanics unobstructed by any existing black-box libraries and frameworks.

This book includes ten meaty chapters and a handy appendix (about how to get an OpenAI API key). The technical explanations throughout the book are made crystal clear by the use of many helpful diagrams. As one reviewer remarked, even for someone building their first agent, "nothing feels like magic or a mystery after the first few chapters!"

Early on, you'll establish the foundational blueprint for your agent by distinguishing rigid developer-defined workflows from true, LLM-directed agent loops. You'll learn how to manage stateless APIs, enforce structured outputs using Pydantic, and implement dynamic tool-calling. Crucially, the book centers on "Context Engineering"—the discipline of systematically structuring information to prevent context rot and the "Lost in the Middle" effect. One technical reviewer particularly appreciated that core concepts like agent loops, context engineering, and evaluation are introduced with clear motivation and practical examples rather than as isolated abstractions.

(continued on last page)

The book covers

- End-to-end construction of a framework-free, autonomous AI agent in Python
- Advanced practical integrations including MCP, vector-based RAG, long-term memory, and sandboxed code execution
- Multi-agent orchestration and automated LLM-as-a-judge evaluation pipelines

About the reader

For Python developers with some knowledge of machine learning who want to build and deeply understand agentic systems without relying on abstract libraries.

About the author

Jungjun Hur is an independent AI and data engineer who specializes in building agentic systems for e-commerce. He is the author of *Practical AI Application Development Using LLMs*. **Younghee Song** is an AI consultant at PricewaterhouseCoopers (PwC). Both authors live and work in South Korea.

FREE eBook
see first page

or go to
https://www.manning.com/freebook

ISBN-13: 978-1-63343-461-5

90000
9 781633 434615

MANNING